

MACHINE LEARNING — IN — BUSINESS FINANCE USING PYTHON

Kian Guan Lim



MACHINE LEARNING
— IN —
BUSINESS FINANCE
USING PYTHON

World Scientific Series on Financial Data Analytics

Print ISSN: 2737-5943

Online ISSN: 2737-5951

Series Editor: Lim Kian Guan (*Singapore Management University, Singapore*)

Published

Vol. 2 *Machine Learning in Business Finance using Python*
by Kian Guan Lim

Vol. 1 *Risk Analytics: From Concept to Deployment*
by Edward H K Ng

MACHINE LEARNING — IN — BUSINESS FINANCE USING PYTHON

Kian Guan Lim

Singapore Management University, Singapore



Published by

World Scientific Publishing Co. Pte. Ltd.

5 Toh Tuck Link, Singapore 596224

USA office: 27 Warren Street, Suite 401-402, Hackensack, NJ 07601

UK office: 57 Shelton Street, Covent Garden, London WC2H 9HE

Library of Congress Control 2025013899

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library.

World Scientific Series on Financial Data Analytics — Vol. 2
MACHINE LEARNING IN BUSINESS FINANCE USING PYTHON

Copyright © 2026 by World Scientific Publishing Co. Pte. Ltd.

All rights reserved. This book, or parts thereof, may not be reproduced in any form or by any means, electronic or mechanical, including photocopying, recording or any information storage and retrieval system now known or to be invented, without written permission from the publisher.

For photocopying of material in this volume, please pay a copying fee through the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923, USA. In this case permission to photocopy is not required from the publisher.

ISBN 978-981-98-1123-6 (hardcover)

ISBN 978-981-98-1168-7 (ebook for institutions)

ISBN 978-981-98-1169-4 (ebook for individuals)

For any available supplementary material, please visit

<https://www.worldscientific.com/worldscibooks/10.1142/14271#t=suppl>

Desk Editors: Nambirajan Karuppiah/Lum Pui Yee

Typeset by Stallion Press

Email: enquiries@stallionpress.com

Printed in Singapore

To
Miranda, Ethan, Reia, and Theo
May they seek the joy in the Lord

This page intentionally left blank

Preface

Major results in mathematical statistics theory such as the Binomial Theorem, the Laplace distribution, the Gaussian or normal distribution, the logarithmic distribution, the Poisson distribution, the Bernoulli distribution, Bayes theorem, and many others were created by giants in mathematics before the 1800s. The early form of central limit theorem was discovered in the 1800s.

In the early 1900s till early 1930s, statistical theories including rigorous formulations of probability distributions by Pearson, the design of experiments and the maximum likelihood method by Fisher, concepts of testing by Neyman and others, were born. The extreme value theory was formulated in the 1920s. The practice of statistics had become rigorous within the framework of probability theory. The latter could have been from the time of Pascal and Fermat, though in modern times, it could have been based on the measure-theoretic approach by Kolmogorov.

The march of statistical applications continuing into modern rigorous and analytical forms would not have been possible without the great efforts and tirelessness of generations of brilliant minds in these fields.

Machine learning is a new wave of approach that might have begun in the late 1950s but picked up momentum since the 1990s. It is a scientific approach that is partly statistical in perspectives and partly engineering in spirit, driven by explosions in data quantities or big data and the commensurate increase in the capacity of computer machines to manage and scrutinize these data. It has come to be somewhat hinged directly with the more general domain of data science and artificial intelligence.

This book is an introduction to machine learning using Python programming language with applications in finance and business. Coverages include the prediction methods of logistic regression, Naïve Bayes, k-Nearest Neighbor, Support Vector Machine, Random Forest, Gradient Boosting, and various types of Neural Networks. Performance measurements and assessments of feature importance are also explained. The book also contains detailed examples of the applications with data. Python codes are explained in a step-by-step manner using Jupyter Notebook so that the readers can practice on their own.

About the Author



Lim Kian Guan is Professor Emeritus of Quantitative Finance at Singapore Management University (2001–2024). Previously he was Professor at National University of Singapore (1980–2001). He obtained his PhD, MS, MA from Stanford University (1981–1986) and BSc (Hons I) from UMIST (1975–1978). He has published several books with World Scientific and de Gruyter Berlin and was an editorial board member of *International Journal of Data Science and Analytics* with Springer. For the detailed CV, please see <https://sites.google.com/site/kianguan1>.

This page intentionally left blank

Acknowledgments

Some of the data exercises created in this book would not have been possible without the availability of data sets on public websites such as Kaggle. The facilitation of various open-source data sets used in this book are acknowledged as follows.

+Yahoo Finance

(Chapter1-1.ipynb to Chapter1-7.ipynb, Chapter8-1.ipynb, Chapter8-2.ipynb, Chapter8-3.ipynb)

Singapore Housing and Development Board – Resale Flat Prices
(Chapter2-1.ipynb)

Kaggle – Credit Card Fraud Detection

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
(Chapter4-1.ipynb)

Kaggle – California Housing Prices

<https://www.kaggle.com/datasets/camnugent/california-housing-prices>
(Chapter4-2.ipynb)

Kaggle – Corporate Credit Rating

<https://www.kaggle.com/datasets/agewerc/corporate-credit-rating>

(Chapter5-1.ipynb, Chapter5-2.ipynb, Chapter5-2M.ipynb, Chapter6-1.ipynb)

Kaggle – Churn Modelling

<https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling>

(Chapter7-1.ipynb to Chapter7-8.ipynb)

Contents

<i>Preface</i>	vii
<i>About the Author</i>	ix
<i>Acknowledgments</i>	xi
<i>Introduction</i>	xvii
Chapter 1 Financial Data Correlations	1
1.1 Portfolio Diversification	3
1.2 Worked Example: Data	4
1.3 Forming Optimal Portfolios	19
1.4 Denoising the Correlation Matrix	21
1.5 Using a More Accurate Forward Predictor	32
1.6 Concluding Thoughts	33
References	34
Chapter 2 Regression and Regularization	35
2.1 Regression and Regularization	39
2.2 Worked Example: Data	44
2.3 Linear Regression Prediction	48
2.4 Fine-Tuning Hyperparameters	50
2.5 Prediction Using Hold-Out Test Set	55
2.6 Cross-Validation	56
2.7 Concluding Thoughts	63
References	64

Chapter 3	Corporate Reporting Data and Analyses	65
	3.1 Financial Statements	66
	3.2 Binary Classification Performance Metrics	70
	3.3 Logistic Regression	79
	3.4 Worked Example: Data	80
	3.5 Dimension Reduction and Principal Component Analysis	87
	3.6 Concluding Thoughts	92
	References	93
Chapter 4	Naïve Bayes, k-NN, and Support Vector Machines	95
	4.1 Naïve Bayes	95
	4.2 k-Nearest Neighbors Algorithm	106
	4.3 Support Vector Machine	109
	4.4 Lagrange Duality Solution	116
	4.4.1 Non-linear kernels in SVM	119
	4.4.2 Linear non-separability in SVM	121
	4.4.3 Other aspects of SVM	123
	4.5 Support Vector Regression	124
	4.6 Worked Examples: Data	126
	4.6.1 Naïve Bayes algorithm	128
	4.6.2 K-Nearest Neighbor algorithm	130
	4.6.3 Support vector machine	132
	4.6.4 Support Vector Regression	133
	4.7 Concluding Thoughts	134
	References	135
Chapter 5	Decision Trees, Random Forest, and Multi-Class Prediction	137
	5.1 Decision Tree Method	138
	5.2 Worked Example DT: Data	146
	5.3 Random Forest	152
	5.4 Multi-Class Classification	160
	5.5 Concluding Thoughts	166
	References	167
Chapter 6	Gradient Boosting, SHAP Values	169
	6.1 Gradient Boosting	169
	6.2 Popular Alternatives	183
	6.3 Worked Example GB: Data	185

6.4	Shapley and SHAP Values	192
6.4.1	SHAP values (Shapley Additive exPlanations)	199
6.5	Concluding Thoughts	203
	References	205
Chapter 7	Artificial Neural Network I	207
7.1	Forward Propagation	215
7.1.1	Activation function $f(\cdot)$	216
7.1.2	Error or loss functions	217
7.2	Backward Propagation	219
7.3	Worked Example: Data	227
7.4	Improving the NN and Understanding the Results	238
7.5	Concluding Thoughts	245
	References	245
Chapter 8	Artificial Neural Network II	247
8.1	Recurrent Neural Network	249
8.2	Worked Example I: Data	258
8.3	Variants of RNN	265
8.4	Worked Example II: Data	269
8.4.1	Bidirectional	276
8.4.2	Gated recurrent unit	277
8.4.3	Convolutional neural network	278
8.5	Concluding Thoughts	283
	References	284
	<i>Index</i>	285

This page intentionally left blank

Introduction

Machine learning is a type of or specific domain of artificial intelligence using computer algorithms in solving specific problems, typically involving optimization, to mainly derive prediction or else classification (including visual, audio, and textual recognition) from a set of data with possibly complex patterns in a high dimension space. Prediction more generally can also mean prediction of a class or category. The solution of the problem leads to better decision-making. The algorithms are largely based on mathematical and statistical models as well as numerical methods of optimization. The applications are often scalable for wide usages.

It is useful to understand the machine learning approach in comparison with the more traditional statistical approach. They have large overlaps but also some differences. Classical statistics and econometrics (including regressions) involve estimation, inference (or testing), prediction, and grouping.

Estimation of embedded model parameters is a frequent practice in the statistical evaluation of a theoretical model whether it is based on economics or based on assumed probability distributions of the underlying variable data or based on both. It is usually accompanied by the testing or inference on the model, whether the model is acceptable or to be rejected. The test statistic used in the testing of the null hypothesis follows a probability distribution whether in finite sample or based on asymptotic (large sample) statistical theory. Hence, a formal process of comparing the test statistic p -value with the assumed level of statistical significance can be done.

In machine learning, however, embedded model coefficients or feature impact are often part of a “black box” – oftentimes inconvenient to enunciate, much less test, such as in neural networks. The impact of features or explanatory variables may however be measured in other metrics, typically not by underlying statistical distributions of the parameters. Machine learning approach became important due to its ability to perform good forecasts or predictions, often based on complex and unstructured data that more traditional statistical approach finds it inconvenient to model. The data set input to machine learning could be very large while smaller data sets typically are not so useful. The latter are, however, applicable in traditional econometrics and statistical inferences.

In data analytics for prediction (or we may simply call it predictive analytics) – that part of machine learning for prediction and classification – there is usually not an economic-theoretical model. Explicitly assumed probability distributions underlying the variable data may or may not be utilized. The basic philosophy is that the realized data should speak loudest for themselves as they are.

Explanatory variables or features are often subject to pre-processing such as cleaning by removing missing data, handling outliers, and converting data types. Then there is feature engineering by reconstructing or extracting new features from the existing data. Often the features are also scaled or standardized to avoid extreme value impact from features with large scales before feeding into the machine learning algorithms.

In an example of the problem of prediction using neural networks, a dependent variable (or target variable or label) is used in conjunction with features. Weights are put on the features and a functional form or activation function is then put on the aggregate weighted features to derive a predicted value. For prediction of classes, the activation function typically shapes the outcome(s) into a probability number that lies between 0 and 1. The highest probability number indicates prediction of that associated class. An optimization criterion such as minimizing least squares between target variable and predicted value is used to reset the embedded weights (parameters) in the machine learning model. There is no testing since there is no specific probability distribution of the estimators. More generally, we can think of a machine learning algorithm as a complex function $f_1(f_2(f_3(\dots|\theta_3)|\theta_2)|\theta_1)\{X\} \rightarrow Y$ mapping a set of features $\{X\}$ to outcome(s) Y whether Y is a numerical target or a classification type. The functions f_k for $k = 1, 2, 3, \dots$ are governed by parameters θ_k (for all k) that are estimated in the process. There are also hyperparameters that do not

affect the functions directly but are used for fine-tuning the optimization process for better prediction capability. Different algorithms produce different mappings, and new methods are being introduced at a rapid rate in the machine learning world.

The approach to prediction in machine learning typically adopts the following pathway. The data set is divided into a training data set (usually involving 50–60% of the data), a validation data set (usually involving 20–30% of the data), and a test data set (remaining part of the data set). If there is no hyperparameter in the model/algorithm, there is no need for a validation data set; just the training data set (involving 60–80% of the data) and test data set (remaining part of the data set) will do. Given hyperparameter values, the algorithm is used to find optimal model parameter values that produces optimal fit between target values and predicted values in the training data set. The optimal fit is based on some specified criterion such as Minimum Mean Square Error (MMSE) in regression prediction or such as Minimum Cross Entropy (MCE) in classification prediction. The fitted parameter values from the training data are then utilized to predict target values in the validation data set based on the validation data set features. The prediction performance in the validation set target values and the predicted values are noted. The use of the validation set is to select the best model and associated best hyperparameter values so that the prediction performance in the validation data set is maximized while utilizing the training data set to compute the model parameters based on the hyperparameters.

The above seems like a long-winded way unlike usual approach in classical regressions, which is to utilize a training data set to estimate the parameters (perhaps including hyperparameters) in an optimal manner, and then use the fitted model to perform out-of-sample test on a separate test data set. The additional use of a validation data set is to check on problems of over-fitting in the training data set with under-fitting in the validation data set. Any such problem is to be mitigated or minimized with possible selection of optimal hyperparameters (“fine-tuning” the hyperparameters). The downside of avoiding a validation is that there is a trade-off loss of robustness since only the test set is left to provide accreditation of the fitted model.

These fine-tuned optimal hyperparameters are then used to fit the model with the test data set using estimated parameters from the training data set. If the test data model performance is similarly good, then the

fitted model can be used in generalized data for good prediction. If not, then the model has to be re-considered.

In evaluating the test data set performance, the model is sometimes re-trained (re-estimating the parameters) using the combined training and validation data sets (a larger sample size). The re-training uses the optimal hyperparameters derived in the validation phase. This is then used to fit the model with the test data set. In the examples in this book except for Chapter 2, for parsimonious reasons, the hyperparameters applied in the training data set are assumed to be already fine-tuned from some earlier validation phase.

In non-time series cases, it is also possible to employ a cross-validation method to perform hyperparameter selection, avoiding the use of a separate validation set. However, in time series prediction, cross-validation may not be appropriate, hence a separate validation data set may still be necessary to fine-tune hyperparameters.

In some machine learning methods, specific probability distributions may be used. A dependent variable (or target variable or label) in a regression model with explanatory variables (or features) is typically modeled with a random disturbance term or innovation or noise added to the explanatory variables. By assuming that the noise follows a particular probability distribution, maximum likelihood methods with desirable large sample properties can be used. This method is also recognized as part of machine learning approach as it involves optimization of a function (in this case, a likelihood function) to obtain the parameter estimates to be used for prediction. As a specific example, the logistic regression for classification is a method recognized in both classical econometrics as well as in machine learning.

In machine learning models, the main objectives are to predict and/or to classify. If the dependent (target) variable is a cardinal measure (interval and ratio scale), e.g., stock price, then the prediction would be the next period stock price. If the dependent (label) variable is a nominal variable or a category or a class, then the prediction given the features associated with each label would be what the label is given a set of new features. When a target or label variable is known and training and validation data sets are used to select optimal model parameters and hyperparameters to make prediction based on test data and future data sets, then it is called supervised machine learning.

In unsupervised machine learning, the data are not labeled or there is no specification of a target variable. Typically, unsupervised learning is

used to form clusters or distinct groupings, reduce dimensions of data, and perform associations, e.g., Amazon's book purchase recommendation based on a customer's prior book orders and the accumulated book order pattern of a large number of other customers. There is no human intervention in the discovery or mining process. It also includes text mining, categorization, and textual analyses that have become entrenched tools for business analyses in the wake of enormous quantities of unstructured data such as emails, news and social media, chats, opinion blogs, product reviews and so on. Unsupervised method such as Principal Component Analysis (PCA) is also commonly used to reduce feature selection or perform feature extraction (combination of features) to enable more effective and efficient predictions.

There are, however, methods of machine learning that work together with classical statistical approach. For example, stationarity of data of the features and of the targets is often of primary importance in classical econometrics, although some specific time series structures could lead to cointegration regressions. For machine learning models, data containing similarities to history is important for trained models to be able to predict future events given test data features. This is akin to out-of-sample prediction in classical econometrics. While strong assumptions of stationarity are typically not required in machine learning, any data deemed non-stationary or having unit roots in time series could best be pre-processed to ensure that the machine learning prediction results can be improved. Sometimes, prediction can still be effective for a non-stationary target when non-stationary features that are used in the machine learning can combine many features and this could help in the prediction without necessarily testing for unit roots or cointegration. Cross-sectional regressions can be combined with a machine learning algorithm to predict the future set of regression coefficients or risk premia.

In addition to maximum likelihood method, the popular GARCH model in finance can in principle be estimated using machine learning method such as SVM and neural networks. Machine learning has been making major and beneficial supplementations to classical statistical methods in the recent two decades; or it may be deemed as competitive alternative methods. However, there are some areas where statistical and theoretical models with testing are still very important – a major advantage of statistical model is to provide policy makers (not just forecasters) with what (future) decision variables to control once the theoretical-economic model is tested.

This book is an introduction to data analytics using Python programming. The book contains worked examples of machine learning models in finance and business. Many of the examples are also related to data strategy in FinTech applications. Details of the codes via *.ipynb files using Jupyter Notebook (a web-based interactive computing platform) are provided in the book for readers to do hands-on practices or experiential learning. Elaborate comments `###` are inserted in the *.ipynb files to facilitate learning of the different steps in the Python codes as well as to provide learning points on outcomes. We use typically open-source Python software libraries and APIs (application programming interface). Demonstration ipynb files and associated data sets in *.csv format are provided in <https://sites.google.com/site/kianguan1/>. We will typically read in the data set as a Pandas DataFrame.

Python can be downloaded from <https://www.python.org/downloads/>, a distribution and tools platform, that offers a range of data science tools, including Jupyter, that run on the Python language. Download the latest version of Anaconda from <https://www.anaconda.com/distribution/>. Ensure you download the version for your device's Operating System.

Several of Python's open-source libraries or packages are commonly used, e.g., Pandas, Numpy, math, Matplotlib, Scikit-Learn (Sklearn), SciPy, Keras, TensorFlow, etc. These libraries or packages contain more efficiently written reusable modules with codes that need not be rewritten from scratch. For installation and import of the libraries and packages, check for latest update in the documentations on public websites, where necessary. Many of the packages call up other numerical methods to search for optimums – we explain some of these, but details of numerical routines (the numerical methods) are outside the scope of this book.

Chapter 1 begins with the central idea of correlation and variance in data behavior and associations. An important application of variance reduction in finance is in investment portfolio diversification. It is shown that optimal portfolio weights computed from past history may not work well in future investment performance. A more fruitful approach is to supplement the portfolio optimization method with machine learning predictions of the stocks' future returns and future return variances. These can then be used to select predictably profitable stocks for inclusion in the portfolio. The predicted variances can be used to compute the forward-looking portfolio variances for the portfolio optimization.

Chapter 2 introduces a simple regression framework and shows the machine learning approach with use of training, validation, and test data

sets, and the idea of selecting optimal hyperparameters. The cross-validation method is also explained. Technical coverages include the concept of regularization and methods such as Lasso, Ridge, and Elastic Net regressions.

Chapter 3 explains key accounting statements such as the balance sheet, income statement, and cashflow statement. There are important machine learning applications using accounting data as features to predict firm performances. Prediction performance metrics are explained. PCA is introduced as a dimension reduction method to obtain fewer features and yet retain most of the data variations when the data set contains a large number of features. Fewer features may enable better prediction and more efficiency in computing. Logistic regression is also introduced to show how prediction of earnings increase or decrease can be done.

Chapter 4 explains classification algorithms such as Naïve Bayes, k -Nearest Neighbors, and Support Vector Machine algorithms. Applications to predicting credit card frauds are shown.

Chapter 5 engages the same data sets in Chapter 3 but employs popular and effective predictive as well as classification algorithm in decision trees and random forest. The ensemble method of the bagging approach of random forest is illustrated in an application to credit analysis. Multi-class prediction is discussed.

Chapter 6 continues with decision trees but uses the gradient boosting approach in adjusting sequential trees to grow from weak predictors to strong predictors. Shap values are explained.

Chapters 7 covers predictions using artificial neural networks. Multi-layer perceptron with its forward passes and backpropagation, the error/loss functions, and the optimization algorithm are discussed. The applications show the importance of having enough depth in the neural networks and having adequate iterations by training with mini-batches and a sufficient number of epochs.

Chapter 8 continues with more complicated neural networks including Recurrent Neural Network (RNN), Long Short-Term Memory Network (LSTM), gated recurrent units, and Convolutional Neural Network (CNN). Applications in finance include predicting of customer retention, stock prices, bankruptcy prediction, and so on.

There is no doubt the field of machine learning is being developed very rapidly at the cutting edges. There are all kinds of new interesting and insightful algorithms. I hope readers will enjoy the learning process charted in this book and step up to greater heights in their future learning journey.

This page intentionally left blank

Chapter 1

Financial Data Correlations

In this chapter, we introduce the idea of financial data correlations. This is the fundamental relationship driving most financial investment and trading models. If every financial data series such as stock returns is probabilistically independent of another, then there are no systematic risk factors affecting all stocks – each stock moves by its own independent unique risk. The latter is not reflective of actual stock market and corporate sector linkages. Listed companies do share common risks such as when they are in the same industry that is affected by an industry change or when they are all affected by the cost of funds when market interest rates change. When stock returns co-move or have non-zero correlations, they can be combined in a portfolio in an optimal way to maximize the investor’s welfare. For example, if an investor wants to keep the risk proxied by portfolio return volatility to a minimum, then the portfolio can be chosen by selecting weights on stocks forming the portfolio such that the portfolio has minimum return volatility (square root of variance). There are many other ways to optimize a portfolio (selecting optimal weights of composition stocks) depending on what is the investor’s objective function. We will discuss these concepts and computations using machine learning approach in this chapter.

Before that we do a quick review of two of the most popular and widely used packages in Python – Pandas and Numpy. Both also work well with the plotting package Matplotlib. Numpy is a library for mathematical computations. Numpy looks at data mainly via arrays (Numpy arrays). A 1-dimensional (1D) array (or a flat array) of [1,2,3,4,5] can be created as “`ar1=np.array([1,2,3,4,5])`” after installing numpy as

“import numpy as np”. The open and close quotation marks “ ” are not part of the command statement. “np.” is important so the code will call up numpy to execute the array command. The array’s dimension is 1 or a rank of 1. When `ar1.shape` is run, the output is (5,) indicating only one-dimension or a list of 5 numbers. If instead, “`ar2=np.array([[1,2,3,4,5]])`” is entered, the output `ar2` is a 2D or rank 2 matrix with output (1,5) when `ar2.shape` is run. Note the second set of brackets makes the entry in () a list of one list, so it becomes 2D matrix with one row and 5 entries in that row. If instead, “`ar3=np.array([[[1,2,3,4,5]]])`” is entered, the output `ar3` is a 3D or rank 3 matrix with output (1,1,5) when `ar3.shape` is run. If “`ar4 = np.array ([[[1,2,3,4,5],[6,7,8,9,10]])`”, then `ar4.shape[0]` outputs the number of rows as 1; `ar4.shape[1]` outputs the number of columns as 2, and `ar4.shape[2]` outputs the third dimension as 5. Output of “`np.array ([[[1,2,3,4,5],[6,7,8,9,10]])`” is (1,2,5).

Output of “`np.array ([ [1], [2], [3], [4], [5] ]).shape`” is (5,1), and so on. Matrix multiplication or `.dot` can be used on numpy arrays with conformable dimensions.

Pandas is a library for data manipulation and is highly compatible with Numpy. It operates on tables such as in an Excel file. A numpy 2-dimensional (2D) or higher dimension array can be directly passed onto Pandas as a dataframe object, e.g., “`dfar2=pd.DataFrame(ar2)`” and printing the dataframe “`Print(dfar2)`” gives

```
“  0 1 2 3 4
   0 1 2 3 4 5 ”
```

with an additional index variable 0 on the leftmost column and named columns 0, 1, 2, 3, 4, respectively, on the first row. The leftmost index basically counts the number of rows but it starts at the number 0 and goes up to the length of the dataframe minus 1. The columns can also be renamed, e.g., “`dfar2.columns=['new1','new2','new3','new4','new5']`” will produce

```
“  new1 new2 new3 new4 new5
   0    1    2    3    4    5 ”
```

if printed. Some commands are Pandas-based, such as `dfar2.info()` and `dfar2.describe()`, and work only with dataframe inputs and not with numpy arrays. For more information on the use of Numpy, Pandas, and plotting, see <https://towardsdatascience.com/the-data-science-trilogy-numpy-pandas-and-matplotlib-basics-42192b89e26/> by H. Parra (2021).

1.1 Portfolio Diversification

Markowitz efficient portfolio frontier (see Markowitz (1952)) represents the boundary of portfolios of risky stocks that have the minimum return variance given the expected portfolio return above the minimum variance portfolio (MVP) return. With a continuous efficient frontier, it is also the maximum expected return given corresponding variance or else volatility (square root of variance) of the portfolio. For clarity, the results are often termed Markowitz mean-variance analysis. The plot of mean return versus volatility (or standard deviation of returns or “risk”) when there is no short-sale constraint is a hyperbola, and is often called the portfolio efficient frontier (for the upper part above the minimum variance). The plot of mean return versus variance of returns when there is no short-sale constraint, however, is a parabola.

There is an exact analytical solution if there are no short-sale constraints, i.e., some stocks could have a negative portfolio weight. However, when there is a short-sale constraint, all stocks must be held in positive or else zero quantities. With short-sale constraint and the capital constraint that all portfolio weights must sum to one (no idle money), the solution to obtain minimum portfolio return volatility, given expected portfolio return above the MVP return, is typically found via numerical optimization methods. In this chapter, we use the Scipy package/library in Python for such optimizations.

Let there be N stocks and a feasible portfolio is formed with weight w_i on stock i . It is feasible based on the constraints $\sum_{i=1}^N w_i = 1$, and $w_i \geq 0$ for every i . Let the weight vector be $w = (w_1, w_2, w_3, \dots, w_N)^T$. Let the expected return¹ of stock i be $E(r_{it}^*)$ at time t . The expected return vector is $\mu = (\mu_1, \mu_2, \mu_3, \dots, \mu_N)^T$, where $\mu_i = E(r_{it}^*)$. If r_{it}^* is stationary, then its unconditional expectation is constant for every t . The covariance matrix of $(r_{1t}^*, r_{2t}^*, r_{3t}^*, \dots, r_{Nt}^*)^T$ for each t is given as $\Sigma_{N \times N}$. A MVP (portfolio with the minimum return variance) can be found as follows.

$$\min_w w^T \Sigma w \text{ subject to } \sum_{i=1}^N w_i = 1 \quad \text{and} \quad w_i \geq 0 \quad \text{for every } i \quad (1.1)$$

The superscript T represents a matrix transpose. The objective function in the solution w_{MVP} is the minimum return variance, $w_{\text{MVP}}^T \Sigma w_{\text{MVP}}$, associated with the MVP. In Eq. (1.1), short-sale constraints are imposed.

¹* or asterisk to r_{it} denotes that the return is adjusted for dividends.

Short-sale of stocks by borrowing faces stiff exchange regulations and itself can be costly due to borrowing fees even if the stock is available for loan by stock dealers. It is also more difficult to borrow stocks when the market anticipates a downturn.

Another type of optimal portfolio is that of maximizing the Sharpe ratio defined as $w^T \mu / \sqrt{w^T \Sigma w}$ which is expected portfolio return per unit of risk or per unit of portfolio return standard deviation. Usually, the numerator is expected portfolio return in excess of the risk-free rate. We use this simplified definition when the risk-free rate may be small and negligible. This is solved as:

$$\begin{aligned} & \min_w \sqrt{w^T \Sigma w} / w^T \mu \\ & \text{subject to } \sum_{i=1}^N w_i = 1 \quad \text{and} \quad w_i \geq 0 \quad \text{for every } i \end{aligned} \quad (1.2)$$

Finally, the minimum variance of a portfolio for a given expected return k (larger than or equal to the expected return of a MVP) can be found by solving:

$$\begin{aligned} & \min_w w^T \Sigma w \\ & \text{subject to } \sum_{i=1}^N w_i = 1 \quad \text{and} \quad w_i \geq 0 \quad \text{for every } i, \text{ and } w^T \mu = k \end{aligned} \quad (1.3)$$

In model (1.3), maximum k for $\sum_{i=1}^N w_i = 1$ and $w_i \geq 0$ occurs when $\mu_m = \max(\mu_1, \mu_2, \mu_3, \dots, \mu_N)$ and $w_m = 1$ while $w_{i \neq m} = 0$. Then $\max k = \mu_m$. For any k that follows $w_{\text{MVP}}^T \mu \leq k \leq \mu_m$, we can find solution $w(k)$ such that the efficient portfolio frontier occurs at expected return k and portfolio return volatility $\sqrt{w(k)^T \Sigma w(k)}$.

1.2 Worked Example: Data

Stock price data are obtained from public source Yahoo Finance. In this first part of the data work, eight of the largest ecommerce listed companies, viz. Alibaba, Amazon, EBay, Rakuten, Suning, Wayfair, Zalando, and JD.Com are examined. The program merges the eight individual stock price data sets in code line [25], and then computes their continuously compounded return rates in code line [28]. Portfolio optimizations based on the daily stock return series are carried out in code lines [36–38]. See demonstration file Chapter1-1portecom.ipynb.

```
[1]: ### This module computes returns and mean-var frontier given each
### "expected" return based on averaged realized return
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

[2]: ### We can import the data set. The dataframe name is now df. By default,
### date columns are represented as object when loading data from a CSV file
df = pd.read_csv('###Alibaba_USD.csv', parse_dates=True)
### If True and parse_dates is enabled, pandas will attempt to infer the
### format of the datetime strings in the columns
### See date grouping codes in https://stackoverflow.com/questions/11391969/
### how-to-group-pandas-dataframe-entries-by-date-in-a-non-unique-column
### If reading a .txt file, other options about in-between spaces may need to
### be specified, e.g. ...header=None, delimiter=r"\s"...
### Columns can be renamed viz. ...pandas.read_csv(..., parse_dates=True, \
### names=['col1', 'col2', 'col3', 'col4', 'col5', 'col6'])
### Note the parsed Date column cannot be renamed

[3]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1965 entries, 0 to 1964
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        1965 non-null   object
1   Open        1965 non-null   float64
2   High        1965 non-null   float64
3   Low         1965 non-null   float64
4   Close       1965 non-null   float64
5   Adj Close   1965 non-null   float64
6   Volume      1965 non-null   int64
dtypes: float64(5), int64(1), object(1)
memory usage: 107.6+ KB

[4]: df.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	19/9/2014	92.699997	99.699997	89.949997	93.889999	93.889999	271879400
1	22/9/2014	92.699997	92.949997	89.500000	89.889999	89.889999	66657800
2	23/9/2014	88.940002	90.480003	86.620003	87.169998	87.169998	39009800
3	24/9/2014	88.470001	90.570000	87.220001	90.570000	90.570000	32088000
4	25/9/2014	91.089996	91.500000	88.500000	88.919998	88.919998	28598000

Code lines [3] and [4] show information about the data type in the various columns read into data frame df. Data type “float64” or 64 bit (double precision) floating point number which is a real number with a decimal point – the decimal point can “float” to a correct position. Floating point numbers can be represented in computer programs as a base 2 (binary)

fraction that is convenient for computations. An integer can typically be converted to a floating number for computations. `df.head()` prints the first five lines of the data frame `df`.

```
[5]: df[["day", "month", "year"]] = df["Date"].str.split("/", expand = True)
    ## Above splits the Date entry into 3 separate columns
    print("\nNew DataFrame:")
    print(df)
    ## Note this split command does not work if ...parse_dates=['date']...
    ## is entered into the pd.read_csv(..)
```

```
New DataFrame:
   Date      Open      High      Low      Close  Adj Close \
0  19/9/2014  92.699997  99.699997  89.949997  93.889999  93.889999
1  22/9/2014  92.699997  92.949997  89.500000  89.889999  89.889999
2  23/9/2014  88.940002  90.400003  86.620003  87.169998  87.169998
3  24/9/2014  88.470001  90.570000  87.220001  90.570000  90.570000
4  25/9/2014  91.089996  91.500000  88.500000  88.919998  88.919998
...
1660 5/7/2022  114.510002  120.529999  112.139999  120.129997  120.129997
1661 6/7/2022  118.930000  120.000000  115.510002  119.120003  119.120003
1662 7/7/2022  120.629997  125.000000  120.629997  122.389999  122.389999
1663 8/7/2022  122.260002  125.839996  120.699997  120.900002  120.900002
1664 11/7/2022  115.459999  115.500002  109.330002  109.570000  109.570000

   Volume day month year
0  271879400  19   9  2014
1   66657800  22   9  2014
2   39009800  23   9  2014
3   32088000  24   9  2014
4   28598000  25   9  2014
...
1660 20989300  5   7  2022
1661 20222700  6   7  2022
1662 24202000  7   7  2022
1663 27201200  8   7  2022
1664 31249900  11  7  2022

[1665 rows x 10 columns]
```

In code line [5], the object of Date can sometimes be usefully separated into day, month, and year. These are then converted to floating numbers in code line [8] should there be a need to use them as numbers for some computations.

```
[8]: cols = df.select_dtypes(exclude=['float']).columns
    df[cols] = df[cols].apply(pd.to_numeric, downcast='float', errors='coerce')
    ## Above selects non-float type as cols and converts the Date, day, mth,
    ## year from objects to float#
    ## errors = 'coerce' sets invalid parsing as NaN such as Date column above
    df.info()
```

Code line [9] selects only rows in the data set `df` that have years (one of the features or columns) between 2017 and 2021 inclusive. This sub data set is now in a data frame called `dfAlibaba`. The original Date column is dropped. The data frame is also redefined to include only four columns of “Adj Close”, “day”, “month”, and “year”. Using “Adj Close” or adjusted closing price of the stock is preferable for computing daily return rates.

The adjustment is for dividend issue and other stock distribution such as stock dividends and stock splits. This is explained as follows.

```
[9]: ### Select only rows where year >= 2017 and <=2021, i.e. 5 years
dfAlibaba = df[(df['year']>=2017)&(df['year']<=2021)]
dfAlibaba = dfAlibaba.drop('Date',axis=1)
### 1 is the axis number (0 for rows and 1 for columns)
### If we do not drop the Date column, Date will be shown as NaN
### Includes only rows where years are between 2017 and 2021 inclusive
### Next remove all columns except keeping Adj Close (closing price including
### dividends and split effects) and day, month, year
dfAlibaba = dfAlibaba[['Adj Close','day','month','year']]
print(dfAlibaba)
```

	Adj Close	day	month	year
576	88.599998	3.0	1.0	2017.0
577	90.510002	4.0	1.0	2017.0
578	94.370003	5.0	1.0	2017.0
579	93.889999	6.0	1.0	2017.0
580	94.720001	9.0	1.0	2017.0
...	...	...	...	...
1830	116.589996	27.0	12.0	2021.0
1831	114.800003	28.0	12.0	2021.0
1832	112.089996	29.0	12.0	2021.0
1833	122.989998	30.0	12.0	2021.0
1834	118.790001	31.0	12.0	2021.0

```
[1259 rows x 4 columns]
```

In U.S., listed or public firm typically pays dividends on a quarterly basis. Suppose on date Y , a firm announces that some dividends are planned to be paid (payment of cash) on date $Y + 20$ days. There are two other important dates in-between, e.g., $Y + 10$ is the ex-dividend date and $Y + 11$ is the record date. Any investor J who buys the stock from investor I on or after the ex-dividend date of $Y + 10$ will not be paid the announced dividends. Investor I who had purchased the share before ex-dividend date and had not sold by $Y + 10$ will receive the dividends even if he/she sold by $Y + 20$. As stock buy/sell transaction will typically take more than a day for settlement, a sell by I at $Y + 10$ would not appear on the record date so the investor I still receives the dividend at $Y + 20$ according to the record at $Y + 11$. On the other hand, investor J who purchased the stock on $Y + 10$ and later would not appear on the record date, so investor J would not receive the dividend at $Y + 20$. The records for dividend distributions are updated with each new forthcoming dividend issue.

The price of the stock typically falls on the ex-dividend date by an amount equal to the dividend (ignoring tax). Thus, it is important to consider adding the dividend so that the computed return rate would not appear to drop due to ex-dividend. There are two ways to compute cum-dividend return rates. The obvious one is to include expected future

dividend payout D_{Y+20} on ex-dividend date, i.e., at $Y + 10$, the return rate is $(P_{Y+10} + D_{Y+20})/P_{Y+9} - 1$. Or the continuously compounded return rate, that has the advantage of a support or potential range of $(-\infty, +\infty)$, i.e., $r_{Y+10} = \ln(P_{Y+10} + D_{Y+20})/P_{Y+9}$ could be computed. Note that the dividend is not exactly collected at $Y + 10$. Another way is to adjust the time series of all daily closing prices as follows.

Subtract D_{Y+20} from the price the day prior to ex-dividend date $Y + 9$. Call this the adjusted closing price at $Y + 9$, $P_{Y+9}^* = P_{Y+9} - D_{Y+20}$. Then the continuously compounded return rate at $Y + 10$ is computed as $r_{Y+10}^* = \ln P_{Y+10}/P_{Y+9}^*$. For daily return, this is approximately the same as r_{Y+10} . Moreover, all closing prices prior to $Y + 10$ are adjusted by the ratio P_{Y+9}^*/P_{Y+9} , i.e., $P_{Y+8}^* = P_{Y+8} \times (P_{Y+9}^*/P_{Y+9})$, $P_{Y+7}^* = P_{Y+7} \times (P_{Y+9}^*/P_{Y+9})$, and so on. So, return rate at $Y + 9$ calculated using the adjusted closing prices is $\ln P_{Y+9}^*/P_{Y+8}^* = \ln(P_{Y+9}/P_{Y+8})$. Earlier return rates are $r_{Y+8}^* = \ln(P_{Y+8}^*/P_{Y+7}^*) = \ln(P_{Y+8}/P_{Y+7})$ and so on.

If a firm announces a 1:1 stock dividend or else a 2:1 stock split, the effect is the same – the investor gets an additional share for every share he/she owns. At effective date of distribution, the closing price of the share will typically drop to half. All prices prior would be adjusted to $P_t^* = P_t/2$.

```
[10]: ### Next we process all the other 7 stocks in ECOMMERCE sector

[11]: dfAmazon = pd.read_csv('###Amazon_USD.csv', parse_dates=True)
dfAmazon[["day", "month", "year"]] = dfAmazon["Date"].str.split("/", expand = True)
cols = dfAmazon.select_dtypes(exclude=['float']).columns
dfAmazon[cols] = dfAmazon[cols].apply(pd.to_numeric, downcast='float', errors='coerce')
dfAmazon = dfAmazon[(dfAmazon['year'] >= 2017) & (dfAmazon['year'] <= 2021)]
dfAmazon = dfAmazon.drop('Date', axis=1)
dfAmazon = dfAmazon[['Adj Close', 'day', 'month', 'year']]

[13]: dfEbay = pd.read_csv('###Ebay_USD.csv', parse_dates=True)
dfEbay[["day", "month", "year"]] = dfEbay["Date"].str.split("/", expand = True)
cols = dfEbay.select_dtypes(exclude=['float']).columns
dfEbay[cols] = dfEbay[cols].apply(pd.to_numeric, downcast='float', errors='coerce')
dfEbay = dfEbay[(dfEbay['year'] >= 2017) & (dfEbay['year'] <= 2021)]
dfEbay = dfEbay.drop('Date', axis=1)
dfEbay = dfEbay[['Adj Close', 'day', 'month', 'year']]

[15]: dfRak = pd.read_csv('###Rakuten_USD.csv', parse_dates=True)
dfRak[["day", "month", "year"]] = dfRak["Date"].str.split("/", expand = True)
cols = dfRak.select_dtypes(exclude=['float']).columns
dfRak[cols] = dfRak[cols].apply(pd.to_numeric, downcast='float', errors='coerce')
dfRak = dfRak[(dfRak['year'] >= 2017) & (dfRak['year'] <= 2021)]
dfRak = dfRak.drop('Date', axis=1)
dfRak = dfRak[['Adj Close', 'day', 'month', 'year']]
```



```
[17]: dfSun= pd.read_csv('###Suning_CNY.csv',parse_dates=True)
dfSun[["day", "month", "year"]] = dfSun["Date"].str.split("/", expand = True)
cols = dfSun.select_dtypes(exclude=['float']).columns
dfSun[cols] = dfSun[cols].apply(pd.to_numeric, downcast='float', errors='coerce')
dfSun = dfSun[(dfSun['year']>=2017)&(dfSun['year']<=2021)]
dfSun = dfSun.drop('Date',axis=1)
dfSun = dfSun[['Adj Close','day','month','year']]

[19]: dfWay= pd.read_csv('###Wayfair_USD.csv',parse_dates=True)
dfWay[["day", "month", "year"]] = dfWay["Date"].str.split("/", expand = True)
cols = dfWay.select_dtypes(exclude=['float']).columns
dfWay[cols] = dfWay[cols].apply(pd.to_numeric, downcast='float', errors='coerce')
dfWay = dfWay[(dfWay['year']>=2017)&(dfWay['year']<=2021)]
dfWay = dfWay.drop('Date',axis=1)
dfWay = dfWay[['Adj Close','day','month','year']]

[21]: dfZal= pd.read_csv('###Zalando_EUR.csv',parse_dates=True)
dfZal[["day", "month", "year"]] = dfZal["Date"].str.split("/", expand = True)
cols = dfZal.select_dtypes(exclude=['float']).columns
dfZal[cols] = dfZal[cols].apply(pd.to_numeric, downcast='float', errors='coerce')
dfZal = dfZal[(dfZal['year']>=2017)&(dfZal['year']<=2021)]
dfZal = dfZal.drop('Date',axis=1)
dfZal = dfZal[['Adj Close','day','month','year']]

[23]: dfJD= pd.read_csv('###Jingdong_USD.csv',parse_dates=True)
dfJD[["day", "month", "year"]] = dfJD["Date"].str.split("/", expand = True)
cols = dfJD.select_dtypes(exclude=['float']).columns
dfJD[cols] = dfJD[cols].apply(pd.to_numeric, downcast='float', errors='coerce')
dfJD = dfJD[(dfJD['year']>=2017)&(dfJD['year']<=2021)]
dfJD = dfJD.drop('Date',axis=1)
dfJD = dfJD[['Adj Close','day','month','year']]
```

In code lines [25–27], the adjusted closing price data sets of the eight stocks are merged so that only all stock prices with the same day, month, and year are included. A few missing day gaps are ignored.

```
[25]: ### merging all 8 data files by same day, month, year -- those that do not overlap are eliminated
dfmerge = pd.merge(dfAlibaba,dfAmazon,on = ['day', 'month','year'])
dfmerge.columns=['AlibabaP','day','month','year','AmazonP'] ## Rename the columns
dfmerge = pd.merge(dfmerge,dfEbay,on = ['day', 'month','year'])
dfmerge.columns=['AlibabaP','day','month','year','AmazonP','EbayP']
dfmerge = pd.merge(dfmerge,dfRak,on = ['day', 'month','year'])
dfmerge.columns=['AlibabaP','day','month','year','AmazonP','EbayP','RakutenP']
dfmerge = pd.merge(dfmerge,dfSun,on = ['day', 'month','year'])
dfmerge.columns=['AlibabaP','day','month','year','AmazonP','EbayP','RakutenP','SuningP']
dfmerge = pd.merge(dfmerge,dfWay,on = ['day', 'month','year'])
dfmerge.columns=['AlibabaP','day','month','year','AmazonP','EbayP','RakutenP','SuningP','WayfairP']
dfmerge = pd.merge(dfmerge,dfZal,on = ['day', 'month','year'])
dfmerge.columns=['AlibabaP','day','month','year','AmazonP','EbayP','RakutenP','SuningP','WayfairP','ZalandoP']
dfmerge = pd.merge(dfmerge,dfJD,on = ['day', 'month','year'])
dfmerge.columns=['AlibabaP','day','month','year','AmazonP','EbayP','RakutenP','SuningP','WayfairP','ZalandoP','JDcomP']
```

10 Machine Learning in Business Finance Using Python

```
In [26]: print(dfmerge)
```

	AlibabaP	day	month	year	AmazonP	EbayP	RakutenP	\
0	80.599998	3.0	1.0	2017.0	37.683498	28.413393	9.825	
1	90.510002	4.0	1.0	2017.0	37.859001	28.337217	10.100	
2	94.370003	5.0	1.0	2017.0	39.022499	28.575266	10.250	
3	93.889999	6.0	1.0	2017.0	39.799500	29.565544	10.520	
4	94.720001	9.0	1.0	2017.0	39.845001	29.279884	10.590	
...	...	...	...	...	...	...	...	
1156	118.640004	23.0	12.0	2021.0	171.068497	64.331001	9.770	
1157	116.589996	27.0	12.0	2021.0	169.664995	65.094376	9.905	
1158	114.800003	28.0	12.0	2021.0	170.660995	65.510750	9.980	
1159	112.089996	29.0	12.0	2021.0	169.201004	65.687474	10.130	
1160	122.989998	30.0	12.0	2021.0	168.644501	66.204720	10.000	
...	...	...	...	...	...	...	...	
	SuningP	WayfairP	ZalandoP	JDcomP				
0	11.436833	35.180000	37.314999	25.184586				
1	11.446534	36.480000	35.919998	25.213848				
2	11.407731	37.169998	36.764999	25.652773				
3	11.233124	37.360001	37.349998	25.623512				
4	11.213723	38.049999	37.484999	25.613758				
...	...	...	...	...				
1156	4.110000	204.369995	70.519997	66.960564				
1157	4.140000	198.880005	70.459999	66.043701				
1158	4.130000	192.860001	70.800003	64.248985				
1159	4.080000	191.720001	70.699997	64.014885				
1160	4.110000	192.809998	71.139999	68.667503				

[1161 rows x 11 columns]

```
[27]: ### Now delete columns day, month, year; but before that keep day,
      ## month, year separately for later concatenation
      dy=dfmerge["day"]
      mth=dfmerge["month"]
      yr=dfmerge["year"]
      del(dfmerge["day"],dfmerge['month'],dfmerge['year'])
      print(dfmerge)
```

Code line [28] computes the continuously compounded return rate that can also be expressed as $\ln [1 + (P_{t+1}/P_t - 1)]$.

```
[28]: dfret=dfmerge.pct_change().apply(lambda x: np.log(1+x))
      ## Lambda is an alternative way of defining function inline using a single line of
      ## python code. percent change = x.
      ## Pandas -- Computes the percentage change from the immediately previous row by default
      ## Note the time series is in ascending order of time (ie later rows are later in time)
      dfret=dfret.iloc[1:len(dfret.index),:]
      ## Remove NaN values at the first row. Note iloc[0:.,:] indicates first row
      dfret.columns=['AlibabaP','AmazonP','EbayP','RakutenP','SuningP','WayfairP','ZalandoP','JDcomP']
      ## Rename the columns
```

The time series of the eight columns of stock returns are shown in code line [29]. Code line [30] uses seaborn package to print the time series graphs of the individual stock returns in two columns and four rows. Code lines [31] and [32] show the covariance and the correlation matrices of the return rates.

```
[29]: print(dfret)
```

```

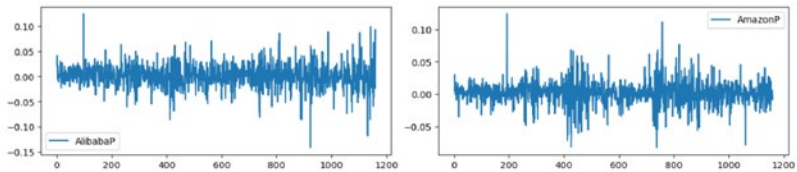
      AlibabaP  AmazonP  EbayP  RakutenP  SuningP  WayfairP  ZalandoP  \
1    0.021329  0.004646 -0.002685  0.027605  0.000848  0.036286 -0.038101
2    0.0041763  0.030270  0.008365  0.014742 -0.003396  0.018738  0.023252
3   -0.005099  0.019716  0.034068  0.026001 -0.015424  0.005099  0.015787
4    0.008801  0.001168 -0.009709  0.006632 -0.001729  0.018300  0.001472
5    0.021205 -0.001281 -0.016394 -0.016183  0.017153  0.075646 -0.004421
...
1156 0.007189  0.000184  0.014748  0.002049 -0.028779  0.002842  0.007973
1157 -0.017599 -0.008212  0.011797  0.013723  0.007273 -0.027230 -0.000851
1158 -0.015472  0.005827  0.006376  0.007543 -0.002418 -0.030737  0.004814
1159 -0.023889 -0.008592  0.005734  0.014918 -0.012180 -0.005929 -0.001414
1160 0.092801 -0.003294  0.004803 -0.012916  0.007326  0.005669  0.006204

      JDcomP
1    0.001161
2    0.017258
3   -0.001141
4   -0.000381
5    0.024079
...
1156 -0.071660
1157 -0.013787
1158 -0.027551
1159 -0.003650
1160 0.070160

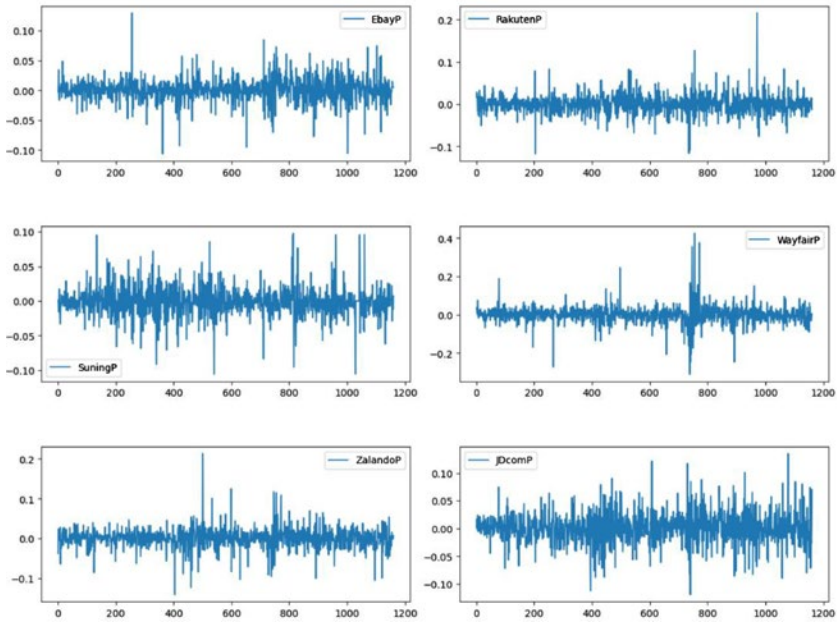
[1160 rows x 8 columns]
```

```
[30]: import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats

fig, axs = plt.subplots(ncols=2, nrows=4, figsize=(12, 12))
index = 0
axs = axs.flatten()
for k in dfret.items():
    sns.lmplot(data=dfret[k], ax=axs[index])
    index += 1
plt.tight_layout(pad=0.4, w_pad=0.5, h_pad=5.0)
```



12 Machine Learning in Business Finance Using Python



[31]: # Finding the simple covariance matrix from a series of returns

```
covar = dfret.cov()
print(covar)
```

	AlibabaP	AmazonP	EbayP	RakutenP	SuningP	WayfairP	\
AlibabaP	0.000568	0.000214	0.000141	0.000105	0.000073	0.000308	
AmazonP	0.000214	0.000366	0.000133	0.000080	0.000027	0.000307	
EbayP	0.000141	0.000133	0.000378	0.000080	0.000028	0.000289	
RakutenP	0.000105	0.000080	0.000080	0.000520	0.000043	0.000202	
SuningP	0.000073	0.000027	0.000028	0.000043	0.000423	0.000103	
WayfairP	0.000308	0.000307	0.000289	0.000202	0.000103	0.002131	
ZalandoP	0.000120	0.000115	0.000113	0.000091	0.000055	0.000335	
JDcomP	0.000430	0.000243	0.000164	0.000116	0.000064	0.000355	
	ZalandoP	JDcomP					
AlibabaP	0.000120	0.000430					
AmazonP	0.000115	0.000243					
EbayP	0.000113	0.000164					
RakutenP	0.000091	0.000116					
SuningP	0.000055	0.000064					
WayfairP	0.000335	0.000355					
ZalandoP	0.000610	0.000169					
JDcomP	0.000169	0.000767					

```
[32]: # Finding the simple correlation matrix from a series of returns
corr_matrix = dfret.corr()
print(corr_matrix)
```

	AlibabaP	AmazonP	EbayP	RakutenP	SuningP	WayfairP	\
AlibabaP	1.000000	0.469633	0.304793	0.192598	0.148897	0.280226	
AmazonP	0.469633	1.000000	0.356632	0.183431	0.067568	0.348132	
EbayP	0.304793	0.356632	1.000000	0.181259	0.070084	0.322172	
RakutenP	0.192598	0.183431	0.181259	1.000000	0.091049	0.191544	
SuningP	0.148897	0.067568	0.070084	0.091049	1.000000	0.108223	
WayfairP	0.280226	0.348132	0.322172	0.191544	0.108223	1.000000	
ZalandoP	0.203696	0.242445	0.236042	0.161165	0.108273	0.293582	
JDcomP	0.651556	0.458592	0.304356	0.184363	0.113146	0.277591	

	ZalandoP	JDcomP
AlibabaP	0.203696	0.651556
AmazonP	0.242445	0.458592
EbayP	0.236042	0.304356
RakutenP	0.161165	0.184363
SuningP	0.108273	0.113146
WayfairP	0.293582	0.277591
ZalandoP	1.000000	0.247294
JDcomP	0.247294	1.000000

Sometimes we may need to check the data to ensure that there are no values that are infinite that could then lead to computational problems in what follows. In code line [33] we do this check. We could also check for how many missing values using `np.isnull(dfret).values.sum()`.

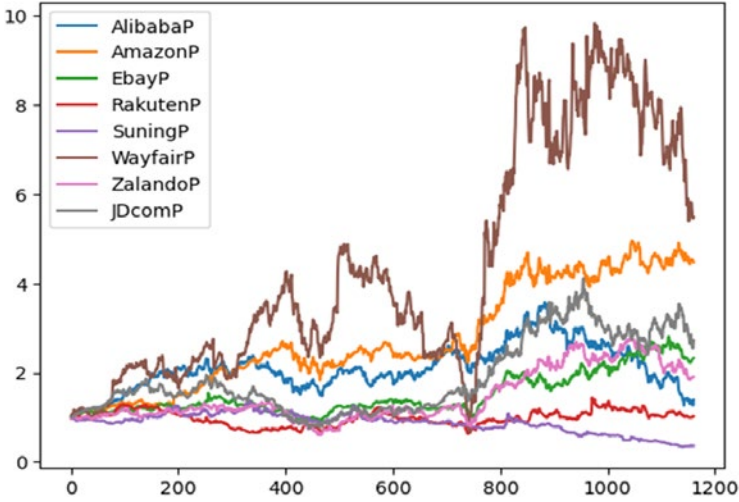
```
[33]: count = np.isinf(dfret).values.sum()
      ## Checking for infinite values using isinf() and displaying the count
      print(count)
```

0

The cumulative return time series of the stocks from 2017 till 2021 are shown in code line [34] and plotted in [35].

```
[34]: ### calculating daily cumulative return over 5 years
dfcumuret=(np.exp(dfret)).cumprod()
      ## cumprod() returns a series of the same length as the original input series,
      ## containing the cumulative product
      print(dfcumuret)
```

```
[35]: fig = plt.figure()
      (np.exp(dfret)).cumprod().plot()
      plt.show()
```



In the following [36], the optimal weights on the eight stocks are found to form the optimal portfolio. The weights are solutions to a minimization problem whereby the objective function is the annualized portfolio return standard error. This follows model (1.1) in Section 1.1. Throughout, the key measurements of mean return and return standard deviation are in annualized terms. These are computed as follows.

For a particular stock i , its annualized return is $252 \times r_{it}^*$ where r_{it}^* is the daily return rate and is assumed to be unconditionally stationary (having same mean and variance for each time). Thus, the mean annual return is $252 \times E(r_{it}^*)$ where $E(\cdot)$ is the expectation operator, and $E(r_{it}^*)$ is estimated using $\frac{1}{T} \sum_{t=1}^T r_{it}^*$ with daily returns r_{it}^* , $t = 1, 2, \dots, T$. The mean annualized return is estimated as $252 \frac{1}{T} \sum_{t=1}^T r_{it}^*$ where 252 is approximately the number of trading days in the U.S. For all stocks, the vector of mean annualized return is $r = \text{np.mean(dfret,axis=0)} * 252$. Thus, r is μ in (1.1). A portfolio with weights vector w would have portfolio mean annualized return as $w^T r$.

For stocks $i = 1, 2, \dots, N$, with return vector r_t^* at day t , each ij th element of covariance of r_{it}^* and r_{jt}^* is estimated as

$$\frac{1}{T-k} \sum_{t=1}^T (r_{it}^* - \mu_i)(r_{jt}^* - \mu_j)$$

where $k = 1$ for $i = j$, and $k = 2$ for $i \neq j$. The covariance matrix may be denoted in matrix form as $\text{var}(r_t^*)$. Annualized portfolio return volatility is then $\sqrt{252 \times (w^T \text{var}(r_t^*) w)}$ or $\text{np.sqrt}(\text{np.dot}(w, \text{np.dot}(w, \text{covar}))) * 252$.

```
[36]: ### Ref: https://www.kaggle.com/code/trangthvu/efficient-frontier-optimization/notebook
import scipy
### No shortsale, so all weights must be between 0 and 1. Set 0 and 1 as the boundaries.
from scipy.optimize import Bounds
bounds = Bounds(0, 1)

### The second boundary is the sum of weights.
from scipy.optimize import LinearConstraint
linear_constraint = LinearConstraint(np.ones((dfret.shape[1],), dtype=int), 1, 1)
### 1,1 in argument of LinearConstraint refers to Lb, Ub in Lb <= A.dot(w) <= Ub;
### if Lb=Ub, it implies equality constraint
### df.shape[0] refers to no. rows, .shape[1] refers to no. cols;
### np.ones fill up with ones
### Above -- np.ones((dfret.shape[1],), dtype=int) is A, i.e. 1 x 8 elements of ones
### since dfret.shape[1] gives dim of cols
### Then A'w = 1 is the constraint, i.e. sum of wts must equal to one

covar = dfret.cov()
r = np.mean(dfret, axis=0) * 252
### axis=0 means to apply calculation "column-wise",
### axis=1 means to: apply calculation "row-wise",
### r is annualized vector mean return based on historical data
### Here mean is calculated for each XYZ stock return time series (column)

def ret(r, w):
    return r.dot(w)
### Note ret(r, w) is defined here. r.dot(w) is matrix multiplication of r and w
def vol(w, covar):
    ### Risk level or volatility
    return np.sqrt(np.dot(w, np.dot(w, covar))) * 252
### same as sqrt of w^T \Sigma w * 252 -- annualized return volatility
def sharpe(ret, vol):
    return ret/vol

### Find a portfolio with the minimum risk.
from scipy.optimize import minimize
### Create x0, the first guess at the values of each stock's weight.
weights = np.ones(dfret.shape[1])
x0 = weights/np.sum(weights)

### Define a function to calculate volatility
portfstderr = lambda w: np.sqrt(np.dot(w, np.dot(w, covar))) * 252
### w is input to function lambda that outputs portfola
res1 = minimize(portfstderr, x0, method='trust-constr', constraints = linear_constraint, bounds = bounds)
### constraint means unit vector .dot(w) = 1; minimize chooses wts w

### Objective function is portfstderr
### 'trust-constr' is to minimize a scalar function subject to constraints
### -- algorithm updates x0 till obj fn portfola is min
### minimize(...) function returns optimal weight w_min
### These are the weights of the stocks in the portfolio with the lowest level of risk possible.
w_min = res1.x
### optimization full output.x gives the solution array
```

```

np.set_printoptions(suppress = True, precision=3)
### Suppress=True means always printing floating point numbers to 3 decimal places

print(w_min)
print('return: % .4f' % (ret(r,w_min)), 'risk: % .4f' % vol(w_min,covar))
### this is min var portfolio
### "print" treats the % as a special character you need to add, so it can know that when you type "f"
### the number (result) that will be printed will be a floating point type, and the ".4" tells
### your "print" to print only the first 4 digits after the point.

[0.036 0.21 0.209 0.165 0.278 0. 0.102 0. ]
return: 0.0622 risk: 0.1908

```

For the MVP solution in [36], the optimal weights indicate 3.6% of total investment wealth allocated to Alibaba, 21% to Amazon, 20.9% to Ebay, 16.5% to Rakuten, 27.8% to Suning, and 10.2% to Zalando. Close to zero % are put into Wayfair and JD.Com. The weights are all positive and sum to one. In [37], the objective function to be minimized is the inverse of the Sharpe ratio – the solution is the same as maximizing Sharpe ratio. This follows model (1.2) in Section 1.1.

```

[37]: ### Define 1/Sharpe_ratio as invSharpe
invSharpe = lambda w: np.sqrt(np.dot(w,np.dot(w,covar))*252)/r.dot(w)
res2 = minimize(invSharpe,x0,method="trust-constr",constraints = linear_constraint,bounds = bounds)
### Objective function is invSharpe -- inverse of Sharpe ratio
### These are the weights of the stocks in the portfolio with the highest Sharpe ratio
### - call the weight vector w_Sharpe

w_Sharpe = res2.x
### constraint means unit vector .dot(w) = 1; minimize chooses wts w
### optimization full output.x gives the solution array of optimal weights in min inverse Sharpe
### ratio or max Sharpe ratio

print(w_Sharpe)
print('return: % .4f' % (ret(r,w_Sharpe)), 'risk: % .4f' % vol(w_Sharpe,covar))
### this is max Sharpe ratio portfolio

print( 1/( np.sqrt(np.dot(w_Sharpe,np.dot(w_Sharpe,covar))*252)/r.dot(w_Sharpe) ) )
### Above is optimized objective function -- the max Sharpe ratio.
### It can also be found using print(sharpe(ret(r,w_Sharpe),vol(w_Sharpe,covar)))

[0. 0.768 0.171 0. 0. 0.03 0.031 0. ]
return: 0.2969 risk: 0.2697
1.1006526861513148

```

The solution to [37] shows the maximized Sharpe ratio is 1.1007, whereas the portfolio return, and standard error are respectively 29.69% and 26.97%. This latter result appears to be superior to that of MVP in (1.1) with a portfolio return of a much lower 6.22% and volatility of 19.08%.

In code line [38], the Markowitz mean-variance efficient portfolio frontier is drawn under the constraints of positive weights as in model (1.3). Different required returns give rise to different minimized volatility portfolios. These are plotted.


```
[38]: w = w_min ### w is now optimal portfolio weights, sum to 1
num_ports = 100
gap = (np.amax(r) - ret(r,w_min))/num_ports
### np.amax in numpy returns max in the array -- since weights sum to 1 and are
### bounded in (0,1), max portf ret is amax(r)
### The above range given by gap starts at ret given by Min Var Portf to Max of all mean returns

all_weights = np.zeros((num_ports, len(dfret.columns)))
### all_weights is 2D 100 x 8 zero matrix
### Note: len(dfret.columns) is 8 -- there are 8 stocks here
### print(np.shape(all_weights)) gives (100,8) - same as print(all_weights.shape) that gives (100,8)

print("*****")
#####
### First note that in Python, a Tuple is a grouping of unnamed, ordered values that can be
### of different types; an array is a collection where elements' values can be changed, and
### are of a single type. All_weights is a 2D array of 100 rows each with 8 cols.
### all_weight[0] below is the first 1D sub-array that is 1st row
### Note: all_weights[0] is [0. 0. 0. 0. 0. 0. 0. 0.], i.e. first row of 100 x 8 all_weight
### Note: all_weights[1] is [0. 0. 0. 0. 0. 0. 0. 0.], i.e. second row of 100 x 8 all_weight
### .....
### Note: all_weights[99] is [0. 0. 0. 0. 0. 0. 0. 0.], i.e. 100th row of 100 x 8 all_weight
### print(all_weights[0].shape) gives (8,) -- a 1-tuple with 8 elements
### print(all_weights.shape[0]) gives 100, i.e. dimension of the rows
### print(all_weights.shape[1]) gives 8, i.e. dim of the cols
#####

ret_arr = np.zeros(num_ports) ### this is a 1-tuple of 100 zeros
### Note: print(ret_arr.shape) gives (100,) -- this is a 1D tuple
### print(ret_arr.shape[0]) gives 100, the number of elements in 1D tuple
### if we print(ret_arr.shape[1]) - this gives "tuple index out of range" as there is no other dim
### If we use instead ret_arr = np.zeros((num_ports,1)), then print(ret_arr.shape)
### gives (100,1), a dataframe with one column

vol_arr = np.zeros(num_ports)

for i in range(num_ports): ## this means looping from i=0 to 1,2,3,4,...,99 (100 loops in total)
    port_ret = ret(r,w) + i*gap
    double_constraint = LinearConstraint([np.ones(dfret.shape[1]),r],[1,port_ret],[1,port_ret])
    ## above means left term in first bracket np.ones(dfret.shape[1]) or A * w <= first term
    ## in 2nd,3rd brackets 1,1;
    ## right term in first bracket r * w <= second terms in 2nd, 3rd brackets port_ret
    ## i.e. constraints port wts sum to 1 and portf exp return == port_ret

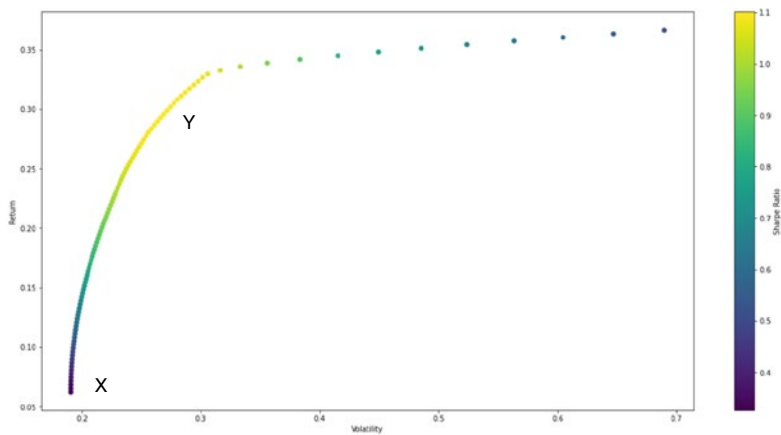
    ## Create x0: initial guesses for weights.
    x0 = w_min
    ## Define a function for annualized portfolio volatility.
    portfvola = lambda w: np.sqrt(np.dot(w,np.dot(w,covar)))*252
    res = minimize(portfvola,x0,method='trust-constr',constraints = double_constraint,bounds = bounds)
    ## Above double constraints mean unit vector .dot(w) = 1; r .dot(w) = port_ret; minimize chooses wts w
    all_weights[i,:]=res.x ### i row x 8 optimal wts (at row i)
    ret_arr[i]=port_ret
    vol_arr[i]=vol(res.x,covar)

## Indented paras after "for i..." form the loop

sharpe_arr = ret_arr/vol_arr ## sharpe_arr is 100 x 1 array since it is ret_arr[100]/vol_arr[100] element by element

plt.figure(figsize=(20,10))
plt.scatter(vol_arr, ret_arr, c=sharpe_arr, cmap='viridis')
## in plt.scatter, c is a scalar or sequence of n numbers to be mapped to colors using cmap
## in plt, for sequential plots, 'viridis' gives colors across the 3D representation of
## vol_arr, ret_arr, sharpe_arr
## c= in front of third dimension sharpe_arr gives the colors in that dimension, otherwise
## dots will be all blue
plt.colorbar(label='Sharpe Ratio')
plt.xlabel('Volatility')
plt.ylabel('Return')
plt.show()

*****
```



The above scatter plot shows the MVP *X* (left lowest point) with a return of 6.22% and volatility of 19.08%. The maximum Sharpe ratio portfolio *Y* has a return of 29.69%, volatility of 26.97%, and a Sharpe ratio of is 1.1007. There are no short sales in the portfolios.

The following codes concatenates the day, month, year and forms a new data set of the eight portfolio returns with 11 columns that is saved in `ret_portecom.csv`, i.e., using `dfret1.to_csv`.

```
[39]: dfret.shape

[39]: (1160, 8)

[40]: t1=pd.concat([dy,mth,yr],axis=1) ### Axis=1 is important -- aligning columns
      t1=t1.iloc[1,: ]
      print(t1)

          day  month  year
1         4.0    1.0  2017.0
2         5.0    1.0  2017.0
3         6.0    1.0  2017.0
4         9.0    1.0  2017.0
5        10.0    1.0  2017.0
...      ...    ...    ...
1156     23.0   12.0  2021.0
1157     27.0   12.0  2021.0
1158     28.0   12.0  2021.0
1159     29.0   12.0  2021.0
1160     30.0   12.0  2021.0

[1160 rows x 3 columns]

[41]: t1.shape

[41]: (1160, 3)

[42]: dfret1=pd.concat([dfret, t1],axis=1)

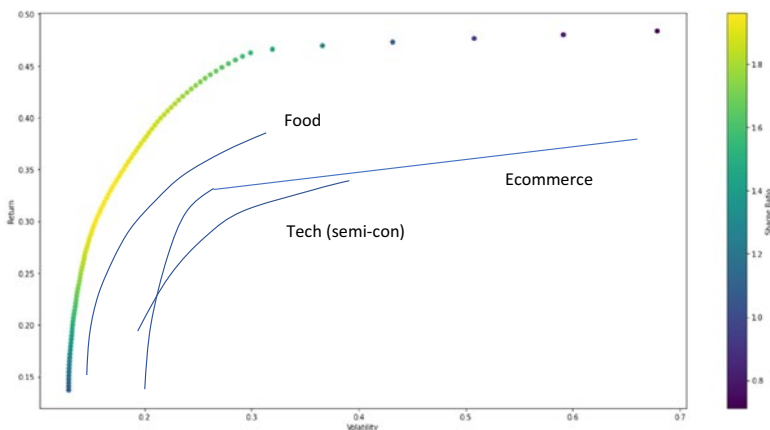
[44]: import pandas as pd
      dfret1.to_csv('ret_portecom.csv')
```

1.3 Forming Optimal Portfolios

Two other industry portfolios – technology (semi-conductor firms) and food – are similarly formed, each with eight stocks. The tech stocks include Broadcomm, Intel, Microchip, Micron, Qualcomm, Samsung, SK Hynix, and SMIC. The food stocks include Tyson, Pepsico, Nestle, Mondelez, Kweichow, Diageo, Danone, and Anheuser-Busch.

The three industry portfolios in data sets `ret_portecom.csv`, `ret_porttech.csv`, and `ret_portfood.csv` with a total of 24 stocks are merged into a larger portfolio for similar optimizations. The data set is further split into two parts: a training set using data from 2017 till 2020, and a test set of data in 2021. But we use only the first 112 data points at beginning of 2021 to about June 2021 – this is to ensure recency when we apply the optimal weights calculated from the training set and apply them to the new return data immediately after the training period.

The outcome of using the training set data to estimate the portfolio mean returns and portfolio return covariance and to form the efficient portfolio is shown in the following figure. The efficient frontiers of the three separate industry portfolios, though using a slightly longer data set including 2021 are also shown. Clearly, the larger combined industry set of stocks produce a better performing efficient frontier – for any given expected return, the portfolio volatility is smaller. This is evidence of the benefit of portfolio diversification when more stock returns are added which have lower variances and low correlations with the others.



However, the above result on expected portfolio return versus portfolio return volatility is based on finding weights within the historical training sample and evaluating the frontier based on those weights. After the optimal weights for every k , as in (1.3), are computed, suppose we apply these weights to fresh data that is out-of-sample in the test data set. Would we obtain the out-of-sample or test set predicted portfolio returns and volatilities that are similar to that mapped in the training set? We use the test set to construct the out-of-sample mean and covariance matrix.

The results are shown in the following. See demonstration file Chapter1-2.ipynb.

```
[21]: ## Below the test set data is used with the optimal wts computed
      ## with training set to form the eff portf frontier
      testcovar=testset.cov()
      testr = np.mean(testset,axis=0)*252

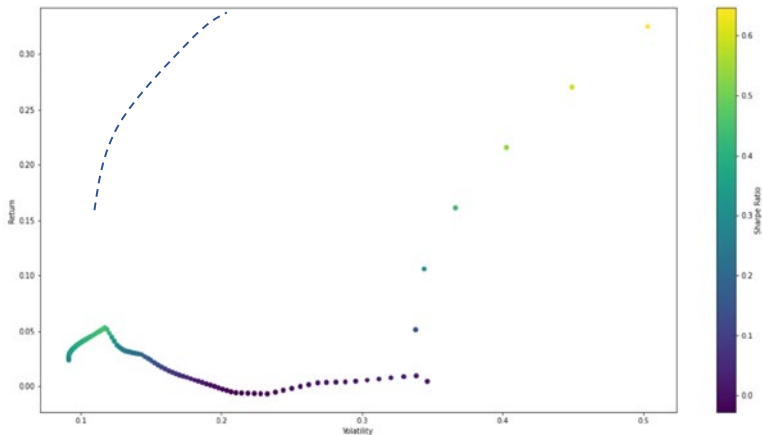
      ## initialize
      testport_ret = np.zeros(num_ports)
      testport_vol = np.zeros(num_ports)

      for i in range(num_ports):
          testport_ret[i] = ret(testr,all_weights[i,:])
          testport_vol[i]= vol(all_weights[i:],testcovar)

      testport_sharpe = testport_ret/testport_vol

      plt.figure(figsize=(20,10))
      plt.scatter(testport_vol, testport_ret, c=testport_sharpe, cmap='viridis')

      plt.colorbar(label='Sharpe Ratio')
      plt.xlabel('Volatility')
      plt.ylabel('Return')
      plt.show()
```



Clearly, the training set mean return μ and portfolio return volatility based on training set Σ would be different from the test set mean return and covariance matrix Σ . Thus, the optimal portfolio weights derived from the training set data may not apply as well to the test set data. The resulting test set portfolio performances (its mean and volatility across test set days) would be likely inferior to those seen in the in-sample training set. This is seen in the portfolio frontier diagram where the solid dots are the test sample performance and the dotted curve is the in-sample efficient frontier.

There are at least two possible reasons why using out-of-sample or test set data would produce substandard performances. In this case, the stocks with high weights computed from the training data set have generally lower returns in the test data set than in the training data set. Thus, the new average portfolio return is lower. The performance errors are created due to the randomness of returns that could produce large deviations from the expectations in training set μ . Another possible reason is that the estimation of the covariance matrix Σ using the training set data could contain random errors and similarly this would affect the out-of-sample portfolio return volatility based on the test set returns.

Both mean return and portfolio return volatility errors could be reduced if the optimal weights could be computed based on predicted returns characteristics of the test data set rather than just based on past training data. One approach to predicting test data return characteristics is to use a multi-factor linear model. In this case, if the factors are correctly found and anticipated, the errors remain only in the residuals of the linear model, and these errors would be smaller. We do not go into multi-factor models here but suggest another method in machine learning to attempt to reduce the error in the use of in-sample estimate of Σ . In a later section, we show how a more accurate forward prediction of future expected return could be used to improve on training set optimal portfolio weights for future investing results.

1.4 Denoising the Correlation Matrix

In finance theory, one often thinks of the stock return covariance and corresponding correlation matrices as constant matrices. However, their sample estimates are random due to small sampling errors. However, with fixed N number of stocks and a time series T that approaches $+\infty$, i.e., $N/T \downarrow 0$, the sample estimate of the correlation matrix $\hat{\Sigma}$ would converge

to the population constant of $\Sigma_{N \times N}$ given stationary time series of the stock returns.

However, suppose N, T are increasingly large, so that N/T does not converge to zero, but instead converge as $N, T \rightarrow +\infty$, to some finite positive number, i.e., $0 < N/T < 1$. In small sample, it is also the case that $0 < N/T < 1$. Then every element in the (small) sample correlation matrix is a random variable. We may treat the sample correlation matrix in this context as a random matrix which is in general a matrix-valued random variable.

A square matrix A is positive definite if for any non-zero conformable vector w , $w^T A w > 0$. If $A_{N \times N} w_{iN \times 1} = \lambda_i w_{iN \times 1}$ for scalar λ_i , then λ_i is called an eigenvalue of A while w_i is the corresponding eigenvector. There are N number of eigenvalues and eigenvectors. The eigenvalues are found by solving the determinant, $|A - \lambda_i I| = 0$. Some of these may be repeated. In general, these eigenvectors are non-unique, so additional constraints to make them normalized and orthogonal to each other are imposed. In other words, for any i, j eigenvectors, w_i, w_j : $w_i^T w_i = 1$, $w_j^T w_j = 1$, and $w_i^T w_j = 0$ for $i \neq j$.

A positive definite symmetric matrix has strictly positive eigenvalues. A matrix is positive definite if it is symmetric, and all its eigenvalues are strictly positive. Hence, sample or else population covariance and correlation matrices of stock returns, that are both symmetric and positive definite, have positive eigenvalues.

Suppose a vector of random variables change over time due to independent and identically distributed noise (with mean 0 and variance of 1) and not signals. Noises, unlike signals, are not systematic factors in the market. Then the Marcenko–Pastur Theorem (see Marchenko and Pastur (1967)) states that as $N, T \rightarrow +\infty$, and N/T converge to a finite positive number in $(0,1)$, then the eigenvalues λ_i of the sample correlation matrix converge to a probability density function (pdf) as follows (and not a degenerate single value distribution of probability one²).

² This would be the case when $N/T \downarrow 0$, then the covariance matrix would be $I_{N \times N}$. Dividing through by the product of standard deviations, the correlation matrix is also $I_{N \times N}$. If we set v_j and λ_j as the j th eigenvector and eigenvalue, then $I v_j = \lambda_j v_j$. Solving, $\lambda_j = 1$. Indeed, for every $j \in [1, 2, \dots, N]$, $\lambda_j = 1$, i.e., eigenvalues are repeated as a single degenerate distribution of one.

The important characterization of the random matrix is that the eigenvalues are random variables.

$$\text{pdf}(\lambda_i) = \frac{\sqrt{(U - \lambda_i)(\lambda_i - L)}}{2\pi\lambda_i\sigma^2 N/T}$$

where $U = \sigma^2 \left(1 + \sqrt{N/T}\right)^2$ and $L = \sigma^2 \left(1 - \sqrt{N/T}\right)^2$, for $\lambda_i \in [L, U]$. Outside of $[L, U]$, $\text{pdf}(\lambda_i) = 0$.

The above result is demonstrated in the following codes in demonstration file Chapter1-3.ipynb whereby $T \times N$ ($T = 50,000$ and $N = 900$) random normal numbers with mean 0 and variance 1 are generated as noises to find estimates of the eigenvalues of the sample correlation matrix of the simulated N number of random numbers.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

[2]: ## To obtain the eigenvalues and eigenvectors (normalized and orthogonal)
## from a matrix, sorting eigenvalues in ascending order
def getPCA(matrix):
    eVal,eVec=np.linalg.eigh(matrix)
    ## x in .eigh(x) is real symmetric or complex Hermitian (conjugate symmetric) array
    ## Returns two objects, a 1-D array eVal =eigenvalues of matrix - eigenvalues
    ## with multiplicity, may not be ordered; and a 2-D square matrix, column v[:, i]
    ## is the normalized eigenvector corresponding to the eigenvalue w[i].
    indices=eVal.argsort() ## Returns the indices that would sort an array.
    eVal,eVec=eVal[indices],eVec[:,indices]
    ## ensures eVal and eVec elements are increasing in order of eVal or eigenvalues
    eVal=np.diagflat(eVal)
    ## Create a two-dim array with the flattened (changing to 1dim) input eVal as a diagonal
    return eVal,eVec

[3]: x = np.random.normal(0, 1, size=(50000,900))
## first argument mean, second std dev. Random matrix dim in size (T,N)

[4]: eVal0,eVec0=getPCA(np.corrcoef(x,rowvar=False))
## matrix in getPca argument is correlation matrix from x.
## If rowvar is True (default), then each row reps a variable with obs in the columns
## Otherwise, the relationship is transposed: each col reps a var, while rows contain obs
## np.corrcoef(x,rowvar=False).shape = (900,900); Len(eVal0) = 900; eVec0.shape is (900,900)
```

Code line [2] produces a user-defined function getPca to derive the 900×1 eigenvalues and corresponding 900×900 eigenvectors (900 columns of eigenvectors with each eigenvector a dimension of 900×1) in [4] based on random correlation matrix of x . (In this case, it is also the covariance matrix of x .) We can think of sample data $x_{50,000 \times 900}$ in [3] as deriving from a random vector of $X_{900 \times 1}$ where the i th column of x represents the time series of the i th row random variable in X . Note that X has zero mean

and a standard deviation of one for all random variables. Taking the i th eigenvector w_i , $w_i^T X$ is the i th principal component of X . It is a linear combination of X with a variance equal to its eigenvalue λ_i . The sum of all the eigenvalues or $\sum_{i=1}^{900} \lambda_i = 900$ which is also the total variance of the correlation matrix Σ , i.e., its trace or sum of the diagonal elements. Hence, the largest principal components explain most of the variances present in the random X .

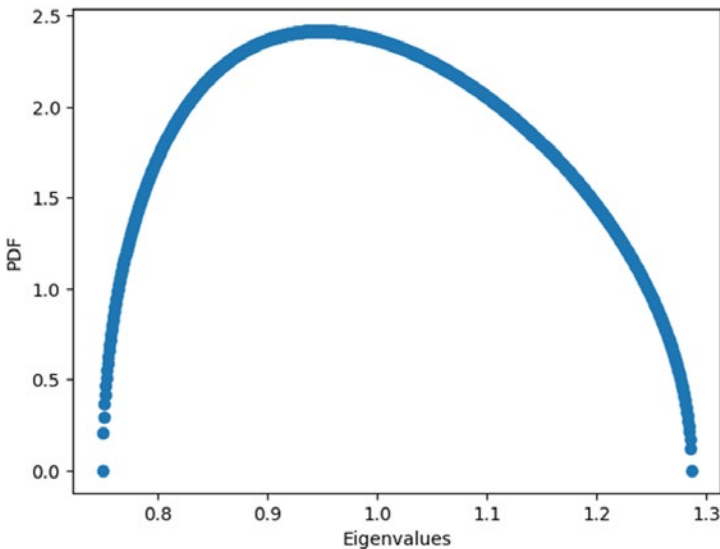
Code lines [5] and [6] compute the approximate theoretical Marcenko–Pastur pdf given $T = 50,000$ and $N = 900$. This is plotted in code line [7].

```
[5]: def randommatpdf(var,q,pts): ###
      eMin, eMax = var*(1-(1./q)**.5)**2,var*(1+(1./q)**.5)**2
      eVal=np.linspace(eMin,eMax,pts)
      ### returns pts (no.) equally spaced vector of nos. starting at eMin to eMax
      pdf=q/(2*np.pi*var*eVal)*((eMax-eVal)*(eVal-eMin))**.5
      pdf=pd.Series(pdf,index=eVal)
      ### Pandas uses Series (similar to 1-dim array in numpy, but essential diff is the presence of
      ### the index in pandas that can be defined, whereas Numpy Array has implicitly def integer index
      return pdf

[6]: pdf0=randommatpdf(1.,q=x.shape[0]/float(x.shape[1]),pts=900)
      ## print(x.shape[0]) = 50,000; print(x.shape[1])=900
      ### Here variance var is entered as 1. pts no. must match simulated N.
      ### float() needs not be used unless item is to be divided
      ### and number needs be a floating type, i.e. with decimal

[7]: plt.scatter(pdf0.index, pdf0) ## This is the theoretical Marcenko-Pastur pdf of the eigenvalues
      plt.xlabel('Eigenvalues')
      plt.ylabel('PDF')

[7]: Text(0, 0.5, 'PDF')
```



Code lines [8] and [9] evaluate the eigenvalues and plot it as a histogram.

```
[8]: ### "print(np.diagonal(eVal0).shape)" gives (900,)  

### "eVal0.shape" gives (900,900)-- 2D numpy array  

eigenvalues=np.diagonal(eVal0)  

### this changes the diag matrix eVal0 back to single column with eigenvalues.shape as (900,)  

### "print(eigenvalues)" is used to check if program line is correct and delivers required output
```

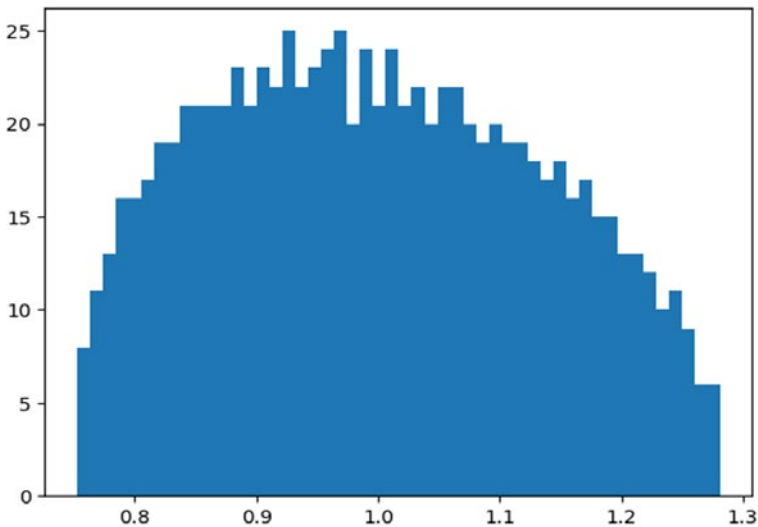
```
[9]: import matplotlib.pyplot as plt  

plt.hist(eigenvalues, bins = 50)  

plt.show()  

### This empirical pdf seems to be explained well by the theoretical pdf
```



Code lines [10] and [11] evaluate the empirical pdf of the eigenvalues generated by the simulated i.i.d. random variables. The kernel refers to the integrand function whereby the integral produces the area under the curve of one. The kernel density estimation provides the empirical probability density function. Different kernel function specification may produce slightly different smoothness in the empirical pdf.

```
[10]: ### kernel density estimation (KDE) is a nonparametric smoothing method to estimate  

### a probability density estimation  

### The y-axis or count of the corresponding Histogram is calibrated to pdf measures  

### such that area under pdf = 1  

### Valid kernels are ['gaussian','tophat','epanechnikov','exponential','linear','cosine',  

### Default is 'gaussian'.
```

```

from sklearn.neighbors import KernelDensity
def fitKDE(obs,bwidth=.25,kernel='gaussian',x=None):
    if len(obs.shape)==1: obs=obs.reshape(-1,1)
    kde=KernelDensity(kernel=kernel,bandwidth=bwidth).fit(obs)
    if x is None:x=np.unique(obs).reshape(-1,1)
    if len(x.shape)==1: x=x.reshape(-1,1)
    logProb=kde.score_samples(x)
    pdf=pd.Series(np.exp(logProb),index=x.flatten())
    return pdf

```

```

[11]: pdf1=fitKDE(eigenvalues,bwidth=.01)
      ### pdf1 is an object wrapping together eigenvalue and its pdf

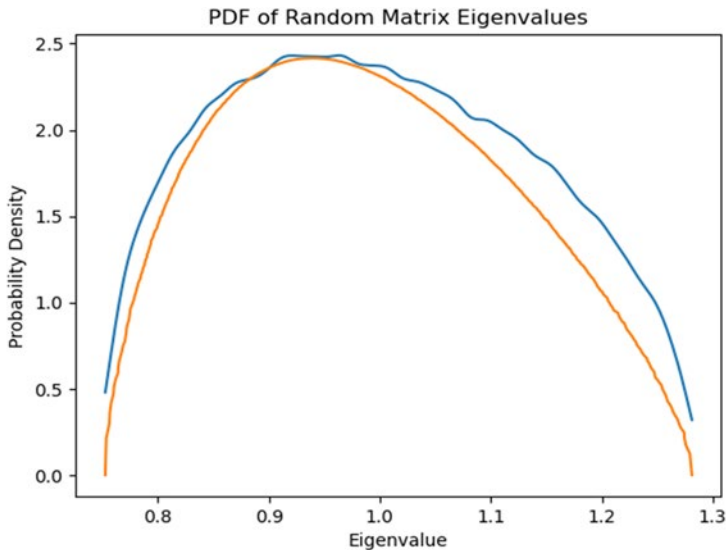
```

```

[12]: import seaborn as sns
      fig, ax = plt.subplots()
      ax = sns.lineplot(x=eigenvalues, y=pdf1, ax=ax) ### prints empirical pdf
      ax = sns.lineplot(x=eigenvalues, y=pdf0, ax=ax) ### prints theoretical pdf
      ax.set_title('PDF of Random Matrix Eigenvalues')
      ax.set_xlabel('Eigenvalue')
      ax.set_ylabel('Probability Density')
      plt.show()

```

Code line [12] plots the empirical pdf of the computed eigenvalues side-by-side the approximate theoretical pdf of the 900 eigenvalues.



```

[13]: print(np.min(eigenvalues),np.max(eigenvalues))
      0.7525367249061625 1.2812509532082264

```

```

[14]: eigenvalues.sum()

```

```

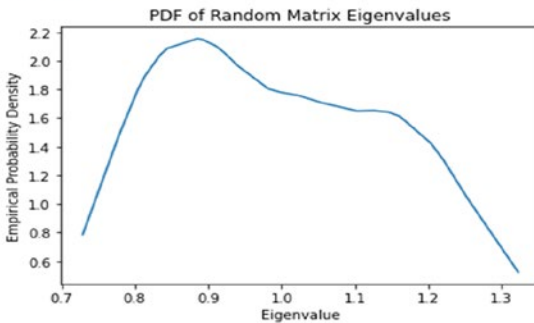
[14]: 899.9999999999998

```

It is seen that for the large $T = 50,000$ and $N = 900$, the empirical pdf looks similar to the theoretical pdf (area under the pdf curve is one).

However, when we use only $N = 24$, $T = 889$ (size of the training set data) similar random normal numbers with mean zero and variance one, the empirical pdf is as follows. See demonstration file Chapter1-4.ipynb. The eigenvalues for the true covariance matrix $I_{24 \times 24}$ based on 889 time values from random generation using mean 0 and variance 1 should be all ones, viz., $Iv_1 = \lambda_1 v_1$, so $\lambda_1 = \lambda_2 = \lambda_3 = \dots = \lambda_{24} = 1$ repeated for 24 times. Yet random cov matrix produces the eigenvalue distribution 0.7–1.3. The average is about 1. This shows the estimated covariance matrix is random. Hence, the empirical pdf does not approximate the theoretical pdf as closely since N, T are small. Nevertheless, it still shows that when the randomness is just independently identically distributed noises, eigenvalues can still range from 0.7 to 1.3. The idea is to look for significantly larger values of eigenvalues that would signal the presence of signals and not just noises.

```
[10]: import seaborn as sns
fig, ax = plt.subplots()
ax = sns.lineplot(x=eigenvalues, y=pdf1, ax=ax) ### prints empirical pdf
ax.set_title('PDF of Random Matrix Eigenvalues')
ax.set_xlabel('Eigenvalue')
ax.set_ylabel('Empirical Probability Density')
plt.show()
```



Suppose we use the actual stock return data of the $N = 24$ stocks over time series $T = 889$ in the training data set to form the correlation matrix $\Sigma_{24 \times 24}$, “corr_matrix”. In the program, this is computed as `corr_matrix = trainingset.corr()`. The eigenvalues and eigenvectors of $\Sigma_{24 \times 24}$ are found using the following. “eigenvalues” show the list of 24 computed eigenvalues. See demonstration file Chapter1-5.ipynb.


```
[7]: ### Create a principal component analysis (PCA) plot for the first two dims
def getPCA(matrix):
    eVal,eVec=np.linalg.eigh(matrix)
    indices=eVal.argsort()
    eVal,eVec=eVal[indices],eVec[:,indices]
    eVal=np.diagflat(eVal)
    return eVal,eVec

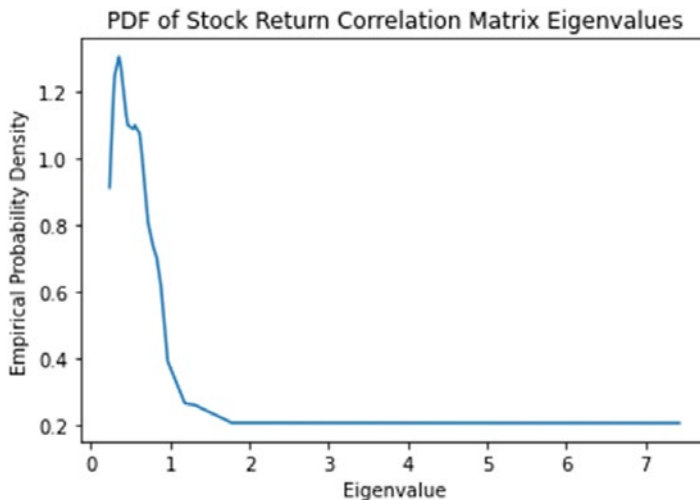
[8]: eVal0,eVec0=getPCA(corr_matrix)

[9]: eigenvalues=np.diagonal(eVal0)
### creates a list of the diagonal elements in original
print(eigenvalues)

[0.23777545  0.24830976  0.29134883  0.30236206  0.35499693  0.36322877
 0.38419966  0.44332669  0.46659641  0.535817   0.55524774  0.61387326
 0.61922829  0.64157765  0.72200588  0.78494843  0.83230546  0.88442046
 0.97049917  1.18529588  1.317256   1.77494235  2.04324288  7.42719501]
```

Note that the correlation matrix “corr_matrix” is the sample correlation matrix of the training set returns, which is equivalent to the covariance matrix of the standardized training set returns. The latter is also the correlation matrix of the standardized training set returns. Hence, “corr_matrix” is correlation matrix of the standardized training set returns. It may be different from the simulated random normal numbers with mean zero and variance one in that the off-diagonal correlation numbers are theoretically not necessarily zeros in the returns sample.

The computed empirical pdf of the eigenvalues is shown as follows.



It is seen that most of the eigenvalues falls below 1.3 and these variances of the principal components could be due to white noise. Only those eigenvalues > 1.3 represent combinations of the standardized stock return random variables (with mean 0 and variance 1) that produce bigger variances that could be due to true market signals and not white noise.

Denoising the empirical correlation matrix is one way to produce a more accurate correlation matrix for portfolio optimization. We show one method of denoising – the constant residual eigenvalue method. This approach is to set a constant eigenvalue for all random eigenvalues that are smaller than or equal to U which we can approximate using 1.3 as seen in the pdf of the random matrix eigenvalues with $N = 24$ and $T = 889$. This constant is the average of all the eigenvalues below 1.3, which is 0.5719. Thus, the total variance (sum of eigenvalues up to U) is preserved. The larger eigenvalues above 1.3 are retained. Empirical covariance is computed using the denoised correlation matrix and the estimated variances from the training data set.

Computation of the denoised correlation matrix “corr2” and the reconstituted denoised covariance matrix “cov2” are shown in the code lines as follows.

```
[15]: ### The 21st to 24th elements on the eigenvalue list are above 1.3
      ### The constant residual eigenvalue method is applied to average as constant for
      ### all eigenvalues below 1.3, or even nullify as they could be just from noises

      small=20 ### eigenvalues elements up to order 20 are all below 1.3
      neweig=eigenvalues.copy() ### rather similar without .copy()
      neweig[:small]=eigenvalues[:small].sum()/small
      eigenvalues1=np.diagflat(neweig)
      ### this is diag 24x24 matrix with same constant in first 20 diagonals
      corr1=np.dot(eVec0,eigenvalues1).dot(eVec0.T)
      ### eVec0.T is transpose of eVec, i.e., corr matrix = eigenvalues1 * eVec0 * eVec0.T
      dd=np.diag(corr1)
      ### after constant residual eigenvalue adjustment, corr1 is not necessarily a corr matrix
      ### need to readjust its diag to ones
      corr2=corr1/dd
      ### divides ith column of corr1 by ith element in list dd
      ### same effect as corr1 .dot(np.diagflat(1/dd))
```

```
[17]: ### Now we use corr2 as denoised correlation matrix of the 24 stock returns
```

```
[19]: ### corr2 is transformed back to corresponding covariance matrix using original variances
      var = trainingset.var()
      sd = np.sqrt(var)
      ### here output comes from pandas -- dataframe, so some numpy commands may not work
      ### as they are not arrays such as np.diagflat()
```

```
[20]: sd1=sd.to_numpy()  ### now sd1 is array

[21]: sd2=np.diagflat(sd1)  ### sd2 is 24 x 24 diagonal matrix with diagonal as std devs

[22]: cov2=(sd2 .dot(corr2)) .dot (sd2.T)
```

Finally, we use the denoised covariance matrix to recompute the optimal weights, “all_weights”, of portfolio based on the training data set with the same portfolio return means but using the denoised covariance matrix, “decovar” in the following code line. See demonstration file Chapter1-6.ipynb.

```
[11]: w = w_min
      num_ports = 100
      gap = (np.amax(r) - ret(r,w_min))/num_ports

      all_weights = np.zeros((num_ports, len(trainingset.columns)))
      ### Note: len(trainingset.columns) is 24 -- there are 24 stocks here
      ### print(np.shape(all_weights)) gives (100,24) -- same as print(all_weights.shape) that gives (100,24)

      print("*****")
      #####

      ret_arr = np.zeros(num_ports)
      vol_arr = np.zeros(num_ports)

      for i in range(num_ports): ## this means looping from i=0 to 1,2,3,4,...,99 (100 loops in total)
          port_ret = ret(r,w) + i*gap
          double_constraint = LinearConstraint([np.ones(trainingset.shape[1]),r],[1,port_ret],[1,port_ret])

          x0 = w_min
          ## Define a function for portfolio volatility.
          portfstderr = lambda w1: np.sqrt(np.dot(w1,np.dot(w1,decovar)))*252)
          optweight = minimize(portfstderr,x0,method='trust-constr',constraints = double_constraint,bounds = bounds)

          all_weights[i,:]=optweight.x  ## 24 x 1 optimal wts at row i, formed using decovar
          ret_arr[i]=port_ret
          vol_arr[i]=vol(optweight.x,decovar)

      sharpe_arr = ret_arr/vol_arr  ## sharpe_arr is 100 x 1 array since it is ret_arr[100]/vol_arr[100] element by element

      plt.figure(figsize=(20,10)) ## optimal frontier using training set and decovar
      plt.scatter(vol_arr, ret_arr, c=sharpe_arr, cmap='viridis')
      ## in plt.scatter, c is a scalar or sequence of n numbers to be mapped to colors using cmap
      ## in plt, for sequential plots, 'viridis' gives colors across the 3D representation of vol_arr, ret_arr, sharpe_arr
      ## c= in front of third dimension sharpe_arr gives the colors in that dimension, otherwise dots will be all blue
      plt.colorbar(label='Sharpe Ratio')
      plt.xlabel('Volatility')
      plt.ylabel('Return')
      plt.show()
```

After that we employ the test data with the optimal weights computed as above to find the new portfolio results.

```
[16]: ### Below the test set data is used with the optimal wts computed
      ### with training set to form the eff portf frontier
      testcovar=testset.cov()
      testr = np.mean(testset,axis=0)*252

      ### initialize
      testport_ret = np.zeros(num_ports)
      testport_vol = np.zeros(num_ports)

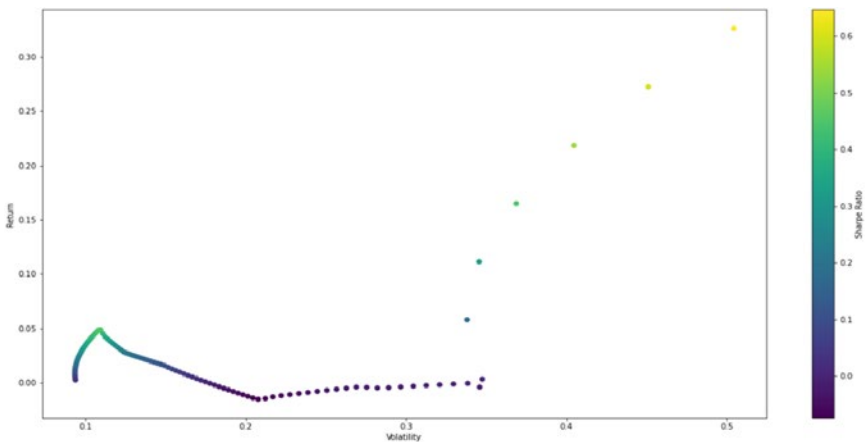
      for i in range(num_ports):
          testport_ret[i] = ret(testr,all_weights[i,:])
          testport_vol[i]= vol(all_weights[i:],testcovar)

      testport_sharpe = testport_ret/testport_vol

      plt.figure(figsize=(20,10)) ### optimal frontier using test set and decovar
      plt.scatter(testport_vol, testport_ret, c=testport_sharpe, cmap='viridis')

      plt.colorbar(label='Sharpe Ratio')
      plt.xlabel('Volatility')
      plt.ylabel('Return')
      plt.show()
```

This is shown as follows.



However, in this case, there is no significant change from the case when the returns covariance matrix is not denoised. This could be due to larger errors in the portfolio expected returns than in the portfolio return variances. Such methods may be more effective in other problems where the means do not change as much.

1.5 Using a More Accurate Forward Predictor

As shown earlier, diversification is limited in terms of being able to produce profitable future portfolio returns if the future returns have means that are not similar to the means based on historical returns used to compute the optimal portfolio weights. In fact, most of the times, accurate predictions of future means count more toward better portfolio performance than diversification itself. This is why prediction of future (expected) returns or similarly prediction of future stock prices is such an important business in financial data science. In this section, we illustrate how a more accurate forward prediction of future expected return would improve the training set optimal portfolio weights for future investing results.

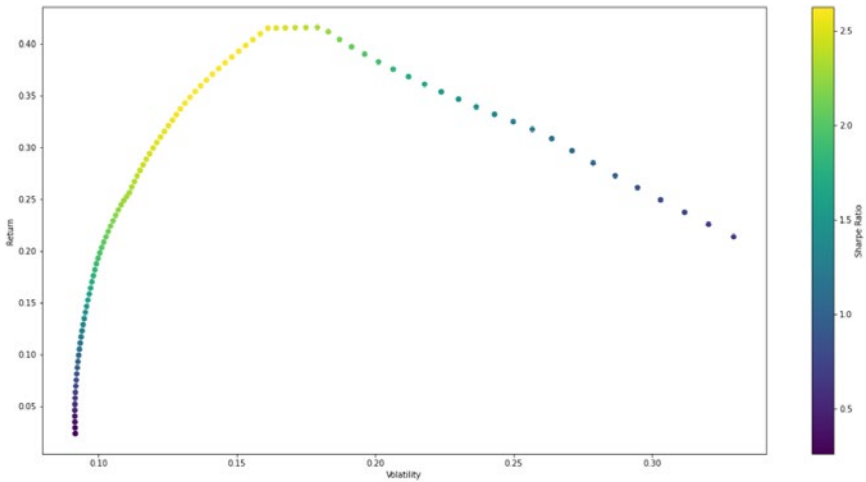
See demonstration file `Chapter1-7.ipynb`. We employ the same training set return data of the 24 stocks. This is used as before in `Chapter1-2.ipynb` to compute the covariance matrix. For the stock mean returns, however, we assume there is a good prediction model. This good prediction model provides predicted future stock returns – and these serve as inputs to the portfolio optimization involving the training set data – the historical mean returns are not used.

Suppose these predicted returns are identical to the mean returns based on the 224 returns observations per stock in 2021. (This is of course knowing about the future – but is used as an illustration here.) This mean vector is then used together with the historical covariance to compute the optimal portfolio weights.

The returns in the first 111 observations of 2021 form the test data set which is a subset of the 224 observations. The test data set return means are therefore randomly close to that based on the 224 observations, but they are not identical.

After the optimal weights for every (required return) k are computed using the training set, suppose we apply these weights to the out-of-sample test date set of 111 observations. This test set is used to construct the out-of-sample mean and covariance matrix of the portfolio returns. The results are shown in the following figure.

Clearly, for whichever set of optimal weight chosen (depending on the target required returns), the performance is close to the *ex-ante* frontier (for volatility < 0.16).



1.6 Concluding Thoughts

Portfolio optimization is a ubiquitous part of buy-side investments. It achieves good in-sample risk-return performances by diversifying risks and attempting to attain the highest expected portfolio return for a given level of estimated portfolio return volatility or risk or to attain the minimum risk for a given level of expected portfolio return. However, there is difficulty of attaining good ex-post portfolio performance results when the ex-post returns or ex-post covariances, particularly the former, differ in a significant way from those used in the training or in-sample. This could happen in times when market shifts or trends change.

Theoretically, minimizing volatility subject to required return may also not be the most preferred outcome. Some investors prefer to maximize the return per unit volatility – Sharpe ratio. It is also possible that portfolio return positive skewness is desired and increasing weights to increase portfolio return skewness may attain better performances although typically the volatility would be higher.

The machine learning approach could attempt to reduce the generalized portfolio performance problem by attempting to form the expected returns and estimated covariances based on predictions rather than based simply on historical data. We can use factor models to try to find better

estimates of future returns. We can also use neural networks or other ML prediction methods to predict next period expected stock returns and use these for constructing optimal weights with the training set data. We can of course choose only stocks with predictably good performances to form the portfolio.

The covariance used for computing the optimal portfolio weights to use in the test data set could also use predictions of variances of individual stock returns from ML models or econometrics model such as the generalized autoregressive conditional heteroskedasticity (GARCH). See Engle (1982) and others. We can assume the historical correlations are more stable, and use these together with the estimated variances to form a covariance matrix for the computation of optimal portfolio weights. In testing, a one period ahead test with a rolling training data set could be more accurate than testing over a time series of test data. For financial investment or trading, it is not sufficient to produce small mean-squared errors in the test; the effect test is to back-test a trading strategy and see if there is significant profit.

For avoiding noise in estimating training covariance matrix, we show how to use the constant residual eigenvalue method to denoise and form a more accurate covariance matrix for the portfolio weight optimization.

References

- Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation, *Econometrica*, 50(4), 987–1007.
- Marchenko, V. A. and Pastur, L. A. (1967). Распределение собственных значений в некоторых ансамблях случайных матриц [Distribution of eigenvalues for some sets of random matrices]. *Mat.Sb. N.S.* (in Russian), 72(114:4), 507–536.
- Markowitz, H. (1952). Portfolio selection. *The Journal of Finance*, 7(1), 77–91. <https://towardsdatascience.com/the-data-science-trilogy-numpy-pandas-and-matplotlib-basics-42192b89e26>.

Chapter 2

Regression and Regularization

In this chapter, we discuss using (multiple) linear regression model to predict a target variable dependent on a number of features (or explanatory variables). We explain the use of non-overlapping data sets for training, validation, and testing for the purpose of prediction of future targets given future feature data. Such new future data outside of the training, validation, and test data sets are called generalized data.

The approach to prediction in machine learning typically adopts the following pathway. The data set is divided into a training data set (usually involving 50–60% of the data), a validation data set (usually involving 20–30% of the data), and a test data set (remaining part of the data set). The test data set is also called the hold-out data set. It is held out, and not involved in the training and validation phases. It is used only when the fitted model is ready for testing to check model usefulness to deploy in future with generalized data.

If there is no hyperparameter in the model/algorithm, there is no need for a validation data set; just the training data set (involving 60–80% of the data) and test data set (remaining part of the data set) will do.

A machine learning model or algorithm contains internal model parameters that have to be estimated or fitted to minimize the prediction or fitting errors between the targets (dependent variable) and the features (explanatory variables). The optimal fit is based on some specified criterion such as minimum mean square error (MMSE) in regression prediction or such as minimum cross entropy (MCE) in classification prediction. Hyperparameters are external to the model/algorithm in the sense that they are not explicitly estimated by using the data. They are exogenously

specified and they control the estimation or fitting process and affects the parameter estimation and model fitting. If there is no hyperparameter in the model or algorithm, the model fitted using the training data set is then used to perform prediction based on features from the test data set. A fitted model means the model parameters and algorithmic structures are determined, and only input features need to be added to obtain the outputs that are predicted targets. The predicted targets or labels in the test data set are compared with the actual targets or labels in the test data set and the accuracy of the predictions is measured by some performance metric. If the latter performance is good, then the fitted model is ready for prediction in generalized data. If not, the model has to be re-considered or improved.

When there is a hyperparameter or there are hyperparameters in the model/algorithm, selection of optimal hyperparameter values (“fine-tuning” the hyperparameters) becomes an additional necessary step in the prediction process. In this situation, an additional non-overlapping validation data set is required.

Different hyperparameter(s) are chosen and each set of the hyperparameter values is used together with the training data set to find the optimized/fitted model. This fitted model (using training data set) is then applied to the validation data set features to obtain predicted targets. These predicted targets are then compared with the actual validation set targets to derive the accuracy measure. Optimal hyperparameters are found by maximizing this prediction accuracy measure. The optimal hyperparameters may be found via a search process or by trial and error. If the training set performance and validation set performance are similarly good, then one may proceed to employing the fitted model on the test data set.

A physical example may be described as follows. A robotic eye is trained to focus on a conveyor belt of manufactured widgets to predict the proportion of defects by learning the shape of the widget. The internal switches on the robotic eye were optimized or set in such a way that the first batch of training samples had prediction errors minimized, such as minimizing the percentage of errors. This was done using a particular speed of the conveyor belt. Using a second batch of validation samples, however, the prediction errors could be significantly larger.

The speed of the conveyor belt is a hyperparameter that may be altered and which will affect the learning of the widget shape parameters and thus the prediction accuracy of the robotic eye. By trying out different conveyor belt speeds, an optimal one could be chosen such that both

training and validation samples yield good prediction accuracies or small prediction errors.

In this chapter we show the use of validation in a multiple linear regression model with regularization. Regularization is a method to constrain a model to fit out-of-sample (validation set) more accurately. Without regularization, the trained model may overfit and this can lead to underfitting or poor prediction performance out-of-sample. In linear regression, regularization takes the form of introducing constraints on the estimated coefficients to ensure they are not too large due to some significant impact of some features that may not occur uniformly or commonly across training and validation sample sets. Regularization is also useful in improving out-of-sample prediction when the model is complex and there are too many features against a finite sample size or where there are high feature correlations that may sometimes produce higher prediction errors. Hyperparameters are used in regression with regularization.

The use of the validation set is to select the best model and associated best hyperparameter values so that the prediction performance in the validation data set is maximized. This result is compared with the training data set performance or in-sample fit on the same set of optimized hyperparameters to decide to proceed with the algorithm on the test data set provided both performances, especially the validation, are satisfactory. Validation also acts as a robustness check.

In a regression model, training is basically to find values of the regression coefficients or parameters that minimize prediction errors in some manner – this is fitting using training data. In the process of fitting, different selections of features may be tried to get the best fitting. Optimal or best fitting is accomplished typically in terms of a maximum fitting R^2 score or else MMSE or least square error. R^2 is the coefficient of determination if it is the case of a linear regression. Maximizing R^2 is equivalent to minimizing a loss function.

After the model is trained or optimally fitted, it is used (using the fitted or estimated coefficients) on the test or hold-out data to check if the predicted target values are close to the actual target values in the test data set. This closeness is also commonly measured by a R^2 test score. If both training and test fittings are consistently good, then the model can be deployed for use in new data or generalized data not seen by the model in training or testing. Of course, in real-life applications, any model needs to be improved or re-trained frequently as more fresh data become available as updated training and test data.

Suppose the regression model or algorithm has hyperparameters. We will study many other algorithms in the rest of the book and will come across hyperparameters such as number of branches and depth in decision tree methods, number of nodes and layers in neural network methods, learning rate in gradient boosting methods, and so on. The training data can be used firstly to check if the predictive model or algorithm is feasible – if the in-sample prediction error is too large, e.g., wrong prediction more than 40% or 50% of the time or having a low R^2 , given reasonable choices of hyperparameters, then another predictive method or model should be considered.

If feasible, the model should next be checked for consistent performance as well as improved performance by optimal choice (fine-tuning) of the hyperparameters. This is done by a separate validation data set. In this context, the sample of size N could be divided into a training data set, e.g., 60% or $0.6 N$, a validation data set, e.g., 20% or $0.2 N$, and a test data set of the remaining 20% or $0.2 N$. For cross-sectional prediction, i.e., data not belonging to a time series, the sample points in the sets could be randomly assigned from the sample of size N . However, for time series, it is more suitable to assign sample points with the earliest time-stamps in the training set, followed by validation set, and the latest data in the test set. The validation data set prediction performance should return consistent performance of similar accuracy as in the training data. An inconsistent performance could be over-fitting or very high prediction accuracy in the training data set but under-fitting or high prediction error in the validation data set.

It is important to recall that fitting in the validation data set is not regressing validation data targets on validation data features. Optimal fitting in validation data set is using trained or fitted model from the training data set with selected hyperparameter(s) to predict validation data targets based just on the validation data features, and then to compare the fit of the actual target values against these predicted ones. If tuning the hyperparameters cannot eliminate the situation of high prediction error in the validation data set, then the predictive model could be problematic.

Suppose there are feasible competing models to perform the prediction of the target variable. Competing models are generally defined to be different algorithms or methods of prediction. These models use the same $0.6 N$ training data set and the same $0.2 N$ validation data set. Now, there is the additional task of selecting the model amongst the competing models before testing (use of the test data set) is done. After fine-tuning the

various models in the validation phase, the best performing model with the highest validation R^2 score is then selected for the testing. If the test data prediction returns similarly good performing results as the validation, then there would have been a reconfirmation, and the model would be suitable for use in a generalized data situation. If the testing returns poor performance such as substandard prediction, then the model has to be re-examined before applying it for generalized data prediction.

It is also possible that after the validation phase, an optional step could be to combine the original training and the validation data into a final training set, and this is used for a final training or fitting the coefficients (re-estimating the model parameters) with the larger sample size. The validation hyperparameters could be used for this purpose. The purpose here is not to change the number of features or to change or re-tune the hyperparameters, but to use the larger combined training data set to re-estimate and get better or less noisy coefficient estimates. This trained model is then used for checking the test R^2 in the test data set.

In this chapter, a useful method of cross-validation in machine learning is also discussed. This method allows validation to be conducted within the training data set itself and thus avoids use of another separate set of data for validation.

2.1 Regression and Regularization

Let Y_k be the k th target variable observed and $(X_{1k}, X_{2k}, X_{3k}, \dots, X_{pk})$ be the set of p features related to the k th target variable. The linear regression is written as

$$Y_k = b_0 + b_1 X_{1k} + b_2 X_{2k} + b_3 X_{3k} + \dots + b_p X_{pk} + \varepsilon_k \quad (2.1)$$

where ε_k is a residual variable that is unobserved. Unlike some classical setup in linear regression where the added residual noise or innovation may take specific distributional assumptions to enable testing, the residual noise in the machine learning method takes a lower profile. We shall employ several competing models in the regression of type in Eq. (2.1). Therefore, we use $0.6 N$ amount of data points for training, $0.2 N$ amount for validation, and the remaining $0.2 N$ for testing.

The linear regression under supervised machine learning in Eq. (2.1) typically aims to find optimal parameter (estimates) $\hat{b}_0, \hat{b}_1, \hat{b}_2, \dots, \hat{b}_p$ so that a loss criterion such as sum of squared errors, $\sum_{k=1}^{0.6N} (Y_k - \hat{Y}_k)^2$ is minimized,

where fitted $\hat{Y}_k = \hat{b}_0 + \hat{b}_1 X_{1k} + \hat{b}_2 X_{2k} + \dots + \hat{b}_p X_{pk}$. Minimizing sum of squared errors is the same as finding estimates with MMSE $\frac{1}{0.6N} \sum_{k=1}^{0.6N} (Y_k - \hat{Y}_k)^2$.

The R^2 -score for the training data set is $1 - \frac{\sum_{k=1}^{0.6N} (Y_k - \hat{Y}_k)^2}{\sum_{k=1}^{0.6N} (Y_k - \bar{Y}_k)^2}$.

The second term is sum of squared errors over total sum of squares in standard least squares regression terminology, and R^2 lies between 0 and +1.

When the trained or fitted coefficients $\hat{b}_0, \hat{b}_1, \hat{b}_2, \dots, \hat{b}_p$ are employed in the prediction of targets Y_k 's in the validation data set using the associated validation set data of X_{jk} 's (for $j = 1, 2, \dots, p$), the prediction yields $Y_k^* = \hat{b}_0 + \hat{b}_1 X_{1k} + \hat{b}_2 X_{2k} + \dots + \hat{b}_p X_{pk}$. The prediction validation score is

$R^2 = 1 - \frac{\sum_{k=1}^{0.2N} (Y_k - Y_k^*)^2}{\sum_{k=1}^{0.2N} (Y_k - \bar{Y}_k)^2}$. In this case, the score R^2 may be sometimes negative if the actual model of the validation data is different so that $\sum_{k=1}^{0.2N} (Y_k - Y_k^*)^2$ is very large. The R^2 score in the validation set is a useful measure of accuracy of the machine learning method when coefficients (or model parameters) trained by a training data set is applied to new data. A somewhat similar though not identical metric in measuring prediction performance is the root mean square error (RMSE) which is

$$\sqrt{\sum_{k=1}^{0.2N} (Y_k - Y_k^*)^2 / (0.2N)}.$$

The validation performance will not be good if the assumption that both data sets come from the same data generating model is incorrect. The use of training and validation data sets is akin to the idea of in-sample and out-of-sample fits. The above comments on the validation data set apply similarly to use of trained or fitted model in prediction in the test data set.

It should be noted that where $\{Y_k, X_{1k}, X_{2k}, X_{3k}, \dots, X_{pk}\}_{k=1,2,\dots,N}$ is not a time series, but a cross-section, then splitting the data set into training, validation, and test subsets can be random. However, if it is a time series, then typically the training set data would precede those of the validation data set. Both should precede that of the test set. Moreover, in the strict sense of prediction based on available information, the features would have time-stamps prior to that of the target variable.

If a training set R^2 or accuracy of fit by the model is high and if a validation set R^2 score is also high, then the fitted model with its fine-tuned hyperparameters is used to check out the test data set. If the test R^2 score is also high or having accurate prediction based on the trained or fitted coefficients, then the model with its trained coefficients may be usable for future prediction of the target variable when a new set of features arrives.

Regressions based on available features can run into overfitting problem when too many features (or explanatory variables) are utilized.

This may produce good fit, e.g., high R^2 in the training data set, but the fitted coefficients may in turn produce inaccurate predictions using the validation and the test data sets.

Consider the multiple linear regression (2.1) where we re-write in short-form its predicted value as $\hat{Y}_k = \sum_j \hat{b}_j X_{jk}$. The sum of squared prediction errors is then $\sum_k (Y_k - \sum_j \hat{b}_j X_{jk})^2$. Suppose an extra feature Z_k for instance or sample point k is added. Then the sum of squared prediction errors is now $\sum_k (Y_k - \sum_j \hat{b}_j X_{jk} - \hat{b}_{p+1} Z_k)^2$. Clearly this latter SSE is smaller, i.e.,

$$\sum_k (Y_k - \sum_j \hat{b}_j X_{jk} - \hat{b}_{p+1} Z_k)^2 \leq \sum_k (Y_k - \sum_j \hat{b}_j X_{jk})^2$$

as the MMSE on the left-hand term is at most as large as the right-hand term when we put $\hat{b}_{p+1} = 0$. Hence unnecessarily increasing the number of features that do not actually affect the target can produce overfitting (higher in-sample training data set R^2). When a validation or test set is used however, there will be underfitting as the additional irrelevant features can distort the prediction and lead to low validation or test score.

The training data set may contain some large outlier instances or sample points that may not occur uniformly or commonly across training and validation sample sets. These kinds of outliers may not occur in the validation or test data sets. By trying to fit with MMSE on training data set with these outliers (that are not removed or not examined), coefficients associated with these outlier features may be fitted with values of larger magnitudes than if the outliers did not occur. This creates overfitting in the training data set (higher R^2) relative to the validation or test set that will see underfitting (lower R^2) when the fitted coefficients are used for prediction (with same hyperparameters).

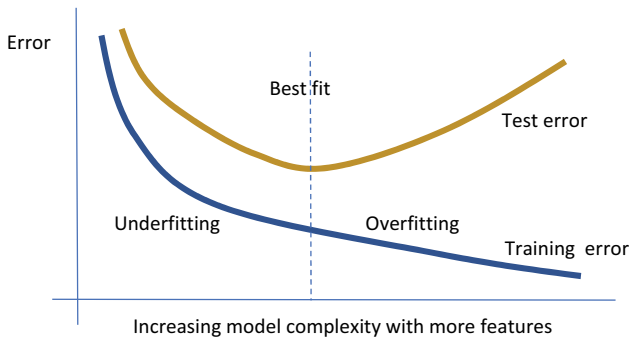
Overfitting could also occur when some features are highly multi-collinear, i.e., having high correlations. We illustrate as follows. Suppose a correctly specified regression is $Y_k = b_0 + b_1 X_k + \varepsilon_k$ where $Y = (5, 5.1, 4.6, 4.9)$. $X = (9.7, 8.5, 7.5)$. A simple regression yields R^2 of 0.18. Consider another feature Z that is highly correlated with X . $Z = (1.3, 0.5, 2.5)$. Their correlation is -0.92 . If we add Z to the regression, i.e., $Y_k = b_0 + b_1 X_k + b_2 Z_k + \text{residual error}$, the R^2 becomes 0.87. When X and Z are added in the regression, their negative correlation smooths out their combined fluctuations, producing aggregated $0.36X_k + 0.38Z_k$ that correlated highly with Y and hence yields a lower R^2 .

The high multi-collinearity could produce overfitting in the training data set but in a different sample such as validation or test data set, the

realizations of X and Z in small sample may not follow the correlation pattern exactly. In such a case, it is more likely that since the correct specification is without Z , the out-of-sample prediction would be poorer with underfitting. Other approaches of regression avoiding poor out-of-sample fit due to multi-collinearity include selection of features, stepwise regression, and principal component regression or regression using key principal components.

Underfitting occurs when the fitted R^2 's or the prediction R^2 are low. Obviously, if training R^2 is too low after optimization, the algorithm is ineffective and should be changed – there is no need to proceed to testing. With increasing number of features, it is possible that the training R^2 will increase and equivalently its training error (this could be defined as $1 - R^2$) will decrease. However, the validation R^2 score and the test R^2 score could still be low.

There is possibly an ideal balance between overfitting and underfitting for the training data as seen in the graph. At the ideal balance, it is likely that the validation and the test data fits will be best (least error). This is seen in the graph below.



To reduce overfitting and enable good validation and test data predictions, besides feature selections, some constraints can be added to the regression. A penalty function is added to reduce the tendencies to yield large-fitted coefficients – this technique is called regularization. There are three common regularized linear regression methods – Ridge regression, Lasso (Least absolute shrinkage and selection operator) regression, and Elastic Net regression. Regularization could shrink some fitted coefficients or even reduce them toward zero, effectively reducing the dimension space of the features since those affected features become impactless if their fitted coefficients are close to zero. They are also called shrinkage

estimators. They reduce the ill effects of excessive number of features, outsized outliers and also high multi-collinearity.

For Ridge regression, see the module `sklearn.linear_model.Ridge` with documentation in https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html. One can also check this source for the explicit codes. This module is the Ridge regression model that minimizes the objective function

$$\sum_{k=1}^n \left(Y_k - \sum_{j=0}^p b_j X_{jk} \right)^2 + \alpha \left(\sum_{j=1}^p b_j^2 \right) \quad (2.2)$$

where $X_{0k} = 1$. The second term is in L^2 -norm (sum of squared values of coefficients – related to measure of Euclidean distance), so Eq. (2.2) is also called a L^2 regularized regression. Note there is no constraint on the intercept. When we use the minimization in `sklearn.linear_model.Ridge` (), the default parameters within () include setting $\alpha > 0$, e.g., $\alpha = 0.1$. We can of course try different values for the alpha hyperparameter to attain low training error.

For Lasso regression using the module `sklearn.linear_model.Lasso`, see documentation in https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html. One can also check this source for the explicit codes. This module is the Lasso regression model that minimizes the objective function

$$\sum_{k=1}^n \left(Y_k - \sum_{j=0}^p b_j X_{jk} \right)^2 + \alpha \sum_{j=1}^p |b_j| \quad (2.3)$$

where $X_{0k} = 1$. The second term is in L^1 -norm (sum of absolute values of coefficients), so Eq. (2.3) is also called a L^1 regularized regression. When we use the minimization in `sklearn.linear_model.Lasso` (), the default parameters within () include setting $\alpha > 0$, e.g., $\alpha = 0.1$. We can of course try different values for the alpha hyperparameter to attain low training error.

Another possible regularized regression is the ‘elastic net’ regression that is the minimization of objective function as follows, where there are two hyperparameters α_1 and α_2 .

$$\sum_{k=1}^n \left(Y_k - \sum_{j=0}^p b_j X_{jk} \right)^2 + \alpha_1 \left(\sum_{j=1}^p b_j^2 \right) + \alpha_2 \sum_{j=1}^p |b_j| \quad (2.4)$$

This is a combination of regularizations from the Ridge and also the Lasso methods.

2.2 Worked Example: Data

We explain the regularized regressions based on a study to predict Singapore Housing Development Board (HDB) resale flat (house) prices. See the publicly accessible data sets at <https://data.gov.sg/collections/189/view>. This method and other related methods in machine learning can be applied to predicting housing prices anywhere as long as there are adequate data pertaining to the house prices and features of each house. See demonstration file Chapter2-1.ipynb.

We use a data set that comprises only 13 features (characteristics). 62,359 samples (sample points) are available in the data set HDBflat_value.csv, each of which is a flat in Singapore. The data cover the period 2017 through to 2023. The target variable is “resale_price_persqm” – the transacted resale price of a 4-room flat in terms of Singapore dollars per square meter of the flat floor area.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv('HDBflat_value.csv')
### Historical data of Singapore HDB Resale prices
### are available at https://data.gov.sg/collections/189/view
df.head()
```

```
df = df.drop(columns=['flat_type', 'resale_price', 'persqm_price_exceed_6500'])
### delete 'district_E' and 'Simplified' columns as they create col dependencies
df = df.drop(columns=['district_E', 'Simplified'])
df.isnull().sum()
### this is to check if there is any missing data in the index
### and other columns -- none or '0'
```

```
### Convert 'year' to no. of years after 2016
df['year'] = df['year'] - 2016
```

```
df=df.iloc[:,0:13]
df.head()
```

	year	height	floor_area_sqm	remaining_lease	district_C	district_N	district_S	district_W	Improved	Model_A	New_Gen	Premium	resale_price_persqm
0	1	3	82	63.500000	1	0	0	0	1	0	0	0	4695.121951
1	1	6	83	61.666667	1	0	0	0	1	0	0	0	7469.879518
2	1	6	85	63.583333	1	0	0	0	1	0	0	0	4705.882353

The features, after some pre-processing, are:

- (1) “year” – number of years after 2016 in which the resale took place, with resale in 2017 indicated as “1”, resale in 2018 indicated as “2”, and so on.
- (2) “height” – the level of the flat. For example, “9” indicates that the flat was in the 9th storey.
- (3) “floor_area_sqm” – the floor area of the flat.
- (4) “remaining_lease” – the number of years left in the lease. All Singapore government flats have a 99 years old lease at the start. By the end of the lease, the flat reverts back to the government.
- (5) “district_C” – location of the flat in the central region of Singapore.
- (6) “district_N” – location of the flat in the northern region of Singapore.
- (7) “district_S” – location of the flat in the southern region of Singapore.
- (8) “district_W” – location of the flat in the western region of Singapore.
- (9) “Improved” – a specific design of the 4-room flat.
- (10) “Model_A” – a specific design of the 4-room flat.
- (11) “New_Gen” – a specific design of the 4-room flat.
- (12) “Premium” – a specific design of the 4-room flat.

Command “df.describe()” in Pandas provides summary descriptive statistics of column variables in the DataFrame. These include mean, standard deviation, minimum, maximum, and the various percentiles for the numerical data.

```
# Let's summarize the data to see the distribution of data
print(df.describe())
### 62,359 rows of data or cases from year 2017 thru 2023 (may not
### be complete data collected here). 'flat_type' all 4-Room.
```

	year	height	floor_area_sqm	remaining_lease	\
count	62359.000000	62359.000000	62359.000000	62359.000000	
mean	4.214516	9.757469	95.475232	78.488792	
std	1.968071	5.589792	6.857665	13.050042	
min	1.000000	3.000000	75.000000	45.750000	
25%	3.000000	6.000000	92.000000	66.416667	
50%	4.000000	9.000000	93.000000	78.500000	
75%	6.000000	12.000000	102.000000	92.666667	
max	7.000000	36.000000	133.000000	97.166667	

	district_C	district_N	district_S	district_W	Improved	\
count	62359.000000	62359.000000	62359.000000	62359.000000	62359.000000	
mean	0.004169	0.617393	0.064289	0.238618	0.020478	
std	0.064437	0.486027	0.245269	0.426242	0.141630	
min	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	0.000000	
50%	0.000000	1.000000	0.000000	0.000000	0.000000	
75%	0.000000	1.000000	0.000000	0.000000	0.000000	
max	1.000000	1.000000	1.000000	1.000000	1.000000	

	Model_A	New_Gen	Premium	resale_price_persqm
count	62359.000000	62359.000000	62359.000000	62359.000000
mean	0.640870	0.111708	0.157058	5027.826256
std	0.479749	0.315009	0.363859	1375.536497
min	0.000000	0.000000	0.000000	2307.692308
25%	0.000000	0.000000	0.000000	4086.021505
50%	1.000000	0.000000	0.000000	4772.277228
75%	1.000000	0.000000	0.000000	5652.173913
max	1.000000	1.000000	1.000000	13310.344830

Code line [8] explores the empirical distributions of each feature. Such preliminary explorations provide some ideas of which features may be more important in the prediction.

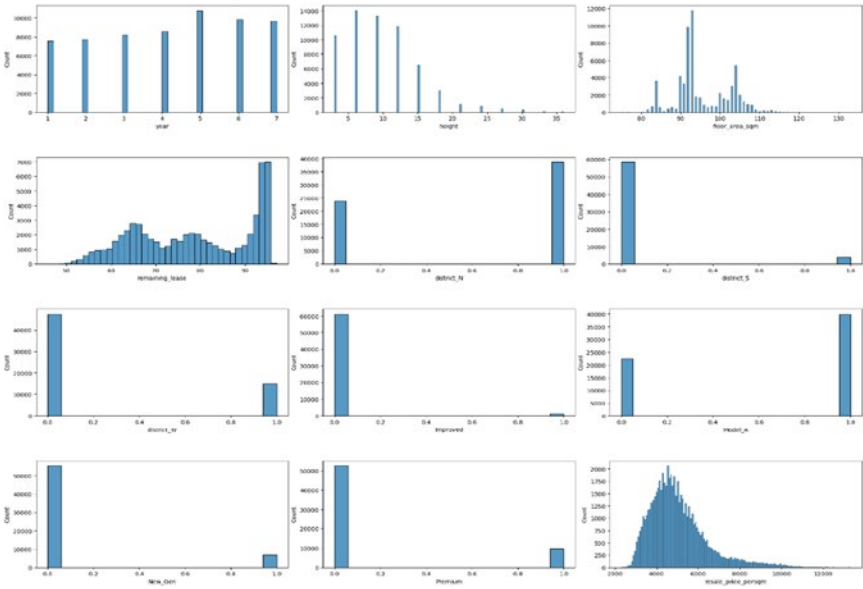
```

### Proportion of '1's in 'district_C' is very small, so we left out of graph
df1 = df.drop(columns=['district_C'])

import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats

fig, axs = plt.subplots(ncols=3, nrows=4, figsize=(20, 15))
index = 0
axs = axs.flatten()
for k,v in df1.items():
    sns.histplot(v, ax=axs[index])
    index += 1
plt.tight_layout(pad=0.4, w_pad=0.5, h_pad=5.0)
### uses seaborn

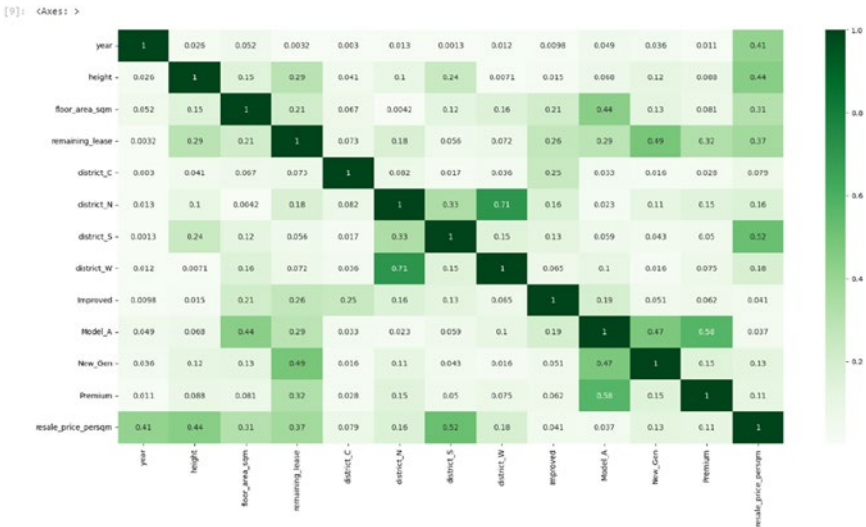
```



Code line [9] shows the correlation heat map amongst the features, including the target variable.

```
### pairwise correlation on the features including 'resale_price_persqm'
plt.figure(figsize=(20, 10))
sns.heatmap(df.corr().abs(), annot=True, cmap="Greens" )
### In heat map, clearly tax and rad are highly correlated
### For more options - see https://seaborn.pydata.org/generated
### /seaborn.heatmap.html
```

In the last row, it is seen that “year”, “height”, “remaining_lease”, and location “district_S” all have high positive correlations with the target variable “resale_price_persqm”.



Next we define the target variable y and also the features X . The vector y and matrix X are then divided into (60%) rows forming corresponding X_{train} , y_{train} in training set, (20%) rows forming corresponding X_{val} , y_{val} in the validation set, and the remaining (20%) rows forming X_{test} , y_{test} in the test set. The rows are randomly assigned to the separate data sets via a random_state. When formed, the training, validation, and test data sets have correspondingly 37,415, 12,472, and 12,472 sample sizes.

```
### Here we define the target var y and the features X
X = df.loc[:, ['year', 'height', 'floor_area_sqm', 'remaining_lease', \
               'district_C', 'district_N', 'district_S', 'district_W', \
               'Improved', 'Model_A', 'New_Gen', 'Premium']]
y = df['resale_price_persqm']
```

```

### Splitting dataset into Training set 60%, Validation Set 20% and Test set 20%
from sklearn.model_selection import train_test_split
X_train, X_valtest, y_train, y_valtest = train_test_split(X, y, test_size = 0.4, \
                                                         random_state = 37)
### X_valtest and y_valtest contains 20% validation and 20% test data
X_val, X_test, y_val, y_test = train_test_split(X_valtest, y_valtest, test_size = 0.5, \
                                                         random_state = 14)
len(X_train), len(y_train), len(X_val), len(y_val), len(X_test), len(y_test)

(37415, 37415, 12472, 12472, 12472, 12472)

```

2.3 Linear Regression Prediction

Continuing with the worked example, a linear regression model is used as a base comparison model to predict the flat resale prices (per square meter) in Subsection 2.2. For comparison with the regularized models with hyperparameters, we employ the training set (60% data) for the initial fitting. This is done in code lines [13] and [14].

```

### see module documentation in https://scikit-learn.org/stable/modules
### /generated/sklearn.linear_model.LinearRegression.html
from sklearn.linear_model import LinearRegression
linreg = LinearRegression()
linreg.fit(X_train, y_train)
### To retrieve the intercept:
print('Intercept (train):', linreg.intercept_)
### To retrieving the slope:
print('Slopes (train):', linreg.coef_)

from sklearn.metrics import r2_score
y_pred_linreg_train = linreg.predict(X_train)
### 'y_pred_linreg_train' is predicted y using the x_train data
r2_score_linreg_train = r2_score(y_train, y_pred_linreg_train)
### r2_score_linreg_train is R-sq in lin reg involving training data set
print('Linreg_train_R2_score: ', r2_score_linreg_train)

Intercept (train): 3422.055996603847
Slopes (train): [ 281.167182    53.05749049  -19.83328235   32.31417559
 1295.99801638  -972.2729605   1601.62306092 -1088.95040275
 -261.64604368  -18.87466348   125.04386431   94.53231616]
Linreg_train_R2_score: 0.6721950473196396

from sklearn.metrics import mean_squared_error
### Predicting RMSE -- the Training set results
rmse_linreg_train = (np.sqrt(mean_squared_error(y_train, y_pred_linreg_train)))
print("Linreg_train_RMSE: ", rmse_linreg_train)

Linreg_train_RMSE: 786.2624684200345

```

In the fitting using the 60% training data set, the fitted R^2 (or R^2 score) is 0.672195 with RMSE of 786.2625.

In code line [16], the fitted model is then used to predict targets in the validation data set employing features from the validation data set. The linear regression validation R^2 score is 0.69452567 with RMSE 769.1847. It is possible, as in this case, that the prediction outcomes are better than in the fitted model.

```
### Predicting R2 Score using Validation Set but Intercept(train)
### and Slopes(train) from Training results
y_pred_Linreg_val = Linreg.intercept_ + np.dot(X_val,Linreg.coef_.T)

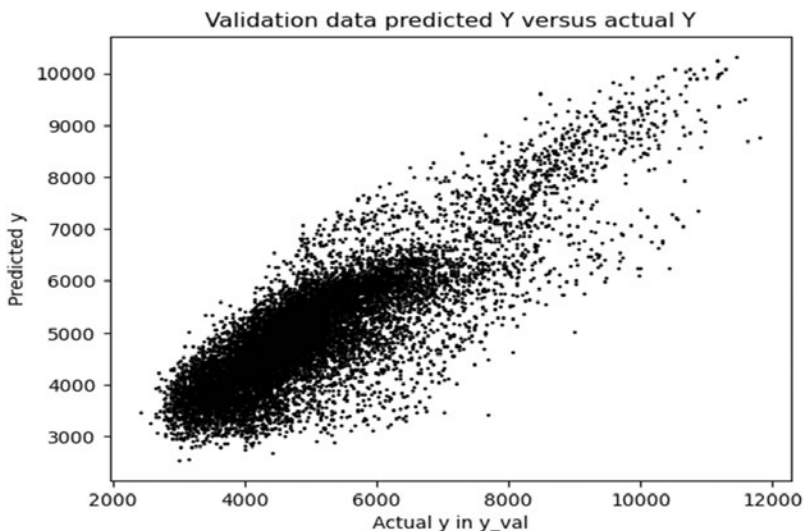
r2_score_Linreg_val = r2_score(y_val, y_pred_Linreg_val)
### r2_score_Linreg_val is R-sq in lin reg involving validation data set
print('Linreg_val_R2_score: ', r2_score_Linreg_val)

rmse_Linreg_val = (np.sqrt(mean_squared_error(y_val, y_pred_Linreg_val)))
print("Linreg_val_RMSE:",rmse_Linreg_val)

Linreg_val_R2_score: 0.6945256718342248
Linreg_val_RMSE: 769.1847041142491
```

The predicted target in the validation set is plotted against the actual target values and this is shown in code line [18].

```
### A scatterplot may be clearer - here matplotlib
plt.scatter(y_val, y_pred_Linreg_val,s=1,color='black')
plt.title("Validation data predicted Y versus actual Y")
plt.xlabel("Actual y in y_val")
plt.ylabel("Predicted y")
plt.show()
```



2.4 Fine-Tuning Hyperparameters

The Ridge, Lasso, and Elastic Net regressions with constraints as in Eqs. (2.2), (2.3), and (2.4) are then used to fit the data in the training set. For each model in Eqs. (2.2), (2.3), and (2.4), different hyperparameter value(s) each time is employed to fit the model using the training set data. The fitted model is then used to perform prediction based on features from the validation set. The validation prediction R^2 score is noted, and the hyperparameter values are iterated till a maximum validation R^2 score is obtained. The optimal (best or “fine-tuned”) hyperparameter value(s) and the associated maximum validation R^2 score are noted for each model.

Code line [21] shows the outcome for the Ridge regression model.

Ridge

```
from sklearn import linear_model
from sklearn.metrics import r2_score

base = 0.6

num = 0.0
while num < 5.0:

    Ridge=linear_model.Ridge(alpha=num,)
    Ridge.fit(X_train, y_train)

    y_pred_Ridge_val = Ridge.intercept_ + np.dot(X_val,Ridge.coef_.T)
    r2_score_Ridge_val = r2_score(y_val, y_pred_Ridge_val)
    rmse_y_pred_Ridge_val = (np.sqrt(mean_squared_error(y_val, y_pred_Ridge_val)))

    if r2_score_Ridge_val > base:
        base = r2_score_Ridge_val
        alpha_Ridge = num
        RI = Ridge.intercept_
        RC = Ridge.coef_
        RMSE = rmse_y_pred_Ridge_val
        num = num + 0.01

print("OPTIMAL HYPERPARAMETER in VALIDATION SCORE")
print("alpha:", alpha_Ridge)
print('Intercept (train):', RI)
print('Slopes (train):', RC)
print("r2_score_Ridge_validation:", base)
print("Pred_RMSE_Ridge_validation:", RMSE)
```

```

OPTIMAL HYPERPARAMETER in VALIDATION SCORE
alpha: 3.5399999999999685
Intercept (train): 3425.557792489438
Slopes (train): [ 281.13954996   53.10300082  -19.86813128   32.30366725
 1262.57317609 -972.43971792  1597.96984447 -1088.88032408
 -255.83874467  -18.01989755   125.29255267   95.05039738]
r2_score_Ridge_validation: 0.6945282132520314
Pred_RMSE_Ridge_validation: 769.1815044611373

```

Optimal alpha with highest R^2 score is 3.54. Compared with the base regression model with no constraint, the validation R^2 score is marginally higher at 0.69452821 while the RMSE is marginally lower at 769.1815. The estimated coefficients for features “district_C”, “district_S”, and “Improved” also have smaller magnitudes due to the Ridge regression constraint. Hence, the constraints lead to better prediction.

It should be noted that while it is convenient to employ an optimization package like Sklearn for the regressions, they use objective functions that are scaled differently to facilitate numerical solutions. For example, sklearn uses $\min \|y - Xw\|^2 + \alpha \|w\|^2$ for Ridge regression just as in Eq. (2.2), with vector w consisting of the parameters b_k 's as elements. But sklearn uses $\min (2n)^{-1} \|y - Xw\|^2 + \alpha \|w\|$ for Lasso regression, which is different from Eq. (2.3) as the alpha in Sklearn Lasso has to be scaled up by $(2n)$ to make it equivalent to the alpha in (2.3), where n is the sample size in the regression. And in sklearn's Elastic Net regression, the object is $\min (2n)^{-1} \|y - Xw\|^2 + 0.5 \times \alpha \times (1 - \text{l1_ratio}) \times \|w\|^2 + \alpha \times \text{l1_ratio} \times \|w\|$, which has different scales as in Eq. (2.4). In the latter, comparing with Eq. (2.4), $\alpha_1 \equiv n \times \alpha \times (1 - \text{l1_ratio})$ and $\alpha_2 \equiv 2n \times \alpha \times \text{l1_ratio}$, where α and l1_ratio are input values to the sklearn optimization.

l1_ratio is called the ElasticNet mixing parameter. If $\text{l1_ratio} = 1$, the penalty or constraint would just be the L1 penalty or just the Lasso regression. If $\text{l1_ratio} = 0$, the penalty or constraint would just be the L2 penalty or just the Ridge regression. The computable range of l1_ratio is between 0 and 1.0 inclusive. If the mixing parameter lies within $(0,1)$, it would be a combination of L1 and L2 constraints.

Thus, for the various regularized regressions, we would not venture into troublesome comparisons of the hyperparameter values across different models. For each model, just as in the Ridge regression above, the optimal or “fine-tuned” hyperparameter is found which led to the maximum

R^2 scores. The latter is what is important for comparisons across the models for their validation data set performances.

Code line [23] shows the outcome for the Lasso regression model.

Lasso

```
from sklearn import linear_model
from sklearn.metrics import r2_score

base = 0.6

num = 0.0
while num < 0.3:

    Lasso=linear_model.Lasso(alpha=num,)
    Lasso.fit(X_train, y_train)

    y_pred_Lasso_val = Lasso.intercept_ + np.dot(X_val,Lasso.coef_.T)
    r2_score_Lasso_val = r2_score(y_val, y_pred_Lasso_val)
    rmse_y_pred_Lasso_val = (np.sqrt(mean_squared_error(y_val, y_pred_Lasso_val)))

    if r2_score_Lasso_val > base:
        base = r2_score_Lasso_val
        alpha_Lasso = num
        LI = Lasso.intercept_
        LC = Lasso.coef_
        RMSE = rmse_y_pred_Lasso_val
        num = num + 0.001

print("OPTIMAL HYPERPARAMETER in VALIDATION SCORE")
print("alpha:", alpha_Lasso)
print('Intercept (train):', LI)
print('Slopes (train):', LC)
print("r2_score_Lasso_validation:", base)
print("Pred_RMSE_Lasso_validation:", RMSE)

OPTIMAL HYPERPARAMETER in VALIDATION SCORE
alpha: 0.043000000000000003
Intercept (train): 3420.4820939032215
Slopes (train): [ 281.15356589   53.07079852  -19.8268077   32.32253673
 1283.35605292 -971.77259527 1600.85485195 -1088.3703868
 -257.48300225 -18.99733798 124.73335914  93.93814603]
r2_score_Lasso_validation: 0.6945260996464644
Pred_RMSE_Lasso_validation: 769.1841654982089
```

Optimal alpha is 0.043. Compared with the base regression model with no constraint, the validation R^2 score is marginally higher at 0.69452610 while the RMSE is marginally lower at 769.1841. The performance, however, is not as good as that of the Ridge regression.

Code line [25] shows the outcome for the Elastic Net regression model.

Elastic Net

```

from sklearn import linear_model
from sklearn.metrics import r2_score

base = 0.6

num1 = 0.0
while num1 < 0.1:

    num2 = 0.0
    while num2 < 1.0:

        ELNet=linear_model.ElasticNet(alpha=num1,l1_ratio=num2)
        ELNet.fit(X_train, y_train)

        y_pred_ELNet_val = ELNet.intercept_ + np.dot(X_val,ELNet.coef_.T)
        r2_score_ELNet_val = r2_score(y_val, y_pred_ELNet_val)
        rmse_y_pred_ELNet_val = (np.sqrt(mean_squared_error(y_val, y_pred_ELNet_val)))

        if r2_score_ELNet_val > base:
            base = r2_score_ELNet_val
            alpha_EN = num1
            l1_ratio_EN = num2
            EI = ELNet.intercept_
            EC = ELNet.coef_
            RMSE = rmse_y_pred_ELNet_val

            num2 = num2 + 0.1
            num1 = num1 + 0.001

print("OPTIMAL HYPERPARAMETER in VALIDATION SCORE")
print("alpha:", alpha_EN)
print("l1_ratio:", l1_ratio_EN)
print('Intercept (train):', EI)
print('Slopes (train):', EC)
print("r2_score_ELNet_validation:", base)
print("Pred_RMSE_ELNet_validation:", RMSE)
OPTIMAL HYPERPARAMETER in VALIDATION SCORE
alpha: 0.001
l1_ratio: 0.8999999999999999
Intercept (train): 3425.7175670985744
Slopes (train): [ 281.13772558   53.10583882  -19.86993016   32.30326366
 1260.46508037  -972.436558   1597.74950211 -1088.86195051
 -255.4296934   -17.97579224   125.29936726   95.06568431]
r2_score_ELNet_validation: 0.6945281801665097
Pred_RMSE_ELNet_validation: 769.1815461159999

```

Optimal alpha is 0.001 and optimal l1_ratio is 0.899. The validation R^2 score is 0.69452818 while the RMSE is marginally lower at 769.1815. The performance is just marginally lower than that of the Ridge regression.

The validation R^2 scores for the linear regression with no constraints, the Ridge, Lasso, and Elastic Net regressions are tabulated below with the optimal hyperparameters shown in the brackets.

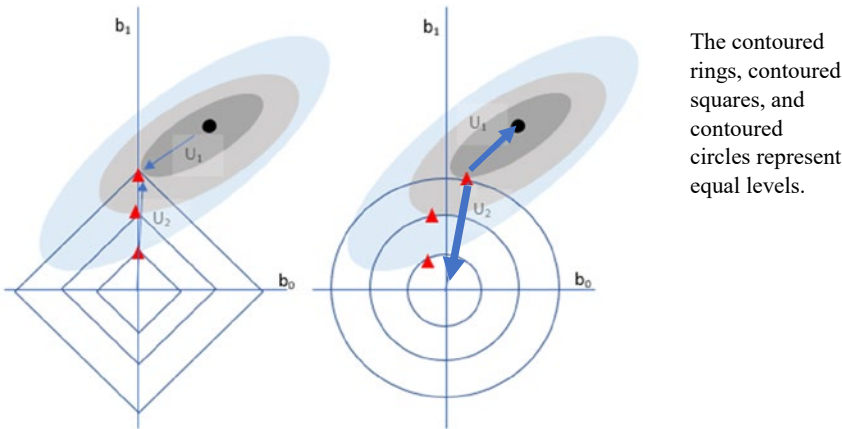
	Validation $(R)^2$
Linear regression (no constraints)	0.69452567
Ridge regression (sklearn $\alpha = 3.54$)	0.69452821
Lasso regression (sklearn $\alpha = 0.043$)	0.69452610
Elastic Net regression (sklearn $\alpha = 0.001$, l1_ratio = 0.899)	0.69452818

In the data studied, the various regressions produce close solutions. In general, however, there are some differences between the Ridge and the Lasso regression due to the difference in their constraints. Their difference can be illustrated geometrically as follows using a two-dimensional case of minimizing the sum of squared errors

$$\sum_{k=1}^n (Y_k - b_0 - b_1 X_{1k})^2$$

that is geometrically an ellipse centered at (b_0, b_1) , taking the values (Y_k, X_k) 's as given.

Under regularization, its minimization is subject to the Lasso type constraints (LHS below) and the Ridge type constraints (RHS below).



It can be seen in the diagrams that the black dot is the (b_0^*, b_1^*) optimal point for the minimization of the unconstrained least squares regression.

U_1 denotes adjusting the (b_0, b_1) estimates toward minimizing the unconstrained objective function $\sum_{k=1}^n (Y_k - \sum_{j=0}^p b_j X_{jk})^2$ while U_2 denotes pulling solution (b_0, b_1) toward minimizing the penalty function. Minimizing the combination of the quadratic function and the penalty constraint is like pulling in opposite directions and the minimum is reached at a point when both contours meet tangentially at points marked as triangles. Higher alphas will pull the solution toward the origin and vice-versa. For a given alpha, the minimum solution will not be away from the tangential point as the sum of the two components will then be larger. It is seen that for Lasso, often the optimal points involve zeros in some coefficients.

In general, for problems with a much larger set of features, Lasso models are adept at feature selection – i.e., selecting a reduced/smaller set of features from a large set. This is due to the large gradient in the penalty function – a feature selection characteristic of the lasso penalty when the coefficients are less than one. An outcome is that typically when two strongly correlated features are pushed towards zero, one may be eliminated. It may therefore ignore or remove some features that may, nevertheless, be interesting or important. In contrast, the ridge regression penalty reduces correlated features together without quickly removing one of them. Thus, Lasso may be more useful with a large set of features where some redundant ones could be eliminated or pruned and which would lead to better prediction. Ridge may be more useful when there is a large number of highly correlated features where shrinking them together without eliminating any would lead to better prediction.

2.5 Prediction Using Hold-Out Test Set

After validation, the best prediction model can be chosen to perform prediction using the test data set or the hold-out data set. This is a final check on the model to see that if it performs just as well in the test set, then it can be rolled out to perform prediction with generalized data.

We perform the optional step of combining the training data set and the validation data set for re-training the model parameters, but using the fine-tuned hyperparameters from the validation phase. The re-trained model is then applied to the prediction of targets from the test data set using the test data features.

Code line [29] shows how the step is done.

```

Ridge=linear_model.Ridge(alpha=alpha_Ridge,)
Ridge.fit(X_trainval, y_trainval)
y_pred_Ridge_test = RI + np.dot(X_test,RC.T)
r2_score_Ridge_test = r2_score(y_test, y_pred_Ridge_test)
rmse_y_pred_Ridge_test = (np.sqrt(mean_squared_error(y_test, y_pred_Ridge_test)))
print("r2_score_Ridge_test:",r2_score_Ridge_test)
print("Pred_RMSE_Ridge_test:", rmse_y_pred_Ridge_test)

Lasso=linear_model.Lasso(alpha=alpha_Lasso,)
Lasso.fit(X_trainval, y_trainval)
y_pred_Lasso_test = LI + np.dot(X_test,LC.T)
r2_score_Lasso_test = r2_score(y_test, y_pred_Lasso_test)
rmse_y_pred_Lasso_test = (np.sqrt(mean_squared_error(y_test, y_pred_Lasso_test)))
print("r2_score_Lasso_test:",r2_score_Lasso_test)
print("Pred_RMSE_Lasso_test:", rmse_y_pred_Lasso_test)

ELNet=linear_model.ElasticNet(alpha=alpha_EN, l1_ratio=l1_ratio_EN,)
ELNet.fit(X_trainval, y_trainval)
y_pred_ELNet_test = EI + np.dot(X_test,EC.T)
r2_score_ELNet_test = r2_score(y_test, y_pred_ELNet_test)
rmse_y_pred_ELNet_test = (np.sqrt(mean_squared_error(y_test, y_pred_ELNet_test)))
print("r2_score_ELNet_test:",r2_score_ELNet_test)
print("Pred_RMSE_ELNet_test:", rmse_y_pred_ELNet_test)

r2_score_Ridge_test: 0.6741670178819545
Pred_RMSE_Ridge_test: 779.7022050419647
r2_score_Lasso_test: 0.6741543806451812
Pred_RMSE_Lasso_test: 779.7173250365355
r2_score_ELNet_test: 0.6741681342698304
Pred_RMSE_ELNet_test: 779.7008693103447

```

In [29], we find the test data prediction scores for all three models since they perform with marginal differences in the validation phase. As seen in the output, the Elastic Net regression model is now marginally better than that of the Ridge regression. The performance at R^2 of over 67% is close to the validation R^2 performance of just over 69%.

2.6 Cross-Validation

Cross-validation is a resampling procedure used to evaluate machine learning models on a limited data sample. k -fold cross validation refers to the number of groups that a given data sample less the hold-out test sample is split into. This method allows validation to be conducted within the training data set itself and thus avoids use of another separate set of data for validation.

k approximately equal subsets of the data sample after leaving out the hold-out set are drawn randomly. Pick a subset as the validation sample. Pick the rest of subsets as training sample. Fit model on training sample

and evaluate prediction R^2 on the validation sample. Pick the next subset as the validation sample, and the rest as training sample, and so on. There will be k number of such fittings and k number of validation R^2 scores using this k -fold cross-validation approach. The mean and variance of the k number of R^2 scores can be computed.

A different set of hyperparameter value(s) is used for each cross-validation mean R^2 score. The optimal hyperparameter value(s) is/are that which yield(s) the maximum mean R^2 score in the cross-validation. This optimal hyperparameter value(s) is/are then used to check the prediction performance of the test data set. The cross-validation procedure essentially replaces $0.6N$, $0.2N$, $0.2N$ slicing of training, validation, and test data sets with $0.8N$, $0.2N$ simpler slicing into training and test data sets. If $k = 5$, then the training data set is divided into five times of $0.64N$ training and $0.16N$ validation.

Repeated k -fold cross-validation occurs when the resampling is done again by randomly splitting the sample into the hold-out and remainder and randomly selecting the k subsets again. Stratified cross-validation occurs when it is ensured that each subset of the data sample contains the same proportion of observations with a given categorical value, e.g., stratification by gender means each subset should have the same proportion of gender mix. Generally, increasing k and/or increasing the number of repetitions will reduce the bias (mean of the scores departure from true mean) but increase the variance or noise of the scores.

There is some similarity of the concept of cross-validation in ML with resampling schemes in classical statistics. The Jackknife is a method when sequentially one observation is left out and the required statistic is computed based on the remaining data. Sample of N requires N number of such computations. Mean and variance of the various computed statistics provide idea of closeness to true statistic and its variance. This idea is similar to cross-validation. Bootstrapping is a statistical procedure that resamples a single dataset to create many simulated samples. This process allows you to calculate standard errors, construct confidence intervals, and perform hypothesis testing for numerous types of the sample statistics. This is similar in concept to repeated cross-validation. While Jackknife and Bootstrapping are resampling procedures to compute sample statistics, cross-validation and repeated cross-validation are procedures to compute mean validation scores.

The discussion on training, validation, and testing can be depicted in Figure 2.1.

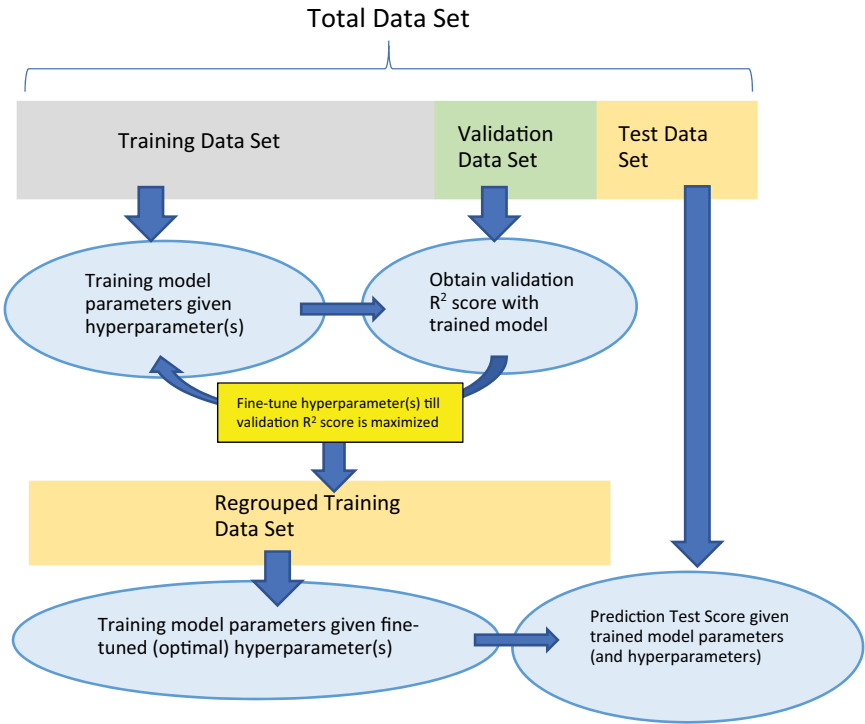


Figure 2.1. Relationship between training, validation, and testing.

The k -fold cross-validation is a common approach or strategy to improve validation effectiveness and thus comes up with an optimal model (with optimized model parameters as well as hyperparameters) for checking prediction using the test data set that has been held-up. This is done particularly with smaller sample size (smaller number of sample points) in the total data set. It is, however, usually not applicable when the data are in time series form as time ordering cannot be randomly re-arranged. In what follows, we also show diagrammatically how hyperparameters can be fine-tuned or optimally selected using this k -fold cross-validation approach.

In Figure 2.2, it is shown that the original training data set and validation data set are combined into a larger training data set. This set is then divided into k -folds. k is typically chosen as 5 or 10 although in theory it can also be a hyperparameter. Suppose we choose $k = 5$. Thus, there are 5 validations. Note that each data point appears only once in the validation

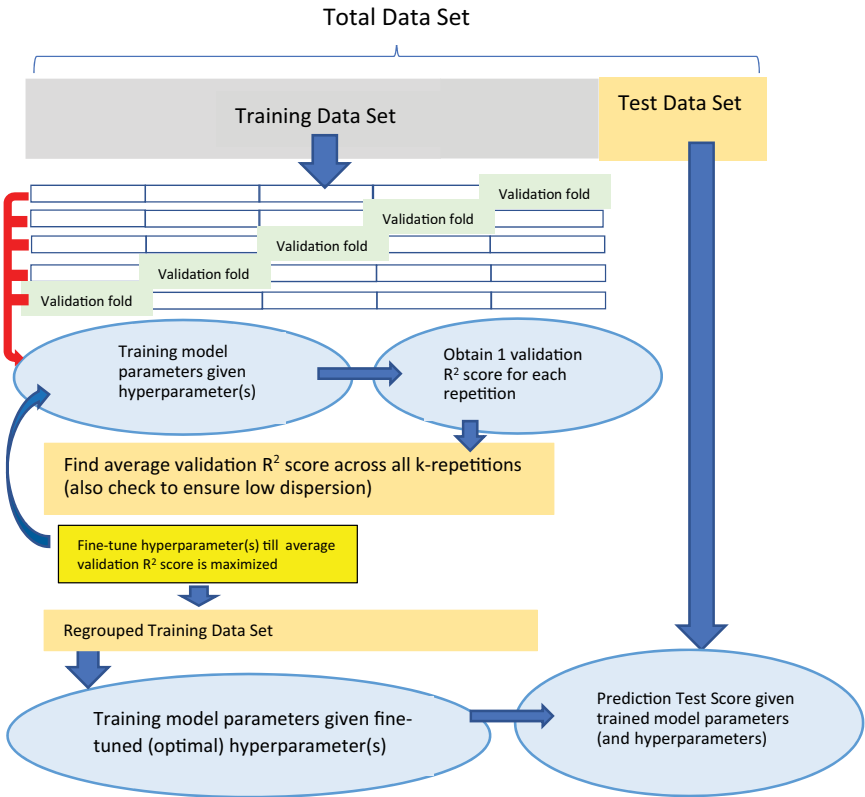


Figure 2.2. k -fold cross-validation.

data set of all the k repetitions. There are k number of validation R^2 scores. These are averaged to get the mean cross-validation scores and a standard deviation or dispersion of the scores is also obtained. The hyperparameters are fine-tuned till the average cross-validation score is maximized. This selected optimal hyperparameter value(s) is then used to train the model parameters in the regrouped training data set. This is then applied to the test data set to check for hold-out data or out-of-sample prediction accuracy as the final step to machine learning in this case.

Code line [30] shows that the data set could be reshuffled anew with a fresh `X_train` and `y_train` training set of 49,887 sample points. Code line [33] provides the cross-validation results for the base case of linear regression without constraint.


```
### Now k-fold cross validation is to be performed on X_train y_train
### reshuffled dataset, leaving test set intact
### Details of other scoring methods and metrics can be found in
### https://scikit-learn.org/stable/modules/model_evaluation.html

from sklearn.model_selection import cross_val_score

scoresLinreg = cross_val_score(estimator = Linreg, X = X_trainval, \
                                y = y_trainval, cv = 5)
### This combined training set X_train, y_train is split into k=5
### (cv=5) folds for each of k=1,2,...,5 repetitions
print(scoresLinreg)
### Score is R2 measure, there are 5 scores since k=cv=5,
### one for each repetition

print("%0.8f mean R2 with a standard deviation of %0.8f" \
      % (scoresLinreg.mean(), scoresLinreg.std()))

[0.66375598 0.67706077 0.67908507 0.67785626 0.69054729]
0.67766107 mean R2 with a standard deviation of 0.00850685
```

The computations according to Figure 2.2 (in this case without tuning any hyperparameter) is performed using `cross_val_score` module in `sklearn`. $k = 5$ is used here (see “ $cv = 5$ ”). The mean validation R^2 score is 0.67766107 with a standard deviation of 0.008507 in the scores. This mean score is slightly lower than 0.69452567 in the case without cross validation, though this result could be more robust with random draws in the training versus test samples.

Code lines [35–39] show the cross-validation scoring results based on $k = 5$ for Ridge, Lasso, and Elastic Net regressions. The optimal hyperparameter is found using `GridSearchCV` in `sklearn.model_selection`.

Summary of the results are shown as follows.

	Mean validation R^2 scores	Std. dev. of validation R^2 scores
Ridge ($\alpha = 0.53$)	0.67766112	0.008509
Lasso ($\alpha = 0.0$)	0.67766107	0.008507
Elastic Net ($\alpha = 0.001$, $l1_ratio = 0.99$)	0.67766111	0.008508

As seen in the cross-validation results using R^2 as the metric or measurement of the accuracy of prediction performance, Ridge regression performs marginally better than regression without constraint and also better than Lasso and Elastic Net, in terms of mean R^2 .

Cross-validation for Ridge Regression

```

### Now k-fold cross validation is to be performed on X_train y_train dataset
### using Ridge regression

from sklearn import linear_model
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

Ridge_range = [i/100.0 for i in range(0,100)]
parameters = {'alpha': Ridge_range}

model_Ridge = Ridge()
Ridge_grid_search = GridSearchCV(estimator=model_Ridge,
                                param_grid=parameters,
                                cv=5)

Ridge_grid_search.fit(X_trainval, y_trainval)

# Print the best hyperparameters and the corresponding mean cross-validated score
print("Best Hyperparameters: ", Ridge_grid_search.best_params_)
print("Best Score: ", Ridge_grid_search.best_score_)
print("Std Err of Scores at Best Hyperparameter: ", \
      Ridge_grid_search.cv_results_['std_test_score'][Ridge_grid_search.best_index_])

Best Hyperparameters: {'alpha': 0.53}
Best Score: 0.6776611238244105
Std Err of Scores at Best Hyperparameter: 0.008508623139690662

```

Cross-validation for Lasso Regression

```

### Now k-fold cross validation is to be performed on X_train y_train dataset
### using Lasso regression

from sklearn import linear_model
from sklearn.linear_model import Lasso

Lasso_range = [i/100.0 for i in range(0,30)]
parameters = {'alpha': Lasso_range}

model_Lasso = Lasso()
Lasso_grid_search = GridSearchCV(estimator=model_Lasso,
                                param_grid=parameters,
                                cv=5)

Lasso_grid_search.fit(X_trainval, y_trainval)

# Print the best hyperparameters and the corresponding mean cross-validated score
print("Best Hyperparameters: ", Lasso_grid_search.best_params_)
print("Best Score: ", Lasso_grid_search.best_score_)
print("Std Err of Scores at Best Hyperparameter: ", \
      Lasso_grid_search.cv_results_['std_test_score'][Lasso_grid_search.best_index_])

Best Hyperparameters: {'alpha': 0.0}
Best Score: 0.6776610737723814
Std Err of Scores at Best Hyperparameter: 0.00850685083401788

```

Cross-validation for Elastic Net Regression

```

### Now k-fold cross validation is to be performed on X_train y_train dataset
### using Elastic Net regression

from sklearn import linear_model
from sklearn.linear_model import ElasticNet

alpha_range = [i/1000.0 for i in range(0,100)]
l1_ratio_range = [i/100.0 for i in range(0,100)]
param_grid = {'alpha': alpha_range, 'l1_ratio': l1_ratio_range}

model_ElNet = ElasticNet()
ELNet_grid_search = GridSearchCV(estimator=model_ElNet,
                                param_grid=param_grid,\
                                cv=5)

ELNet_grid_search.fit(X_trainval, y_trainval)

# Print the best hyperparameters and the corresponding mean cross-validated score
print("Best Hyperparameters: ", ELNet_grid_search.best_params_)
print("Best Score: ", ELNet_grid_search.best_score_)
print("Std Err of Scores at Best Hyperparameter: ", \
      ELNet_grid_search.cv_results_['std_test_score'][ELNet_grid_search.best_index_])

Best Hyperparameters: {'alpha': 0.001, 'l1_ratio': 0.99}
Best Score: 0.6776611126209133
Std Err of Scores at Best Hyperparameter: 0.008508289316292484

```

Using the process as in Figure 2.2, the eventual use of the best method, whether linear regression or the regularized regression, together with the optimally selected hyperparameter(s), may then be applied to the regrouped training data and the computed model is then applied in the final step to the test data for prediction accuracy assessment. This is shown in code lines [41–43].

```

### Computing for Ridge
Ridge2=linear_model.Ridge(alpha=0.53)
Ridge2.fit(X_trainval, y_trainval)
y_pred_Ridge2_test = Ridge2.intercept_ + np.dot(X_test,Ridge2.coef_.T)
r2_score_Ridge2_test = r2_score(y_test, y_pred_Ridge2_test)
rmse_y_pred_Ridge2_test = (np.sqrt(mean_squared_error(y_test, y_pred_Ridge2_test)))
print("r2_score_Ridge2_test:",r2_score_Ridge2_test)
print("Pred_RMSE_Ridge2_test:", rmse_y_pred_Ridge2_test)

r2_score_Ridge2_test: 0.6741727289444343
Pred_RMSE_Ridge2_test: 779.6953718670477

### Computing for Lasso
Lasso2=linear_model.Lasso(alpha=0.0)
Lasso2.fit(X_trainval, y_trainval)
y_pred_Lasso2_test = Lasso2.intercept_ + np.dot(X_test,Lasso2.coef_.T)
r2_score_Lasso2_test = r2_score(y_test, y_pred_Lasso2_test)
rmse_y_pred_Lasso2_test = (np.sqrt(mean_squared_error(y_test, y_pred_Lasso2_test)))
print("r2_score_Lasso2_test:",r2_score_Lasso2_test)
print("Pred_RMSE_Lasso2_test:", rmse_y_pred_Lasso2_test)

r2_score_Lasso2_test: 0.6741702504690269
Pred_RMSE Lasso2 test: 779.6983373219088

```

```

### Computing for Elastic Net
ELNet2=linear_model.ElasticNet(alpha=0.001,l1_ratio=0.99,)
ELNet2.fit(X_trainval, y_trainval)
y_pred_ELNet2_test = ELNet2.intercept_ + np.dot(X_test,ELNet2.coef_.T)
r2_score_ELNet2_test = r2_score(y_test, y_pred_ELNet2_test)
rmse_y_pred_ELNet2_test = (np.sqrt(mean_squared_error(y_test, y_pred_ELNet2_test)))
print("r2_score_ELNet2_test:",r2_score_ELNet2_test)
print("Pred_RMSE_ELNet2_test:", rmse_y_pred_ELNet2_test)

r2_score_ELNet2_test: 0.6741727740241151
Pred_RMSE_ELNet2_test: 779.6953179298503

```

2.7 Concluding Thoughts

There are clearly advantages in the use of regularized regressions as a comparison to the linear regression model without constraint when the model may have too many features and can be overfitted using the training data. There are many studies involving proprietary data sets showing advantages of or better performances of regularizations – see Jorge Chan-Lau (2017), and Xin and Khalid (2018).

When the prediction problem does not have too many features, then using regularizations may not significantly improve prediction performance. Often, data features that may have large differences in their values require scaling of the features to improve the performance of the method. A `min_max_scaler = preprocessing.MinMaxScaler()` can be used where each feature is scaled to $(X - X.min)/(X.max - X.min)$, i.e., within (0,1). Or else, a normalizing method such as `sklearn.preprocessing.StandardScaler(*, copy = True, with_mean = True, with_std = True)` can be used. The features are transformed by subtracting the sample mean from each and dividing by the corresponding standard deviation. Scaling is not done in the example in this chapter as the features are mostly binary cases of “1”s or “0”s and the others do not have extreme values. Scaling is also typically not critical for linear regression and for logistic regression with linear loadings in the features as the weights can adjust themselves instead of having to scale the features. However, scaling in most other nonlinear ML algorithms can be helpful. For cross-sectional data with no time sequencing, the features can be scaled before partitioning into training and test data sets. This makes results invariant to the random partitioning. For time-sequenced data where training data should typically precede test data, both the training data and the test data should be scaled separately in order to allow consideration of drifts in data over time.

In this chapter, we show how a validation data set is used to fine-tune hyperparameters to obtain maximum performance in the prediction of validation targets based on training the parameters in the training data set.

We also show how when the data sample size is small, a cross-validation method can be used to accompany regularized regressions. The advantage of cross-validation is that its results are likely to be more robust, e.g., in selecting optimal hyperparameter(s) by considering average of the validation R^2 scores. However, cross-validation may take up more computing time with the number of repetitions, especially with larger data sets.

The data example in this chapter is to show how regularization can be done. By today's data standards, for predicting real estate values professionally, a lot more features can and should be added such as: recently transacted values of similar neighboring plots or blocks or units, distances/connections to transport nodes such as highways, train/bus stations, distances/connections to markets/shops/hospitals/entertainment centers, distances to childcare facilities, distances to seaside or vista points, neighborhood crime rates/theft rates, parking facilities and estate road and amenities conditions, electricity/gas supply conditions, weather conditions including flooding, hurricane situations, and so on. These additional features are likely to produce better regularized regression predictions.

References

- Jorge A. Chan-Lau (2017). *Lasso Regressions and Forecasting Models in Applied Stress Testing*. IMF Working Paper, Institute for Capacity and Development.
- Seng Jia Xin and Kamil Khalid (2018). Modelling house price using ridge regression and Lasso regression. *International Journal of Engineering and Technology*, 7, 498–501.

Chapter 3

Corporate Reporting Data and Analyses

If we have observations of the speed performances of sports cars based on pre-determined road conditions, we can try to predict the speed performances of the next batch of sports cars based on similar road conditions. However, our predictions will reasonably be improved if we also know how the cars were made, their engine structures, gear transmission dynamics, wheel types, parts, and so on. Similarly, we could improve prediction of a company's performance – its earnings, stock returns (for a listed stock), and so on – not by just knowing the past historical data of these performances but also by knowing the structure and business operations of the company. The latter are captured largely in accounting statements that are mandatory reporting statements to be made by the company, at least for listed companies that tap on funding from the public.

A company can produce its accounting statements as frequently as monthly for its internal use, but official submissions to the regulators as public documents for public access, and hence availing such data for data analyses, are typically on standard time frames in the U.S. The U.S. Securities and Exchange Commission (SEC) requires publicly listed firms to submit annual reports called 10-K forms/filings that include audited financial statements. The firms are also required to disclose corporate performance information on an ongoing quarterly basis in 10-Q forms.

In this chapter, we show how accounting variables can be used to predict future (the following year's) earnings or profit margins.

3.1 Financial Statements

There are three key financial reporting statements in the corporate reports: the Balance Sheet, the Income Statement, and the Cashflow Statement. The Balance Sheet is a snapshot of a company’s assets and liabilities at a specific time point. The Income Statement is a record of revenues, expenses, and net profits incurred by a company over the specified reporting period, e.g., a year or a quarter. The Cashflow Statement is a report of the aggregated sources and uses of cashflows of the company, including operating cashflows, investing cashflows, and financing cashflows.

Z Inc.
Balance Sheet
(As of) December 31, YY, \$ millions

<u>Assets</u>		<u>Liabilities and Equity</u>	
Current Assets		Current Liabilities	
Cash	10	Accounts Payable	10
Accounts Receivable	30	Short-term Debt ⁺	10
Inventories	20	Taxes Payable	<u>5</u>
Prepaid Expense*	<u>10</u>		
Total Current Assets	<u>70</u>	Total Current Liabilities	<u>25</u>
Investments**	<u>10</u>	Long-Term Debt	<u>45</u>
		Total Liabilities	<u>70</u>
Property, Plant, & Equipment		Stockholder’s Equity	
Land & Buildings	50	Paid-in Capital	70
Equipment	20	Retained Earnings	<u>20</u>
Less accumulated dep’n ⁺⁺	<u>(10)</u>	Total Owner’s Equity	<u>90</u>
Net PP&E	<u>60</u>		
Intangible Assets			
Goodwill	10		
Tradenames	<u>10</u>		
Total Intangible Assets	<u>20</u>		
Total Assets	<u>160</u>	Total Liabilities and Equity	<u>160</u>

*Notes:**Insurance coverage, leased office equipments, etc., where expenses have been paid but advantages will accrue over time in incomes. These are recognized as assets.

** Long-term over 1 year, e.g., long bonds.

⁺Example is bank overdrafts payable in less than a year, whereas long-term debts are due in more than a year.

⁺⁺ dep’n stands for “depreciation”: this amount is not expensed yet, i.e., not yet a cash outflow, but is typically kept to pay for replacement of the capital in due time.

The basic ingredients of a Balance Sheet are as follows in a fictitious firm Z. The units are in \$ millions. Year is YY.

The net working capital (NWC), i.e., the company's total current assets less total current liabilities, can be computed from the Balance Sheet. Positive NWC enables the company to fund its current operations on an ongoing daily basis as the current assets can be easily liquidated for cash or enable more short-term loans.

The Balance Sheet also indicates the Invested Capital (total long-term debt and equity) = NWC + Investments + PP&E + Intangibles. The company's net profits should be measured versus the invested capital to see the return rate of capital among other measures. The amount of long-term debt versus equity also provides a measure of the leverage, whether the company is excessively using debt to fund its business operations and whether this may become detrimental when interest costs of debts rise.

The basic ingredients of an Income Statement are as follows in the fictitious firm Z. The units are in \$ millions. Report is for Year ended December 31, YY.

Z Inc.
Income Statement
For the year ended December 31, YY, \$ millions

Sales	110
Cost of Goods Sold (COGS)	<u>80</u>
Gross Profit (Total Income)	<u>30</u>
Selling, General & Admin Expenses (SG&A)	12
Depreciation & Amortization* (D&A)	<u>5</u>
Total Expenses	<u>17</u>
Operating Profit/Income (EBIT**)	<u>13</u>
Net Non-operating Revenue*	3
Less Interest Expense	<u>(4)</u>
Net Income before tax	12
Less Income Tax	<u>(2)</u>
Net Income (EAIT++)	<u>10</u>

Notes: *Depreciation is expensing a fixed asset to reflect its deterioration and to save the cash and accumulate over time to pay for replacement of the capital in due time. Amortization is the spreading of cost of an intangible asset over time.

**Earnings before interest and tax.

*Non-operating revenue includes interest received from investments, and gains (losses) in investments. It may also include extraordinary item such as a one-time sale of some plants or buildings.

++Earnings after interest and tax. Note here income tax rate is 2/12 or about 16 2/3%.

The Income (or Profit & Loss) Statement reflects if the company is profitable in that period. The time series of income statements also show the trend in sales and various costs, and thus allow a gauge of the sustainability of the business operations. Ratios of Income Statement items and also ratios of Income Statement items versus Balance Sheet items, such as net income over total equity (return on equity, ROE), net income over sales, operating margin (operating profit/gross profit), show how profitable the business is.

These can be compared across different companies to see how well they perform relative to one another. Other financial ratios such as sales in Income Statement over total assets in the Balance Sheet (asset turnover ratio) indicate the company's efficiency or effectiveness in producing sales from assets. One key ratio in the valuation of a company's share for mergers and acquisition, takeover, or leveraged buyout of a company's shares is its price earnings ratio (P/E) where P is the share price and E is the earnings per share (EPS). EPS is net earnings (EAIT, excluding extraordinary items) divided by total shares outstanding. For companies not facing credit risk, a low P/E often signal a good buy.

Z Inc.

Cashflow Statement

For the year ended December 31, YY, \$ millions

Add Net Income	10
Add Non-cash charges (D&A)	5
Less Increase in Current Assets	(20)
Add Increase in Current Liabilities	<u>15</u>
Operating Cashflow	<u>10</u>
Less Capital Expenditures	(20)
Add Sale of PP&E	<u>0</u>
Investing Cashflow (net CAPEX)	<u>(20)</u>
Add Additional (Seasoned) Equity Issue	15
Add New Long-term Debt Issue	15
Less Dividends to Equity Holders	<u>(5)</u>
Financing Cashflow	<u>25</u>
Total Cashflow	<u>15</u>

The financial analyses of a company is not complete without the Cashflow Statement. A cashflow statement informs how much cash is entering and exiting the company business during a given period. It indicates if there is enough cash or liquidity to fund the company's operating expenses and pay its debts. If not, the Operating Cashflow would be negative. It complements the other two statements. Cashflow reporting is harder to manipulate than that of net income – the revenues and costs in the Income Statement are subject to accrual basis in accounting that allows discretion by the company to report revenues (accrued revenues) before the money is received or delay reporting of cost and thus may provide a better optics of positive net income when the cashflow could be negative.

The total cashflow of the company is the sum of Operating Cashflow, Investing Cashflow, and Financing Cashflow. Together they reflect the cash situation of the company and if there are enough cashflows to continue smooth operations of the business. Low or negative cashflows may adversely affect the company's stock prices. However, there are peculiar situations where a company may appear to have negative cash flows due to its decision to expand and incur large capital expenses with the view of huge payoffs in the near future – this may on the contrary augur great stock returns in future. In the example of Z Inc., there is a capital expenditure of \$20 million. This amount would appear in the Balance Sheet as an addition to PP&E and an increase in Long-Term Debt. It does not appear in the Income Statement. The changes in current assets and in current liabilities are not reflected in the Income Statement but are reflected in the Balance Sheet as adjustments in Current Asset and in Current Liability from the previous reporting year.

Often, free cashflow (FCF) is also calculated. This is Operating Cashflow less Investing Cashflow (net CapEx) with after tax interest payment added. FCF is the cashflow available for the company to repay creditors or pay dividends and interest to investors. FCF per share can augment use of the EPS as it can better reflect availability of cash, though the number may be lumpy and uneven as CapEx is lumpy over time. In the above example, $FCF = 10 - 20 + 4(1 - 2/12) = -6(2/3)$, which is generally not a positive signal. The positive total cashflow in the example is due to new issues of additional equity and long-term debt of \$15 million each.

The three key accounting statements are inter-related. Income statement's net income and non-cash charges are key components of the

operating cash flows in the Cashflow statement. Net income in the Income Statement has direct impact on stock prices and hence equity value. This equity value is reflected in mark-to-market valuation of stocks in the Balance Sheet. Changes in asset and liabilities in the Balance Sheet are also reflected in the Cashflows. Total Cashflow (including paid interest) is approximately reflected as change in the Cash item in the Balance Sheet. In the rest of this chapter, we show how to employ accounting variables to predict earnings change using logistic regression. Performance measures of any prediction are discussed next.

3.2 Binary Classification Performance Metrics

In statistical hypothesis testing, when a null hypothesis that is true is rejected (not accepted), it is called a Type 1 error. When a null hypothesis that is false is accepted (not rejected), it is called a Type 2 error.

In a binary classification problem, there are two classes or types of the targets or labels in the sample. Suppose we define Type A to be the positive class and Type B to be the negative class. The definition of whether a type is a positive or a negative case is subjective. For example, a company that defaults (within a specific future horizon) may be Type A and one that does not is Type B. A true positive (TP) is an outcome whereby the model correctly predicts that the class/type belongs to the positive case, i.e., correctly predicting that a firm will default. A true negative (TN) is an outcome whereby the model correctly predicts that the class/type belongs to the negative case, i.e., correctly predicting that a firm will not default.

A false negative (FN) is an outcome where the model incorrectly predicts that the class/type belongs to the negative class, i.e., incorrectly predicting a firm will not default when it does default as in a positive case. This may be called a prediction Type 1 error if we associate the positive class with the classical idea of null hypothesis. A false positive (FP) is an outcome where the model incorrectly predicts that the class/type belongs to the positive class, i.e., incorrectly predicting a firm will default, when it does not. This may be called a prediction Type 2 error. However, it may be redundant to invoke use of Type 1, 2 errors in the prediction context.

The confusion matrix refers to the matrix containing the numbers of cases (values) in each of the four situations after prediction (see Figure 3.1).

		Predicted values	
		Positive	Negative
Actual values	Positive	TP 10	FN 10
	Negative	FP 80	TN 900

Figure 3.1. Confusion matrix.

Suppose there are a total of 1000 cases/firms in the cross-sectional sample during a specific time period. Number of TP cases is 10. This is simply written as $TP = 10$. $FP = 80$. $FN = 10$, and $TN = 900$. Note that the number of positive cases and number of negative cases are imbalanced, i.e., 20 versus 980.

For imbalanced cases, the accuracy measure, depicting the percent of correct predictions of positive or negative cases, may not be very informative. In this confusion matrix,

$$\text{Accuracy} = (TP + TN) / (TP + FN + FP + TN) = 910 / 1000 = 91\%$$

When knowing 980 out of 1000 cases are negative cases, one could use a naïve approach of predicting that all cases as negative. This leads to $TP = 0$ and $TN = 980$, so accuracy is 98%. Even if someone attempts to predict the 20 positive cases (but getting them wrong), then $FN = 20$ (all actual positive cases are predicted as negative cases) and $TP = 0$. With 20 predictions on positive cases, then $FP = 20$, and $TN = 960$. Accuracy would still be a high 96%. Such high accuracies are due to the imbalance.

Naïve prediction of negative cases or trusting high accuracy in imbalanced data prediction could be bad if the context is to be able to predict the default (positive) cases and where inability to predict such cases that would default is very costly. Imbalanced data as such can be helped by randomly deleting some negative cases or resampling on positive cases.

Besides the misleading high accuracy measure in such a situation, other predictive performance measures as follows could possibly provide more information on the predictive ability of the algorithm.

Another prediction performance measure is precision, which is

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

that shows what percent of the predicted positive cases turns out to be correctly positive. Here, it is $10/(10 + 80) = 1/9$ or about 11.11%. This provides an indication of the performance of the model if predicting positive cases is important. A naïve approach could not be of use when precision is measured.

Another prediction performance measure is recall, which is

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

that shows the percentage of the actual number of positive cases that were predicted correctly. Here, it is $10/(10 + 10) = 50\%$. Recall is sometimes also called the true positive rate (TPR) or sensitivity.

Whether precision or recall is a more informative measure depends on whether FP or FN is more of a problem, respectively. Suppose a bank is making a prediction if the firms it lends money to will default or not in the short run, i.e., predicting if the firms are positive cases (default cases). If a prediction is correct, i.e., TP, the bank can either recall the loan before the firm defaults or else hedge the default by buying credit protection from a secondary market, such as credit default swaps. If the prediction is incorrect, i.e., FN, which is predicting no default when the firm defaults, then it is costly to the bank due to the default losses. In this situation, a high FN is bad. Thus, a large recall (ratio) $\text{TP}/(\text{TP} + \text{FN})$ with small FN is an important prediction performance metric for this bank. Recall (or TPR) marks the percentage of the actual number of default firms that were predicted correctly. In this situation, an FP (predicting a firm to default when it does not) may only cost the bank insurance payment fees in insuring the credit risk, which is a smaller sum. Thus, high FP or lower precision in $\text{TP}/(\text{TP} + \text{FP})$ is not as harmful. In other words, in this situation, higher recall is more important and more informatively significant to the bank than higher precision.

Suppose a company selling a line of sports products besides advertisements on social media decides to use signboard advertisements in selected towns to reach out to its potential customers. Each town is either one with

potential customers receptive to signboard advertisements (positive case) or not (negative case). The company makes predictions on which town is positive and which is negative.

When a positive case is predicted, the company puts in signboard advertising expenses for that town. Otherwise, it does not. If it is correctly predicted, i.e., TP, then the company will make profits. However, if the prediction is incorrect, i.e., FP, then the company would have sunk in a lot of advertising costs but got little in sales and profit. This situation of low precision or low $TP/(TP + FP)$ due to high FP produces losses and is bad for the company. On the other hand, if the company predicts a negative case, it does not put up any signboard advertisements. However, if it is an FN, there will be missed opportunities, but the cost of this would be less than if it were an FP. Thus, low recall $TP/(TP + FN)$ with a large FN is not as harmful or bad for the company. In other words, in this situation, higher precision is more important and more informatively significant to the company than higher recall. For the company, a higher percentage of the correctly predicted positive cases is more critical. The prediction of the signboard response could be done based on a historical database of potential customers and their features, and survey of how they responded to such advertisements.

Consumer business such as Spotify or Netflix and others would do well to predict consumer preferences correctly. For example, requiring high precision is important when high FP is a problem such as recommending music (film) of type A (positive class) wrongly to new platform subscribers who dislike music (film) of type A – this may cause business loss due to customer dissatisfaction and churning.¹ In other situations, requiring high recall is important when high FN is a problem such as not being able to correctly screen patients for allergies (positive class) to some treatment drugs. Such FN may result in fatal treatments using those medications/drugs.

Sometimes, a mixture measure is used, such as maximizing the F1-score which is the harmonic mean of precision and recall, i.e.,

$$F1\text{-score} = 2/(\text{recall}^{-1} + \text{precision}^{-1}).$$

¹Based on the history of past subscribers, supervised learning on them may lead to predictive analytics that recommends music (film) types to new subscribers with the associated profiles (features). For existing subscribers with log-in history, unsupervised learning may make further recommendations such as highly correlated music (film) types to each subscriber based on the listening history.

F1-score increases with increases in recall and precision. This is useful when both recall and precision need to be high.

The above prediction performance metrics of precision, recall, and F1-score are measured with respect to the positive (P class) or type A. If we define type B as the P case, then the precision, recall, and F1-score may be different numbers with respect to type B. Suppose we define TP with respect to type A (P class) as TP_A and TP with respect to type B (P class) as TP_B . Similarly, we define FP with respect to type A (P class) as FP_A and FP with respect to type B (P class) as FP_B .

If precision, recall, and F1-score with respect to type A as P are $prec_A$, rec_A , and $f1_A$, and precision, recall, and F1-score with respect to type B as P are $prec_B$, rec_B , and $f1_B$, and suppose there are n_A number of cases that are of type A and n_B number of cases that are of type B, then

$$\text{Macro average of precision} = (prec_A + prec_B) / 2$$

$$\text{Macro average of recall} = (rec_A + rec_B) / 2$$

$$\text{Macro average of F1-score} = (f1_A + f1_B) / 2$$

that are based on simple arithmetic averages. The weighted averages of these measures are, respectively, $n_A / (n_A + n_B) \times prec_A + n_B / (n_A + n_B) \times prec_B$, $n_A / (n_A + n_B) \times rec_A + n_B / (n_A + n_B) \times rec_B$, and $n_A / (n_A + n_B) \times f1_A + n_B / (n_A + n_B) \times f1_B$.

In the example of Figure 3.1, precision with respect to A is $10 / 90$ but precision with respect to B is $900 / 910$. Hence, the macro average of precision is 55%. The recall with respect to A is $10 / 20$, but recall with respect to B is $900 / 980$. The macro average of recall is 70.9%. Hence, averaging generally reduces the extreme effect of imbalanced data. The weighted average precision is $(20 / 1000) \times (10 / 90) + (980 / 1000) \times (900 / 910) = 0.97$, a very high precision number. This reflects the big problem when using imbalanced data.

Another type of averaged prediction performance is

$$\text{Micro average precision} = (TP_A + TP_B) / (TP_A + FP_A + TP_B + FP_B)$$

$$\text{Micro average recall} = (TP_A + TP_B) / (TP_A + FN_A + TP_B + FN_B)$$

The above two measures are equal and the same as accuracy. Such “averaged” measures, with respect to alternative definitions of positive and negative cases, are useful when there is no discrimination between the

cases. They are used more to compare with macro average measures. In the case in Figure 3.1, when the data are imbalanced, micro averages tend to produce higher scores, e.g., 0.91 here compared to the macro averages for precision and for recall of 0.55 and 0.71, respectively.

Another measure is specificity or true negative rate (TNR)

$$\text{Specificity} = \text{TN}/(\text{TN} + \text{FP})$$

This is the percentage of the actual number of negative cases that were predicted correctly. The false positive rate (FPR)

$$\text{FPR} = 1 - \text{specificity} = \text{FP}/(\text{TN} + \text{FP})$$

FPR is the percentage of the actual number of negative cases that were predicted as FP, i.e., wrongly predicted. Note that TPR and FPR both lie between 0 and 1, possibly inclusive.

In a binary classification, the model typically will output a predicted probability of positive class (where the positive class is appropriately defined). It is typically the case to use a hyperparameter threshold of 0.5, whereby if the predicted probability is $>$ ($\leq$) 0.5, then the case is assigned as a predicted positive (negative) case. The threshold can however be adjusted to improve performance if the types are highly imbalanced. For example, in the 1000 firms' study, given the predicted probabilities by the algorithm, precision can be raised if the threshold for predicting positive is raised so that FP, which is a relatively large number to TP, is decreased as less cases are then predicted as positive due to the increased threshold. Less of the negative cases are incorrectly predicted as positive – hence smaller FP. Thus, changing threshold is an interesting way to find more information about the predictive ability of the algorithm.

One of the most important more comprehensive performance metrics in a classification model's performance is the AUC (area under the curve) of the ROC (receiver operating characteristics) curve. It may also be called the AUROC (area under the receiver operating characteristics).

The ROC is a curve that plots the TPR against FPR at various threshold values. This is shown in Figure 3.2.

Given the predicted probabilities, as the threshold is increased toward 1, less cases are predicted as P, hence TP decreases toward zero, and FN increases (since $\text{TP} + \text{FN}$ add up to a fixed number of actual

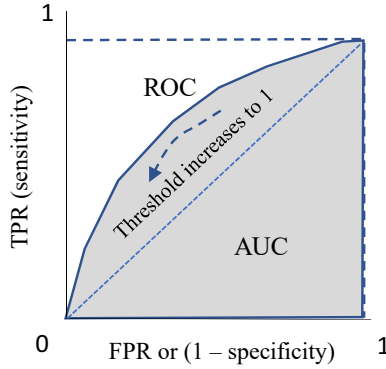


Figure 3.2. Receiver operating characteristics curve.

positive cases). Thus, TPR or sensitivity $TP/(TP + FN)$ goes toward 0. Similarly, as the threshold is increased toward 1, more cases are predicted as N, hence TN increases, and FP decreases (since $TN + FP$ add up to a fixed number of actual negative cases). Thus, FPR or $FP/(TN + FP)$ also goes toward 0. In other words, as threshold increases, sensitivity (recall) drops while specificity increases, i.e., detection of positive cases becomes less frequent due to more conservatism while detection of negative cases becomes more frequent.

Conversely, as the threshold approaches zero, more cases are predicted as P, hence TP increases, and FN decreases (since $TP + FN$ add up to a fixed number of actual positive cases). Thus, TPR or sensitivity $TP/(TP + FN)$ increases toward 1. Similarly, as the threshold is decreased toward 0, less cases are predicted as N, hence TN decreases, and FP increases (since $TN + FP$ add up to a fixed number of actual negative cases). Thus, FPR, $FP/(TN + FP)$, also goes toward 1. In other words, as threshold decreases, sensitivity (recall) increases while specificity decreases, i.e., detection of positive cases becomes more frequent due to less conservatism, while detection of negative cases becomes less frequent.

In the diagram, any point on the diagonal dotted line represents a situation when $TPR = FPR$. Points of ROC curve above this diagonal line represent the situations $TPR > FPR$, where the proportion of correctly predicted positive cases of all positive cases is larger than the proportion of incorrectly predicted positive cases of all negative class.

When $TPR > FPR$, solving $TP / (TP + FN) > FP / (TN + FP)$ gives $TP \times TN > FP \times FN$. Here, there is a high chance that the model can distinguish the positive cases from the negative cases because the model can detect more numbers of TPs and TNs (in product term) than FPs and FNs. On the diagonal line when $TP \times TN = FP \times FN$, the model is not able to distinguish between positive and negative cases, i.e., akin to predicting the classes randomly.

Across all thresholds, higher AUC (where $AUC > 0.5$), or higher averages of $TP \times TN$ than averages of $FP \times FN$, the better is the algorithm's ability to distinguish between positive cases as positive and negative classes as negative. The ideal situation is for the ROC curve to be sharply rising near $FPR = 0$, where TPR is close to 1 on the left region. There are more detailed analyses of the ROC (see Nahm (2022)). $AUC = 1$ means perfect classification.

The AUC is based on the ROC using TPR and FPR measures based on a specific class as positive. It turns out that if we compute TPR and FPR based on the other binary class as positive instead, we would still obtain the same AUC measure. Hence, it does not make any difference which case we treat as positive or which negative for the computation of the area under the ROC.

For competing predicting models, the one with a higher AUC is the better model that can predict. AUC measures predictive performance in a more comprehensive way by considering all thresholds and not just the usual threshold of 0.5. When AUC is below 0.5 or close to 0.5, then the algorithm is not effective in prediction. It is possible for ROC curve to be partly below and partly above the diagonal. If this flip occurs around the threshold of typical 0.5, then the algorithm is also not effective. AUC is a single performance metric. Whether sensitivity or specificity is more important in the prediction problem does not enter into the AUC metric. This may be done separately in cost-benefit extension of the confusion matrix.

In a multi-class classification, the computations will be based on TP, FN, FP, and TN computed differently for each of the category/class. Take, for example, a prediction of one of the three classes, viz., whether a business class airline passenger prefers Eastern (E), Western (W), or Mediterranean (M) food on board in order to make optimal food meals stock-up before flight. The numbers of (actual preference j , predicted preference k) cases are indicated in the following (j, k) box.

		Predicted Preference		
		E	W	M
Actual Preference	E	X_{EE}	X_{EW}	X_{EM}
	W	X_{WE}	X_{WW}	X_{WM}
	M	X_{ME}	X_{MW}	X_{MM}

The TP, FN, FP, and TN for the E class (those who prefer E are the positive cases; those who do not prefer E are the negative cases) are X_{EE} , $(X_{EW} + X_{EM})$, $(X_{WE} + X_{ME})$, and $(X_{WW} + X_{WM} + X_{MW} + X_{MM})$. The TP, FN, FP, and TN for the W class (those who prefer W are the positive cases; those who do not prefer W are the negative cases) are X_{WW} , $(X_{WE} + X_{WM})$, $(X_{EW} + X_{MW})$, and $(X_{EE} + X_{EM} + X_{ME} + X_{MM})$. The TP, FN, FP, and TN for the M class (those who prefer M are the positive cases; those who do not prefer M are the negative cases) are X_{MM} , $(X_{ME} + X_{MW})$, $(X_{EM} + X_{WM})$, and $(X_{EE} + X_{EW} + X_{WE} + X_{WW})$. Thus, we can compute an AUC for each class, and a clearly better performing model would produce higher AUC in as many classes.

Besides the performance confusion matrix, there are also a couple of other common prediction performance metrics. The Kappa value is a metric that compares an observed accuracy with respect to that of an expected accuracy based on random chance. For example, a machine learning method could produce an observed accuracy $A = (TP + TN) / (TP + FN + FP + TN)$, and the sample observations could have P number of positives and N number of negatives in the sample size of $P + N = T$. Based on this *a priori* sample distribution, a random guess with a pick from the bin would suggest probability P / T designation of P and probability N / T designation of N . An optimal decision to maximize probability of correct guess would choose, for example, N for every draw if $N / T > P / T$. Then the random but rational guess would have $TP = 0$, $TN = N$, and an expected accuracy of $A^* = (0 + N) / T$. In this case, the Kappa value is $(\text{observed accuracy } A - A^*) / (1 - A^*)$, where $A > A^*$. This measure indicates the improvement in the accuracy via the machine learning method as a fraction of the inaccurate segment. There is generally no definitive benchmark for what constitutes good performance for a Kappa value. Across different machine learning methods, obviously the higher the Kappa value of a particular method, then that method would be considered to have performed better.

3.3 Logistic Regression

Logistic regression or logit model can be used to estimate the probabilities of events, given attributes or features that can possibly determine such probabilities. If an event is a binary classification of either type A or type B (positive or negative; yes or no; default or no default; contagious or not contagious; criminal or not criminal; exit or no exit, pass or fail, and so on), we can define the i th case as type A with label as $Y_i = 1$. Without loss of generality, if the i th case is not type A, then it is type B with label $Y_i = 0$.

Then the logistic/logit regression model can be applied:

$$E(Y_i | X_i) = \text{Prob}(Y_i = 1 | X_i) = \exp(\beta^T X_i) / [1 + \exp(\beta^T X_i)] = 1 / [1 + \exp(-\beta^T X_i)]$$

where X_i is a vector of feature variables associated with case i . β is vector of coefficients on X_i . See Lim (2015) for an application.

For example, in studying many firms during a fiscal year, firm i could see an earnings increase ($Y_i = 1$) while another firm j could see an earnings decrease ($Y_j = 0$). The latter includes case of unchanged earnings. Now, $\exp(\beta^T X_i) / [1 + \exp(\beta^T X_i)]$ is the logistic (cumulative) distribution function that lies between 0 and 1. This function is also the probability of a Bernoulli distribution with binary outcome 1 and with probability $[\exp(\beta^T X_i) / [1 + \exp(\beta^T X_i)]]$ for outcome 1. Then the likelihood function of an observed training sample of N cases (N_1 cases of outcome 1 and $N - N_1$ cases of outcome 0) is

$$L = \prod_{i=1}^N [F(\beta^T X_i)]^{Y_i} [1 - F(\beta^T X_i)]^{1-Y_i}$$

where $F(\beta^T X_i) = \text{Prob}(Y_i = 1 | X_i)$, and $\sum_i Y_i = N_1$. X_i and Y_i are from the training set. The loss function (same as negative of log likelihood function in classical statistics) is

$$\text{loss} = \sum_{i=1}^N (Y_i \log F + (1 - Y_i) \log(1 - F))$$

which is to be minimized. This is done via a numerical iterative procedure to find the optimal parameters $\hat{\beta}$ that is then used to evaluate the predicted probability of $\hat{Y}_k$ based on X_k from the test set cases k . In sklearn LogisticRegression app, the options can be set to (loss function) of `multi_class = "multinomial"`. This includes the binary and hence the

binomial situation. Solver can be by the “newton-cg” method. Apps metrics, `classification_report`, accuracy score, `confusion_matrix`, etc., from `sklearn` can then be applied to find the various prediction performance results. Also, it should be noted that by default `sklearn LogisticRegression` applies L2 regularization in the estimation. For `sklearn LogisticRegression`, when the estimated probability of

$$F(\hat{\beta}^T X_i) = \text{Prob}(Y_i = 1 \mid X_i)$$

exceeds 0.5, the predicted class is considered as $Y_i = 1$. Otherwise, it is $Y_i = 0$.

However, just as in linear regression, logistic regression faces some problems when there is large multi-collinearity among the features – even failure to converge to the optimal solution. Large multi-collinearity among features (or regressors in classical terminology) can be detected using “`variance_inflation_factor`” app in `statsmodels`. Each feature is regressed, via linear regression, on all the other features, and the variance-inflation-factor ($\text{VIF} = 1 / (1 - R^2)$) can be computed, where R^2 is the coefficient of determination of the linear regression. If $R^2 > 0.9$ (very high collinearity), then $\text{VIF} > 10$.

With multi-collinearity, one problem is that prediction (or forecasting in classical statistics) can become unstable and can produce bad results as the coefficients or weight vector on features for prediction can be unstable. Secondly, since these weight vectors β can change in unstable ways, it is also not accurate to try to find out which features X_i are more important, i.e., have bigger weights in the prediction.

To more conveniently assess the logistic regression estimation results and check on the statistical significance of the various feature impact (eliminating those that are multi-collinear and also which may have negligible impact), logit model regression in `statsmodels` is used to complement the prediction exercise besides using `sklearn LogisticRegression`.

3.4 Worked Example: Data

U.S. listed company annual (end December) accounting ratios from 1971 till 2015 are used in this demonstration.² Due to the proprietary nature of the data, they are not provided herewith. Only the python notebook `Chapter3-1.ipynb` is provided.

²The data set is `co_fin_ratios.csv` from the author’s earlier research file and obtained from subscribed WRDS.

Some companies appeared in the data set after 1971 while some showed data for only a few years. Some companies whose accounting ratios were not complete were not included.

The data consist of 35,074 rows of annual company data and 47 columns of accounting ratios. Some rows are different annual data of the same company. The last column “NPM_FWD_CHG” (net profit margin forward change) is not an accounting ratio. It is 1 if the net profit margin (typically net profit over sales) increased in the following year. It is 0 otherwise. In the reported sklearn metrics, label of 1 is the negative case and label of 0 is the positive case (because sklearn lines up the confusion matrix starting with first row as 0 or the positive case). Note that this forward variable is of course not observed at the time stamp of the features, but it is used for training and for testing the predictive abilities of the model using the features.

The purpose of the exercise is to harness the companies’ accounting ratios each year to see if they can help predict if the following year’s net profit margin would increase or not. See Xi *et al.* (2022) for a similar kind of accounting study.

Code line [4] shows that in the data set there are 17,689 annual cases of NPM increase and 17,385 cases of no NPM increase. Thus, the data set is balanced in the two categories.

```
[4]: ### exploring the data/ .value_counts - shows counts of unique values
data['NPM_FWD_CHG'].value_counts()

[4]: NPM_FWD_CHG
1      17689
0      17385
Name: count, dtype: int64
```

The columns with objects such as “datadate”, “fyear”, “tic”, and “Company Name” are dropped, leaving 47 accounting ratios to form the feature space.

Logit regression using statsmodel is first performed on the entire data set with “NPM_FWD_CHG” as the target variable or label. This is to have an initial idea of how the various features impact or affect the label. Using statsmodel here (and sklearn logistic regression later) more conveniently provides regression estimates and their p -values so that exceedingly insignificant features may be discarded due to their redundancy. Note that in linear regression, multi-collinear features often have estimated coefficients that are not statistically significantly different from zero due to the large standard errors of the coefficient estimators. Hence, some multi-collinear features may also be discarded. See code line [11]. Only a part of the output table is shown in the following to economize on space.

```
[11]: ### now perform logit regression using statsmodel package
      ### using statsmodel here (and sklearn logistic regression later)
      ### more conveniently provides regression estimates and p-values
      ### so that exceedingly insignificant features can be discarded
      ### due to their redundancy or multicollinearity with other features
      import statsmodels.api as sm
      result=sm.Logit(y,X).fit()
      print(result.summary())
```

Optimization terminated successfully.

Current function value: 0.689962

Iterations 9

Logit Regression Results						
=====						
Dep. Variable:	NPM_FWD_CHG	No. Observations:	35074			
Model:	Logit	Df Residuals:	35026			
Method:	MLE	Df Model:	47			
Date:	Sun, 22 Sep 2024	Pseudo R-squ.:	0.004541			
Time:	14:15:15	Log-Likelihood:	-24200.			
converged:	True	LL-Null:	-24310.			
Covariance Type:	nonrobust	LLR p-value:	2.449e-24			
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	-0.0412	0.018	-2.315	0.021	-0.076	-0.006
CASH_DEBT	-0.0153	0.003	-4.431	0.000	-0.022	-0.009
CURRENT	-0.0038	0.006	-0.608	0.543	-0.016	0.008
PCHG_CURRENT	0.0014	0.001	1.026	0.305	-0.001	0.004
QUICK	-0.0083	0.007	-1.226	0.220	-0.022	0.005
PCHG_QUICK	-0.0022	0.002	-1.364	0.172	-0.005	0.001
DAYS_SALEAR	0.0001	3.72e-05	3.811	0.000	6.89e-05	0.000
PCHG_DAYS_SALEAR	-0.0027	0.001	-3.229	0.001	-0.004	-0.001
DEBT_EQ	-7.146e-05	0.000	-0.504	0.614	-0.000	0.000
PCHG_DEBT_EQ	0.0003	0.001	0.355	0.722	-0.001	0.002
DEPR_PPE	0.0026	0.005	0.525	0.600	-0.007	0.012
PCHG_DEPR_PPE	-0.0005	0.001	-0.889	0.374	-0.002	0.001
DIV_CASH	0.0041	0.006	0.640	0.522	-0.008	0.017
EQ_FA	1.23e-05	4.2e-05	0.293	0.770	-7e-05	9.46e-05
PCHG_EQ_FA	4.624e-06	4.26e-05	0.109	0.914	-7.89e-05	8.81e-05
INV_TURN	0.0002	0.000	1.541	0.123	-4.47e-05	0.000
PCHG_INV_TURN	-0.0004	0.000	-0.907	0.364	-0.001	0.000
INVT_AT	0.3920	0.093	4.205	0.000	0.209	0.575
PCHG_INVT_AT	0.0214	0.010	2.178	0.029	0.002	0.041
LTDEBT_EQ	6.145e-06	0.000	0.014	0.989	-0.001	0.001
NI_CF	0.0010	0.001	1.317	0.188	-0.000	0.002
PCHG_OPMBD	0.0012	0.000	2.748	0.006	0.000	0.002
PCHG_PTPM	-0.0003	0.000	-1.097	0.273	-0.001	0.000
REPAY_DEBT	0.0001	0.000	1.020	0.308	-9.96e-05	0.000
ROA	0.0004	0.000	0.874	0.382	-0.000	0.001
SALE_AT	0.0351	0.011	3.097	0.002	0.013	0.057
PCHG_SALE_AT	0.0007	0.002	0.409	0.682	-0.003	0.004
SALE_CASH	8.135e-06	7.6e-06	1.070	0.284	-6.76e-06	2.3e-05
SALE_FA	-0.0013	0.001	-2.177	0.030	-0.002	-0.000
SALE_INVT	-0.0001	7.97e-05	-1.356	0.175	-0.000	4.81e-05
PCHG_SALE_INVT	0.0043	0.003	1.405	0.160	-0.002	0.010
SALE_NWC	7.757e-05	3.42e-05	2.265	0.024	1.04e-05	0.000

PCHG_SALE_NWC	-0.0003	0.000	-1.051	0.293	-0.001	0.000
SALE_RECT	-4.43e-05	5.6e-05	-0.791	0.429	-0.000	6.55e-05
TIE	-2.543e-05	9.83e-06	-2.588	0.010	-4.47e-05	-6.17e-06
PCHG_TIE	0.0001	0.000	0.867	0.386	-0.000	0.000
WKCAP_AT	0.0004	0.000	1.185	0.236	-0.000	0.001
PCHG_WKCAP_AT	-0.0003	0.000	-0.991	0.322	-0.001	0.000
PCHG_AT	3.149e-05	0.000	0.283	0.777	-0.000	0.000
PCHG_COGS	-9.369e-05	0.000	-0.308	0.758	-0.001	0.001
PCHG_DEPR	0.0007	0.001	0.973	0.331	-0.001	0.002
PCHG_DLT	2.494e-05	3.38e-05	0.738	0.460	-4.13e-05	9.11e-05
PCHG_INVT	-0.0075	0.004	-1.980	0.048	-0.015	-7.74e-05
PCHG_SALE	0.0009	0.001	0.849	0.396	-0.001	0.003
PCHG_WORKINGCAP	-3.247e-05	0.000	-0.171	0.864	-0.000	0.000
CHG_DIVIDEND	0.0009	0.002	0.395	0.693	-0.003	0.005
PE_RATIO	0.0002	0.000	1.792	0.073	-1.7e-05	0.000
PCHG_PE_RATIO	6.914e-05	0.000	0.418	0.676	-0.000	0.000

The logistic regression shows that estimated probability $\text{Prob}(Y_i = 1 | X_i)$ is found via maximization of likelihood function or minimization of loss function in estimating $\hat{\beta}$. The estimated probability is also

$$F(\hat{\beta}^T X_i) = 1 / [1 + \exp(-\hat{\beta}^T X_i)]$$

where X_i is the vector of 47 features for a particular company in a particular year and $\hat{\beta}$ is the vector of 47 estimated coefficients, as shown in the Logit Regression Results Table. In other words,

$$\hat{\beta}^T X_i = \hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \hat{\beta}_2 X_{i2} + \cdots + \hat{\beta}_k X_{ik} + \cdots + \hat{\beta}_{47} X_{i47}$$

where the k th element of $\hat{\beta}$ is scalar $\hat{\beta}_k$ and its associated feature of the i th case in the logistic regression is X_{ik} . If $\hat{\beta}_k > 0$, then when X_{ik} increases, *ceteris paribus*, $F(\hat{\beta}^T X_i)$ or estimated $\text{Prob}(Y_i = 1 | X_i)$ increases. The latter implies that there is more likelihood that earnings would increase. On the contrary, if $\hat{\beta}_k \leq 0$, then when X_{ik} increases, $F(\hat{\beta}^T X_i)$ or estimated $\text{Prob}(Y_i = 1 | X_i)$ decreases. The latter implies that there is more likelihood that earnings would stagnate or decrease.

The results in the table indicate that the coefficients on DAYS_SALEAR, INVT_AT, PCHG_INVT_AT, PCHG_OPMBD, SALE_AT, SALE_NWC are significantly positive at p -value $< 5\%$ and increase in their associated feature values would increase the likelihood of positive NPM change the following year. The coefficients on CASH_DEBT, PCHG_DAYS_SALEAR, SALE_FA, TIE, PCHG_INVT are significantly negative at p -value $< 5\%$ and increase in their associated feature

values would reduce the likelihood of positive NPM change the following year.

Some of these results are easier to explain. For example, increase in PCHG_OPMBD or percentage change in operating profit margin before depreciation often augurs improvement in NPM. Increase in sales to asset (SALE_AT) and increase in sales to net working capital (SALE_NWC) signals asset and funding efficiency and future improved NPM. Increase in percentage of days' sales in accounts receivable (PCHG_DAYS_SALEAR), which is the number of days in a year divided by the accounts receivable turnover ratio or the average time to collect cash on receivables, implies weaker collection policy and hence lower future NPM expectation. Increase in sale of fixed asset SALE_FA is often linked with financial troubles (other than strategic business realignment) at the firm and hence would imply poor NPM the following year. Percentage increase in investments (PCHG_INVT) would typically mean short-run drain on NPM but perhaps with long run increases when the investments become successful. Some of the other accounting ratio such as times interest earned (TIE) impacts are harder to explain with the diverse set of firms.

Next, the data are randomly split into 70% training set (24,551 rows) and 30% test set (10,523 rows).

```
[13]: from sklearn.linear_model import LogisticRegression
      from sklearn import metrics
      X_train, X_test, y_train, y_test = \
          train_test_split(X, y, test_size=0.3, random_state=0)
      logreg = LogisticRegression()
      lresult=logreg.fit(X_train, y_train)
```

sklearn LogisticRegression is then applied to the training set fitting and then the prediction of the test set. One difference between the outputs from sklearn Logistic Regression and statsmodel logit regression is that the former adds L2 regularization in the estimation, so their results may differ by a little. We shall assume that if there are hyperparameters, they were fine-tuned in a validation phase, and what we do here is to employ a regrouped training data (earlier training data with validation data) for refitting (using fine-tuned hyperparameters) via “logreg” in [13] and then testing via “logreg.predict” and “logreg.score” in [14].

The accuracy of the test set prediction is shown to be about 53%. See code line [14].

```
[14]: y_pred = logreg.predict(X_test)
      print('Accuracy of logistic regression classifier \
            on test set: {:.2f}'.format(logreg.score(X_test, y_test)))

      Accuracy of logistic regression classifier    on test set: 0.53
```

The confusion matrix and the various prediction performance metrics are shown in [17,18]. Note for confusion matrix in [17] here in sklearn, the number of true positives (“NPM_FWD_CHG” = 0 or predicted label ‘0’) is 2384. The number of true negatives (“NPM_FWD_CHG” = 1 or predicted label ‘1’) is 3090. Hence, accuracy is $(2,524 + 3,090) / 10,523 = 53.34\%$.

```
[17]: from sklearn.metrics import confusion_matrix
      confusion_matrix = confusion_matrix(y_test, y_pred)
      print(confusion_matrix)

      [[2384 2837]
       [2077 3225]]
```

In other words, the first line of the confusion matrix denotes the case “NPM_FWD_CHG” = 0 as true positive or the case that next net profit margin does not increase. The second line denotes the case “NPM_FWD_CHG” = 1 as true negative or the case that next net profit margin increases.

```
[18]: from sklearn.metrics import classification_report
      print(classification_report(y_test, y_pred, digits=4))
      ### classification_report(y_test, y_pred, digits=8) gives more decimal places
```

	precision	recall	f1-score	support
0	0.5344	0.4566	0.4925	5221
1	0.5320	0.6083	0.5676	5302
accuracy			0.5330	10523
macro avg	0.5332	0.5324	0.5300	10523
weighted avg	0.5332	0.5330	0.5303	10523

The tables show the following for precision. For the 5221 positive cases of no increase in NPM, the percentage of the predicted positive cases that turns out to be correctly positive is $2384 / (2384 + 2077) = 53.44\%$.

For the 5302 negative cases (or increase in NPM), the percentage of the predicted negative cases that turns out to be correctly negative is $3225 / (3225 + 2837) = 53.20\%$. These are the precision numbers corresponding to positive case (row “0”) and negative case (row “1”).

In terms of recall, the tables show that for the 5221 positive cases of no increase in NPM, recall is $2384 / 5221 = 45.66\%$, which indicates a very low percentage that is correctly predicted as being positive. But for the 5302 negative cases of increase in NPM, recall is $3225/5302 = 60.83\%$, indicating 60.83% is correctly predicted as being negative.

It may be argued that ability to predict negative cases (increase in NPM) may be more important. However, it is much better to be able to predict both negative and positive cases.

```
[19]: import sklearn.metrics as metrics
      ## calculate the fpr and tpr for all thresholds of the classification
      preds_logreg = logreg.predict_proba(X_test)[:,-1]
      ## preds_logreg or logreg.predict_prob computes 10,523 prob numbers

      fpr, tpr, thresholds = metrics.roc_curve(y_test, preds_logreg)
      ## matches y_test of 1's and 0's versus pred prob of 1's for each of
      ## the 10,523 test cases
      ## sklearn.metrics.roc_curve(y_true, y_score,...) requires y_true as
      ## 0,1 input and y_score as prob inputs
      ## this metrics.roc_curve returns fpr, tpr, thresholds (Decreasing
      ## thresholds used to compute fpr and tpr)

      roc_auc_logreg = metrics.auc(fpr, tpr)
      ## sklearn.metrics.auc(fpr,tpr) returns AUC using trapezoidal rule
      ## Compute Area Under the Curve (AUC) using the trapezoidal rule.
      ## This is a general function, given points on a curve. For computing
      ## the area under the ROC-curve, see roc_auc_score.
      roc_auc_logreg

[19]: 0.5527816854878569
```

Code line [19] output shows how the AUC of the ROC is computed as 55.28%. The plotting of the ROC curve is shown in the output of code line [20].

```
[20]: import matplotlib.pyplot as plt
      plt.rc("font", size=14)
      plt.title('Receiver Operating Characteristic')
      plt.plot(fpr, tpr, 'b', label = 'AUC = %.2f' % roc_auc_logreg)
      plt.legend(loc = 'lower right')
      plt.plot([0, 1], [0, 1], 'r--')
      plt.xlim([0, 1])
      plt.ylim([0, 1])
      plt.ylabel('True Positive Rate')
      plt.xlabel('False Positive Rate')
      plt.show()
```



3.5 Dimension Reduction and Principal Component Analysis

If we were able to use all 47 features in the logistic regression, it may be overfitting. One way to reduce the dimensionality and overfitting is to prune the number of features. Chen *et al.* (2022) and others showed how careful selection of the features (sometimes using stepwise approach by removing insignificant features that do not affect R^2) could be important.

A common dimension reduction technique is principal component analysis (PCA) whereby the 47 features/variables could be collapsed (reduced) to a smaller number of variables. Each of the resulting variable is a principal component (PC) that is a linear combination of the 47 variables. One nice feature of PCA is that each PC is orthogonal to another (zero correlation) and with enough number of PCs, their total variance could match close to if not equal to the total variance of all the individual features. The orthogonality would alleviate the multi-collinearity issue in regression problems, including the logistic regression.

A brief theory of PCA can be explained as follows. Suppose the N features are random variables $x_1, x_2, x_3, \dots, x_N$. Now, let $X_{N \times 1} = (x_1, x_2, x_3, \dots, x_N)^T$ be a random vector. Let the mean of $X_{N \times 1}$ be $\mu_{N \times 1}$. Let the covariance matrix of X be $\Sigma_{N \times N}$. Let $\lambda_1, \lambda_2, \lambda_3, \dots, \lambda_N$ be the N eigenvalues and $v_1, v_2, v_3, \dots, v_N$ be the N corresponding eigenvectors (dimension $N \times 1$) of Σ such that $\Sigma v_j = \lambda_j v_j$ for any $j = 1, 2, \dots, N$. It can be shown that the eigenvalues of Σ , a real symmetric positive-definite matrix, are all (real) positive, though some may not be unique.

The solutions to the eigenvalues and eigenvectors are constrained by normalized orthogonal eigenvectors so that $v_j^T v_j = 1$ for all j , and $v_j^T v_k = 0$ for $j \neq k$. Let $\Lambda_{N \times N} = \text{diag}(\lambda_1, \lambda_2, \lambda_3, \dots, \lambda_N)$, a diagonal matrix. Without loss of generality, let the diagonal elements be arranged in descending order so that the left-top element λ_1 is the largest eigenvalue and the right-bottom element λ_N is the smallest eigenvalue.

Let $\Gamma_{N \times N} = (v_1, v_2, v_3, \dots, v_N)$ be such that $\Gamma^T \Gamma = I_{N \times N}$, the N -dimension identity matrix. Now, let random variable $Y_j = v_j^T (X - \mu)$ which has a 1×1 dimension. This linear combination of the N detrended random features is called the j th scalar PC of X . There are N such PCs since there are N number of eigenvectors v_j for $j = 1, 2, \dots, N$. Let $Y_{N \times 1} = (Y_1, Y_2, Y_3, \dots, Y_N)^T$ be vector of the Y_j 's.

Properties of the PC vector Y can be seen as follows. Given Γ , its expectation or mean, $E(Y) = \Gamma^T E(X - \mu) = 0$. Variance of vector Y is $\text{var}(Y)$ or $[\text{cov}(Y_j, Y_j)]_{N \times N} = E(Y Y^T) = E(\Gamma^T (X - \mu)(X - \mu)^T \Gamma) = \Gamma^T \Sigma \Gamma$ given Γ . But by the definition of eigenvalues and eigenvectors, $\Sigma \Gamma = \Gamma \Lambda$. Hence,

$$\text{var}(Y) = E(Y Y^T) = \Gamma^T \Sigma \Gamma = \Gamma^T \Gamma \Lambda = \Lambda$$

Clearly, the variance of the j th PC Y_j is the j th diagonal element of $\Lambda_{N \times N}$ or λ_j . Different PCs are also uncorrelated with each other. If the PCs are normally distributed, then they are independent.

From the way we construct Λ , the first PC, Y_1 , has the largest variance followed by the variance of the second PC Y_2 , and so on.

Since $\Gamma^T \Gamma = \Gamma \Gamma^T = I_{N \times N}$, trace of Λ or $\text{tr}(\Lambda) = \text{tr}(\Gamma^T \Sigma \Gamma) = \text{tr}(\Sigma \Gamma \Gamma^T) = \text{tr}(\Sigma) = \sum_{j=1}^N \text{var}(x_j)$ or the total variance of the features. Note that the trace operation is invariant under circular shifts shown above. Therefore, the sum of all the variances of the PCs is equal to $\text{tr}(\Lambda)$ which is also the total variance of the features. The contribution of each PC variance is also called the explained variance (information explained using the eigenvectors) and is often expressed as a ratio to the total variance, i.e., $\text{var}(Y_j) / \sum_{j=1}^N \text{var}(x_j)$. The sum of these ratios across all eigenvalues equal to one.

Since we do not know the exact Σ nor mean μ of the vector X , they must be estimated using the time series of the features, assuming the features are stationary. Estimate of μ is $\hat{\mu} = (\hat{\mu}_1, \hat{\mu}_2, \dots, \hat{\mu}_N)$ where $\hat{\mu}_j = \frac{1}{T} \sum_{t=1}^T x_{jt}$. The j th random feature takes realized time series values $x_{j1}, x_{j2}, \dots, x_{jT}$. T is the sample size. Estimate of Σ is $\hat{\Sigma} = [\widehat{\text{cov}}(X_i, X_j)]$ where $\widehat{\text{cov}}(x_i, x_j) = \frac{1}{T} \sum_{t=1}^T (x_{it} - \hat{\mu}_i)(x_{jt} - \hat{\mu}_j)$. Any computed eigenvalue $\hat{\lambda}_j$ and eigenvector $\hat{V}_j$ would be based on sample estimates $\hat{\Sigma}$ and $\hat{\mu}$. Solution of

$\hat{\lambda}_j$ is via a polynomial equation involving determinant $\det(\hat{\Sigma} - \hat{\lambda}_j I) = 0$. Eigenvector $\hat{V}_j$ is then solved in the second step. Once the eigenvectors are estimated, the j th PC based on realized features data is $Y_j = \hat{V}_j^T (X^* - \hat{\mu})$, where X^* is the realized values of X .

We will perform a PCA and possible dimension reduction of the feature space. Dimension reduction occurs if we choose only a subset of the PCs incorporating PCs with the largest variances. First, we standardize the features. See code line [23].

```
[23]: from sklearn.preprocessing import StandardScaler
      scaler = StandardScaler()
      #`StandardScaler` in `sklearn.preprocessing` module is used to
      # standardize features by removing the mean and scaling to unit variance.
      X1=scaler.fit_transform(X)
```

Then we check the degree of multi-collinearity that may affect the logistic regression. Code lines [26] and [27] show that there are 53 correlations of pairs of features in X that have correlation exceeding 0.9.

```
[26]: # check correlation of features after standardization
      cor=X1.corr()
```

```
[27]: count = np.count_nonzero(cor > 0.9)
      print(count)
```

53

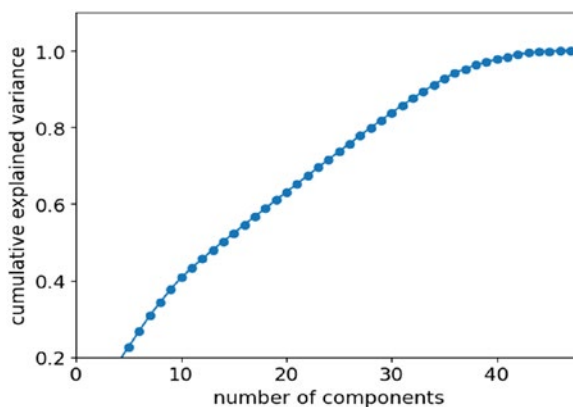
In code line [29], all 47 PCs are extracted after the features are normalized. The normalization or standardization is important as it then assigns “equal weights” to all features, so they are on the same scale. This is particularly important in feature space where some features with large measures may swarm those with smaller measures and produce distorted combined features.

```
[29]: from sklearn.decomposition import PCA
      pca = PCA(n_components = 47)
      pc = pca.fit_transform(X1)
      print('explained variance ratio: %s' %pca.explained_variance_ratio_)
      # % is used for string formatting placed in front of following numbers

explained variance ratio: [0.05183334 0.04664873 0.04402974 0.04330785 0.04119352 0.04107028
0.03985298 0.03542389 0.033818 0.03049519 0.02525618 0.02363967
0.02324064 0.02235872 0.02181575 0.02179855 0.02147621 0.02141851
0.02131531 0.02129143 0.0212768 0.02123959 0.02113885 0.021079
0.02092839 0.02087299 0.02062019 0.02056914 0.01983438 0.01934671
0.01905125 0.01866613 0.01784745 0.01716822 0.01650159 0.01479251
0.01069929 0.01028556 0.00737799 0.00710649 0.00668856 0.00589251
0.0042024 0.00306136 0.00134249 0.00068641 0.00043914]
```

The “pca.explained_variance_ratio” is the ratio of each eigenvalue to the sum of all eigenvalues which represents the total sample variances of all the feature variables. The individual ratios add up to one. The cumulative contribution of the explained variances ratios is shown in [31].

```
[31]: plt.plot(range(1,48),np.cumsum(pca.explained_variance_ratio_))
plt.scatter(range(1,48),np.cumsum(pca.explained_variance_ratio_))
plt.xlim(0,48)
plt.ylim(0.2,1.1)
plt.xlabel("number of components")
plt.ylabel("cumulative explained variance")
```



The PCs are used to form the new 47 features in pc_X. See code line [32].

```
[32]: pc_X = pd.DataFrame(pc,columns = \
    ['pc_1','pc_2','pc_3','pc_4','pc_5','pc_6','pc_7','pc_8',\
    'pc_9','pc_10','pc_11','pc_12','pc_13','pc_14','pc_15',\
    'pc_16','pc_17','pc_18','pc_19','pc_20','pc_21','pc_22',\
    'pc_23','pc_24','pc_25','pc_26','pc_27','pc_28','pc_29',\
    'pc_30','pc_31','pc_32','pc_33','pc_34','pc_35','pc_36',\
    'pc_37','pc_38','pc_39','pc_40','pc_41','pc_42','pc_43',\
    'pc_44','pc_45','pc_46','pc_47'])
```

We can experiment with a smaller number of PCs and check their performances in the predictions. Here, we utilize all PCs to perform the same logistic regression and predictions. One problem with regression using the PCs is that it is now difficult to interpret the impact of each PC which is a linear combination of many accounting ratios.

Then accuracy measurement of the test cases now using features `pc_X` is performed in code lines [35] and [36].

```
[35]: from sklearn import metrics
X_train, X_test, y_train, y_test = train_test_split(pc_X, y, \
                                                    test_size=0.3, random_state=0)
logreg = LogisticRegression()
lresult=logreg.fit(X_train, y_train)

[36]: y_pred1 = logreg.predict(X_test)
print('Accuracy of logistic regression classifier on test set: \
      {:.2f}'.format(logreg.score(X_test, y_test)))

Accuracy of logistic regression classifier on test set:      0.54
```

There is an improvement of about 1% relative to the 53% in the feature space before PCA transformations.

The confusion matrix and classification reports are shown via code lines [37] and [38].

```
[37]: from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, y_pred1)
print(confusion_matrix)

[[2647 2574]
 [2268 3034]]

[38]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred1, digits=4))
```

	precision	recall	f1-score	support
0	0.5386	0.5070	0.5223	5221
1	0.5410	0.5722	0.5562	5302
accuracy			0.5399	10523
macro avg	0.5398	0.5396	0.5392	10523
weighted avg	0.5398	0.5399	0.5394	10523

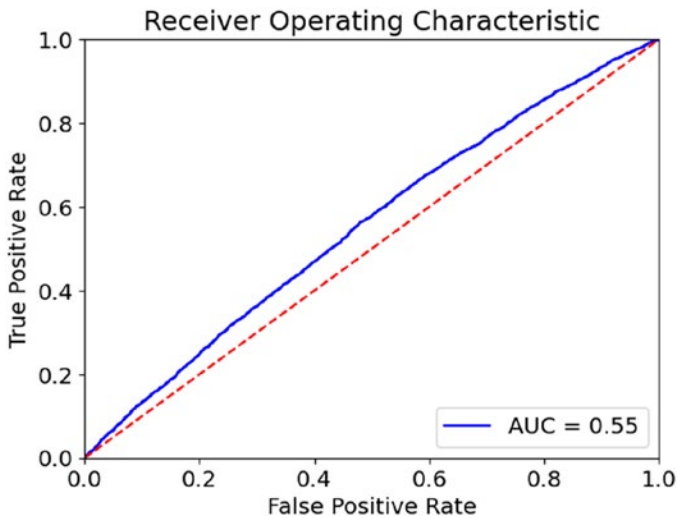
The area under ROC is now 55.34%, showing a marginal improvement from ROC in the original feature space. See code lines [40] and [41].

```
[40]: ### Instead of above first two commands, we can use
from sklearn.metrics import roc_auc_score
logit_roc_auc = roc_auc_score(y_test, logreg.predict_proba(X_test)[:,:1])
print(logit_roc_auc)
### But we still require the 3rd command metrics.auc(fpr, tpr) to draw the ROC

0.5534208432402845
```



```
[41]: import matplotlib.pyplot as plt
plt.rc("font", size=14)
plt.title('Receiver Operating Characteristic')
plt.plot(fpr, tpr, 'b', label = 'AUC = %0.2f' % roc_auc_logreg)
plt.legend(loc = 'lower right')
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.show()
```



It is seen that PCA transformed features in this case (eliminating multi-collinearity) provide marginally better prediction results. There is no guarantee that PCA transformation will produce significantly better results in prediction even when multi-collinearity is reduced or eliminated. However, it is correct to say that reducing multi-collinearity (with or without PCA) generally should improve statistical tests and interpretations of significance of features.

3.6 Concluding Thoughts

In this chapter, we see that the machine learning method can be deployed to predict if a company's next year (period) earning will rise or not. The recall is above 60%, while the precision is about 53%. These results can certainly be improved with more thorough selection and pre-processing of the company accounting ratios as the features.

We also demonstrate how to use standardization on the features and then construct PC variables that are not correlated. The latter may be important if the features are highly correlated (high multi-collinearity) and dampen the predictive accuracies.

The logistic regression is employed and is seen as an efficient tool to predict binary classifications. The logistic regression (together with regularization) could be used as a basic benchmark when comparing more sophisticated machine learning methods in the later chapters.

Perhaps more important than just statistical performance, the outputs such as in the confusion matrix could be used for analysing economic consequences. Conditional probabilities based on numbers in the confusion matrix can be used as a basic measure of expected outcomes. For example, 2697 predicted cases of “NPM_FWD_CHG” > 0 (label “1” or negative cases) when in fact label is “0” or no NPM increase means that conditional probability of wrong prediction based on label being “0” is $2697 / (2524 + 2697) = 51.65\%$. This high conditional probability may imply trading losses if stock is purchased based on predicted next year $\text{NPM} > 0$. It is also seen that sometimes recall is an economically more significant measure in a bank loan context, while at other times, precision is more important in sales targeting.

References

- Chen Xi, Yang Ha Cho, Yiwei Dou, and Baruch Lev (2022). Predicting future earnings changes using machine learning and detailed financial data. *Journal of Accounting Research*, 60(2).
- Francis Sahngun Nahm (2022). Receiver operating characteristic curve: Overview and practical use for clinicians. *Korean J Anesthesiol*, 75(1), 25–36.
- Kian Guan Lim (2015). *Financial Valuation and Econometrics*, 2nd Edition, Chapter 24. World Scientific.

This page intentionally left blank

Chapter 4

Naïve Bayes, k-NN, and Support Vector Machines

In this chapter, we shall study several algorithms that perform prediction of numerical targets as well as classifications. Naïve Bayes, k-NN (k-Nearest Neighbors), and Support Vector Machine (SVM) are supervised learning algorithms that are commonly used for classification of targets. They can also be used for regression in predicting continuous target values. For the case of SVM, regression is often called Support Vector Regression (SVR).

4.1 Naïve Bayes

The Naïve Bayes classifier (or classification algorithm) is based on Bayes' Rule or Bayes' Theorem. The theorem states that the conditional probability of event $\{Z = 1\}$ given event Y has occurred is

$$P(\{Z = 1\}|Y) = P(\{Z = 1\} \cap Y) / P(Y)$$

where probability of event Y in the context of a universe set Ω in which Y resides is $P(Y)$, and $P(\{Z = 1\} \cap Y)$ denotes the probability of event that $\{Z = 1\}$ and Y occurred jointly. Examples of $P(Y)$ are that the probability of random draw of a blue ball from a bag containing 30 blue balls, 30 red balls, and 40 green balls is 30/100 or 0.3, the random draw of a point falling in interval $[0.3, 0.5]$ in $\Omega = [0, 1]$ is 0.2, and so on.

Consider the bag with the same 100 balls. 10 of the 30 blue balls are painted with a black dot, 20 of the 30 red balls are painted with a black dot, and 30 of the 40 green balls are painted with a black dot. Let $\{Z = 1\}$ represent the event that the ball drawn contains a black dot. $P(\{Z = 1\}) = 0.6$ since altogether 60 balls have black dots.

Let Y represent the event that a blue ball is drawn. Suppose in a drawn ball, we are given the information that it is blue, then the probability that it also carries a black dot is $P(\{Z = 1\}|Y) = 1/3$ or 10 out of 30 blue balls. $P(\{Z = 1\}|Y)$ can also be computed as $P(\{Z = 1\} \cap Y) / P(Y) = 0.1/0.3 = 1/3$ since $P(\{Z = 1\} \cap Y) = 10/100$ and $P(Y) = 30/100$.

Now, think of event $\{Z = 1\}$ as a class/type, e.g., those who are marksmen (sharp shooters in national service who can score at least 21 out of 25 shots at training targets), and event $\{Z = 0\}$ as the rest of the universe, i.e., those who are not marksmen. In the context of sequential events, $P(\{Z = 1\})$ is sometimes called the prior probability or probability with no other information connected to it except for its sample count in the population Ω . Similarly, $P(\{Z = 0\})$ is also a prior probability of event $\{Z = 0\}$. The Bayes' Theorem is very useful for updating probabilities with sequential information inputs.

Now, think of Z as a random variable that can take the value 1 or else 0, i.e., there is a binary classification. Bayes' Theorem can be expressed as:

$$\begin{aligned} P(Z|Y) &= P(Y|Z) \times P(Z) / P(Y) \\ &= P(Y|Z) \times P(Z) / [P(Y|Z) \times P(Z) + P(Y|Z^c) \times P(Z^c)] \end{aligned}$$

where Z^c is complement of event Z , i.e., $Z^c = \Omega \setminus Z$ (Ω less Z), and $P(Y) = P(Y \cap Z) + P(Y \cap Z^c)$. This latter version of Bayes' Theorem is useful in the context of updating prior information or probability of Z , $P(Z)$. Given new information in the form of event Y , the updated or posterior probability of Z is $P(Z|Y)$. Sometimes it may not be convenient to obtain $P(Z \cap Y)$ directly. Hence, the computation of posterior $P(Z|Y)$ may be better done via $P(Y|Z) \times P(Z) / [P(Y|Z) \times P(Z) + P(Y|Z^c) \times P(Z^c)]$ where $P(Y|Z)$ and $P(Y|Z^c)$ are more readily available.

A real example is in medical testing, where a patient has *a priori* (without any testing for further information) a probability $P(Z) = x$ of having the medical condition Z . x (%) could be obtained from some statistical averages over reports of such medical conditions in various countries. A medical test on the patient could produce information Y , e.g., a positive test case. It may not be easy to obtain total information across all countries of the % of all tests producing Z and Y , i.e., $P(Z \cap Y)$. However, it may

be easier to obtain summary reports on $P(Y|Z)$ and $P(Y|Z^c)$, e.g., percentage of cases that have Z and tested positive, and percentage of non- Z and tested positive (false positive).

The Naïve Bayes classification algorithm works using the Bayes Theorem:

$$P(Z|Y) = P(Y|Z) \times P(Z) / P(Y)$$

where $Z = 1$ or else 0 are the two class types we seek for a case under testing and Y is some information/features or evidence we can use to estimate/predict Z .

Suppose it is a prediction of next stock price index change employing sentiment analysis (a kind of natural language processing technique) based on density of mood words prevalent in stock market reports, assuming words like “optimistic”, “growth”, “boom”, etc. would more likely lead to stock index rises, while “pessimistic”, “poor”, “cautious”, etc. would likely lead to stock index drops. Let $\{Z = 1\}$ be the event of next stock price index increase. Therefore, $\{Z = 0\}$ is the event of next stock price index decrease or no change. Let features $F_1, F_2, \dots, F_n$ be mood words in a particular day. These are used for predicting if the stock price index will rise or fall the next day. This illustration here is based on single words; advanced sentiment analysis dives into phrases, sentences, and texts. However, the approach is similar.

Since the mood words are single elements, it is more convenient to use sets to represent collections of these elements. Now, all the features found in a day constitute event set $Y = \{F_1, F_2, \dots, F_n\}$, the evidence or signal for the day. A feature or mood word can occur many times on a day due to many reports or appearing many times in a single report, but we could just count it as one occurrence for that day.

$$P(Z | F_1, F_2, \dots, F_n) = P(F_1, F_2, \dots, F_n | Z) \times P(Z) / P(Y)$$

Naïve Bayes typically naively makes a simplifying assumption. The conditional probability of $P(F_j | Z)$ is assumed to be independent of any other feature $F_k, k \neq j$. In other words, for any j ,

$$P(F_j | Z, F_1, F_2, \dots, F_{j-1}, F_{j+1}, \dots, F_n) = P(F_j | Z)$$

This assumption makes the computations more tractable. It is however, simplifying, because it is possible that a mood word “downward” (as in downward trend) is likely associated with “poor” (as in poor

performance), and both words are not independent given $Z = 0$. The simplification is not ideal and may create ineffectiveness in the prediction.

By Bayes' Theorem,

$$\begin{aligned} P(F_1, F_2, \dots, F_n | Z) &= P(F_1 | F_2, \dots, F_n, Z) \times P(F_2, F_3, \dots, F_n | Z) \\ &= P(F_1 | F_2, \dots, F_n, Z) \times P(F_2 | F_3, \dots, F_n, Z) \times P(F_3, F_4, \dots, F_n | Z) \\ &= P(F_1 | F_2, \dots, F_n, Z) \times P(F_2 | F_3, \dots, F_n, Z) \times P(F_3 | F_4, \dots, F_n, Z) \\ &\quad \times P(F_4, \dots, F_n | Z) \\ &= \dots\dots\dots \\ &= P(F_1 | F_2, \dots, F_n, Z) \times P(F_2 | F_3, \dots, F_n, Z) \times P(F_3 | F_4, \dots, F_n, Z) \\ &\quad \times P(F_4 | F_5, \dots, F_n, Z) \times P(F_5 | F_6, \dots, F_n, Z) \times \dots \times P(F_{n-1} | F_n, Z) \times P(F_n | Z) \end{aligned}$$

Using the simplifying assumption on the last line,

$$\begin{aligned} P(F_1, F_2, \dots, F_n | Z) &= P(F_1 | Z) \times P(F_2 | Z) \times P(F_3 | Z) \times P(F_4 | Z) \times P(F_5 | Z) \times \dots \\ &\quad \times P(F_{n-1} | Z) \times P(F_n | Z) \\ &= \prod_{j=1}^n P(F_j | Z) \end{aligned}$$

Then,

$$P(Z|F_1, F_2, \dots, F_n) = \prod_{j=1}^n P(F_j|Z) \times P(Z) / P(Y) \quad (4.1)$$

Since there are only two classes/types $\{Z = 1\}$ and $\{Z = 0\}$, we can use the training data set to estimate $\hat{P}(Z = 1)$ (percentage of days when next day stock price index increases) and $\hat{P}(Z = 0)$ (percentage of days when next day stock price index decreases or remains constant) respectively, where $\hat{P}(Z = 1) + \hat{P}(Z = 0) = 1$. The method can of course be extendible to consider not just two classes or types (sometimes also called “categories”), such as using features to predict whether a test student obtains grade in class/type “A+”, “A”, “A–”, “B+”, or “B” based on the student’s features such as background strength, number of hours put into study, and level of motivation, etc.

Similarly, for all features $\{F_1, F_2, \dots, F_n\}$ in the training data set, we can estimate $\hat{P}(F_j|Z=1)$ (percentage of days of occurrences of F_j when next day stock price index increases) and $\hat{P}(F_j|Z=0)$ (percentage of

days of occurrences of F_j when next day stock price index decreases or remains constant). Now, suppose a sample point $Y^* = \{F_1, F_3, \dots, F_m\}$ ($m \leq n$) from the test sample is chosen. We suppose we do not know the test point's Z value (the type) and want to use Naïve Bayes algorithm to predict its Z value. Then

$$P(\{Z = 1\} | F_1, F_3, \dots, F_m) = \hat{P}(F_1|Z = 1) \times \hat{P}(F_3|Z = 1) \cdots \times \hat{P}(F_m|Z = 1) \\ \times \hat{P}(Z = 1)/P(Y^*)$$

and

$$P(\{Z = 0\} | F_1, F_3, \dots, F_m) = \hat{P}(F_1|Z = 0) \times \hat{P}(F_3|Z = 0) \cdots \times \hat{P}(F_m|Z = 0) \\ \times \hat{P}(Z = 0)/P(Y^*).$$

$P(Y^*)$ does not need to be computed above as it is the same for both the quantities above. We find

$$\text{Max } \{P(\{Z = 1\} | F_1, F_3, \dots, F_m), P(\{Z = 0\} | F_1, F_3, \dots, F_m)\}$$

given Y^* . The predictor (of class Z , whether 1 or 0) for this test sample point with Y^* is

$$\arg \max_{Z=1,0} P(\{Z\} | F_1, F_3, \dots, F_m) = \arg \max_{Z=1,0} \prod_j \hat{P}(F_j|Z) \times \hat{P}(Z) \quad (4.2)$$

The accuracy of the Naïve Bayes algorithm is then evaluated based on the performance of each of the predictors of each sample point/case in the test dataset. For small data sets, the Naïve Bayes method is shown to perform reasonably well.

In Eq. (4.1), for estimating $\hat{P}(Z = 1)$ and $\hat{P}(Z = 0)$, we can use their frequency counts/total number of sample points or cases in the training sample. These are unconditional probability estimates. These estimates add to one. There are several methods for estimating $\hat{P}(F_j|Z = 1)$ and $\hat{P}(F_j|Z = 0)$ for every j in the training data set, depending on what we assume is the probability distribution of the features F_j . Some common distribution assumptions are the categorical, the Gaussian/normal distributions, and the multinomial. The Naïve Bayes predictor derived from

each of these distributions of the features are in turn called the Categorical NB, the Gaussian NB, and the Multinomial NB.

In the sentiment analyses using mood words as features, it is natural to assume that the features $F_j, j = 1, 2, \dots, n$ follow the categorical distribution. This means that for any sample point (day), given Z , the probability of a feature F_j occurring is $P(F_j|Z)$ and the probability of the feature not occurring is $1 - P(F_j|Z)$. Thus, any feature F_1 through F_n can either occur or not occur. The categorical variable of F_j or no F_j can be hot-encoded as dummy variable 1 or 0. If there is only one feature in the dataset, then this is also a Bernoulli Naïve Bayes. In any one day, different categorical variables F_1, F_2 , etc. can occur together.

For example, suppose we have 700 days in our study for the training set and 300 other days for the test set. Suppose the dictionary contains 100 mood words. If in the training set of 700 days, 500 have $Z = 1$ or index rise the day after, and 200 have $Z = 0$ which represents index decrease or staying constant the day after. Of the 500, 200 days have reports containing “optimistic”, 100 days have reports containing “growth”, and so on. For estimating $\hat{P}(F_j|Z=1)$, we can use their frequency counts of number of days of occurrence of F_j /total number of days given Z in the training sample. Here, $\hat{P}(F_j = \text{“optimistic”} | Z=1) = 200/500 = 0.4$. $\hat{P}(F_j = \text{“growth”} | Z=1) = 100/500 = 0.2$. Of the other 200 days, 4 days have reports containing “optimistic”, and 10 days have reports containing “growth”, and so on. Then, $\hat{P}(F_j = \text{“optimistic”} | Z=0) = 4/200 = 0.02$. $\hat{P}(F_j = \text{“growth”} | Z=0) = 10/200 = 0.05$. Note that both “optimistic” and “growth” can occur together in any one day.

Sometimes, the probability estimates may be too small or close to zero and can make the prediction in Eq. (4.2) difficult because of the nearly zero probability on the right-side. To make the numerical computation more robust, a smoothing parameter $\alpha > 0$ (usually ≤ 1) is added in the estimation of $\hat{P}(F_j|Z=1)$ and $\hat{P}(F_j|Z=0)$. For example, $\hat{P}(F_j = \text{“boom”} | Z=1) = (1 + \alpha)/(200 + \alpha n)$ where n is the number of features. This adjustment can be non-negligible when n is small.

In the general execution of Eq. (4.2), we assume that the training set is large enough so that $\hat{P}(F_j|Z) > 0$ for every possible F_j (in the dictionary) in each of condition $Z = 1$ or $Z = 0$. We also assume that in the test data set (and foreseeable generalized data set), observed Y^{**} for one day or observed dictionary mood words for that day consisting of

$F_5, F_7, F_8, \dots, F_m$ for example have computed $\hat{P}(F_5|Z=1)$, $\hat{P}(F_7|Z=1)$, $\hat{P}(F_8|Z=1)$, $\dots$, $\hat{P}(F_m|Z=1)$, and also $\hat{P}(F_5|Z=0)$, $\hat{P}(F_7|Z=0)$, $\hat{P}(F_8|Z=0)$, $\dots$, $\hat{P}(F_m|Z=0)$ obtained from the training set computations. $\hat{P}(Z=1)$ and $\hat{P}(Z=0)$ are also obtained from the training data set. Then (4.2) can be used to predict the test set Z (whether 1 or 0) based on these test set features Y^{**} .

Sometimes the features may appear to be organized into sub-categories, e.g., daily reports could come from three sources, viz., news, analysts' reports, and social media. Each source would have features such as the mood words; the same mood words could occur in more than one source. The information Y contains not $\{F_1, F_2, \dots, F_n\}$, but news – $\{F_1, F_2, \dots, F_n\}$, analysts' reports – $\{F_1, F_2, \dots, F_n\}$, social media – $\{F_1, F_2, \dots, F_n\}$, i.e., including sources and their mood words. We could use instead news- F_j , analysts'- F_j , social media- F_j for all j , as separate complex features. In this way, the same mood word F_j occurring in different sources are treated as being different pieces of information. We can extend Eq. (4.1) to

$$\begin{aligned} &P(Z | \text{news-}F_1, \text{news-}F_2, \dots, \text{news-}F_n, \text{analysts'-}F_1, \\ &\quad \text{analysts'-}F_2, \dots, \text{analysts'-}F_n, \text{social media-}F_1, \text{social} \\ &\quad \text{media-}F_2, \dots, \text{social media-}F_n) \\ &= \prod_{j=1}^n P(\text{news} - F_j | Z) \times \prod_{j=1}^n P(\text{analysts'} - F_j | Z) \\ &\quad \times \prod_{j=1}^n P(\text{social media} - F_j | Z) \times P(Z) / P(Y) \end{aligned}$$

For estimating $\hat{P}(\text{news} - F_j | Z=1)$, we can use their frequency counts of number of days of occurrence of news- F_j /total number of days given Z in the training sample. There is some expansion in the number of features, but the computational method remains the same as in the categorical NB.

Now, suppose there are other non-natural language features G_j that are continuous, such as volatility ($j=1$), volume traded on index futures ($j=2$), lagged index change ($j=3$), etc., so that it is not feasible to count category frequency to estimate $\hat{P}(G_j|Z)$. In such a situation, we can use the Gaussian Naïve Bayes (GNB) method which employs the Gaussian or normal probability distribution suited for continuous variables/features. GNB assumes that $P(G_j|Z)$ (for each continuous variable/feature G_j)

follows a Gaussian/normal probability distribution. Suppose Z has two classes/types/categories, $Z = 1$ or $Z = 0$. The conditional probability density function of each feature $G_j, j = 1, 2, \dots, n$, is given by

$$P(G_j|Z) = \frac{1}{\sqrt{2\pi\sigma_{Zj}^2}} \exp\left(-\frac{1}{2}\left[\frac{G_j - \mu_{Zj}}{\sigma_{Zj}}\right]^2\right)$$

where $\mu_{Zj} = E(G_j|Z)$ and $\sigma_{Zj} = \sqrt{\text{var}(G_j|Z)}$. These can be estimated using the sample mean and sample standard deviation of the feature j values given Z , $\hat{\mu}_{Zj}$ and $\hat{\sigma}_{Zj}$. If it is further assumed that these parameters are independent of Z , then $\mu_{Zj} = \mu_j = E(G_j)$ and $\sigma_{Zj} = \sigma_j = \sqrt{\text{var}(G_j)}$, and these can be estimated using the unconditional sample mean and sample standard deviation of the feature j values. And if the assumption is that these parameters are independent of both Z and j , then $\mu_{Zj} = \mu = E(G)$ and $\sigma_{Zj} = \sigma = \sqrt{\text{var}(G)}$ where no subscripts for G indicate that the sampling mean and variance are taken over all features, across all j .

Then, the predictor (of class Z , whether 1 or 0) for each test sample case with Y (associated lagged features $\{G_j\}$) is

$$\begin{aligned} & \arg \max_{Z=1,0} P(\{Z\}|G_1, G_2, \dots, G_n) \\ &= \arg \max_{Z=1,0} \prod_{j=1}^n \frac{1}{\sqrt{2\pi\hat{\sigma}_{Zj}^2}} \exp\left(-\frac{1}{2}\left[\frac{G_j - \hat{\mu}_{Zj}}{\hat{\sigma}_{Zj}}\right]^2\right) \times \hat{P}(Z) \end{aligned} \quad (4.3)$$

where $\hat{P}(Z)$ is the frequency count given Z obtained from the training data set, and $\hat{\mu}_{Zj}$, $\hat{\sigma}_{Zj}$ are estimates from the training data where Z is known or given. Y is the information containing all the non-natural language features. Note that with GNB, there is no need to first standardize/normalize the features first (using all of sample data) before training if the test data features also have the same mean and variance as those in the training data since $\hat{\mu}_{Zj}$, $\hat{\sigma}_{Zj}$ estimates from the training data would suitably represent parameters from the test data set. However, if the mean and variance from the test data set are different from those in the training data set (even if features from both sets are Gaussian) for example when we are dealing with time-sequenced data, it may be more robust to first standardize the training and the test data features separately before applying the GNB method. After standardizing, the $\hat{\mu}_{Zj}$, $\hat{\sigma}_{Zj}$ estimates of the standardized features are 0 and 1 respectively, for both the training and test set data.

For mixed type of data in features, e.g., some features F_j are categorical while others G_j are continuous variables, we can apply the NB method as follows. Following Eq. (4.1):

$$\begin{aligned} P(Z | F_1, F_2, \dots, F_u, G_1, G_2, \dots, G_v) \\ = \prod_{j=1}^u P(F_j | Z) \prod_{j=1}^v P(G_j | Z) \times P(Z) / P(Y) \end{aligned}$$

where $Y \equiv \{F_1, F_2, \dots, F_u, G_1, G_2, \dots, G_v\}$. The independence of all features conditional on Z is assumed. Then, the predictor (of class Z , whether 1 or 0) for each test sample case with Y is

$$\begin{aligned} \arg \max_{Z=1,0} P(\{Z\} | F_1, F_2, \dots, F_u, G_1, G_2, \dots, G_v) \\ = \arg \max_{Z=1,0} \prod_{j=1}^u \hat{P}(F_j | Z) \prod_{j=1}^v \frac{1}{\sqrt{2\pi\hat{\sigma}_{Zj}^2}} \exp\left(-\frac{1}{2}\left[\frac{G_j - \hat{\mu}_{Zj}}{\hat{\sigma}_{Zj}}\right]^2\right) \times \hat{P}(Z) \end{aligned}$$

The maximization can also be taken over the logarithm.

Now, suppose the continuous features do not actually follow Gaussian distributions. Then the Gaussian NB may yield biased predictions due to the incorrect distributional specification. Suppose the probability distribution of the features is unknown. In this case we may use the Multinomial NB. Unlike categorical variables where features $\{F_j\}$ can occur together in a test sample point where each feature F_j de facto takes dummy value 1 if the feature occurs or dummy value 0 if the feature does not occur, a feature with multinomial distribution basically can take on only one of several possible occurrences, e.g., the outcome of drawing a blue or red or green ball from a bag, and each draw or trial is repeated a number of times so that the feature can be characterized as 5 blue, 2 red, 3 green in 10 draws. The next feature could be 4 blue, 3 red, 3 green. Yet the next could be 6 blue, 1 red, 3 green, and so on.

If the training data set under condition of $Z = 1$ yields most cases with features having more blue balls, while under condition of $Z = 0$ yields most cases with features having more red balls, then the observed features in a test case with more blue could likely predict that the case is $Z = 1$. For each case, there could be multiple features each with a different multinomial distribution, but here we use only one such multinomial feature.

More formally, suppose for a sample point (day) in the training data set, let V_u denote the event that volatility is up and V_d denote the event that

volatility is down. Let L_u denote the event that trading volume is up and L_d denote the event that trading volume is down. Let I_u denote the event that the index is up and I_d denote the event that index is down. Then the trading feature have one of 2^3 or eight possibilities, viz., (V_u, L_u, I_u) , (V_u, L_u, I_d) , (V_u, L_d, I_u) , (V_u, L_d, I_d) , (V_d, L_u, I_u) , (V_d, L_u, I_d) , (V_d, L_d, I_u) , and (V_d, L_d, I_d) . Suppose the day is broken into three sessions: early morning, late morning, and early afternoon in the trading sessions of the exchange. Then the above possibilities can be measured three times during the day. These possibilities combined into a feature become a multinomial distribution with three trials. The feature is used to predict the outcome $Z = 1$ or $Z = 0$ the following trading day.

Under condition of $Z = 1$ (next day index rises), the probabilities of combined events (V_u, L_u, I_u) , (V_u, L_u, I_d) , (V_u, L_d, I_u) , (V_u, L_d, I_d) , (V_d, L_u, I_u) , (V_d, L_u, I_d) , (V_d, L_d, I_u) , (V_d, L_d, I_d) , are estimated by frequency out of the total of all combined events in the training data, i.e., each with probability $\hat{\theta}_1, \hat{\theta}_2, \hat{\theta}_3, \hat{\theta}_4, \hat{\theta}_5, \hat{\theta}_6, \hat{\theta}_7, \hat{\theta}_8$, where $\sum_{i=1}^8 \hat{\theta}_i = 1$. Similarly, under condition of $Z = 0$ (next day index falls or maintains), the probabilities of combined events (V_u, L_u, I_u) , (V_u, L_u, I_d) , (V_u, L_d, I_u) , (V_u, L_d, I_d) , (V_d, L_u, I_u) , (V_d, L_u, I_d) , (V_d, L_d, I_u) , (V_d, L_d, I_d) , are estimated by frequency out of the total of all combined events in the training data, i.e., each with probability $\hat{\gamma}_1, \hat{\gamma}_2, \hat{\gamma}_3, \hat{\gamma}_4, \hat{\gamma}_5, \hat{\gamma}_6, \hat{\gamma}_7, \hat{\gamma}_8$, where $\sum_{i=1}^8 \hat{\gamma}_i = 1$. Assume these combined events are independent of each other and the session in which they occurred.

Suppose for a test sample point (day), the combined event of $E = \{(V_u, L_u, I_u), (V_u, L_u, I_d), (V_d, L_u, I_u)\}$ are observed in the previous day sessions. The multinomial probability of this happening conditioned on $Z = 1$ the next day is

$$P(E|Z=1) = n! \times \left[\frac{\left(\hat{\theta}_1^{x_1} \hat{\theta}_2^{x_2} \hat{\theta}_3^{x_3} \hat{\theta}_4^{x_4} \hat{\theta}_5^{x_5} \hat{\theta}_6^{x_6} \hat{\theta}_7^{x_7} \hat{\theta}_8^{x_8} \right)}{x_1! x_2! x_3! x_4! x_5! x_6! x_7! x_8!} \right]$$

where $x_1 + x_2 + \dots + x_8 = n$. Here, $n = 3$, $x_1 = 2$, $x_5 = 1$, and the remaining x_j 's are zeros. The multinomial probability of this happening conditioned on $Z = 0$ the next day is

$$P(E|Z=0) = n! \times \left[\frac{\left(\hat{\gamma}_1^{x_1} \hat{\gamma}_2^{x_2} \hat{\gamma}_3^{x_3} \hat{\gamma}_4^{x_4} \hat{\gamma}_5^{x_5} \hat{\gamma}_6^{x_6} \hat{\gamma}_7^{x_7} \hat{\gamma}_8^{x_8} \right)}{x_1! x_2! x_3! x_4! x_5! x_6! x_7! x_8!} \right]$$

The predictor of Z becomes

$$\arg \max_{Z=1,0} P(\{Z\}|E) = \arg \max_{Z=1,0} [P(E|Z)] \times \hat{P}(Z) \quad (4.4)$$

The accuracy of the Naïve Bayes algorithm is then evaluated based on the performance of each of the predictors of each sample point/case in the test dataset, using the trained $\hat{\theta}_j$, $\hat{\gamma}_j$ for $j = 1, 2, \dots, 8$ and also $\hat{P}(Z)$ from the training set data.

When sample data is split into training and test data sets, the Naïve Bayes algorithm is first applied to the training data set to compute the classification accuracy. In GNB, the data set possibly also provides estimation of the feature parameters of means and variances in order to compute the conditional probabilities for predicting the labels or targets. If there are no competing models or no hyperparameters to optimize, the estimated parameters are then applied to predict cases from the test features. If there are hyperparameters, such as the number of categories, we assume validation has been done and that we are working on a re-grouped training set using the optimal hyperparameters and applying the trained or estimated model to perform predictions on the test data set. The predictions are compared with the test labels to find the various performance metrics as well as the ROC AUC.

Naïve Bayes is typically used for classification or prediction of classes/types. However, the method may be adjusted for regression prediction of quantitative or numerical values. The GNB, making use of the Gaussian or normal probability distribution, is a type of Naïve Bayes method that can be applied to predict quantitative/numerical targets. Here, the targets Z^* are not classes/types. Z^* is a continuous variable, e.g., the next day stock index value. Suppose we use features such as volatility ($j = 1$), volume traded on index futures ($j = 2$), and lagged index change ($j = 3$) to help predict the next day index level Z^* . There are possibly several methods and research on new methods is still ongoing.

One method is to partition the continuous values of Z^* into a finite number of q intervals where the mid-point of an interval serves as discrete value approximations to the continuous values in the interval. Denote these discrete points as $\{Z_k^{**}\}$ for $k = 1, 2, \dots, q$. They are de facto classes or types. Then we use the features to predict which class/type will occur. For a sample point in the test data, from the right-side of Eq. (4.3), find $P(Z_k^{**})$, given the features, for the different possible values of Z_k^{**} . We may need to also estimate probability of the features. Then predict Z^*

using $\sum_{k=1}^q Z_k^{**} \times P(Z_k^{**})$. More sophisticated methods use non-parametric kernel density estimation. See also Frank *et al.* (1999).

4.2 k-Nearest Neighbors Algorithm

Another popular method for predicting classification is the k-NN algorithm. See Figure 4.1.

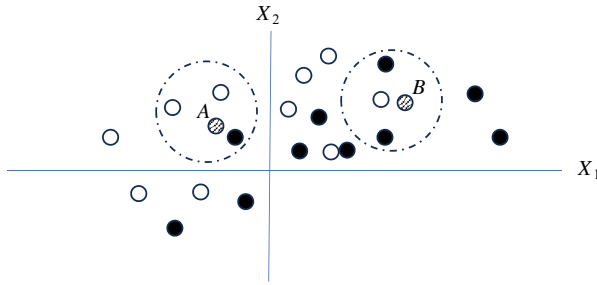


Figure 4.1. Example of k-NN algorithm.

In the diagram, the training data are 20 sample points represented by the black circles (target value/label 1) and the white circles (target value/label 0). Each sample point has two features represented by values in (X_1, X_2) . The feature values X_k are obtained by transforming the raw features using the standard scalar (where all transformed values X_k have mean 0 and standard deviation 1) or else by MinMaxScaler (where all transformed values X_k lie within $[0,1]$). This transformation is done over the whole sample for cross-sectional data. The transformation is useful in k-NN algorithm so that outliers or some features with much larger values on a larger scale would not create bias when the distances are measured.

Sometimes categorical variables are found in available features, e.g., $X_3 = \text{male}$ or else $X_3 = \text{female}$. While categorical variables can be a problem in k-NN since they do not operate on a quantitative basis with ratio or else relative comparisons, they may be usable if we provide a suitable dummy. In this case, we can expand gender to two dummy features, where $X_{31} = \alpha$ if male and 0 if female, and $X_{32} = 0$ if male and α if female. α should be chosen to be of the same order of magnitude as the rest of the standardized/scaled features. Too small an α close to zero may render the categorical/qualitative feature ineffective. Too large an α may unintentionally outsize the effect of categorical/qualitative feature.

Another example is X_4 = unemployed or blue-collar or else white-collar worker. In k -NN this may be fixed as $X_{41} = 1$ if unemployed and 0 otherwise, $X_{42} = 1$ if blue-collar and 0 otherwise, and finally $X_{43} = 1$ if white-collar and 0 otherwise. Using dummies will of course increase the number of features and expand the dimensionality of the dataset, which makes computations more tedious.

In the diagram, the test set data are entered one at a time. Sample point A from the test set is shown with the given features in X_{A1} and X_{A2} . The distance between any pair of points u and v is measured typically using the Euclidean distance which is

$$\sqrt{\sum_{k=1}^2 (X_{uk} - X_{vk})^2}$$

where (X_{u1}, X_{u2}) are the feature values of point u , and (X_{v1}, X_{v2}) are the feature values of point v . Sometimes the square of this distance is used as the distance metric. Other distance metrics include the Manhattan distance $\sum_{k=1}^2 |X_{uk} - X_{vk}|$, the Minkowski distance $\left(\sum_{k=1}^2 |X_{uk} - X_{vk}|^p\right)^{1/p}$ for some $p \geq 1$, and so on.

If $k = 1$ or using 1 Nearest-Neighbor algorithm, then only the nearest neighbor (of the training sample points) to A , i.e., shortest distance from A , will be picked. In this case, it is a black circle. The prediction for test case A is therefore black or target value/label 1. If $k = 3$ or using 3 Nearest-Neighbor algorithm, then only the nearest 3 neighbors to A (of the training sample points) will be picked. In this case, the 3 neighbors are within the dotted circle showing 2 white and 1 black circle. The probability of white circle for the test point is $2/3$ and probability of black circle for the test point is $1/3$. Since the majority is white (or the higher probability > 0.5), the 3-NN algorithm for test case A is white or target value/label of 0. Note that in k -NN algorithm, there is de facto no active computations done on the training data, so the training is sometimes called lazy learning or fitting. The use of the training data is delayed until a test query is made.

For test sample point B , using 1 Nearest-Neighbor, the prediction of B is white or label 0 since the white circle (of the training sample points) is nearest. But for 3-Nearest-Neighbor, the prediction of B is black or label 1 since within the dotted circle of the 3 closest neighbors (of the training sample points), there is a majority of black circles (or probability $2/3 > 0.5$ of black circles).

In using the k-NN algorithm, one important and about the only hyperparameter (there may be other hyperparameters if some other more complicated distance metric is used) to choose is k . In general, for k-NN algorithm, k is chosen to be odd so as to avoid a tie. We may have to re-run the algorithm several times for different hyperparameter values of k (tuning k) before we reach a satisfactory prediction or classification accuracy and good AUC. When k is decreased toward 1, the prediction for each case becomes unstable as prediction is based only on what happens to be the nearest neighbor of the test sample point. The prediction is myopic and may miss the correct affiliation as it may happen that there are more of the incorrect categories that are closer. But as k is increased, the prediction becomes more stable due to majority voting/averaging phenomenon, but this prediction can worsen as k becomes too large when the expanding neighborhood gets in more of both categories or types of the sample points so that it can flip sides. The k-NN algorithm also becomes slower as the number of features used for computing distances increases.

The hyperparameter of k could be selected using a validation data set. Another way of tuning k is to implement cross-validation, e.g., splitting the data set into 5 equal-sized groups and using 4 for training k while keeping 1 as the validation set. After optimal k is selected, the k-NN algorithm can then be applied to the test set to check the test sample prediction performance.

In the case of regression where k-NN is used to predict a target value based on the test sample point's features, the average of the values of the training cases in the nearest-neighbor dotted circle will be used as the prediction. Note that k-NN is a non-parametric algorithm. There is no need for any assumption of probability distributions on the features, and therefore avoids distributional misspecification errors.

If a test point sits in the k -neighborhood of m number of white circles and $k-m$ of black circles, then the predicted probability of a white circle of the test point may be construed as m/k while the probability of a black circle as predicted value is $1 - m/k$. Typically $m/k > 0.5$ (a hyperparameter threshold) implies prediction of white, otherwise black. But for constructing the Receiver Operating Characteristics (ROC) Curve in a binary classification, the hyperparameter threshold of white circles can be varied between 0 and 1 so that only when the predicted probability of white is higher than the threshold, the prediction is white, otherwise black. Thus, ROC can be constructed for the k-NN algorithm.

4.3 Support Vector Machine

Another algorithm for making class/type prediction is the SVM. We first provide the ideas behind the SVM method before we dive more into the theory.

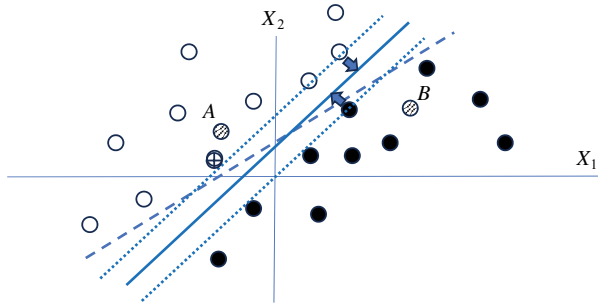


Figure 4.2. Example of SVM algorithm.

Suppose we try to predict the targets/labels/classes of the cases or instances represented as black or white circles. The diagram shows a simple example of cases with two features with values X_1 and X_2 . In the training of the SVM, a (linear or nonlinear) boundary is drawn to separate the two classes/groups as far as is possible. In this example, the colored circles are easily separated by a linear straight line.

As shown in Figure 4.2, the maximum separation is attained via the solid boundary line with positive slope. Both the colored circles are clearly separated. There, the nearest white circle to the boundary and the nearest black circle to the boundary have equal distances represented by the arrows. The linear SVM provides for a solid line boundary with the maximum margin (sum of perpendicular distances from the nearest feature points on either class to the line). Any shift in the boundary (see dashed line) would end up having at least one colored circle being closer to the boundary, e.g., the white circle shown with a cross. The latter means that the margin or gap has shrunk and is not an optimal boundary.

In general, this boundary is called the separating hyperplane or optimal decision plane (to separate the two classes). It separates the two classes as far as is possible. The separating hyperplane comes with two parallel hyperplanes (“margin” hyperplanes) on either side (denoted by the dotted lines) that touch the closest points (from the boundary) of the two classes. These closest points from both classes are called the support vectors as they help determine the boundary. In other words, if these

support vectors are shifted, then the boundary will also change. To recall, the perpendicular gap or distance between the dotted lines, or similarly, twice the distance between a supporting vector and the boundary line is called the margin.

Once the SVM has fitted the training data set, i.e., determined the boundary to separate the classes in the training data set, and supposing validation was done to fine-tune the hyperparameters, then the test data sample points, e.g., A and B, can be easily predicted as being in the white circle and the black circle group/class respectively. Of course, the prediction need not be always accurate as we suppose that the test sample target values are not observable during the prediction. The latter is called a generalization or out-of-sample error when trained/fitted model is used to predict the class of test sample points.

Now, suppose we use the same example of the small-sized 20 training data set seen in the k-NN algorithm. Figure 4.3 shows that it is not possible to find a linear straight line to separate the two classes/groups of colored circles. When we try to draw a straight line to keep the black circles on one side (see dotted lines in the diagram), it will always rope in some white circles as well. Vice-versa for trying to keep the white circles on one side.

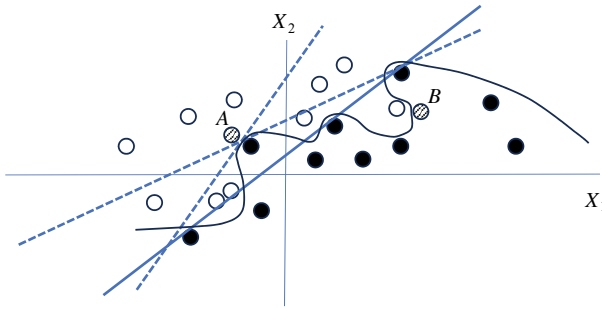


Figure 4.3. Example of SVM with nonlinear boundary.

In SVM there are at least two ways to proceed. Firstly, we could still apply linear boundary as in the solid line but allow some classification errors in training. In this case, there is one mis-classified black circle (black circle in the white circled upper-half plane) and one mis-classified white circle (white circle in the black circled lower-half plane). Secondly, we may be able to find a polynomial function (shown in the curved line) to separate the colored circles. This involves transforming the original N training data points $(X_1^{(1)}, X_2^{(1)})$, $(X_1^{(2)}, X_2^{(2)})$, $(X_1^{(3)}, X_2^{(3)})$, ..., $(X_1^{(N)}, X_2^{(N)})$ to new points $\phi(X_1^{(1)}, X_2^{(1)})$, $\phi(X_1^{(2)}, X_2^{(2)})$, $\phi(X_1^{(3)}, X_2^{(3)})$, ..., $\phi(X_1^{(N)}, X_2^{(N)})$ that may be in the

same 2-dimensional space or in a larger dimensional space. What is significant here is that under ϕ transformation, the new points $\phi(X_1^{(1)}, X_2^{(1)})$, $\phi(X_1^{(2)}, X_2^{(2)})$, $\phi(X_1^{(3)}, X_2^{(3)})$, ..., $\phi(X_1^{(N)}, X_2^{(N)})$ becomes linearly separable. After the training or fitting of a decision boundary, a test data sample point t 's features are subject to the transformation so that $\phi(X_1^{(t)}, X_2^{(t)})$ can be checked if it falls in which separable groups to predict its affiliation.

Often there are complicated grouping patterns such as the following (see Figure 4.4) whereby a transformation must be applied to the features to separate the classes. This transformation involves a higher-dimensional feature space. Such a transformed feature space with a higher dimension than the original one for the purpose of separating the classes/types is often called a “kernel trick”.

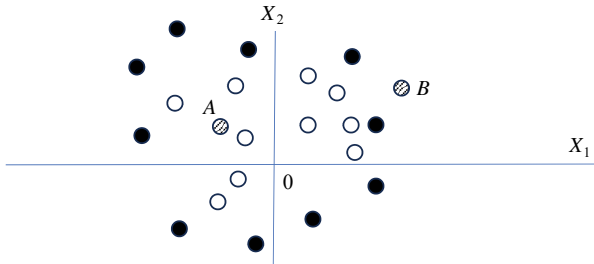


Figure 4.4. Example of SVM with non-separating pattern.

As a specific (need not be optimal) solution, we could use the transformation $\phi(X_1^{(j)}, X_2^{(j)}) = (X_1^{(j)}, X_2^{(j)}, X_3^{(j)})$ for the j th training data set sample point, where the third coordinate is $X_3 = X_1^2 + X_2^2$. By extending the feature space, we can obtain a 3-dimensional representation of the targets/labels in Figure 4.5.

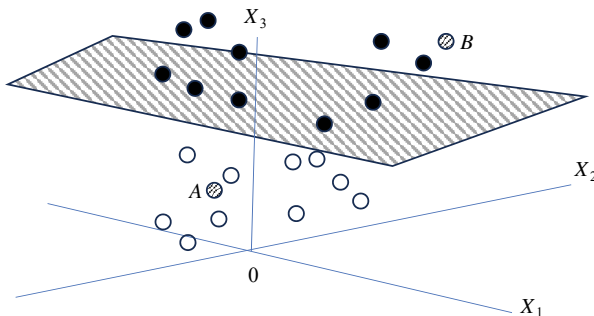


Figure 4.5. Kernel trick.

Now the black circles will have higher X_3 values since they lie on an outer circle in the $X_1 - X_2$ axes. (Drawing may not be exactly to scale.) In the 3-dimensional representation, a plane (shaded) can be seen to separate the black circles from the white circles below. Clearly now A and B from the test sample can be easily predicted as belonging to the white and black circle groups respectively.

Depending on the problem, the binary or dichotomous classes could be diseased patient versus non-diseased patient using medical and physical attributes to predict, spam or no-spam emails using natural language processing of the words to predict, rating R or non-R classification using film visual features, detection of presence or absence of specific object using the object features, weather prediction of rain or no-rain using weather conditions as features, stock decrease or no decrease using market conditions and the firm conditions as features, and so on. In the categorical nature of the class/type, there is no loss of generality in assigning numerical value $+1$ or -1 to the j th sample point category/type, i.e., let $Y_j = +1$ or -1 depending on which of the binary class/type.

Consider again the two-dimensional features in an earlier example. We want to draw a hyperplane (a straight line in 2-dimension) to separate the black circles (target/label $Y_j = -1$) from the white circles (target/label $Y_j = +1$). Moreover, we want a straight line with the widest margin (wide “street”) with the two “margin” lines (shown as bold lines) touching the support vectors or the points on the lines. The double-headed line in Figure 4.6 shows the extent of the margin.

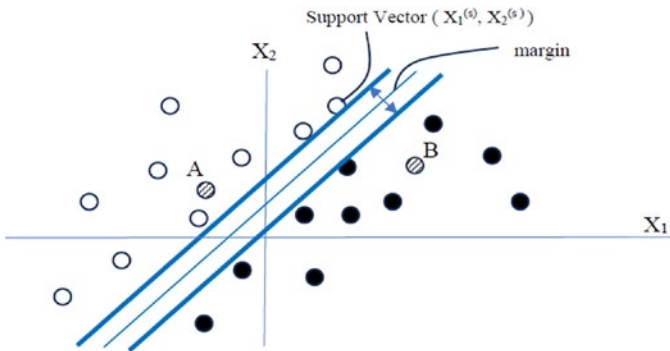


Figure 4.6. Support vector and margin.

In the 2-D case here, the straight lines can be represented algebraically as equations $aX_1 + bX_2 = c$ for different scalar values of c . This is a more

general representation (allowing b to be possibly 0) of the familiar $X_2 = c/b - a/b X_1$ where slope is $-a/b$ and the intercept on the X_2 axis is c/b .

Let the supporting vector on white circle lie on line $AX_1 + BX_2 - C = +\delta$. Hence, slope of “margin” line is $-A/B$. Intercept of line is $(C + \delta)/B$. Let the supporting vector on black circle lie on line $AX_1 + BX_2 - C = -\delta$. Hence, slope of margin line is $-A/B$. Intercept of line is $(C - \delta)/B$. The separating line in the middle of the “margin” lines is $AX_1 + BX_2 - C = 0$. ($B > 0$, $C > 1$, and $A < 0$ so that the slopes and intercepts are correct as in the diagram.) The white circles or points have features that obey $AX_1 + BX_2 - C \geq +\delta$, i.e., the white circles lie above (on the positive side of) the “margin” line $AX_1 + BX_2 - C = +\delta$. The black circles or points have features that obey $AX_1 + BX_2 - C \leq -\delta$, i.e., the black circles lie below (on the negative side of) the “margin” line $AX_1 + BX_2 - C = -\delta$.

Consider any perpendicular line from a point on $AX_1 + BX_2 - C = +\delta$ to a point on $AX_1 + BX_2 - C = -\delta$. The slope of this perpendicular line is $-1/(-A/B) = B/A$. One such perpendicular line is $X_2 = [B/A] X_1$ or $AX_2 - BX_1 = 0$. The intersection of this perpendicular with line $AX_1 + BX_2 - C = +\delta$ is the point with co-ordinates $(A[C + \delta]/[A^2 + B^2], B[C + \delta]/[A^2 + B^2])$. The intersection of this perpendicular with line $AX_1 + BX_2 - C = -\delta$ is the point with co-ordinates $(A[C - \delta]/[A^2 + B^2], B[C - \delta]/[A^2 + B^2])$. Therefore, the distance between these two intersection points is the shortest distance between the two margin lines. This distance is

$$\sqrt{\left(\frac{2\delta A}{A^2 + B^2}\right)^2 + \left(\frac{2\delta B}{A^2 + B^2}\right)^2} = \sqrt{\frac{(2\delta)^2}{A^2 + B^2}} = \frac{2\delta}{\sqrt{A^2 + B^2}}$$

Let vector $W = (A, B)$ and the separating line can be written as $W \bullet X - C = 0$, where X is vector (X_1, X_2) , and operation “ $\bullet$ ” denotes the dot product. Then $W \bullet W = A^2 + B^2$ where $W \bullet W$ is also matrix multiplication of transpose of W with W . In Euclidean space, $W \bullet W = \|W\|^2$ under Euclidean norm $\|\cdot\|$. So, the perpendicular distance between two margin lines is $2\delta/\|W\|$ (for the two-dimensional case here). The task in this linear SVM is to find the minimum $\|W\|$ for the maximum margin $2\delta/\|W\|$ of the wide “street” sandwiched between the two “margin” lines. This is to separate the two sets of colored circles as much as possible in order to provide a clear decision boundary for test or generalized cases.

More formally, the theory of SVM can be explained as follows.¹ Suppose a training sample consists of N number of data or sample points. Each sample point has p features and the j th sample point can be characterized in Euclidean space as a p -dimensional vector $X_j \equiv (X_1^{(j)}, X_2^{(j)}, X_3^{(j)}, \dots, X_p^{(j)}) \in R^p$. In addition, each point belongs to one of two classes denoted by $Y_j = +1$ or $Y_j = -1$. (If subscript j is not highlighted, then the point is any from the training data set.)

Let $W \in R^p$ be a p -dimensional vector representing coefficients of or weights on a p -dimensional vector X , and W_0 denote a constant scalar. Suppose the separating hyperplane is $W^T X + W_0 = 0$. Then a unit vector perpendicular² to the separating hyperplane is $W/\|W\|$. Similarly, $-W/\|W\|$ is an opposite unit vector perpendicular to the same separating hyperplane.

If X_j is a point in R^p and X_0 is its perpendicular projection onto the separating hyperplane (X_0 is on the separating hyperplane), then $X_j = X_0 + \gamma_j W/\|W\|$ where $\gamma_j > 0$ is the distance of the perpendicular projection of X_j . Let X_j be a sample point or vector on the positive side or class $Y_j = +1$.

Now, $X_0 = X_j - \gamma_j W/\|W\|$. Since X_0 is on the separating hyperplane, $W^T X_0 + W_0 = 0$. Thus, $W^T (X_j - \gamma_j W/\|W\|) + W_0 = 0$. Or, $W^T X_j + W_0 = \gamma_j W^T W/\|W\| = \gamma_j \|W\|$. Note that $\gamma_j \|W\|$ is equal to δ in the earlier two-dimensional case.

Hence, for the points on the positive side with class $Y_j = +1$,

$$\gamma_j = \frac{W^T X_j + W_0}{\|W\|}$$

¹See also Vladimir N. Vapnik (1998). *Statistical Learning Theory*. John Wiley, and V. Vapnik and R. Izmailov (2018). Rethinking statistical learning theory: learning using statistical invariants. *Machine Learning*, 1–43. V. Vapnik is a pioneer of the SVM methods.

²Cosine law states that in a triangle with sides having lengths a, b, c , with c the largest, then $c^2 = a^2 + b^2 - 2ab \cos \theta$ where θ is the angle between the sides having lengths a and b . But vectors u and v are such that the vector $v - u$ forms the third side of a triangle. Then $\|v - u\|^2 = \|u\|^2 + \|v\|^2 - 2\|u\|\|v\|\cos \theta$ by the cosine law. Or, $(v - u) \cdot (v - u) = u \cdot u + v \cdot v = 0$ if and only if (iff) u and v are perpendicular, i.e., $\cos \theta = 0$. Hence, $u \cdot v = 0$ iff u and v are perpendicular. Now, any two points X_a and X_b on the separating hyperplane obeys $W^T X_a + W_0 = 0$ and $W^T X_b + W_0 = 0$, so $W^T (X_a - X_b) = 0$. But $(X_a - X_b)$ is a vector lying on the hyperplane. Hence, W is a perpendicular vector to $(X_a - X_b)$. Thus W is perpendicular to the separating hyperplane. W has length $\sqrt{(W \cdot W)} = \|W\|$. Hence, $W/\|W\|$ has unit length.

If, however, X_j is a sample point or vector on the negative side or class $Y_j = -1$. Then $X_j = X_0 - \gamma_j W/\|W\|$ where $\gamma_j > 0$ is the distance of the perpendicular projection of X_j . Now, $X_0 = X_j + \gamma_j W/\|W\|$. Since X_0 is on the separating hyperplane, $W^T X_0 + W_0 = 0$. Thus, $W^T(X_j + \gamma_j W/\|W\|) + W_0 = 0$. Or, $W^T X_j + W_0 = -\gamma_j W^T W/\|W\| = -\gamma_j \|W\|$.

Hence, for the points on the negative side with class $Y_j = -1$,

$$\gamma_j = \frac{W^T X_j + W_0}{\|W\|}$$

Assume the classes are separable by the separating hyperplane. For class $Y_j = +1$ points on the positive side of the separating hyperplane, we let $\gamma_j = \frac{W^T X_j + W_0}{\|W\|} > 0$ be a positive signed distance. For class $Y_j = -1$ points on the negative side of the separating hyperplane, we let $\gamma_j = \frac{W^T X_j + W_0}{\|W\|} < 0$ be a negative signed distance. We can call the distances $\gamma_j > 0$ as positive SVM scores, and the distances $\gamma_j < 0$ as negative SVM scores.

Then for the points on the positive side with class $Y_j = +1$,

$$Y_j \gamma_j = Y_j \frac{W^T X_j + W_0}{\|W\|} > 0$$

For the points on the negative side with class $Y_j = -1$, negative Y_j multiplied by negative $\frac{W^T X_j + W_0}{\|W\|}$ gives

$$Y_j \gamma_j = Y_j \frac{W^T X_j + W_0}{\|W\|} > 0$$

For all sample points j in the training data set, their absolute perpendicular distances $\gamma_j = \left| \frac{W^T X_j + W_0}{\|W\|} \right|$ from the separating hyperplane should be above or equal to a minimum $\gamma > 0$. Twice γ is the margin of the SVM. Hence,

$$Y_j \frac{W^T X_j + W_0}{\|W\|} \geq \gamma \text{ for all } j$$

But a constant c times W , W_0 gives $(cW^T X_j + cW_0)/\|cW\| = (W^T X_j + W_0)/\|W\|$. Hence, there is no unique solution (W, W_0) linking a support vector X_s to minimum projection distance γ . Suppose the solution γ exists and is unique given the sample data set. We can find a unique

solution (W, W_0) by fixing $\|W\| = 1/\gamma$ given the data set. This is fixing $\|W\| \gamma = 1$.

Suppose the solution to a SVM separating hyperplane $W^T X + W_0 = 0$ is

$$\max_{W, W_0, \gamma} \gamma \quad \text{subject to } Y_j \frac{W^T X_j + W_0}{\|W\|} \geq \gamma \quad \text{for all } j$$

For unique solution (W, W_0) , the fixing of $\|W\| \gamma = 1$ allows the program to be re-written as:

$$\max_{W, W_0} 1/\|W\| \quad \text{subject to } Y_j(W^T X_j + W_0) \geq 1 \quad \text{for all } j.$$

This is equivalent to a quadratic program:

$$\min_{W, W_0} \frac{1}{2} \|W\|^2 \quad \text{subject to } Y_j(W^T X_j + W_0) \geq 1 \quad \text{for all } j. \quad (4.5)$$

To solve the constrained optimization shown above more readily, we make use of the Lagrange duality. This involves the Lagrange primal form and the dual Lagrange form to facilitate numerical computations of a solution. The Lagrange duality concept and solution is explained in the following sub-section.

4.4 Lagrange Duality Solution

A numerical solution of W, W_0 (each is p -dimensional) may be too cumbersome as the inequality constraints $Y_j(W^T X_j + W_0) \geq 1$ needs to be checked for each j (with $p \times 1 X_j$ and $1 \times 1 Y_j$) given a potential solution (W, W_0) . For this context, a neater solution is to make use of the Lagrangian. Recall program (4.5) is to maximize the margin of the SVM with the separating hyperplane as $W^T X + W_0 = 0$ for some optimal W and W_0 . The primal representation of program Eq. (4.5) is

$$\min_{W, W_0} \max_{\lambda \geq 0} \frac{1}{2} \|W\|^2 + \sum_{j=1}^N \lambda_j (1 - Y_j(W^T X_j + W_0)) \quad (4.6)$$

where $N \times 1$ vector $\lambda = (\lambda_1, \lambda_2, \lambda_3, \dots, \lambda_N)^T$, and $\lambda_j \geq 0$ (defined to be positive) is a Lagrange multiplier on the inequality constraint $1 - Y_j(W^T X_j + W_0) \leq 0$. The Lagrange multiplier acts like a penalty if the constraint is violated such as when $1 - Y_j(W^T X_j + W_0) > 0$. In this case $\lambda_j > 0$ is selected to be

sufficiently high, hence the max function, to penalize the large positive value of $1 - Y_j(W^T X_j + W_0)$, in order to force the subsequent minimization over (W, W_0) to then ensure the constraint apply. λ_j is also the “shadow price” of the constraint $1 - Y_j(W^T X_j + W_0) \leq 0$. It is the amount of decrease in the objective function $\frac{1}{2}\|W\|^2$ with a marginal relaxation of the constraint. It would be incorrect to use minimization over λ_j since a violation in $1 - Y_j(W^T X_j + W_0) > 0$ means $\lambda_j = 0$. This is contradictory as zero shadow price means the constraint is already met, which is not the case here. Note that the Lagrangian and use of the Lagrange multipliers allow optimization in principle over an unconstrained program.

The dual Lagrange form of Eq. (4.6) is

$$\max_{\lambda \geq 0} \min_{W, W_0} \frac{1}{2} \|W\|^2 + \sum_{j=1}^N \lambda_j (1 - Y_j (W^T X_j + W_0)) \quad (4.7)$$

Note that we typically can construct a dual for a primal program. The dual program $\max_{\lambda \geq 0} \min_{\Theta} L(\Theta, \lambda) \leq \min_{\Theta} \max_{\lambda \geq 0} L(\Theta, \lambda)$, the primal program. An example in which inequality could occur is when the primal goes to infinity with a $\lambda_j \uparrow$ due to a stubborn violation of constraint. The dual Lagrange form in Eq. (4.7) has a finite solution where

$$\min_{W, W_0 | \lambda} \frac{1}{2} \|W\|^2 + \sum_{j=1}^N \lambda_j (1 - Y_j (W^T X_j + W_0)) \quad (4.8)$$

is performed with a given λ as a kind of first stage optimization before the second stage optimization over $\lambda \geq 0$ (some elements of λ can be $= 0$). For our context of solving the SVM problem, when an optimal interior solution exists,

$$\max_{\lambda \geq 0} \min_{\Theta} L(\Theta, \lambda) = \min_{\Theta} \max_{\lambda \geq 0} L(\Theta, \lambda).$$

for a general parameter Θ and Lagrangian L .

In the first stage solution in Eq. (4.8), the first order conditions for minimum are:

$$\frac{\partial L(\lambda)}{\partial W} = W - \sum_{j=1}^N \lambda_j Y_j X_j = 0$$

and

$$\frac{\partial L(\lambda)}{\partial W_0} = \sum_{j=1}^N \lambda_j Y_j = 0$$

Fixing these optimal values for W^* in Eq. (4.8), we have the second stage optimization in $\lambda \geq 0$:

$$\begin{aligned}
& \max_{\lambda \geq 0 | W^*} \frac{1}{2} \left(\sum_{j=1}^N \lambda_j Y_j X_j \right) \bullet \left(\sum_{j=1}^N \lambda_j Y_j X_j \right) + \sum_{j=1}^N \lambda_j \left(1 - Y_j \left(\sum_{j=1}^N \lambda_j Y_j X_j \right)^T X_j \right) \\
& \quad - W_0^* \sum_{j=1}^N \lambda_j Y_j \\
& \equiv \max_{\lambda \geq 0 | W^*} \frac{1}{2} \left(\sum_{j=1}^N \sum_{k=1}^N \lambda_j Y_j Y_k X_j^T X_k \right) + \sum_{j=1}^N \lambda_j - \sum_{j=1}^N \lambda_j Y_j \left(\sum_{j=1}^N \lambda_j Y_j X_j \right)^T X_j \\
& \equiv \max_{\lambda \geq 0 | W^*} \frac{1}{2} \left(\sum_{j=1}^N \sum_{k=1}^N \lambda_j \lambda_k Y_j Y_k X_j^T X_k \right) + \sum_{j=1}^N \lambda_j - \sum_{j=1}^N \sum_{j=1}^N \lambda_j Y_j X_j^T X_j \\
& \equiv \max_{\lambda \geq 0 | W^*} \sum_{j=1}^N \lambda_j - \frac{1}{2} \sum_{j=1}^N \sum_{k=1}^N \lambda_j \lambda_k Y_j Y_k X_j^T X_k \tag{4.9}
\end{aligned}$$

subject to $\sum_{j=1}^N \lambda_j Y_j = 0$ and $\lambda_j \geq 0$ for all j .

The numerical solution of dual in Eq. (4.9) is tractable given observations $Y_j Y_k X_j^T X_k$ for every j, k . The solutions λ_j^* are then applied to obtain $W^* = \sum_{j=1}^N \lambda_j^* Y_j X_j$. Note that W^* is solution to the separating hyperplane $W^{*T} X + W_0^* = 0$. Note that for uniqueness of solution in W^* , W_0^* , with setting $\gamma \|W^*\| = 1$, and $Y_j (W^{*T} X_j + W_0^*) \geq 1$ for all j , we have support vector(s) on the positive side ($Y_j = +1$) “margin” hyperplane as $W^{*T} X_j + W_0^* = 1$ or $\min(W^{*T} X_j + W_0^* | Y_j = +1) = 1$, or $\min(W^{*T} X_j | Y_j = +1) = 1 - W_0^*$.

Support vector(s) on the negative side ($Y_j = -1$) “margin” hyperplane are $W^{*T} X_j + W_0^* = -1$ or $\max(W^{*T} X_j + W_0^* | Y_j = -1) = -1$, or $\max(W^{*T} X_j | Y_j = -1) = -1 - W_0^*$. Then W_0^* can be found as

$$W_0^* = -[\min(W^{*T} X_j | Y_j = +1) + \max(W^{*T} X_j | Y_j = -1)]/2$$

The score of any sample point can be measured as $W^{*T} X_j + W_0^*$ which is the distance of $\gamma_j = \frac{W^{*T} X_j + W_0^*}{\|W^*\|}$ in terms of $\gamma = 1/\|W^*\|$, hence $\gamma_j / \gamma = W^{*T} X_j + W_0^*$. This score can be positive (on the positive side of the separating hyperplane) or negative (on the negative side of the separating hyperplane).

When it comes to prediction of a class given a generalized data point with features X_g (including predictions on test data), the SVM score of the generalized data point is measured using the trained SVM, i.e., score = $W^{*T} X_g + W_0^*$. If the score is $\geq +1$, the point or case is predicted to be in the class $Y_g = +1$. If the score is ≤ -1 , the point or case is predicted to be in the class $Y_g = -1$.

4.4.1 Non-linear kernels in SVM

So far we have used a linear separating hyperplane, $W^T X_0 + W_0 = 0$ based on the p -dimensional feature vector $X_k \equiv (X_1^{(k)}, X_2^{(k)}, X_3^{(k)}, \dots, X_p^{(k)}) \in R^p$. Suppose the optimization in Eq. (4.9) fails as a linear separating hyperplane cannot be found. In Eq. (4.9), the term $X_i^T X_j$ involving the dot product of X_i and X_j is called a linear kernel on X_i and X_j .

We saw earlier how we can possibly use a nonlinear transformation (nonlinear kernel) to transform the features or feature vectors X_i and X_j . Nonlinear SVM kernels that can be used for classification include the following.

The polynomial kernel $K(X_i, X_j) = (\gamma X_i \bullet X_j + c)^d$ where the degree of the polynomial d , gamma γ , and c are hyperparameters. Sometimes a simpler version of $\gamma = 1$ or $\gamma = 1/N$ is used as a default case in many programs. As an illustration, suppose point X_i is $(x_1^{(i)}, x_2^{(i)})$ with two dimensions. Let $K(X_i, X_j) = (X_i \bullet X_j + 1)^2$ where $d = 2$. Then $K(X_i, X_j) = (x_1^{(i)}x_1^{(j)} + x_2^{(i)}x_2^{(j)} + 1)^2 = 2 x_1^{(i)}x_1^{(j)} + 2 x_2^{(i)}x_2^{(j)} + 2 x_1^{(i)}x_1^{(j)}x_2^{(i)}x_2^{(j)} + (x_1^{(i)}x_1^{(j)})^2 + (x_2^{(i)}x_2^{(j)})^2 + 1 = (\sqrt{2} x_1^{(i)}, \sqrt{2} x_2^{(i)}, \sqrt{2} x_1^{(i)}x_2^{(i)}, x_1^{(i)2}, x_2^{(i)2}, 1) \bullet (\sqrt{2} x_1^{(j)}, \sqrt{2} x_2^{(j)}, \sqrt{2} x_1^{(j)}x_2^{(j)}, x_1^{(j)2}, x_2^{(j)2}, 1)$. Clearly, the polynomial kernel now expands X_i from two to six dimensions. The higher dimension allows for more space in-between sample point features and hence easier linear separation; this is called the “kernel trick”.

We can let the dimension expansion (enlarging feature space) be transformation function $\phi(x_1^{(i)}, x_2^{(i)}) = (\sqrt{2} x_1^{(i)}, \sqrt{2} x_2^{(i)}, \sqrt{2} x_1^{(i)}x_2^{(i)}, x_1^{(i)2}, x_2^{(i)2}, 1)$. Hence, polynomial kernel on 2-dimensional X_i, X_j , $K(X_i, X_j) = \phi(x_1^{(i)}, x_2^{(i)}) \bullet \phi(x_1^{(j)}, x_2^{(j)}) = \phi(X_i) \bullet \phi(X_j)$. The latter is a linear kernel on $\phi(X_i), \phi(X_j)$.

Without elaborating on some highly technical details, we state that there is a Mercer’s Theorem³ showing that for symmetrical real-valued

³Mercer, J. (1909). Functions of positive and negative type and their connection with the theory of integral equations. *Philosophical Transactions of the Royal Society A*, 209, (441–458): 415–446.

$K(X_i, X_j)$, a representation $K(X_i, X_j) = \phi(X_i) \bullet \phi(X_j)$ for function $\phi(\cdot)$ can be found.

Using a non-linear kernel $K(X_i, X_j)$, we are *de facto* trying to solve for a (linear) separating hyperplane on linear $\phi(X_i)$, $\phi(X_j)$ for some function $\phi(\cdot)$ associated with the non-linear kernel. See Eq. (4.10).

$$\max_{\lambda \geq 0 | W^*} \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j Y_i Y_j \phi(X_i) \bullet \phi(X_j) \quad (4.10)$$

Another popular non-linear kernel is the Gaussian Radial Basis Function (RBF) $K(X_i, X_j) = \exp(-\gamma \|X_i - X_j\|^2)$ or $\exp(-\gamma(X_i - X_j) \bullet (X_i - X_j))$ where $\gamma = 1/(2\sigma^2)$. Here, γ (gamma) > 0 is a hyperparameter and σ need not be constrained to be standard deviation of the standardized features. In Sklearn, by default, $\gamma = 1/(N\sigma^2)$. Another example would be to set γ to a positive number. High gamma leads in general to more accuracy in classification but perhaps overfitting and more bias. Notice that if we expand the exponential function into an infinite Taylor series, then the RBF is like expanding $K(X_i, X_j)$ into an infinite number of dimensions involving $\|X_i - X_j\|^4$, $\|X_i - X_j\|^6$, $\|X_i - X_j\|^8$, and so on. Hence, the RBF is often able to perform well on highly nonlinear data points as it can project them to a higher number of dimensions. Another non-linear kernel is the sigmoid or hyperbolic tangent kernel $K(X_i, X_j) = \tanh(\gamma X_i \bullet X_j + c)$ where gamma γ and c are hyperparameters. Sometimes a simpler version of $\gamma = 1$ or $\gamma = 1/N$ is used as a default case in many programs. This kernel form is popular in neural networks but is generally not as effective in SVM.

The choice of what type of kernel depends on the nature of the data and the problem. Linear kernel is used when the data is approximately linearly separable. Polynomial kernel is used when the data has a curved border. Gaussian kernel is used when the data appear to have complicated overlaps and no clear boundaries.

In general, when we apply a particular kernel function as in the constrained optimization in Eq. (4.10), we check for the performance results in prediction accuracy and the classification errors on the validation or test sample data set. There is no need to explicitly determine or check the transformation function of $\phi(\cdot)$ in $K(X_i, X_j) = \phi(X_i) \bullet \phi(X_j)$.

For SVM in classification prediction, the training data set is used to estimate optimally the separating hyperplane parameters with pre-selected sets of kernel parameters and associated hyperparameters. Then a validation set can be used to fine-tune the hyperparameters by seeking best

performance metrics. After that, the fine-tuned hyperparameters can be used to train a re-grouped or enlarged training data set, and then this trained model is applied to the test set to check performance metrics for robustness.

4.4.2 Linear non-separability in SVM

Suppose the optimization in Eq. (4.9) still fails as a linear separating hyperplane cannot be found despite suitable transformations of features or feature vector X_i into $\phi(X_i)$ with perhaps a higher dimension. We can see this in Figure 4.7 whereby any transformations $\phi(X_i)$ of a sample point into two dimensions (Z_1, Z_2) could not allow linear separation.

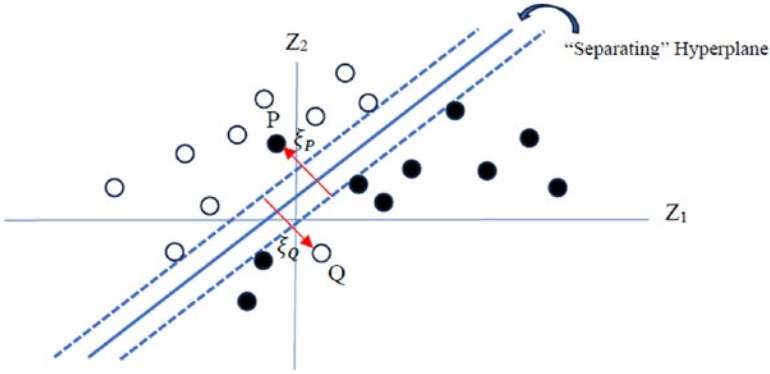


Figure 4.7. Non-separability in SVM.

In general, denote the sample point feature vector as Z_j . In forming the “separating” hyperplane or optimal decision boundary, there will still be support vectors touching the “margin” lines whereby according to (4.7), $Y_j(W^{*T}Z_j + W_0^*) = 1$. And most sample points would have $Y_j(W^{*T}Z_j + W_0^*) > 1$. However, as seen in Figure 4.7, there would be points such as P , Q that violate the constraints. In particular, the distance of P away from its “margin” into the opposite side is ξ_P , and the distance of Q away from its “margin” into the opposite side is ξ_Q .

To avoid such violations, we put slack variables ξ_j into the constraints such that $Y_j(W^{*T}Z_j + W_0^*) \geq 1 - \xi_j$, and $\xi_j \geq 0$ for all j . This is also called soft margin classification. If all $\xi_j = 0$, then we are back to the solution with separation or hard margin classification.

The dual program to solve in the context of allowing some points to have $\xi_j > 0$, similar to Eq. (4.8), is

$$\min_{W, W_0, \xi | \lambda, \mu} \frac{1}{2} \|W\|^2 + \alpha \sum_{j=1}^N \xi_j + \sum_{j=1}^N \lambda_j (1 - \xi_j - Y_j (W^T Z_j + W_0)) - \sum_{j=1}^N \mu_j \xi_j \quad (4.11)$$

given λ and μ , where $\alpha > 0$ is a hyperparameter denoting the penalty to put on $\sum_{j=1}^N \xi_j > 0$. $N \times 1$ vector $\lambda = (\lambda_1, \lambda_2, \lambda_3, \dots, \lambda_N)^T$, and $\lambda_j \geq 0$ (defined to be positive) is a Lagrange multiplier on the inequality constraint $Y_j (W^T Z_j + W_0) \geq 1 - \xi_j$ for all j . $N \times 1$ vector $\mu = (\mu_1, \mu_2, \mu_3, \dots, \mu_N)^T$, and $\mu_j \geq 0$ (defined to be positive) is a Lagrange multiplier on the inequality constraint $\xi_j \geq 0$ for all j . $N \times 1$ vector $\xi = (\xi_1, \xi_2, \xi_3, \dots, \xi_N)^T$. A large penalty α tends to reduce $\sum_{j=1}^N \xi_j$ as a choice, i.e., small error rates of training misclassifications, while a small penalty α and large $\sum_{j=1}^N \xi_j$ results in larger margin and large error rates of training misclassifications. α is to be fine-tuned for optimal performance in a validation data set before final testing is done. If we re-arrange the objective function as $\frac{1}{2} \|W\|^2 + \alpha' \sum_{j=1}^N (1 - Y_j (W^T Z_j + W_0) + \eta^2)$ for some real η and given λ_j 's, or equivalently in the minimizer to find W , W_0 , $\frac{1}{2\alpha'} \|W\|^2 + \sum_{j=1}^N (1 - Y_j (W^T Z_j + W_0))^+$ where the superscript “+” denotes the argument taking a value larger or equal to zero, then $(1 - Y_j (W^T Z_j + W_0))^+$ is called a hinge loss function.

Program Eq. (4.11) can be re-written as:

$$\min_{W, W_0, \xi | \lambda, \mu} \frac{1}{2} \|W\|^2 + \sum_{j=1}^N \xi_j (\alpha - \lambda_j - \mu_j) + \sum_{j=1}^N \lambda_j (1 - Y_j (W^T Z_j + W_0)) \quad (4.12)$$

The first order conditions for minimum are:

$$\frac{\partial L(\lambda, \mu)}{\partial W} = W - \sum_{j=1}^N \lambda_j Y_j Z_j = 0$$

$$\frac{\partial L(\lambda, \mu)}{\partial W_0} = \sum_{j=1}^N \lambda_j Y_j = 0$$

and
$$\frac{\partial L(\lambda, \mu)}{\partial \mu} = \alpha - \lambda - \mu = 0$$

Fixing these optimal values for W^* in Eq. (4.12), we have the second stage optimization in $\lambda \geq 0$:

$$\max_{\lambda \geq 0 | W^*} \sum_{j=1}^N - \frac{1}{2} \sum_{j=1}^N \sum_{k=1}^N \lambda_j \lambda_k Y_j Y_k Z_j^T Z_k \quad (4.13)$$

subject to $\sum_{j=1}^N \lambda_j Y_j = 0$, $\alpha = \lambda_j + \mu_j$, $\lambda_j \geq 0$ and $\mu_j \geq 0$ for all j . The constraints here can be simplified to $\sum_{j=1}^N \lambda_j Y_j = 0$ and $\alpha \geq \lambda_j \geq 0$ for all j . The solutions W^* , W_0^* can be similarly obtained as in the separating context. Validation or test sample points on the positive side of the “separating” hyperplane $W^{*T} Z_j + W_0^* = 0$ are predicted as $Y_j = +1$, whereas points on the negative side of the “separating” hyperplane are predicted as $Y_j = -1$.

4.4.3 Other aspects of SVM

The SVM method does not explicitly compute the probability of a test point Z being in one class/type or another. However, we can use a logit model to estimate the probability given the predicted class being $+1$ or -1 against the features used in SVM for the prediction. A common method is to use the SVM score for each sample point as the feature for predicting the class using the logit model. The predicted probabilities can then be used to build the ROC curve given different probability thresholds.

The SVM method is natively a binary classification algorithm as its separating hyperplane offers two obvious sides of the classifications. However, SVM may be extendible to multi-class separation although the methods typically involve multiple binary classifications in a heuristic approach. These approaches are applicable to ML algorithms that are natively binary in its classifications such as the logistic or logit prediction.

Suppose there are $n > 2$ classes for prediction. In the One-vs.-Rest (OvR) or One-vs.-All (OvA) approach, a SVM binary classification is performed for each of the n classes versus all the remaining classes combined. The training data set will be utilized n times where each time, it is a prediction of class j versus the rest (non class j). This builds up n prediction models. Let the trained j th model provide predicted probability P_j of a generalized validation or test case or instance being of class j

(and $1 - P_j$) being not of class j . For all $j = 1, 2, \dots, n$, find j with the highest P_j . That j is the predicted class for the instance in the multi-class prediction problem.

In the same n class prediction problem, another heuristic approach is the One-vs.-One (OvO) method. Here, a binary classification is performed for each pair of the n classes. There are $n(n - 1)/2$ such pairs. The classification model training for each pair will use only data that contain these two classes. In each of the pair classifications, the predicted class is noted for an instance from the validation or test data set. Of the $n(n - 1)/2$ events each with a predicted class, the class with the highest number of times of being predicted (between 0 and $n - 1$) is the predicted class for the instance in the multi-class prediction problem. For other details, see Wang (2005).

4.5 Support Vector Regression

When the SVM method is used for regression in making numerical prediction, the method is modified to an algorithm called SVR. Just as in multiple linear regression where the objective is to minimize the errors (via metric such as mean squared distances) between the predicted hyperplane and the targets of the training data points, the objective in SVR is also to minimize errors between the predicted hyperplane and the training data targets but setting the errors to non-zero values only when they exceed a given hyperparameter epsilon $\varepsilon > 0$. Within this epsilon space or tube, no penalty is associated with the error or deviation of actual target value Y_j from the fitted hyperplane $\hat{Y}_j = W \bullet X_j - W_0$.

Just as in SVM, the SVR finds a maximum margin (equivalently minimum $\frac{1}{2}\|W\|^2$) where the “margin” hyperplanes have given vertical displacements $\varepsilon > 0$ on either side of the fitted hyperplane in the middle between the two margin hyperplanes. As seen in Figure 4.8 (where for illustration a simple one-dimension feature case is shown), there are two possible fitted lines L_1 and L_2 . Each has two “margin” hyperplanes with vertical displacements ε on either side. The pre-determined epsilon ε is large enough to contain all sample points within the epsilon tube, so the maximization of margin has zero penalty. Hence the margin is maximized by line L_2 . Line L_2 has a wider margin than line L_1 , i.e., $m_2 > m_1$. Hence, maximizing margin subject to displacements of $\varepsilon > 0$ has a tendency to lead to a flatter fitted hyperplane.

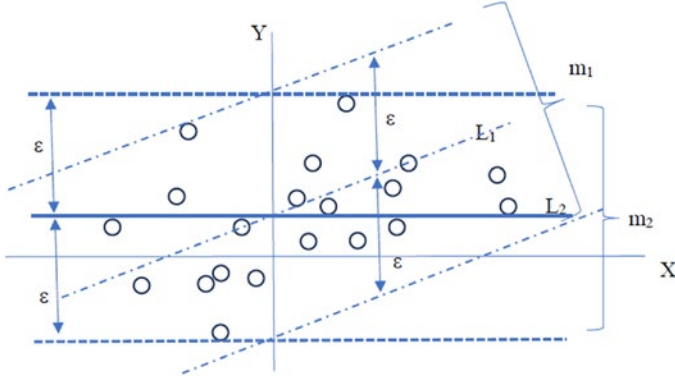


Figure 4.8. Maximizing margin in support vector regression.

However, when epsilon ε is smaller, it may not be possible to contain all the sample points within the epsilon tube. As with the case of imperfect separation in SVM classification, there may be points lying outside the epsilon space or tube. These outside points (see * points in Figure 4.9) are penalized with a cost α for the error or deviation ζ_i beyond the “margin” hyperplanes. Thus, if line L_2 is fitted, the added cost is $\alpha \sum_{i=1}^3 \zeta_i$ in this case. The idea in finding the optimal fitted hyperplane is in

$$\min_{W, W_0, \zeta_i} \frac{1}{2} W \bullet W + \alpha \sum_{i=1}^N \zeta_i \quad (4.14)$$

subject to $|Y_i - (W \bullet X_j - W_0)| - \varepsilon - \zeta_i \leq 0$ and $\zeta_i \geq 0$ for $i = 1, 2, \dots, N$. Note that in SVR, unlike SVM, Y_i is the numerical target value, not the class/type dummy variable of +1 and -1.

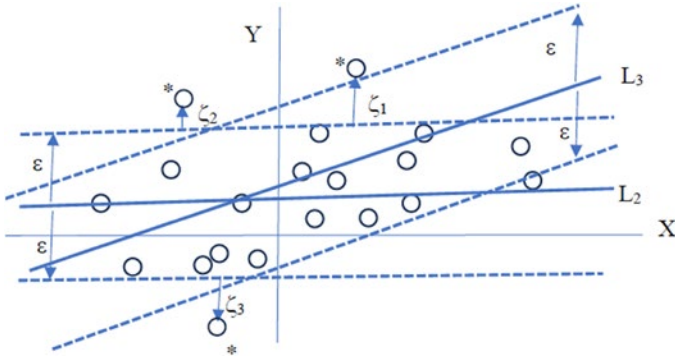


Figure 4.9. Support vector regression with constraints.

Note that in Figure 4.9, a better fit with a lower objective function in Eq. (4.14) can be found in line L_3 . The hyperplane L_3 has lowest $\frac{1}{2}W \bullet W + \alpha \sum_{i=1}^N \zeta_i$ as the tilting of the slope and intercept shift adjust the epsilon tube such that the outside points now have less deviation from the “margin” lines and hence less penalties.

In general, increasing epsilon ε would reduce sum of ζ_i deviations but when the increase is too large and no deviations exist, the fitted slope becomes flat or zero slope. Sometimes increasing epsilon ε may cause prediction to deteriorate if catching too many points in the tube including outliers loses the slope pattern of the majority of the points. In this latter case, there is usually poor prediction on test data. On the other hand, when epsilon shrinks toward zero, the SVR can become overly sensitive to outliers. It approaches a linear regression. In sklearn, the default epsilon is 0.1 and the default α (or “C” in sklearn) is 1.0.

Just as in SVM, the dual problem of finding the optimal fitted hyperplane in SVR can be formulated in terms of $(X_i \bullet X_j)$. Nonlinear kernels $K(X_i, X_j) = \phi(X_i) \bullet \phi(X_j)$ can also be used in SVR to obtain fitted hyperplane after the original data points are suitably transformed via $\phi(\cdot)$. The latter may help in better fitting with lower penalty costs in the objective function. See also Smola and Scholkopf (2004).

4.6 Worked Examples: Data

The data set is found in public website: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>. It was collected by Worldline in collaboration with the Machine Learning Group of Université Libre de Bruxelles. The dataset shows European credit cardholders’ 284,807 transactions over two days in September 2013, out of which there were 492 fraudulent cases, accounting for about 0.17%⁴ of the transactions. Credit card fraudulent transactions occur when unauthorized person(s) somehow gain access to use the card to make purchases.

According to the source, the data are numerical continuous variable features that are a PCA transformation from the original confidential client background information. There are 28 such features. In addition to these, two actual variables were added: “Time” variable indicates time in seconds that elapsed between each transaction and the first transaction in

⁴Due to the huge type imbalance, it may be misleading just to consider Confusion Matrix accuracy, the AUC should also be considered.

the dataset and “Amount” indicates dollar transaction amount. The target or class/type variable “Type” is a dummy variable that takes the value 1 if the case is fraudulent, and 0 otherwise.⁵

The “Time” variable is not included in the study as it does not appear to be a useful feature. The 28 PCA-transformed variables and the “Amount” are standardized via the StandardScaler and these 29 features are used to predict Type 1 (Fraud) or Type 0 (no Fraud). The prediction results are run in Chapter4-1.ipynb using Naïve Bayes, k-Nearest Neighbor, SVM, and also the Logistic Regression algorithms. The main code lines are shown as follows.

```
In [1]: ### Data from https://www.kaggle.com/datasets/mlg-ulb/
        creditcardfraud
        ### Credit Card Fraud Prediction/Detection
        import pandas as pd
        import numpy as np

In [2]: dataX=pd.read_csv('creditcard.csv',header=0)
        dataX=dataX.dropna() # Remove missing values or NA
        print(dataX.shape)
        print(list(dataX.columns))

Output: (284807, 31)
['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11',
'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20', 'V21',
'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount', 'Type']
```

There are 284,807 records/cases and 31 fields. Columns 2 to 30 will be used as features, while column 31 provides the target variable values or types. “Type” = 1 indicates fraud case while “Type” = 0 indicates no-fraud case. In [4], the percentage of fraud case is found to be very small at about 0.173%. The data classifications are imbalanced. To mitigate the imbalance problem that can produce high accuracy in prediction without any algorithm by just always picking the class that has very high proportion,

⁵A similar study by Suraj Patil, V. Nemade, and P.K. Soni, (2018). Predictive modelling for credit card fraud detection using data analytics. *Procedia Computer Science* 132, 385–395, indicated client’s checking account history, pattern of credit card payments, abnormal use and amounts of credit card use, and client’s possession of property, car, and insurance could have predictive influence on the probability of a fraud.

one way is to create a more balanced sample by deleting sample points with the higher proportion of the same targets/classes.

Another way is to augment (using resampling schemes) sample points with the lower proportion of the same targets/classes. Here, we randomly select 492 sample points from the high density cases with Type 0 to match with the existing 492 sample points from the low density cases with Type 1.

```
In [4]: data=dataX.groupby("Type").sample(n=492, random_state=1)
      ### creates 492 or n number of each of Type 1 and 0
```

Now, there are a total of 984 cases with 50% of the cases involving credit card frauds. The features are normalized (with mean 0 and standard deviation 1) in code line [10] as part of the data preprocessing. Having data of similar scale is important for some machine learning algorithm that are affected by relative magnitudes of features.

```
In [10]: # Normalization of all features
```

```
from sklearn.preprocessing import StandardScaler #from
sklearn import preprocessing done earlier
scaler = StandardScaler().fit(data1) # this computes means and
sd to standardize
data2 = scaler.transform(data1) # this does the standardization
with computed means, sds
data3=pd.DataFrame(data2,index=data1.index,
columns=data1.columns)
# above step converts array from last step back to pandas
dataframe, also puts back indexes V's
```

Next, the data is split into 80% (787 cases) for training and remaining 20% (197 cases) for testing. We skip validation and assume the test result is done using optimized hyperparameter(s), or that validation and hyperparameter tuning was already done.

4.6.1 Naïve Bayes algorithm

Code line [15] shows the application of the sklearn Naïve Bayes algorithm to fit the training set data, then perform the prediction on the test data (X_{test}). The accuracy is then reported as 91.878% using the sklearn metrics on `accuracy_score`.

```
In [15]: # Train the model
gnb = GaussianNB()
gnb.fit(X_train, y_train)

# Evaluate the model
y_pred_NB = gnb.predict(X_test)

from sklearn import metrics
from sklearn.metrics import accuracy_score
Accuracy_NB = metrics.accuracy_score(y_test, y_pred_NB)
print("Naive Bayes Accuracy:", Accuracy_NB)

Naive Bayes Accuracy: 0.9187817258883249
```

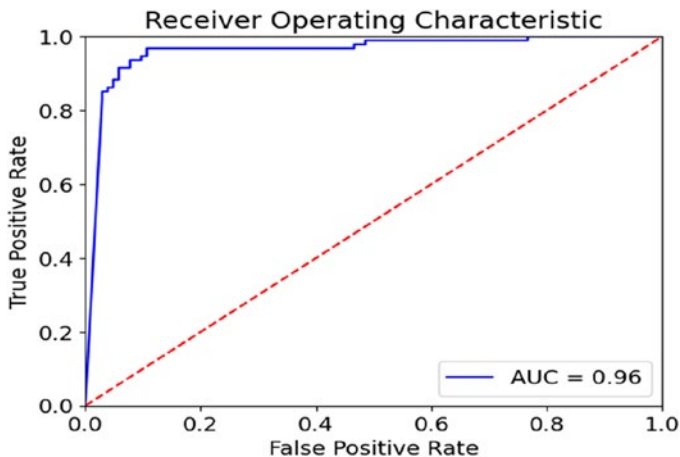
The confusion matrix, the classification report, and the ROC curve are obtained via code lines [16–19]. ROC area is 96.075%.

```
In [16]: from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, y_pred_NB)
print(confusion_matrix)

[[98  5]
 [11 83]]
```

```
In [17]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_NB))
```

	precision	recall	f1-score	support
0	0.90	0.95	0.92	103
1	0.94	0.88	0.91	94
accuracy			0.92	197
macro avg	0.92	0.92	0.92	197
weighted avg	0.92	0.92	0.92	197



4.6.2 K-Nearest Neighbor algorithm

Code lines [20–22] show the application of the sklearn KNeighbors Classifier employing $k = 3$ to fit the training set data, then perform the prediction on the test data (X_test). The accuracy is then reported as 92.893% using the sklearn metrics on accuracy_score.

```
In [20]: from sklearn.neighbors import KNeighborsClassifier
kNN= KNeighborsClassifier(n_neighbors = 3, algorithm='auto', metric='minkowski', p=2,
metric_params=None, n_jobs=None, weights='uniform')
# argument: Minkowski (default metric) and p=2 gives Euclidean distance
# default weight is uniform
# algorithm auto uses 'best' search method to solve min distances
# n_jobs (default none means 1): number of parallel processes to solve
kNN.fit(X_train, y_train)
## Note the kNN.predict process next involves intense computations

Out[20]: KNeighborsClassifier(n_neighbors=3)

In [21]: y_pred_kNN = kNN.predict(X_test)

In [22]: from sklearn import metrics
from sklearn.metrics import accuracy_score
Accuracy_kNN = metrics.accuracy_score(y_test, y_pred_kNN)
print(Accuracy_kNN)

0.9289340101522843
```

Code lines [23] and [24] show that the training score is higher than the test score which is to be expected since the testing is done on generalized (out-of-sample) data. Both training fitting accuracy score and testing accuracy score are close so there is no overfitting in the training.

```
In [23]: print('Test set score: {:.16f}'.format(kNN.score(X_test, y_test)))
## same computation as combined two steps above

Test set score: 0.9289340101522843

In [24]: # Compare with training score
print('Training set score: {:.16f}'.format(kNN.score(X_train, y_train)))

Training set score: 0.9504447268106735
```

The confusion matrix, the classification report, and the ROC curve are obtained via code lines [26–28], [33], and [34].

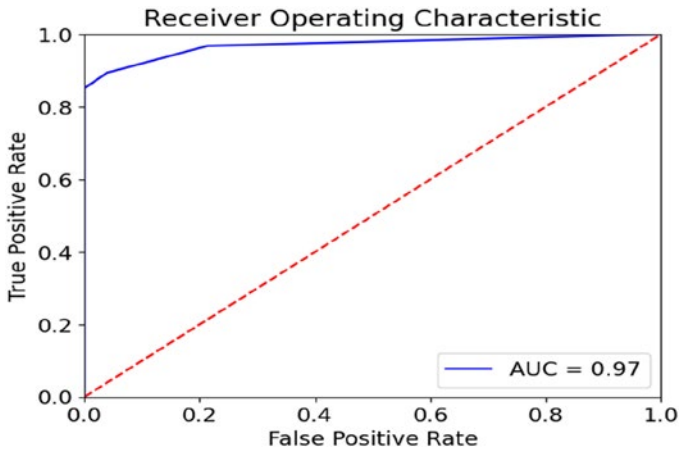
```
In [26]: from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, y_pred_kNN)
print(confusion_matrix)

[[99  4]
 [10 84]]
```

```
In [27]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_kNN))
```

	precision	recall	f1-score	support
0	0.91	0.96	0.93	103
1	0.95	0.89	0.92	94
accuracy			0.93	197
macro avg	0.93	0.93	0.93	197
weighted avg	0.93	0.93	0.93	197

```
In [28]: Prob1=kNN.predict_proba(X_test)[:,:0]
### gives the probabilities for target variable being 0
```



The area under ROC for $k = 3$ is 97.041%, about 1% higher than that of GNB method. When k is increased to $k = 5$, the area increases marginally to 97.707%, after which this area decreases when k is further increased. The accuracy also increases to 94.923% when $k = 5$, after which it decreases as k increases. We assume that $k = 5$ would have been the optimal hyperparameter to be used if we had used a validation subset.

4.6.3 Support vector machine

Code line [57] shows the application of the sklearn SVM employing the linear kernel with hyperparameter $\alpha = 1.0$ (“C” in the ipynb file) to fit the training set data. These hyperparameters are assumed to be optimal from earlier validation. Then the model is for prediction on the X_test. Code lines [58–60] shows the confusion matrix, accuracy, and classification report. The accuracy is reported as 96.447%.

Linear Kernel

```
In [57]: from sklearn import svm
svm1 = svm.SVC(kernel='linear', C = 1.0, probability=True)
### C is regularization para. Default C=1.
svm1.fit(X_train,y_train)
y_pred_svm = svm1.predict(X_test)
```

```
In [58]: from sklearn.metrics import accuracy_score,confusion_matrix
confusion_matrix(y_test,y_pred_svm)
```

```
Out[58]: array([[102,  1],
               [ 6, 88]], dtype=int64)
```

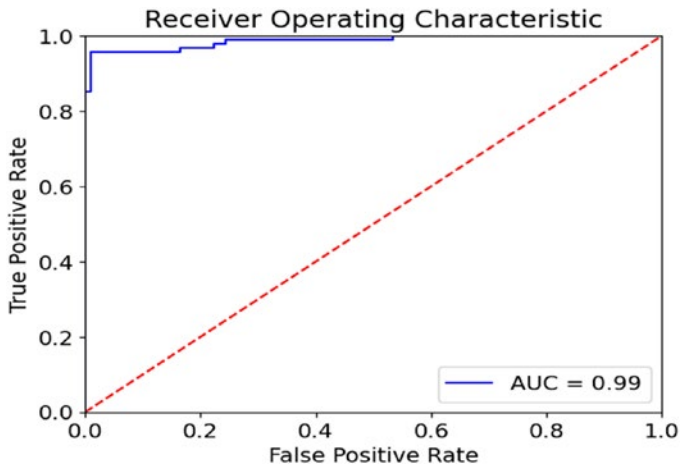
```
In [59]: accuracy_score(y_test,y_pred_svm)
```

```
Out[59]: 0.9644670050761421
```

```
In [60]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_svm))
```

	precision	recall	f1-score	support
0	0.94	0.99	0.97	103
1	0.99	0.94	0.96	94
accuracy			0.96	197
macro avg	0.97	0.96	0.96	197
weighted avg	0.97	0.96	0.96	197

Code lines [61] and [62] yield the ROC area and graph. The area is 98.657% which is the higher than those of the Naïve Bayes and the k-NN methods.



Employing other kernels such polynomial, sigmoid, and rbf in the SVM produced lower accuracies and lower area under ROC curves.

4.6.4 Support Vector Regression

The exercise here is contained in Chapter4-2.ipynb and uses the data set “california_housing.frame.csv”. As seen in code line [8], after removing outliers and censored data, there are 19615 rows with eight columns of features and one column of target variable “MedHouseval” (median house value in block in USD).

In code line [17], the features are pre-processed and scaled to within (0,1). In code line [18], the data set is randomly split into 80% or 15,692 training data points and 3,923 test data points. In code lines [20] and [21], linear kernel SVR ($\epsilon = 0.56$, α or “C” = 25) shows a R^2 training score of 61.97% and R^2 test score of 62.14%.

```
In [20]: SVR1 = SVR(kernel = 'linear', epsilon=0.56, C=25)
SVR1.fit(X_train, y_train)
y_pred_SVR1_train = SVR1.predict(X_train)
y_pred_SVR1_test = SVR1.predict(X_test)
```

```
In [21]: r2_score_SVR1_train = r2_score(y_train, y_pred_SVR1_train)
print('R2_score (train): ', r2_score_SVR1_train)
### R2_score (train) is the R-square in the linear regression involving only the training data set
r2_score_SVR1_test = r2_score(y_test, y_pred_SVR1_test)
print('R2_score (test): ', r2_score_SVR1_test)
```

```
R2_score (train): 0.6197322462567303
R2_score (test): 0.6214216496582796
```

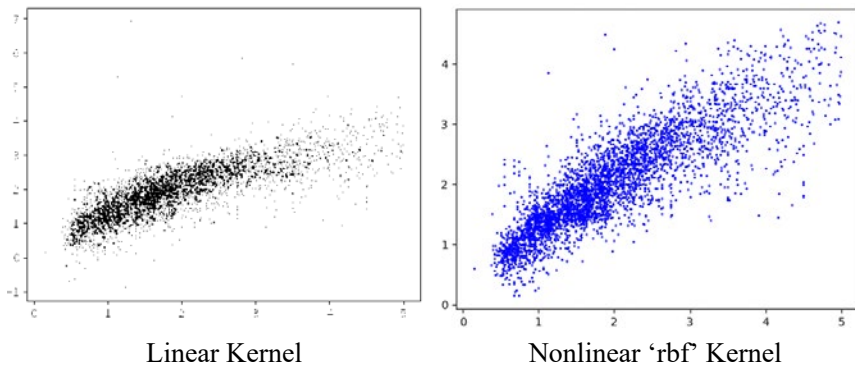
Code lines [24] through [27] show the results of other nonlinear kernels. The best result is shown in code lines [26] and [27] with the “rbf” kernel.

```
In [26]: SVR3 = SVR(kernel = 'rbf', epsilon=0.56, C=25)
SVR3.fit(X_train, y_train)
y_pred_SVR3_train = SVR3.predict(X_train)
y_pred_SVR3_test = SVR3.predict(X_test)

In [27]: r2_score_SVR3_train = r2_score(y_train, y_pred_SVR3_train)
print('R2_score (train): ', r2_score_SVR3_train)
### R2_score (train) is the R-square in the linear regression involving only the training data set
r2_score_SVR3_test = r2_score(y_test, y_pred_SVR3_test)
print('R2_score (test): ', r2_score_SVR3_test)

R2_score (train): 0.7286284928673199
R2_score (test): 0.7356554348857722
```

R^2 test score is increased to 73.57%. The predicted target versus actual target value plots for the linear kernel and the more effective nonlinear “rbf” kernel are shown below.



4.7 Concluding Thoughts

Naïve Bayes method is popular as it has lower computational time and the method scales up effectively with multi-class predictions as with binary predictions. It can also work with many features/dimensions. However, its independence assumption may cause issues in classification if the features are highly correlated. A common mistake is not to ensure that all features present in the testing sample set are also used for fitting in the training data set; use of such a feature in the test set would render it a “zero frequency” by which it has no part to play in the prediction of the test sample

point. Numerical or regression prediction using Naïve Bayes is not a natural concoction and may produce ineffective predictors.

k -NN has lazy training as active computations take place only when a test sample point is introduced and then the nearest neighbors are searched within the training data set. Each new test sample point gives rise to new constructions of neighborhood and the resulting prediction. The method is completely non-parametric with no distributional assumptions. It also extends readily to multi-class problems and can be used for both classification and numerical prediction.

However, k -NN has drawbacks in some problems. Firstly, since the method uses a distance measure such as Euclidean distance, it is sensitive to scale so that features with outsized or else miniature scale should be standardized or normalized. The algorithm is slow when the dimension or number of features increases as the neighborhood computations become large. In the same vein, tuning the k hyperparameter also involves intense computations when the dimension is large. Imbalanced data may cause bias due to biased neighborhood. It also pays much less attention to outliers not in the neighborhood.

Contrary to k -NN, SVM can easily handle many features. The number of features can also be larger than the number of training data points. The ability to use different specifications of kernel functions provides good possibility of finding separating boundaries or separating hyperplanes after kernel transformation of the original data points. The transformation often expands the feature dimension. This can perform reasonably well in complex data patterns.

However, SVM may require high computational time for a large data set. Overfitting often occurs especially when the number of features is large relative to the size of the data set. Regularization is thus important in SVM. SVM may also be sensitive to outliers.

References

- Frank E., L. Trigg, G. Holmes, and I. H. Witten (1999). Technical note: Naïve Bayes for regression. *Machine Learning*. Kluwer Publishers Boston.
- Smola A. J. and B. Schölkopf (2004). A tutorial on support vector regression. *Statistics and Computing*, 14, 199–222.
- Wang L. (2005). *Support Vector Machines: Theory and Applications*. Springer, Verlag.

This page intentionally left blank

Chapter 5

Decision Trees, Random Forest, and Multi-Class Prediction

Nonparametric estimation is a statistical method that estimates the functional form of data fits when parametric modelling is absent and where there is no guidance from theory. Examples of nonparametric estimation are k-NN, SVM, and kernel estimation method akin to fitting histograms on frequency counts to get a distributional form.

Decision Tree (DT) is a method to predict the value of a target variable (whether predicting a class/category or a continuous value) by a non-parametric way without employing some parametric functional form relating target to features such as in a logistic regression approach. A DT contains decision rules that are conditions to test each sample point or instance on the training data set. For example, to partition red strawberries from red rambutans (an Australian friend of mine years ago called the latter “hairy strawberries”), one could use the decision rule that if the fruit has thick hairs, then it is rambutan; otherwise, it is strawberry. Another example could be trying to partition between crocodiles and alligators. A decision rule could be that if the creature has rounded snout, then it is an alligator; otherwise, it is a crocodile. A difficult example is identifying a mugger from a police lineup where the other people in the lineup are innocent. One could use a decision rule based on observed features of the lineup suspects, such as whether the suspect has a proven alibi at the time of the crime. This may reduce the number of suspects, but there could still be many who are left, so more decision rules must be added sequentially to filter out the innocent ones.

The DT approach is typically supervised learning with a training data set, a validation data set and a test data set. In this chapter, we look at the usefulness of DT and the related Random Forest (RF) method of predicting binary classification. It is called the Categorical Variable DT when predicting binary or categorical groups. When used for predicting continuous value outputs, the DT may be called a Continuous Variable DT. Categorical Variable DT (and related RF) is generally found to be more accurate relative to use of Continuous Variable DT as the latter involves many more permutations of the target variable. However, due to the non-parametric nature, DT methods can produce unstable outcomes when data features have high variabilities as the decision rule outcomes on the partitioning can change sharply from one test data point to another.

5.1 Decision Tree Method

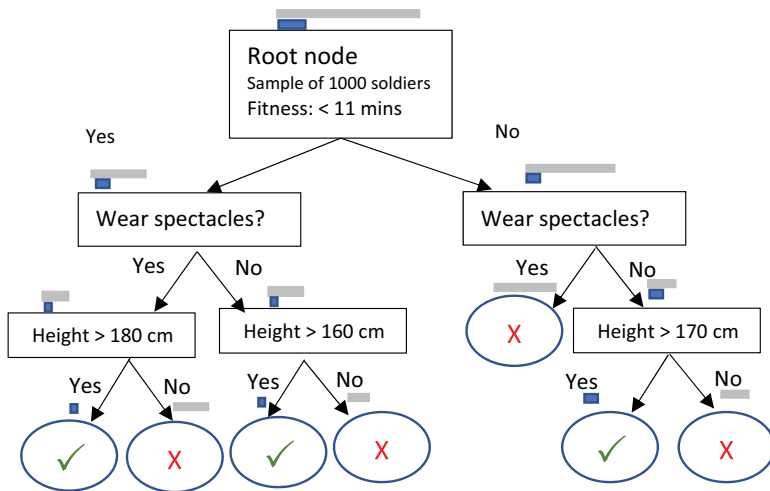
Suppose we use DT to predict if a case belongs to the positive or the negative class, being binary partitions of all cases or instances, i.e., any case could be put into the positive or the negative class, and that the classes are reasonably balanced. Balance refers to the number of cases in the positive class being of similar magnitude to the number of cases in the negative class. Let each case, besides its positive or negative label, also carry a set of attributes or features that can help in predicting its type/label.

We illustrate as follows how a DT can help to predict (1) if a soldier who passed BMT is a marksman (defined as one who shoots on target at least 84% of the times or 21 hits out of 25 shots at training targets), with only preliminary training, given the soldier's fitness (time in minutes to complete a 2.4 km run), height (in cm) and whether the soldier needs to wear spectacles or not and (2) if a person would get a certain lung disease by age 60 given the person's fitness measured by number of hours of exercise a week, whether a regular smoker (at least one cigarette a day) or not, and whether there is a hereditary factor, i.e., if the person's parent (one or both) has had such a disease.

A trained DT based on a training sample of past 1000 soldiers for (1) could be as follows. The first root node is also a decision node on using fitness to split the total sample of 1000 soldiers. Value of a node refers to the number of positive cases (training sample marksmen) and number of negative cases (non-marksmen). For the illustration, this set of values is indicated by the relative length of the dark shade versus lighter grey bars above the decision node. The decision node is a node whereby

a decision is made, e.g., if a sample point or case or instance (a soldier in the training data set) has fitness-run below 11 minutes (or else equal and above).

A feature is chosen (in some optimal way to be explained later) and a threshold of the feature is chosen. The latter refers to some values of the feature as the splitting point to divide the data associated with the feature into two subsets. The splitting or branching is based on “Yes” or “No” to the outcome of the decision rule. If the feature is categorical, then the splitting is based on “Yes” or “No” to the categorical rule. The decision node is associated with a decision rule. Data is split into subsets and moves along the branches to the next (lower) levels of decision nodes until the leaf or terminal nodes are reached. At each leaf (end) node, the model makes a prediction.



The rectangular boxes represent the decision nodes where splitting of the sample then occurs along the subsequent branches. The ovals at the end of the tree (upside down) represents the leaf nodes or terminal nodes. These carry the prediction or final decision whether prediction (tick) is a marksman or (cross) not a marksman. In this example, at each terminal node or leaf, the remaining sub-sample is either all marksman or not marksmen. Hence, there is no more sub-dividing. The DT could also end in a leaf with no further extension or decision box if no other features can be of help in further separation of the types in the leaf.

Recall that the bars above the boxes or ovals represent the number of marksmen (dark shade) and non-marksmen (lighter shade) from the historical sample used as training set. With each decision and split, the numbers of marksmen and non-marksmen are split into the next set of nodes.

The prediction rules according to this DT are: if fitness is < 11 mins, no spectacles, and height > 160 cm, or if fitness is < 11 mins, spectacles, and height > 180 cm, or if fitness is ≥ 11 mins, no spectacles, and height > 170 cm, then marksman; otherwise not in the other feature conditions.

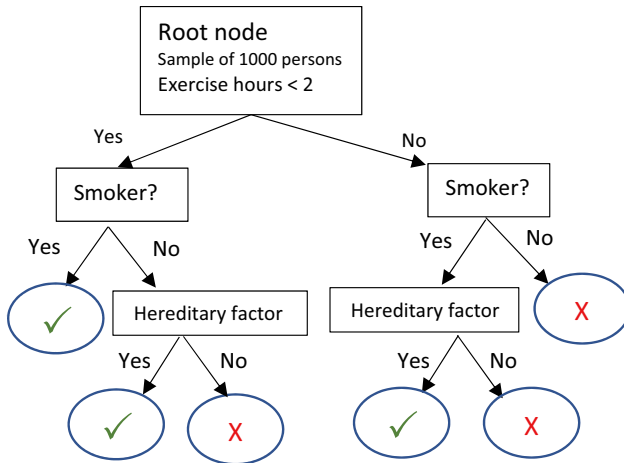
In general, it is seen that with a given training set, one could continue to search for new features (associated with the targets) such that a chain of rules would lead to perfect discrimination – separating the targets into the different categories. This could be over-fitting and may not produce a good accuracy on the test set. Some regularizations/constraints on the hyperparameters may be introduced such as limiting the depth (levels) of the DT instead of the default maximum (8 to 32 in the sklearn app), limiting to no decision node based on a feature if the improvement in the differentiation is minimal, limiting the total number of nodes, limiting to a minimum number of sample points or cases on any node and not to split further, and so on.

Another example is a DT based on a training sample of past 1000 medical insurance buyers over a long period. Assume no moral hazard problem and that each insurance buyer starts off under the insurance scheme with no sign of contracting the disease.

A tick at the leaf denotes prediction of getting the disease; otherwise, it is a cross. The prediction rules according to this DT are: if exercise hours is < 2 per week, if smoker, or, if exercise hours is < 2 per week, if non-smoker but with hereditary factor, then disease is predicted. Or else, if exercise hours is ≥ 2 per week, smoker and with hereditary factor, then disease is predicted. Otherwise, disease is not predicted.

Next, we explain how the DT chooses which feature to create the split, how is the split condition chosen, and when does the splitting stop. We continue with the marksman example.

At the root node, the sample (size N) provides a measure of Gini impurity that is defined as $1 - \sum_{k=1}^m p_k^2$ where m is the number of different categories in the sample and p_k is the empirical probability of being in category k . Thus, in the marksmen case, if the original sample has 200 marksmen and 800 non-marksmen, then if the positive case 1 (marksmen) has probability $p_1 = 200/1000 = 0.2$, and the negative case 2



(non-marksman) has probability $p_2 = 0.8$. The Gini impurity would be $1 - (0.2^2 + 0.8^2) = 0.32$ in this binary classification. Note that if $p_1 = p_2 = 0.5$, then we have a maximum Gini impurity. But $p_1 = p_2 = 0.5$ is also the case for maximum entropy (uncertainty in distributional outcome) in the log loss or (cross-) entropy measure of $-\sum_{k=1}^m p_k \ln(p_k)$. Hence, lower Gini impurity is related to lower entropy or better resolution of the classification rules. The idea is to develop rules leading to the lowest Gini impurity or else lowest entropy in the next nodes. For binary classifications, $m = 2$, so depending on the optimization criterion, we either minimize $1 - \sum_{k=1}^2 p_k^2$ or else the cross-entropy measure $-\sum_{k=1}^2 p_k \ln(p_k)$. Both these measures are close approximations of each other. Computing Gini impurity is more efficient as it does not involve finding logarithms in the cross-entropy measure.

The minimization of loss function as Gini impurity is done by (1) working on each feature considering splitting at different points – for a continuous variable feature, it means discretizing it and considering splits at each discrete points, e.g., splits at < 9 mins, < 9.1 mins, < 9.2 mins, ..., < 11 mins, < 11.1 mins, ..., < 14 mins etc., and then (2) computing the Gini impurities in the new next layer binary nodes under each split, and (3) finding that split that produces the lowest weighted Gini impurities lower than that in the previous node (hence, the largest gain in information – largest drop in entropy in the new split). For discrete valued features – the potential split point could be the mid-points of adjacent discrete values of the feature.

For example, the following could be a result of the split. The numbers in the brackets () denote number of marksmen on the left and number of non-marksmen on the right. There are a total of 200 marksmen and 800 non-marksmen in the training data set. Consider the split at $\text{Fitness} < 10.9$ minutes. The Gini impurity for the “Yes” branch is $1 - (100/420)^2 - (320/420)^2 = 0.36281$. The Gini impurity for the “No” branch is $1 - (100/580)^2 - (480/580)^2 = 0.28358$. Then the weighted average of Gini impurities in this next layer of binary nodes under the split is $(420/1000) \times 0.36281 + (580/1000) \times 0.28358 = 0.31790$.

Values Gini impurity	Yes	No	Weighted Gini impurity
If < 10.9 min	(100, 320) [0.36281]	(100, 480) [0.28358]	$(420/1000) \times 0.36281$ + $(580/1000) \times$ $0.28358 = 0.31790$
If < 11 min	(120, 300) $[1 - (120/420)^2 -$ $(300/420)^2$ $= 0.40816]$	(80, 500) $[1 - (80/580)^2 -$ $(500/580)^2$ $= 0.23781]$	$(420/1000) \times 0.40816$ + $(580/1000) \times$ $0.23781 = 0.30936$
If < 11.1 min	(110, 340) [0.369383]	(90, 460) [0.273719]	$(450/1000) \times 0.369383$ + $(550/1000) \times$ $0.273719 = 0.31677$

Suppose the split at $\text{Fitness} < 11.0$ min gives the lowest weighted Gini impurity or biggest drop in impurity from the previous root node of 0.32. Suppose also that considering splits in all the other features do not produce any bigger drop in impurity. Then the first decision node is whether $\text{Fitness} < 11.0$ min. Note that the Gini impurity is the loss function to be minimized in this DT.

Building the DT then continues to the next level (down) when the next best information gain feature would be used to create the next split, and so on. In case a leaf (final node) ends without fully discriminating against the two (or more) categories due to regularization (non-overfitting), the empirical probability in the sample in that leaf can be used to make the prediction. The splitting of the sample into smaller subsets along each branch is called recursive distribution of the sample. In a binary classification, if a terminal leaf has H number of positive cases and G number of negative cases, then the (empirical) probability of a positive case in the prediction would be $H/(H + G)$ and that of a negative case would be $G/(H + G)$.

Once the DT is trained using the training set, the hyperparameters are fine-tuned in a validation data set for optimal prediction. Then the DT could be re-trained using a re-grouped training set (including validation data). We assume the validation has been done, and we proceeded with the training as seen earlier and then it is used next to predict the outcomes/categorizations in the test data set. All the test data X -variables (features) are used to decide on a positive or a negative case based on the trained decision rules. In the testing, the test sample point labels/types are not used for the prediction but are used afterward to measure the prediction performance.

We briefly explain the Continuous Variable DT as follows. Suppose in the same marksmen problem above, we want to predict the actual shooting score – how many shots on target out of a total of 25 shots in the shooting range test. This is a discrete variable – but any continuous variable in the finite initial sample is considered a discrete value (no doubt with decimal places in the numbers).

Continuous Variable DT works slightly differently whereby the loss criterion is squared error (default case) – using `DecisionTreeRegressor` instead of `DecisionTreeClassifier` (for classification) in `sklearn`. Continuous Variable DT is sometimes called a regression tree model. Suppose we plot the actual shoot score of the 1000 cases/instance against one of the features – Fitness.

The diagram serves only to illustrate as there would be 1000 points or instances. Suppose at $\text{Fitness} < 11.0$ min, we obtain two partitions. On the left we have mean square error

$$\text{mse}_1 = \frac{\sum_{j=1}^L (Y_j - \bar{Y}^L)^2}{L}$$

where L is the number of sample points with feature $\text{Fitness} < 11.0$ min and on the right

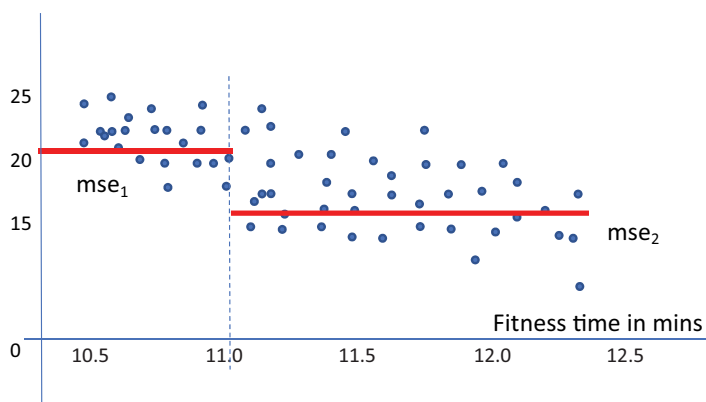
$$\text{mse}_2 = \frac{\sum_{j=1}^R (Y_j - \bar{Y}^R)^2}{R}$$

where R is the number of sample points with feature $\text{Fitness} \geq 11.0$ min. Total $L + R = 1000$ from the total number of sample points/instances before the split from the earlier node. $\bar{Y}^L$ and $\bar{Y}^R$ are respectively the left sub-sample and right sub-sample means. Weighted variance (mean squared error)

or loss function in this case is $L/(L + R) \times \text{mse}_1 + R/(L + R) \times \text{mse}_2$. This is the same as

$$\left[\sum_{j=1}^L (Y_j - \bar{Y}^L)^2 + \sum_{j=1}^R (Y_j - \bar{Y}^R)^2 \right] / (L + R)$$

The decision rule to split, i.e., at Fitness < 11 min, is chosen such that this loss function is the minimum across other possible splits and across all other feature splits and is also a variance reduction from the previous higher-level node.



The next level split is then done on the left sub-sample, and another next level split is done separately on the right sub-sample using the next feature, and so on. The potential split points in the features are typically obtained as the mid-points of two adjacent discrete values of observations of the feature under consideration.

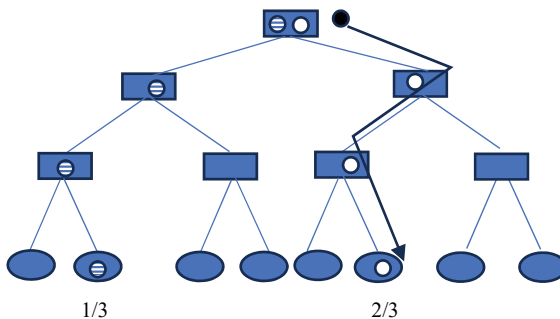
Finally, when all levels are done and the leaf node is on instances with the same shooting score, the DT can then be used to perform prediction of a future soldier's shooting score based on a new soldier's features. If the leaf ends with a small set of different shooting scores – then the mean can be taken as the predicted score given the decision rules along that DT ending in that leaf.

The method of splitting a feature via threshold to form a decision rule on a DT by minimizing a loss criterion is also called a Classification And Regression Tree (CART) where more generally, the nodes carry not just the value information but also estimated values and derivatives of loss function computed at nodal level.

Just as in a categorical variable DT, once the DT is trained using the training set, it is then used to predict the outcomes in the test data set. All the test data X -variables (features) are used to decide on the predicted shooting score based on the trained decision rules. In the testing, the test sample point labels/types are not used for the prediction but are used afterward to measure the prediction performance.

The prediction in testing is illustrated as follows. The following is a trained DT showing the root node on top down to the leaf nodes. The DT and decision rules were arrived at during the training by making use of the knowledge of the training set labels.

A trained sample point from the training data set may be the white circle. Given its features and the decision rules on the features, it ended up in leaf node indicating 2/3 (if it is a categorical variable DT, i.e., label is categorical). The trained DT indicates that the leaf (terminal node) with 2/3 meant there were 2/3 positive cases and 1/3 negative case there in the training set. Essentially all the original training data set instances were distributed into the (terminal) leaves of the trained DT. Another training sample point could be the striped circle ending up in leaf node indicating 1/3. In this case, this test instance is predicted to be a positive case with only 1/3 probability, but a negative case with 2/3 probability.



Suppose the black circle is a test sample point or instance, and the decision rules in the fitted or trained tree led it to land on the leaf with 2/3 indicating it has a probability of 2/3 being a positive case. In this case of predicted probability, since the usual threshold is 0.5, predicted Prob. (label $Y = 1$) = 2/3 implies it is predicted as category $Y = 1$.

If it is a continuous variable DT, then instances would end in a leaf with a numerical value, e.g., 12. This would mean the predicted target value is 12.

Note that for complex data, it is sometimes possible for the algorithm to use the same feature (used earlier for splitting closer to the root node) to split again at some points along a sub-tree. The way the splitting is done is based on the “greedy” algorithm approach that looks for the best option at hand without looking down the road if other not immediately best steps could instead be taken that may actually produce overall lower Gini impurities in all leaves.

DTs and their ensemble seen later are not as sensitive to multi-collinearity in features as in other methods such as linear models or logistic regression. This is because for any highly collinear features, the DT proceeds by choosing only one to perform the decision rule. The other one does not at the same time affect the outcome of the DT.

DT can also be used to predict multi-class problems, e.g., predicting if a test sample point belongs to class A, class B, or class C. The approach is the same, except now each node contains value function with three numbers, viz., number of case A, number of case B, and number of case C. At each decision split into the next level of two nodes, the lowest weighting of Gini impurity $1 - \sum_{k=1}^m p_k^2$ is selected. The downward splitting of nodes continues until the leaf or end node. At each leaf, if Gini impurity is not zero (i.e., zero implies perfect identification of A or B or C in the training data), empirical probabilities of the cases are used and the class with the highest empirical probability (i.e., having the highest number of sample points belonging to that class in that leaf) is the predicted class.

5.2 Worked Example DT: Data

This data set `corporate_rating2.csv` is obtained on public website from Kaggle <https://www.kaggle.com/datasets/agewerc/corporate-credit-rating>.

The dataset contains 2029 credit ratings issued by major agencies from 2010 to 2016 on large US companies. For each credit rating, each company shows 25 accounting features. The other six columns in the data set consist of rating, name of firm, symbol of firm in exchange trading, the rating agency, date of rating, and the sector of the firm. I added a new column called “Class” that contains “1” if the credit rating indicates investment grade, i.e., “AAA”, “AA”, “A”, or “BBB”. Any rating below “BBB” – considered as speculate grade – has a class value of “0”. There are 1165 instances of Class “1” and 864 instances of Class “0”.

The idea in this exercise using `Chapter5-1.ipynb` is to use the company financial/accounting variables/ratios to try to predict which binary class of

investment grade or speculative grade the firm belongs to, using the published class as target for supervised training. A particular company may have several data points in the sample set as it might have obtained credit ratings, hence, a classification as “1” or “0”, at different calendar dates/times.

When adequately trained, the predictive model can be used to help new firms getting listed as well as investors/creditors of such firms to use the firm’s internal accounting ratios (reported as accurately as possible in a format close to the published ones) to help predict which category the firm should belong to. Obviously if identified as investment grade, the firm would find it easier to float new shares (IPOs) and issue debts at a cheaper interest cost.

In code line [4], the features information is shown.

```
[4]: # Display the structure
df_ratios.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2029 entries, 0 to 2028
Data columns (total 32 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Rating                                     2029 non-null   object
1   Class                                     2029 non-null   int64
2   Name                                       2029 non-null   object
3   Symbol                                    2029 non-null   object
4   Rating Agency Name                       2029 non-null   object
5   Date                                      2029 non-null   object
6   Sector                                    2029 non-null   object
7   currentRatio                             2029 non-null   float64
8   quickRatio                               2029 non-null   float64
9   cashRatio                                2029 non-null   float64
10  daysOfSalesOutstanding                   2029 non-null   float64
11  netProfitMargin                          2029 non-null   float64
12  pretaxProfitMargin                       2029 non-null   float64
13  grossProfitMargin                        2029 non-null   float64
14  operatingProfitMargin                    2029 non-null   float64
15  returnOnAssets                           2029 non-null   float64
16  returnOnCapitalEmployed                  2029 non-null   float64
17  returnOnEquity                           2029 non-null   float64
18  assetTurnover                             2029 non-null   float64
19  fixedAssetTurnover                       2029 non-null   float64
20  debtEquityRatio                          2029 non-null   float64
21  debtRatio                                2029 non-null   float64
22  effectiveTaxRate                          2029 non-null   float64
23  freeCashFlowOperatingCashFlowRatio       2029 non-null   float64
24  freeCashFlowPerShare                     2029 non-null   float64
25  cashPerShare                             2029 non-null   float64
26  companyEquityMultiplier                  2029 non-null   float64
27  ebitPerRevenue                           2029 non-null   float64
28  enterpriseValueMultiple                   2029 non-null   float64
29  operatingCashFlowPerShare                 2029 non-null   float64
30  operatingCashFlowSalesRatio               2029 non-null   float64
31  payablesTurnover                         2029 non-null   float64
```


Standard scaling is done in code line [8] on the features, excepting the Class target variable. The scaling reduces the variances of the features.

```
[8]: # Normalization
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit(df_ratios_1)
df_ratios_1 = scaler.transform(df_ratios_1)
df_ratios_1=pd.DataFrame(df_ratios_1,\
    columns=['currentRatio','quickRatio','cashRatio','daysOfSalesOutstanding',\
    'netProfitMargin','pretaxProfitMargin','grossProfitMargin',\
    'operatingProfitMargin','returnOnAssets','returnOnCapital',\
    'returnOnEquity','assetTurnover','fixedAssetTurnover','debtEquityRatio',\
    'debtRatio','effectiveTaxRate','freeCashFlowOperatingCashFlowRatio',\
    'freeCashFlowPerShare','cashPerShare','companyEquityMultiplier',\
    'ebitPerRevenue','enterpriseValueMultiple','operatingCashFlowPerShare',\
    'operatingCashFlowSalesRatio','payablesTurnover'])
```

This scaled data is then combined/concatenated with the Class variable in [13]. Note that we do not scale the Class variable here.

```
[13]: TT=pd.concat([df_ratios['Class'],df_ratios_new],axis=1)
TT.shape
TT.tail()
```

```
[13]:
```

	Class	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding	netProfitMargin
2024	1	0.186827	0.135444	1.783407	-0.074822	-0.036575
2025	0	-0.012870	-0.041268	-0.066739	-0.068183	-0.073966
2026	0	-0.060074	-0.054997	-0.120995	-0.041872	-0.047159
2027	0	-0.059442	-0.057857	-0.099558	-0.045460	-0.031518
2028	0	-0.055507	-0.049416	-0.129463	-0.040959	-0.084191

5 rows × 26 columns

The total sample is then randomly split into 75% training set (1521 sample points) and 25% test set (508 sample points).

```
[15]: Train, Test = train_test_split(TT,
                                     test_size=0.25,
                                     random_state=0)

X_train, y_train = Train.iloc[:,1:26], Train.iloc[:,0]
### choose the 25 features
X_test, y_test = Test.iloc[:,1:26], Test.iloc[:,0]
```

```
[16]: X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

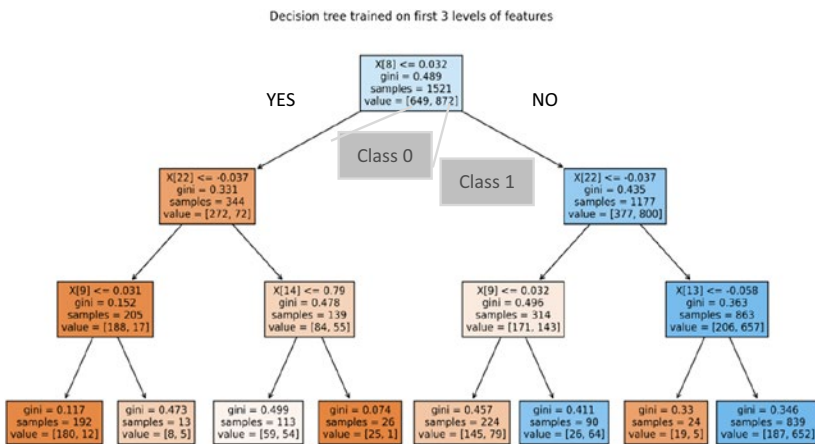
```
[16]: ((1521, 25), (1521,), (508, 25), (508,))
```

The sklearn DecisionClassifier algorithm is then performed on the training data set. See code line [19]. In this preliminary DT classification, only a depth of three levels is applied. The root node and decision nodes and the branching are shown.

Decision Tree

```
[19]: from matplotlib import pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
### The following is experimental DT with only depth of 3 levels
dt1 = DecisionTreeClassifier(max_depth=3,criterion='gini').fit(X_train, y_train)

fig, axes = plt.subplots(nrows = 1,ncols = 1,figsize = (15,8), dpi=300)
plot_tree(dt1, filled=True)
plt.title("Decision tree trained on first 3 levels of features")
plt.show()
```



The text version of the tree is shown as follows.

```
[20]: from sklearn.tree import export_text
text_representation = export_text(dt1)
print(text_representation)
```

```

| --- feature_8 <= 0.03
| | --- feature_22 <= -0.04
| | | --- feature_9 <= 0.03
| | | | --- class: 0
| | | | --- feature_9 > 0.03
| | | | | --- class: 0
| | | --- feature_22 > -0.04
| | | | --- feature_14 <= 0.79
| | | | | --- class: 0
| | | | --- feature_14 > 0.79
| | | | | --- class: 0
| | --- feature_8 > 0.03
| | | --- feature_22 <= -0.04
| | | | --- feature_9 <= 0.03
| | | | | --- class: 0
| | | | --- feature_9 > 0.03
| | | | | --- class: 1
| | | --- feature_22 > -0.04
| | | | --- feature_13 <= -0.06
| | | | | --- class: 0
| | | | --- feature_13 > -0.06
| | | | | --- class: 1

```

There are 649 instances of Class “0” out of 1521 in the training sample or a fraction of 0.4267. Hence, the Gini impurity at root node is computed as $1 - 0.4267^2 - 0.5733^2 = 0.489$. The first decision rule at the root node is found in the feature eight – Operating Profit Margin – a key accounting variable of a firm. The rule based on the scaled number is that if its value < 0.032 , then go to the left branch, if not go to the right branch.

The Gini impurity calculated based on the left branch now is 0.331 with a sample value [272, 72], i.e., 272 cases of Class 0 from the root node of 649 and 72 cases of Class 1 from the root node of 872. The Gini impurity calculated based on the right branch now is 0.435 with a sample value [377, 800], i.e., 377 cases of Class 0 from the root node of 649 and 800 cases of Class 1 from the root node of 872. The mean (weighted average) Gini impurity in the first depth level after the root node is $(344/1521) \times 0.331 + (1177/1521) \times 0.435 = 0.4115$, representing a Gini impurity decrease of $0.489 - 0.4115 = 0.0775$. This feature $X[8]$ at the split of 0.032, amongst all features and their split, has the largest decrease in Gini impurity. This is thus selected as the first node (root node) decision rule.

It is seen that the splitting seems to push more Class 1 to the right sub-trees. Intuitively lower operating profit margin indicates a weaker

firm (Class 0) and hence to the left there are now more firms in Class 0 than in Class 1.

On the left branch, the next decision node is based on feature 22 “Enterprise Value Multiple” – which is EV/EBITA, the total value of a company (EV) (market cap plus debts and cash) relative to its earnings before interest, taxes, depreciation, and amortization (EBITDA). Lower multiple may indicate a weaker firm (this may conversely indicate a better value for acquisition). If this multiple is ≤ -0.037 (recall this is standard-scaled), then the branch goes to the next level on the left with sample value [188, 17] – a high proportion of Class 0 versus Class 1. There is a further Gini impurity reduction of $0.331 - 0.152 = 0.179$. The average/mean Gini impurity decrease, or Gini importance is the (relative sample size) weighted average of the Gini impurity decreases (in absolute values) for both sides of the branching. Basically, as discussed, the decision node decides on the feature that yields the largest weighted average Gini impurity decreases.

If we use only this low depth of 3, then the testing results may not be accurate. For example, our decision rules would be (see the left-most leaf on the third level) predicting “0” (since 180 versus 12 is obviously a higher odds for “0”) if $X[9] \leq 0.031$, $X[22] \leq -0.037$, and $X[8] \leq 0.032$, and so on for the other rules. This predicted probability of Class 0 is $180/192 = 93.75\%$. But it could be higher if more depth is used.

In code line [22], we develop the full DT without constraining the depth – letting the tree nodes expand till all leaves are pure. All features are also considered in each level of splitting with the greedy algorithm seeking the one with largest weighted average Gini impurity decrease. The DT text codes are shown in [22] and the structure of the DT is shown graphically in [25] with 22 levels of depth.

```
[22]: from sklearn.tree import DecisionTreeClassifier
DT_model = DecisionTreeClassifier(criterion='gini')
### note list of default parameters used in above:
### max_depth = none, nodes expanded till all leaves are pure or
### contain < min_samples_split_samples = 2 by default
### max_features = none (default) means max_features = n_features (all).
### When max_features < n_features, the algorithm will select max_features
### at random at each split before finding the best split among them
### min_impurity_decrease = 0 (default case) -- means a node will be split
### if it induces impurity decrease >= this value
```

```
DT_model.fit(X_train, y_train)
y_pred_DT = DT_model.predict(X_test)
Accuracy_DT = metrics.accuracy_score(y_test, y_pred_DT)
print("DT Accuracy:", Accuracy_DT)
```

DT Accuracy: 0.7066929133858267

```
[26]: from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, y_pred_DT)
print(confusion_matrix)
```

```
[[143  72]
 [ 77 216]]
```

```
[27]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_DT))
```

	precision	recall	f1-score	support
0	0.65	0.67	0.66	215
1	0.75	0.74	0.74	293
accuracy			0.71	508
macro avg	0.70	0.70	0.70	508
weighted avg	0.71	0.71	0.71	508

Code lines [22–29] show an accuracy of 70.669%, precision of 75% and recall of 74% for the case of Class 1. The precision and recall for Class 0 are 65% and 67% respectively. High recall in case 0 is important for this situation of being able to predict a low rating firm correctly. Predicting low rating firm to be from a high rating means a bank may lend and may suffer losses. The AUC based on Class 0 is 70.12%.

5.3 Random Forest

RF or Random DTs is an ensemble (collection or group) method for classification as well as regression prediction. In a RF, many, e.g., 500, DTs are run independently (or in parallel). Each of the 500 DTs is done on a sample bootstrap aggregation (“bagging”), i.e., if the training sample is 1521, after one DT is done as seen earlier, the next DT is constructed by resampling on the 1521 sample with replacement, i.e., some cases/firms may appear more than once in a resampled sample of 1521. See also chapters 6, 7 of Geron (2019).

Each of the trained 500 DTs gives a possibly different set of decision rules due to the resampling within the training data set and random selection of a smaller set of features. The X_{train} , and correspondingly X_{test} , i.e., the training data set, change for each re-sampling. Hence, there are 500 different DTs and thus 500 different sets of decision rules. These are applied to the fixed 508 firms in y_{train} and y_{test} , i.e., the test data set.

In the testing, each of the 500 trained DTs will, using its own trained decision rules, yield its prediction vector on the test sample of the same 508 firms. Each of these prediction vector from each of the 500 DTs are possibly different.

Suppose the trained 500 DTs produce 500 vectors of the 508×1 predictions on the 508 test set firms' target labels of "0"s or "1"s. Suppose sum of predicted test cases is shown as follows.

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ \vdots \\ 0 \end{bmatrix} + \dots + \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 1 \end{bmatrix} = \begin{bmatrix} 312 \\ 145 \\ 98 \\ 436 \\ 24 \\ 210 \\ 56 \\ \vdots \\ 396 \end{bmatrix}_{508 \times 1}$$

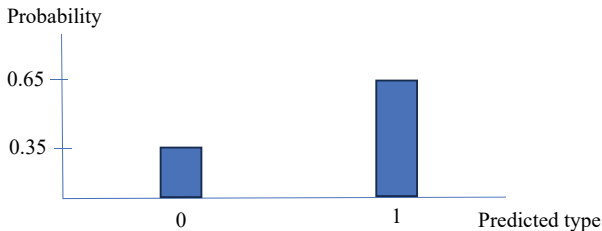
Dividing the 508×1 vector by 500 to show the averaged "vote" or probability of being "1"s – we obtain:

$$P = \begin{bmatrix} 0.624 \\ 0.29 \\ 0.196 \\ 0.872 \\ 0.048 \\ 0.42 \\ 0.112 \\ \vdots \\ 0.792 \end{bmatrix}$$

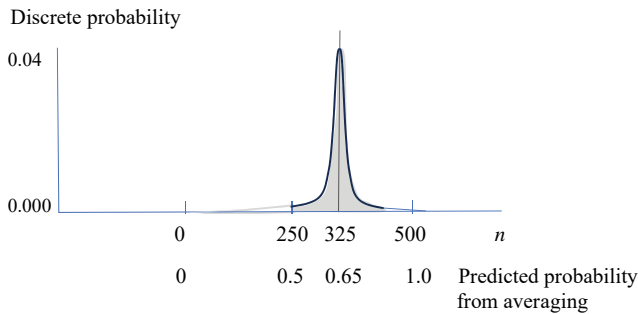
This means that the trained 500 DTs predicted, on averaging, based on the test set, that the first firm in the test set has predicted probability of 0.624. The second firm in the test set has predicted probability of 0.29, and so on. Using probability > 0.5 as threshold, for Class “1”, first firm is predicted as Class “1” and second firm is predicted as Class “0”. Using the actual test y -value or actual firm’s “1”, “0” status, the confusion matrix of the RF can thus be produced, and the various prediction performance metrics can be computed. With a different threshold, e.g., 0.65, using probability > 0.65 as prediction of “1”, the prediction of the first firm is “0” and that of the second firm is “0”. With a threshold, e.g., 0.25, using probability > 0.25 as prediction of “1”, the prediction of the first firm is “1” and that of the second firm is “1”. Hence with different thresholds, and thus different corresponding confusion matrix, the ROC and its AUC can be computed.

In a RF Regression context, each predicted vector (508×1 vector of values) of the test set sample, based on each of the 500 trained DTs, is noted and together they are averaged to find the RF predicted probabilities (508×1 vector). Just like an orchestra, an ensemble produces good accuracy even if individual DTs may falter or are weak performers. The averaging can produce better results as it reduces individual DT variance in the prediction outcomes. The intuition for this is explained as follows.

Consider the single DT. After training, the probability of correctly predicting “1” from a sample point drawn from the test data set could be 0.65 (accuracy of 65%). Suppose we build a second tree, a third tree, and so on (using bagging with random resampling and random selection of subsets of features for training different trees). The accuracies of the other trees could be slightly different such as 68%, 72%, 65%, 71%, etc. We assume for illustration that all the DTs have accuracies of 65%. For the situation of a single DT for prediction, given the test sample point is type “1”, the probability distribution of the likely prediction is Bernoulli, seen as follows.



When 500 DTs in a RF are considered, and assuming they are independent trees, then the number of “1”s predicted follow a binomial distribution. Let n be number of “1”s predicted and $(500 - n)$ be the number of “0”s predicted for the same test sample point or instance of type “1”. The probability of n occurring is $P(n) = {}^{500}C_n(0.65)^n(0.35)^{500-n}$ where ${}^{500}C_n = 500!/(n!(500 - n)!)$. This distribution can be shown as follows using a normal distribution as approximation with mean of the distribution of n as $500(0.65) = 325$, and variance as $500(0.65)(0.35) = 113.75$ or standard deviation of 10.67.



The probability of getting an average number of votes of “1” out of 500 to be more than 50% (shown in the second X -axis) in this test sample point is $\text{Prob}(n > 250)$ which is close to 1 (indicated by the shaded area).

Hence, it is seen that whereas using a single DT can end up with a high chance of wrong prediction of 30%, the ensemble method would have a close to zero chance of wrong prediction in this case.

We continue with the credit rating accounting data set used earlier in Section 5.2. First, we apply the RF method. The sample is similarly split into 25% test size and 75% training size. The following shows code lines from Chapter5-2.ipynb file.

Split the dataset: Creating a Training Set and a Test Set

- 25% of the data will be used as the test set and 75% of the data would be used to train

```
[14]: Train, Test = train_test_split(TT,
                                     test_size=0.25,
                                     random_state=0)

X_train, y_train = Train.iloc[:,1:26], Train.iloc[:,0] ### choose the 25 features
X_test, y_test = Test.iloc[:,1:26], Test.iloc[:,0]
```



```
[15]: X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

```
[15]: ((1521, 25), (1521,), (508, 25), (508,))
```

Sklearn RandomForestClassifier is used with specification of 500 DTs (`n_estimators = 500`). RF method, besides bagging, also possibly randomizes by choosing a subset of the total set of 25 features for each DT. We set `max_features = "sqrt"` which means that each DT randomly uses $\sqrt{25} = 5$ features of the 25 total in constructing the branching in each of the 500 DTs. Therefore, it is important to have many DTs or `n_estimators`. See code line[17]. The number of trees (`n_estimators`) and the number of features to use for each DT are two key hyperparameters in RF.

Random Forest

```
[17]: # Random Forest
from sklearn.ensemble import RandomForestClassifier

RF_model = RandomForestClassifier(n_estimators=500, \
                                random_state=1, max_features="sqrt", max_depth=34)
### default n_estimators/trees = 100; default criterion = 'gini'
### max_features -- The number of features to consider when
### looking for the best split:
### default case is "sqrt"; or "log2", or None (all features),
### unlike DecisionTree Classifier where default case = "none"
### If max_depth=None, then nodes are expanded until all leaves
### are pure or until all leaves contain less than
### min_samples_split samples -- usu 2
### random_state -- resamples each of the n_estimators number of DTs
### bootstrapping - bagging, default=true
### oob_scorebool, default=False -- Whether to use out-of-bag samples
### to estimate the generalization score. Only available if
### bootstrap=True, i.e., if there is resampling using bootstrap.

RF_model.fit(X_train,y_train)
y_pred_RF = RF_model.predict(X_test)
Accuracy_RF = metrics.accuracy_score(y_test, y_pred_RF)
print("RF Accuracy:",Accuracy_RF)
```

```
RF Accuracy: 0.7952755905511811
```

In this investment grade/speculative grade prediction problem, the accuracy of the RF here is 79.53%. The Confusion matrix and the prediction performance classification report are shown as follows. In sklearn, the confusion matrix is usually shown with first line as positives or taking the lower y-value, `y = "0"` in this case.

```
[18]: from sklearn.metrics import confusion_matrix
      confusion_matrix = confusion_matrix(y_test, y_pred_RF)
      print(confusion_matrix)

[[153  62]
 [ 42 251]]
```

```
[19]: from sklearn.metrics import classification_report
      print(classification_report(y_test, y_pred_RF))
```

	precision	recall	f1-score	support
0	0.78	0.71	0.75	215
1	0.80	0.86	0.83	293
accuracy			0.80	508
macro avg	0.79	0.78	0.79	508
weighted avg	0.79	0.80	0.79	508

Compare for class $y = 1$, precision of 80% and recall of 86% in RF versus 75% and 74% for a single DT. For class $y = 0$, RF also has higher precision of 78% and higher recall of 71% compared to those of a single DT of 65% and 67% respectively.

The predicted probability of Class 1 for each of the 508 test cases is given by code lines [20] and [21].

```
[20]: ### Assuming your target is (0,1), then the classifier would output
      ### a probability matrix of dimension (N,2).
      ### The first index refers to the probability that the data belong to class 0,
      ### and the second refers to the probability that the data belong to class 1.
      ### Each row sums to 1.
      ### "RF_model.predict_proba(X_test)[:,:1]" will print a single array of 508
      ### numbers - the second column or the predicted prob of test cases being 1.
      ### "RF_model.predict_proba(X_test)" prints array([[0.242, 0.758], ... for 508 pairs
```

```
[21]: import sklearn.metrics as metrics
      # calculate the fpr and tpr for all thresholds of the classification
      preds_RF = RF_model.predict_proba(X_test)[:,:1]
      print(preds_RF)
```

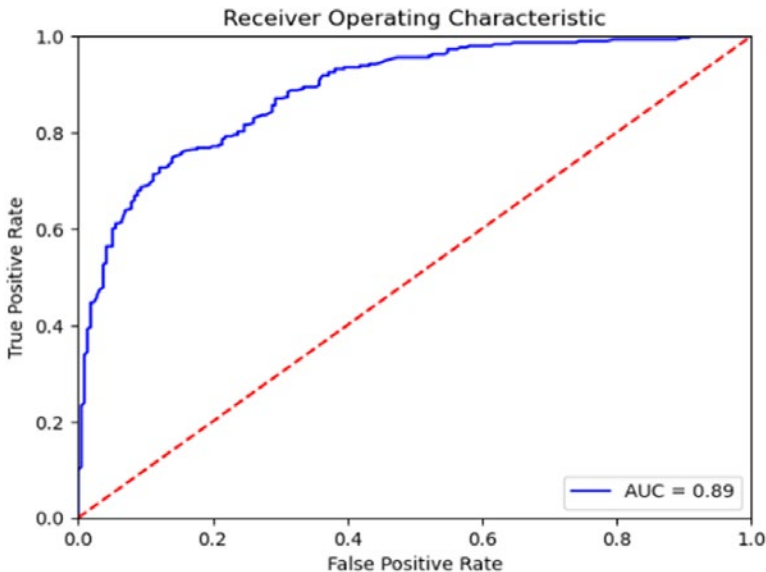
0.758	0.866	0.49	0.508	0.98	0.888	0.836	0.722	0.628	0.82	0.6	0.816
0.712	0.798	0.78	0.508	0.25	0.878	0.348	0.138	0.63	0.954	0.368	0.96
0.522	0.634	0.752	0.76	0.204	0.23	0.794	0.36	0.354	0.734	0.636	0.156
0.862	0.14	0.652	0.872	0.892	0.868	0.286	0.21	0.894	0.162	0.944	0.736
0.674	0.58	0.836	0.938	0.794	0.636	0.226	0.68	0.534	0.718	0.882	0.552
0.866	0.088	0.73	0.432	0.156	0.598	0.398	0.268	0.936	0.922	0.464	0.33
0.834	0.968	0.096	0.386	0.796	0.452	0.846	0.872	0.966	0.046	0.94	0.774
0.664	0.78	0.948	0.688	0.32	0.59	0.772	0.148	0.594	0.05	0.502	0.09

The probabilities allow computation of the Receiver Operating Characteristic curve when the threshold for predicting the class is varied from 0 to 1. Code lines [22–23] show computations of the AUC in ROC.

```
[22]: fpr, tpr, thresholds = metrics.roc_curve(y_test, preds_RF)
      ## matches y_test of 1's and 0's versus pred prob of 1's for each of
      ## the 508 test cases
      ## sklearn.metrics.roc_curve(y_true, y_score,...) requires y_true as
      ## 0,1 input and y_score as prob inputs
      ## metrics.roc_curve returns fpr, tpr, thresholds (Decreasing thresholds
      ## used to compute fpr and tpr)
      roc_auc_RF = metrics.auc(fpr, tpr)
      ## sklearn.metrics.auc(fpr,tpr) returns AUC using trapezoidal rule
      roc_auc_RF
```

```
[22]: 0.8868640368283197
```

```
[23]: import matplotlib.pyplot as plt
      plt.title('Receiver Operating Characteristic')
      plt.plot(fpr, tpr, 'b', label = 'AUC = %0.2f' % roc_auc_RF)
      plt.legend(loc = 'lower right')
      plt.plot([0, 1], [0, 1], 'r--')
      plt.xlim([0, 1])
      plt.ylim([0, 1])
      plt.ylabel('True Positive Rate')
      plt.xlabel('False Positive Rate')
      plt.show()
```



With RF, the AUC is 88.69%. This compares well with 70.12% AUC for a single DT.

With RF, we can also analyse the “Feature Importance”. For each of the 500 DTs, the features that are involved in the splitting and hence Gini importance (mean decrease in Gini impurity) from the decision node down to the next two branched nodes are noted.

Over all the 500 DTs, the total Gini importance contributed by each of the 25 features are tabulated, including contributions by the same feature more than once in a DT. This feature total is then divided by the total across the totals of the 25 features. This ratio is called the Feature Importance of the particular feature. The ratios of all features sum to 100%. The higher the % of each feature in this contribution to overall Gini importance or overall Gini weighted impurity decrease, the more important is this feature in the RF algorithm. The examining of feature importance is shown as follows in code line [25].

```
[25]: RF_model.feature_importances_ ### they sum to one

[25]: array([0.03539456, 0.03190046, 0.04191705, 0.02772411, 0.05589552,
            0.04333299, 0.02547268, 0.031284 , 0.06059753, 0.05753511,
            0.05522376, 0.02494506, 0.03374492, 0.03015326, 0.03604814,
            0.0375338 , 0.03397923, 0.03182288, 0.03024578, 0.03071357,
            0.05349687, 0.03554667, 0.0806157 , 0.0473035 , 0.02757288])
```

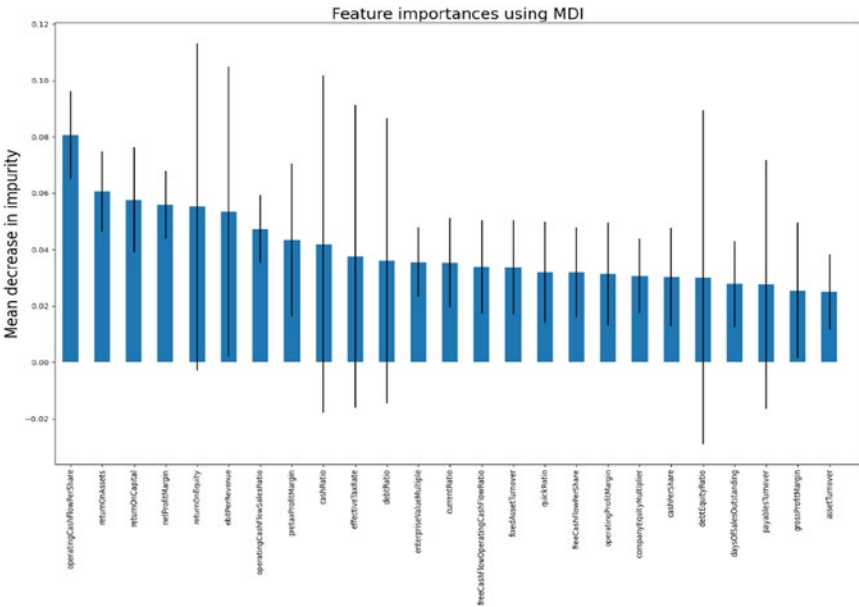
The feature importance output is more conveniently visualized. See code line [26].

```
[26]: import pandas as pd

forest_importances = pd.Series(importances, index=X_train.columns)
forest_importances.sort_values(ascending=False,inplace=True)

fig, ax = plt.subplots(figsize=(15,12))
forest_importances.plot.bar(yerr=std, ax=ax)
### thin line indicates 1 std err from the mean either way
### -- doesn't mean mean decrease is neg
ax.set_title("Feature importances using MDI",fontsize=20)
### MDI is mean decrease in impurity
ax.set_ylabel("Mean decrease in impurity",fontsize=20)
ax.set(xticks=[ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,
                24,25]))
### define own x-ticks to avoid clutter, entry must be a list
fig.tight_layout()

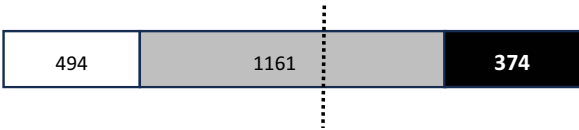
### Note: Negative feature importance value means that feature makes the loss go up.
### Either model is underfitting with not enough iteration and not enough splitting
### use of feature or feature should be removed.
```



The output shows that in order of Feature importance, the features are Operating Cashflow per share, Return on Assets, Return on Capital, Net Profit Margin, Return on Equity, EBIT per Revenue, and so on. These are key accounting variables/ratios of the firm that largely determine the credit quality and hence if the firm is of investment grade or of speculative grade.

5.4 Multi-Class Classification

Sometimes binary classification may not be the best way to classify if the types have the following distributions where the numbers indicate the instances.



These classes are akin to the Class 2 for white sector, Class 1 for grey sector, and Class 0 for black sector. Banks facing potential borrowing

firms would try to accurately predict their unknown rating status, especially if they have bad ratings belonging to Class 0 or the black sector. Banks would also like to predict if the potential client is a good firm of Class 2 so that more favorable lending rates can be given to maintain longer client relationship. Hence, it is more relevant to perform multi-class (3-class) prediction in this case.

We continue with the corporate ratings data set in Section 5.2 but now re-classify the firms into 3 classes, viz., Class 2 for AAA, AA, A ratings, Class 1 for BBB, BB ratings, and Class 0 for B, CCC, CC, C, and D ratings. The code file is Chapter5-2M.ipynb and the data set is corporate_rating3.csv. The 2029 cases are now divided into the 3 classes as follows, shown in code line [4].

```
[4]: Class
     1    1161
     2     494
     0     374
```

Standard scaling is done, as in the binary classification, to the 25 features in code line [7]. The data set is similarly divided into 75% training data and 25% test data (assuming validation phase has been done). There are 508 test cases and the distribution of these in Classes 2, 1, 0 is shown as follows.

```
[16]: # Display the no. of cases in classes '0', '1', '2'
      pd.Series(y_test).value_counts()

[16]: Class
     1    284
     2    132
     0     92
```

The RF method is performed on this multi-class prediction problem. Code line [18] shows that with ensemble of 1000 DTs, the accuracy is 69.29%.

Random Forest

```
[18]: # Random Forest
      from sklearn.ensemble import RandomForestClassifier

      RF_model = RandomForestClassifier(n_estimators=1000, random_state=1, \
                                       max_features="sqrt", max_depth=34, criterion="entropy")
```



```
RF_model.fit(X_train,y_train)

y_pred_RF = RF_model.predict(X_test)
Accuracy_RF = metrics.accuracy_score(y_test, y_pred_RF)
print("RF Accuracy:",Accuracy_RF)

RF Accuracy: 0.6929133858267716
```

Code lines [21] and [22] show the computed confusion matrix and the classification report for this multi-class prediction.

```
[21]: from sklearn.metrics import confusion_matrix
confusion_matrix_RF = confusion_matrix(y_test, y_pred_RF)
print(confusion_matrix_RF)

[[ 36  54   2]
 [ 12 249  23]
 [   0  65  67]]
```

The first, second, and third row (from top to bottom) in the confusion matrix denotes actual Class 0, 1, and 2 test instances. The first, second, and third columns (from left to right) denotes the predicted Class 0, 1, and 2, respectively. The recall for Class 0 is $36/(36 + 54 + 2) = 39.1\%$. The precision for Class 0 is $36/(36 + 12 + 0) = 75\%$. The recall for Class 2 is $67/(67 + 65 + 0) = 50.75\%$. The precision for Class 2 is $67/(67 + 23 + 2) = 72.8\%$.

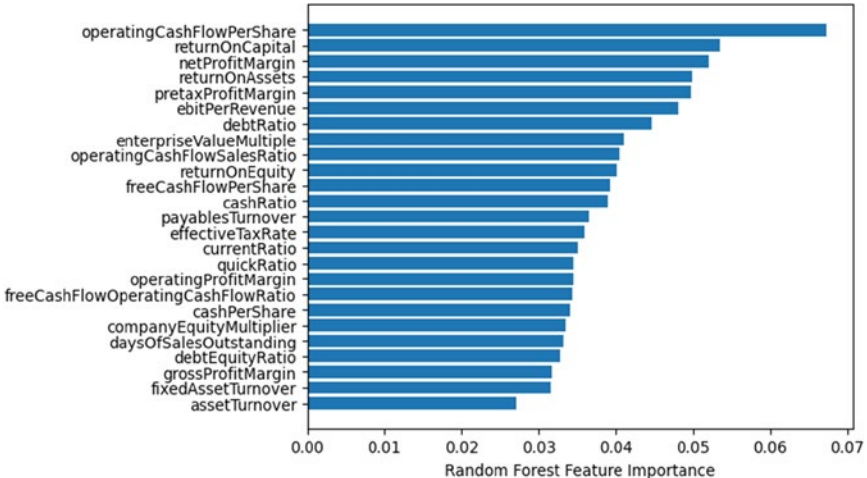
```
[22]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_RF))
```

	precision	recall	f1-score	support
0	0.75	0.39	0.51	92
1	0.68	0.88	0.76	284
2	0.73	0.51	0.60	132
accuracy			0.69	508
macro avg	0.72	0.59	0.63	508
weighted avg	0.70	0.69	0.68	508

The recall for Class 0 appears low and can be improved via more effective algorithms.

The feature importance is computed via code line [26] in a different presentation as follows. They are largely similar to those in the binary classification. Other measures and issues of feature importance are discussed in chapter 8 of De Prado (2018) and in Altmann *et al.* (2010).

```
[26]: ### another representation
sorted_idx = RF_model.feature_importances_.argsort()
plt.barh(X_train.columns[sorted_idx], RF_model.feature_importances_[sorted_idx])
plt.xlabel("Random Forest Feature Importance")
plt.figure(figsize=(20, 15), dpi=80)
fig.tight_layout()
```



For multi-class classification, it is not a natural step to construct a single ROC curve since the ROC curve involves the true positive rate plot versus the false positive rate and these come more naturally from a binary classification table. However, we can construct 3 separate ROCs, one for each class as the positive case.

Code line [27] shows how to pull out the predicted probability vectors “preds_RF[:,0]”, “preds_RF[:,1]”, “preds_RF[:,2]” for Class 0, 1, and 2, respectively. Each vector gives the probability of class in the 508 instances.

```
[27]: import sklearn.metrics as metrics
preds_RF = RF_model.predict_proba(X_test)
### Target here is ('0','1','2'). RF_model.predict_proba(X_test) produces
### 508 x 3 matrix of 3 cols. First col preds_RF[:,0] row i gives
### predicted prob that case i belongs to class '0'
### Second col preds_RF[:,1] row i gives predicted prob that case i
### belongs to class '1'. Third col preds_RF[:,2] row i gives predicted
### prob that case i belongs to class '2'. All elements in row i,
### i.e., preds_RF[i,0] + preds_RF[i,1] + preds_RF[i,2] = 1
### We need these probs to compare against varying thresholds to give ROC

preds_RF_0 = preds_RF[:,0]
preds_RF_1 = preds_RF[:,1]
preds_RF_2 = preds_RF[:,2]
```

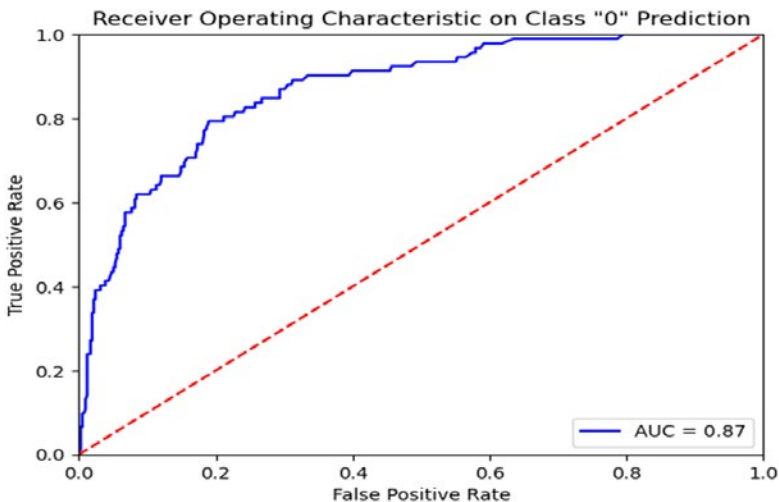

Code line [28] binarizes each value of either “1”, “2”, or “3” in `y_test` to three columns. In the first column, the value is “1” if the test instance is of Class 0, otherwise it is “0”. In the second column, the value is “1” if the test instance is of Class 1, otherwise it is “0”. In the third column, the value is “1” if the test instance is of Class 2, otherwise it is “0”. Thus “`yy`” is a 508×3 matrix.

```
[28]: from sklearn.preprocessing import label_binarize
yy = label_binarize(y_test, classes=[0,1,2])
### yy[:,0] is 508 x 1 where kth row = 1 if it belongs to class 0, o.w. 0
### yy[:,1] is 508 x 1 where kth row = 1 if it belongs to class 1, o.w. 0
### yy[:,2] is 508 x 1 where kth row = 1 if it belongs to class 2, o.w. 0
```

To construct the ROC for Class 0, we match the binarized “`yy[:,0]`” (first column) with “`preds_RF[:,0]`”, the predicted probability that each row instance is in Class 0. This is shown in code line [29] and the ROC is plotted as follows. The area under the ROC or AUC is 87.0%.

```
[29]: import sklearn.metrics as metrics
### calculates the fpr and tpr for all thresholds of the classification 0
### prediction on y_test
fpr_0, tpr_0, thresholds = metrics.roc_curve(yy[:,0], preds_RF_0)
### matches y_test of 1's and 0's versus pred prob of 1's for each of the 508
### test cases/instances for '0' class
### sklearn.metrics.roc_curve(y_true, y_score,...) requires y_true as 0,1 input
### and y_score as prob inputs
### metrics.roc_curve returns fpr, tpr, thresholds (Decreasing thresholds used
### to compute fpr and tpr)
roc_auc_RF_0 = metrics.auc(fpr_0, tpr_0)
### sklearn.metrics.auc(fpr,tpr) returns AUC using trapezoidal rule
roc_auc_RF_0
```

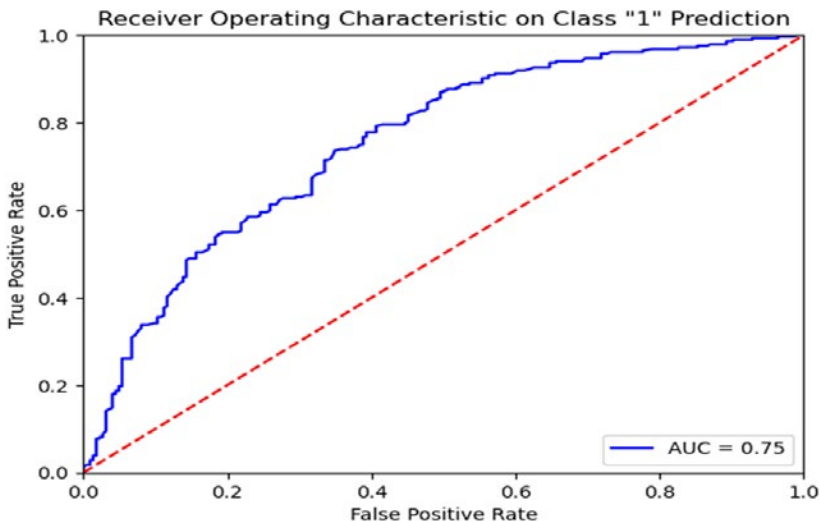
```
[29]: 0.8695521530100334
```



To construct the ROC for Class 1, we match the binarized “yy[:,1]” (first column) with “preds_RF[:,1]”, the predicted probability that each row instance is in Class 1. This is shown in code line [31] and the ROC is plotted as follows. The area under the ROC or AUC is 75.4%.

```
[31]: import sklearn.metrics as metrics
      ## calculate the fpr and tpr for all thresholds of the classification 1
      ## prediction on y_test
      fpr_1, tpr_1, thresholds = metrics.roc_curve(yy[:,1], preds_RF_1)
      roc_auc_RF_1 = metrics.auc(fpr_1, tpr_1)
      roc_auc_RF_1
```

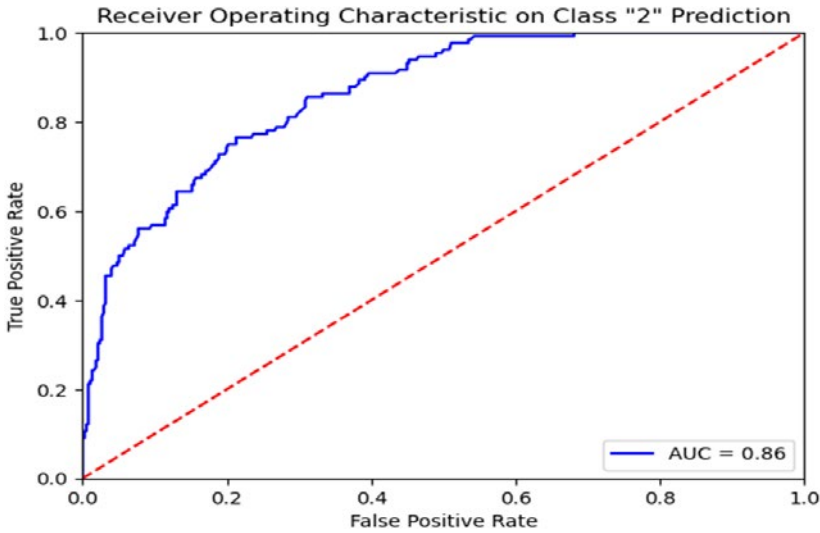
```
[31]: 0.7543228118712275
```



Finally, to construct the ROC for Class 2, we match the binarized “yy[:,2]” (third column) with “preds_RF[:,2]”, the predicted probability that each row instance is in Class 2. This is shown in code line [33] and the ROC is plotted as follows. The area under the ROC or AUC is 86.4%.

```
[33]: import sklearn.metrics as metrics
      ## calculate the fpr and tpr for all thresholds of the classification 2
      ## prediction on y_test
      fpr_2, tpr_2, thresholds = metrics.roc_curve(yy[:,2], preds_RF_2)
      roc_auc_RF_2 = metrics.auc(fpr_2, tpr_2)
      roc_auc_RF_2
```

```
[33]: 0.8635658446163765
```



5.5 Concluding Thoughts

DTs are a non-parametric method to predict the type or label of a case or instance based on the features that come with the instance. In this chapter, we see how the accounting features such as accounting and financial ratios of a firm can be used to predict the credit rating of the firm.

When the ensemble approach is taken as in the RF algorithm, RF reduces the variance of outcomes in a single DT. RF also randomizes selection of features. RF thus handles high dimensionality well as it can reduce use of all features; it also handles missing values since any missing value amounts to having not selected that feature for that row. However, a RF prediction outcome is also harder to explain or interpret since there is no fixed set of features and weights as it is a collection of different DTs.

We also see how in the problem of a bank predicting a firm's credit rating and thus the willingness to lend, it is more relevant to perform multi-class (3-class) prediction in this case. It is more important to predict low credit-rating firms and avoid such loans to reduce potential default loss. Secondly, it is also important to predict high credit-rating firms and encourage such loans to build up a healthy pipeline of lending profits.

There is another popular ensemble technique to create an ensemble or collection of predictors. The idea is also to construct many DTs – but the difference is that the target variable for each subsequent DT is different.

The idea with each subsequent DT is to solve for the net error left from the past DTs and to build stronger and more accurate DT starting from weaker ones. The latter technique is called gradient boosting and is covered in the next chapter.

References

- Altmann A., Laura Tolosi, Oliver Sander, and Thomas Lengauer (2010). Permutation importance: A corrected feature importance measure. *Bioinformatics*, 26 (10).
- Geron A. (2019). *Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow*, 2nd ed. O'Reilly Publisher.
- De Prado, M. L. (2018). *Advances in Financial Machine Learning*. Wiley.

This page intentionally left blank

Chapter 6

Gradient Boosting, SHAP Values

Another important class of algorithms that uses Decision Trees (DTs), besides Random Forest (RF), is the Gradient Boosting (GB) method.

6.1 Gradient Boosting

We have seen how a single DT can be unstable. Small changes to the training set such as changing of a few sample points or instances and its related features/explanatory variables, or choice of different sub-sets of features (or omitting some) can produce different trees with different predictions on a test sample. A RF on the collection or ensemble of independently trained DTs may “average” the variability of each DT and yield a more accurate forecast. RF also randomizes selection of features and resamples for computing different DTs – the bagging approach.

Like the RF method, the GB method is also an ensemble of DTs. However, RF and GB differ in the way they combine the DTs. RF combines all independently generated DTs at the end while the GB method combines the DTs as each subsequent DT is generated in a dependent way from the previous DTs. Each subsequent DT may have a different set of target values, and the DT is also called a boosting tree. The idea of GB is to start with weak trees (trees with large prediction errors) and then strengthens them as the algorithm proceeds to “solve” for the net errors left from the past DTs.

There are GB methods for continuous or discrete target variable fitting and GB methods for binary and multi-class classifications. We first provide an illustration of the GB methods for continuous or discrete target variable fitting.

A GB method (or GB Trees) initially constructs a weak predictor and checks for the training error on each sample point. In regression, the initial weak predictor could be just the sample mean $\sum_{j=1}^N Y_j / N$ as predictor for every Y_j where Y_j is the target value for the j th training sample point. Call this initial predictor $F_0 \equiv \bar{Y}$. The initial errors of fitting for each sample point in the training set data are $e_{(0)j} = Y_j - \bar{Y}$, for each j in the training sample.

GBT then builds a first DT, DT(1), to fit/predict $e_{(0)j}$ using features of each sample point j . This mapping or DT works by using each feature value X_{jk} (each feature k) to decide which next sub-group the sample point j belongs to, until sample point j is led by its features $\{X_{jk}\}$ and by the DT(1) decision rules to a terminal leaf with a fitted/predicted value. The fitting follows the splitting criterion of minimizing weighted Mean Squared Errors (MSE) in DT. Let the terminal value for a $e_{(0)j}$ be h_{1j} .

The process can be depicted as follows in Figure 6.1.

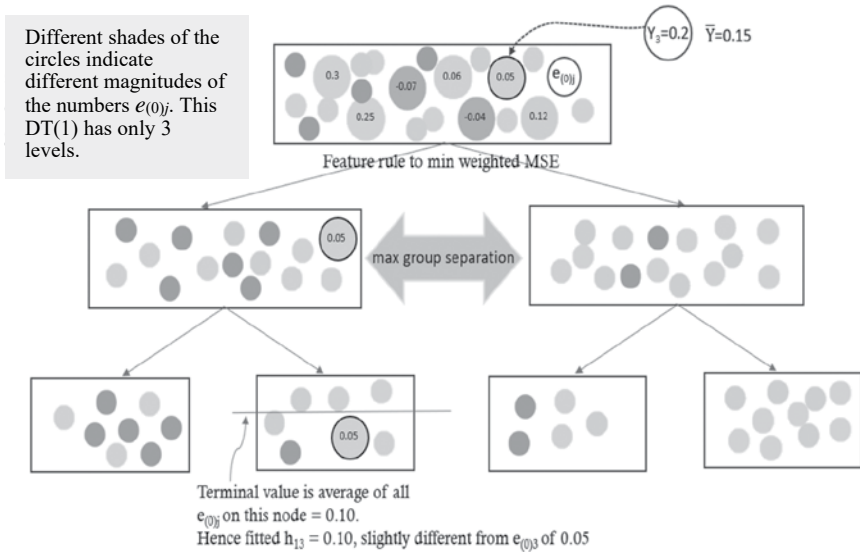


Figure 6.1. The process of GB.

In general, we expect to see small differences in targets of $e_{(0)j}$ in DT(1) and the predicted values of h_{1j} . But a good training tree would make this difference close to zero. Even if the DT(1) should in this training data set produce h_{13} exactly equal to $e_{(0)3} = 0.05$, when the leaf node (final node on tree)

is thoroughly resolved to contain only the point or points predicting the same $e_{(0)j}$ values of 0.05, the DT(1) decision rules when applied to a test sample point with approximately similar features may still yield a slightly different predicted h_{1j} as the features are not identical to that in the training sample point. And if the features are identical, but the test sample point has a slightly different target value than that in the training sample point, then there is still a slight error in prediction.

In GB, to prevent overshooting (rising above the target from below or falling below the target from above), a learning rate (hyperparameter) $\alpha \in (0, 1]$, e.g., $\alpha = 0.1$, is imposed on the predicted value h_{1j} within the algorithm. This regularization does not typically occur in a standalone DT. In the case here, the GB algorithm produces the fitted value/predictor of $e_{(0)j}$ as $F_{1j} = \alpha h_{1j}$. The updated value of the prediction of Y_j is $F_0 + F_{1j}$. The fitting/prediction error from DT(1) is then $e_{(1)j} = Y_j - F_0 - F_{1j} = e_{(0)j} - F_{1j}$, for each j in the training sample. Note how the training data set enables use of the label Y_j to determine the next errors $e_{(1)j}$ (for each sample point j in training set) to fit.

Suppose some measure of the total errors $e_{(1)j}$ for all j is not small enough. GBT then builds a second DT in the next stage to train a better DT. And then a third tree to improve on the training, and so on. Thus, unlike the RF method where independent DTs are produced for “averaging”, GBT adjusts the algorithm for the next and subsequent DTs (hence dependence of the DTs) to improve on the training based on fitting errors in earlier DTs.

In DT(2), let the terminal (predicted) value for a $e_{(1)j}$ be h_{2j} . The GB algorithm produces the fitted value/predictor of $e_{(1)j}$ as $F_{2j} = \alpha h_{2j}$. The updated value of the prediction of Y_j is $F_0 + F_{1j} + F_{2j}$. The fitting/prediction error from DT(2) is then $e_{(2)j} = Y_j - F_0 - F_{1j} - F_{2j} = e_{(1)j} - F_{2j}$, for each j in the training sample.

Suppose some measure of the total errors $e_{(2)j}$ for all j is still not small enough. The third DT(3) is built to predict $e_{(2)j}$ for all j . The GB algorithm produces the fitted value/predictor of $e_{(2)j}$ as $F_{3j} = \alpha h_{3j}$. The updated value of the prediction of Y_j is $F_0 + F_{1j} + F_{2j} + F_{3j}$. The fitting/prediction error from DT(3) is then $e_{(3)j} = Y_j - F_0 - F_{1j} - F_{2j} - F_{3j} = e_{(2)j} - F_{3j}$, for each j in the training sample. When the maximum number of DTs as in DT(n) is reached or till the measure of errors $e_{(n)j}$, for all j , becomes smaller than a pre-determined size, then the GB tree prediction is $F_0 + F_{1j} + F_{2j} + F_{3j} + \dots + F_{nj}$ for each Y_j . The boosting regression trees are additive.

Note that in a GB Tree algorithm, the DTs are constructed sequentially in a dependent way whereby previous training errors are considered in building the next tree. Finally, all the trees are added up to form the prediction. The outputs for each of the sequential trees are in a fixed order. This is unlike the RF where each DT can be independently evaluated and hence can be computed in parallel. GB algorithms typically start with weak trees, i.e., trees that produce large errors. Then, stronger trees are subsequently built to predict those errors and add the predicted errors up to form the final prediction. In this sense, subsequent trees put more “weight” or consideration to sample points in previous trees with larger errors for subsequent error predictions (as larger errors carry more “weight” for fitting in squared error sense for optimal splitting). Computationally, GBT is generally more intensive.

We provide a numerical example of the above procedure as follows. Suppose $N = 4$ and the set of target values, initial predictor (sample mean), and initial fitting errors are shown.

Y_j	0.5	1.0	1.2	1.3
F_0	1.0	1.0	1.0	1.0
$e_{(0)j}$	-0.5	0.0	0.2	0.3

Next, DT(1) computes h_{1j} , hence F_{1j} . Note $F_{1j} = \alpha h_{1j}$ where $\alpha = 0.1$ here. Next error is $e_{(1)j} = e_{(0)j} - F_{1j}$.

h_{1j}	-0.8	-0.5	0.4	0.9
F_{1j}	-0.08	-0.05	0.04	0.09
$e_{(1)j}$	-0.42	0.05	0.16	0.21

Next, DT(2) computes h_{2j} , hence F_{2j} . Next error is $e_{(2)j} = e_{(1)j} - F_{2j}$.

h_{2j}	-0.5	0.4	0.7	1.5
F_{2j}	-0.05	0.04	0.07	0.15
$e_{(2)j}$	-0.37	0.01	0.09	0.06

We can see that if the algorithm works, $e_{(n)j}$ gets smaller toward zero.

More formally, a GB Model develops stronger trees sequentially by minimizing a loss function at each new tree. The loss function in a GB algorithm is directly connected with setting up the next target variable (the “error”) to fit, in the subsequent boosting trees in a GB approach. We just saw how the subsequent target = “error” = original target less updated fitted/predicted value, viz., $e_{(1)j} = Y_j - (F_0 + F_{1j})$, $e_{(2)j} = Y_j - (F_0 + F_{1j} + F_{2j})$, $e_{(3)j} = Y_j - (F_0 + F_{1j} + F_{2j} + F_{3j})$, and so on.

Subsequent trees put more “weight” or consideration to sample points in previous trees with larger errors such as in squared error sense. The loss function as in MSE puts more importance to fit outliers as these impute larger losses. However, if outliers are erroneous, e.g., due to data entry error, then such fitting would produce poor forecasts as the model overfitted outliers. Using mean absolute error (MAE) as loss function would be more robust to effect of outliers in fitting but may not perform well if the outliers reflect genuine higher risk cases.

We assume outliers are removed or that they posed as genuine possibilities. Suppose the loss function is MSE or is quadratic in the fitted/predicted “error”. We minimize the MSE or equivalently the sum of squared errors (with a scalar $\frac{1}{2}$ multiple) $L = \frac{1}{2} \sum_{j=1}^N \left(Y_j - \sum_{i=0}^n F_{ij} \right)^2$ (for an optimal n number of GB trees) where N is the sample size of the training data set. Typically, $n < N$. At each boosting stage, e.g., in the algorithm of the m th ($m \leq n$) tree, the training loss function would be $\frac{1}{2} \sum_{j=1}^N \left(Y_j - F_0 - F_{1j} - \dots - F_{m-2,j} - F_{m-1,j} \right)^2$. To find the minimum of the loss function, supposing $F_{m-1,j}$ is some function of the features X_{jk} , we could set the following derivatives to zeros, i.e.,

$$\begin{aligned} \frac{\partial F_{m-1,j}}{\partial X_{jk}} \frac{\partial}{\partial F_{m-1,j}} \frac{1}{2} \sum_{j=1}^N \left(Y_j - F_0 - F_{1j} - \dots - F_{m-2,j} - F_{m-1,j} \right)^2 \\ = - \frac{\partial F_{m-1,j}}{\partial X_{jk}} \sum_{j=1}^N \left(Y_j - F_0 - F_{1j} - \dots - F_{m-2,j} - F_{m-1,j} \right) = 0. \end{aligned}$$

In principle, the minimum loss could be attained when $\sum_{j=1}^N F_{m-1,j} = \sum_{j=1}^N Y_j - F_0 - F_{1j} - \dots - F_{m-2,j}$. However, as we are dealing with DTs, it is

not possible to explicitly derive an analytical function of $F_{m-1,j}$ in terms of X_{jk} for every j such that $\sum_{j=1}^N F_{m-1,j} = \sum_{j=1}^N Y_j - F_0 - F_{1j} - \dots - F_{m-2,j}$. But we can try to find for each j , $F_{m-1,j}$ that is close to $Y_j - F_0 - F_{1j} - \dots - F_{m-2,j} = e_{(m-2)j}$ using the DT algorithm. And we can try to find for each j , F_{mj} that is close to $Y_j - F_0 - F_{1j} - \dots - F_{m-2,j} - F_{m-1,j} = e_{(m-1)j}$ and so on.

Note that the “error” $e_{(m-1)j}$ as the target value (for every j) of the m th boosting tree, $e_{(m-1)j} = Y_j - F_0 - F_{1j} - \dots - F_{m-1,j}$, is also element in the negative of the gradient term $\frac{\partial L}{\partial F_{m-1,j}}$

$$-\sum_{j=1}^N (Y_j - F_0 - F_{1j} - \dots - F_{m-2,j} - F_{m-1,j})$$

with respect to the loss function. Hence, this approach via reducing “errors” or reducing the gradients toward zeros is thus called GB method. Reducing the loss function is directly connected with setting up the next target errors $e_{(m-1)j}$ for fitting.

Visually, GB regression trees for fitting training data can be seen as follows in Figure 6.2. Note that the features are typically randomly selected for different boosting trees. The quantities on the top line $\{Y_j - \bar{Y}\}$, $\{Y_j - F_0 - F_{1j}\}$, $\{Y_j - F_0 - F_{1j} - F_{2j}\}$, etc., denote the target errors $e_{(0)j}$, $e_{(1)j}$, $e_{(2)j}$ for the trees DT(1), DT(2), DT(3), and so on. The bottom line quantities $\{F_{1j}\}$, $\{F_{2j}\}$, $\{F_{3j}\}$, etc. denote the predicted errors of the DTs.

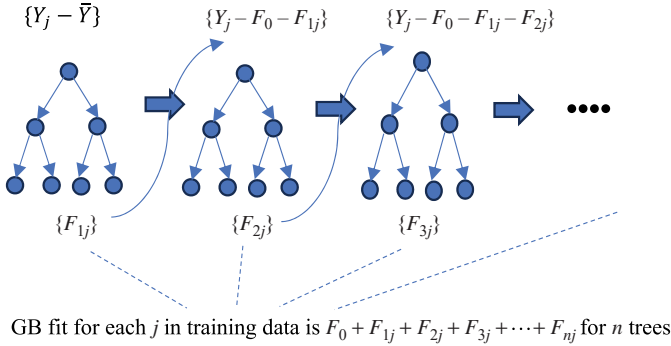


Figure 6.2. GB regression.

For the test sample or prediction of any test sample point given its features, a regression DT(1) is illustrated as seen in Figure 6.3.

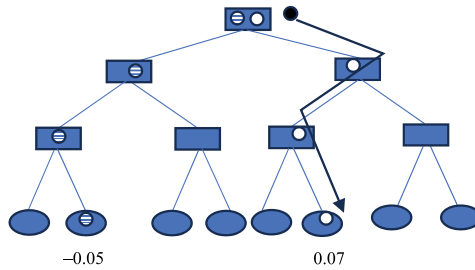


Figure 6.3. Illustrating prediction of a test sample point.

The training was done by training data points starting from the root node. The test point (black circle)'s features X_{jk} were used in the DT(1) rules to lead to 0.07.

The trained GB trees DT(1) to DT(n) are used to predict each test sample point j via $\hat{F}_{1j} = 0.07$ from DT(1), $\hat{F}_{2j}$ from DT(2), and so on, without employing the test point's label. Then the GB predictor for the test point j is $\hat{F}_{0j} + \hat{F}_{1j} + \hat{F}_{2j} + \hat{F}_{3j} + \dots + \hat{F}_{nj}$. Start term $\hat{F}_{0j}$ could be the same as F_{0j} as it does not involve information from the test point label. This is seen in Figure 6.4.

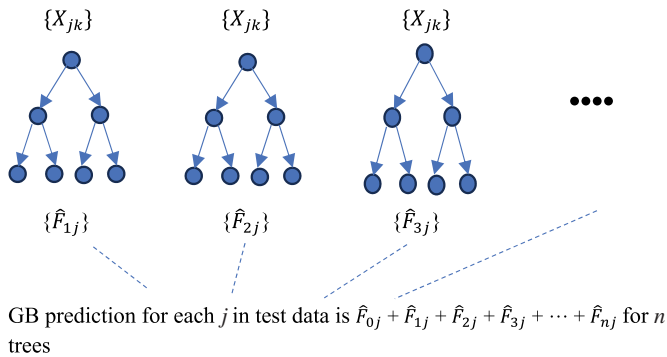


Figure 6.4. GB prediction of a test instance.

The loss function is typically synonymous with the splitting criterion in the case of a single DT or RF. However, in a GB algorithm, the loss function, and the splitting criterion for building decision rules in each of the DTs may be different. In the sklearn GradientBoostingRegressor, the loss function could be squared error L as seen earlier. The splitting criterion can be “MSE” as seen in minimizing weighted MSE in RF, but it can

also be Friedman MSE which is finding the split (optimal feature and its threshold) yielding maximum information gain

$$\frac{W_L W_R}{W_L + W_R} (\bar{Y}^L - \bar{Y}^R)^2$$

where $\bar{Y}^L$ is mean of L training sample points' labels $e_{(m-1)j}$ (for some j) on left node on the m th tree at any level, $\bar{Y}^R$ is mean of R training sample points' labels $e_{(m-1)j^*}$ (for some other j^*) on right node on the m th tree at the same level, w_L is weight given by $\sum_{j=1}^L \hat{p}_j (1 - \hat{p}_j)$, w_R is weight given by $\sum_{j^*=1}^R \hat{p}_{j^*} (1 - \hat{p}_{j^*})$, and $\hat{p}_j$, $\hat{p}_{j^*}$ are probability estimates of label $y = 1$ from a pre-trained training set logistic regression of labels of 1 or 0 on the features (or a simpler implementation such as empirical probabilities). The information gain is larger for the same $(\bar{Y}^L - \bar{Y}^R)^2$ when the uncertainties contained in w_L and w_R are larger, i.e., $\frac{w_L w_R}{w_L + w_R}$ is larger.

We next provide explanation of the GB method for binary classification. Here, instead of fitting sequential errors of continuous variable predictions, we fit sequential residuals that are differences between the observed classification value (1 or 0) and the predicted probability of the case in Class 1.

As more training proceeds with sequential or boosting DTs, the probability estimates get more accurate, the residuals (or pseudo residuals – since the probability estimates are model driven and not explicitly observed attributes) reduce toward zero, and eventually the probability estimate for each case j is used to decide if a target belongs to one category or the other. If the case or target has probability $> 1/2$, then prediction is that it is in Class 1. If the case or target has probability $\leq 1/2$, then prediction is that it is in Class 0.

We provide a numerical example of a typical GB classification algorithm as follows. The targets are credit ratings of firms, belonging to either investment grade (BBB and above) or else speculative grade (BB and below). Suppose there are 10 firms, $N = 10$. Each firm has features that are accounting variables of the firm. Let the four available features be X_{1j} , X_{2j} , X_{3j} , and X_{4j} , where $j = 1, 2, \dots, 10$. Let firm j have target rating of $Y_j = 1$ for investment grade, or else $Y_j = 0$ for speculative grade. The target values of the firms are shown below.

j	1	2	3	4	5	6	7	8	9	10
Y_j	0	1	0	0	0	1	1	1	0	0

The empirical unconditional probability of $Y_j = 1$ for each j is 0.4 since 4 out of 10 in the sample have values as 1. A GB method initially constructs a weak initial predictor of the probability of $Y_j = 1$ for each Y_j , as 0.4. Note that at the end of the GB algorithm, the final prediction of Y_j is 1 if the final predicted probability is greater than $\frac{1}{2}$, and the final prediction of Y_j is 0 if the final predicted probability is less than or equal to $\frac{1}{2}$.

Call the initial predicted probability p_0 , of $Y_j=1$, to be 0.4. The initial pseudo residuals in the probability fitting of the training data is actual $Y_j - p_0$, either $1 - 0.4 = 0.6$ or $0 - 0.4 = -0.4$. The first DT(1) is used to fit/predict $Y_j - p_0$. The DT(1) is shown as follows. Three features were randomly selected to form the decision rules. (The feature values are just examples.) The splitting criterion is Friedman MSE. This is shown in Figure 6.5.

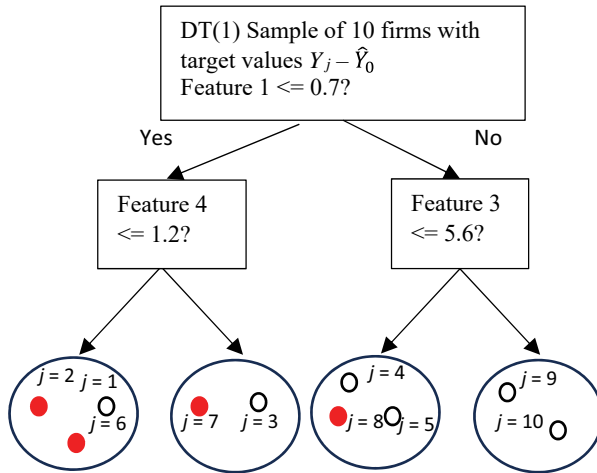


Figure 6.5. Splitting in a GB tree.

In the terminal nodes, the coloured circles represent firms with pseudo residual 0.6 ($Y_j = 1$) while the uncoloured circles represent firms with pseudo residual -0.4 ($Y_j = 0$). The splits are performed based on the Friedman MSE criterion or else weighted MSE criterion on the pseudo residuals. Thus, at the terminal nodes based on the randomly selected features, we obtain some separation of those firms with 0.6 versus those with -0.4 . The initial log odds corresponding to $p_0 = \sum_{j=1}^{10} Y_j / 10$ is $x_0 = \log\left[\frac{p_0}{(1-p_0)}\right]$. Note that the training set allows identification of j (labels) amongst the circles.

To continue with the next boosting tree DT(2), however, the pseudo residuals have to be updated. This requires specification of a loss function L . A typical loss function for classification problems is the log loss function. Note that the loss criterion L is different from the splitting criterion in the GB trees.

$$L = -\sum_{j=1}^N \left(Y_j \log p + (1 - Y_j) \log(1 - p) \right) > 0 \quad (6.1)$$

where $p \in [0, 1]$ is probability of $Y_j = 1$. There is no loss of generality with any constant multiple on L as these loss functions will have the same solution of p for minimum when L is convex ($\partial^2 L / \partial p^2 > 0$). Log-loss measures “distance” of the predicted probability p to the target values Y_j in the binary classification. Eq. (6.1) can also be expressed as monotonic transform

$$\exp(L) = 1 / \left(p^{\sum_{j=1}^N Y_j} (1 - p)^{\sum_{j=1}^N (1 - Y_j)} \right) > 1 \quad (6.2)$$

In Eq. (6.2), higher likelihood function of $p^{\sum_{j=1}^N Y_j} (1 - p)^{\sum_{j=1}^N (1 - Y_j)}$ implies that L (loss) is smaller. Lower likelihood function of $p^{\sum_{j=1}^N Y_j} (1 - p)^{\sum_{j=1}^N (1 - Y_j)}$ implies that L (loss) is higher. Higher likelihood function and hence smaller loss L implies more accurate prediction of p . Inaccurate predictions are penalized with higher loss values.

We want to find a way to minimize the loss function such that the boosting trees could increment the predicted probability. This can be done using log odds in the argument.

If we let $x = \log$ (odds), i.e., $\log p / (1 - p)$, then $p = e^x / (1 + e^x)$. Then the log loss function can be written as $L = -\sum_{j=1}^N (Y_j x - \log(1 + e^x))$ in terms of x .

At any node when number of sample points on the node is $n \leq N$, i.e., it calculates the loss on that node:

$$L(x) = -\sum_{j=1}^n (Y_j x - \log(1 + e^x)) = -\sum_{j=1}^n Y_j x + n \log(1 + e^x).$$

Using Taylor series expansion around x_0 , an initial estimate of the log odds, we have

$$L(x) = L(x_0) + (x - x_0) \left[-\sum_{j=1}^n Y_j + \frac{ne^{x_0}}{(1 + e^{x_0})} \right] + \frac{1}{2} (x - x_0)^2 \left[\frac{ne^{x_0}}{(1 + e^{x_0})^2} \right]$$

Higher terms involving higher derivatives of $\frac{ne^x}{(1+e^x)^2}$ and so on are small and are ignored. This approximation of the log loss function in x allows taking the derivative

$$\left. \frac{dL}{dx} \right|_{x_0} = 0 + \left[-\sum_{j=1}^n Y_j + \frac{ne^{x_0}}{(1 + e^{x_0})} \right] + (x - x_0) \left[\frac{ne^{x_0}}{(1 + e^{x_0})^2} \right] = 0$$

that is set to zero on the right to obtain the optimal x . The second order condition satisfies the case for minimum L . Then,

$$(x - x_0) = \frac{\sum_{j=1}^n Y_j - \frac{ne^{x_0}}{(1 + e^{x_0})}}{\frac{ne^{x_0}}{(1 + e^{x_0})^2}}$$

or,

$$x = x_0 + \frac{\sum_{j=1}^n (Y_j - p_0)}{np_0(1 - p_0)}$$

where $Y_j - p_0$ is the initial pseudo residual. Note that $p_0 = \frac{e^{x_0}}{(1 + e^{x_0})}$. The new value of $x = x_1$ is meant to reduce the loss function optimally in DT(1). The new updated x_1 increases from x_0 if at the particular terminal node of DT(1), there are more Y_j 's = 1 than that at the start. It decreases from x_0 if there are less Y_j 's = 1 than that at the start. Obviously, for any j , a x_{1j} different from x_0 (higher for some j , lower for others) indicates the DT(1) is working well toward separating the types.

The updating involves terms in the gradient dL/dx . However, the algorithm typically puts in a learning rate, e.g., $\alpha = 0.1$, so

$$x_{1j} = x_0 + \alpha \frac{\sum_{j=1}^n (Y_j - p_0)}{np_0(1 - p_0)}$$

A subscript j is added to x_1 to indicate that now each new log odd may be different for different case or sample point as each may reside in different lower nodes, i.e., in DT(1), $x_{1j} \neq x_{1j^*}$ for different instances j and j^* .

Recall each final splitting produces two terminal nodes with differential pseudo residual values in the two nodes. The $\sum_{j=1}^n Y_j$ will be different for each node depending on n points in that node and the sum of Y_j 's with target value 1. From this new set of x_{1j} 's for every j in training set, the updated probability estimate is

$$p_{1j} = \frac{e^{x_{1j}}}{1 + e^{x_{1j}}}$$

and the updated pseudo residuals are $Y_j - p_{1j}$ for every j . These are then used as labels to train DT(2).

Note that by this stage, the different sample points Y_j may have different corresponding pseudo residuals. Refer to the example in Figure 6.5. The new labels to the same training sample points are shown as follows. Note that $x_0 = \log_e(0.4/0.6) = -0.40547$.

On the left-most leaf in DT(1),

$$\alpha \frac{\sum_{j=1}^n (Y_j - p_0)}{np_0(1 - p_0)} = 0.1 \times \frac{0.6 + 0.6 - 0.4}{3(0.4)(0.6)} = 0.111111$$

On the second leaf from the left,

$$\alpha \frac{\sum_{j=1}^n (Y_j - p_0)}{np_0(1 - p_0)} = 0.1 \times \frac{0.6 - 0.4}{2(0.4)(0.6)} = 0.041667$$

On the third leaf from the left,

$$\alpha \frac{\sum_{j=1}^n (Y_j - p_0)}{np_0(1 - p_0)} = 0.1 \times \frac{0.6 - 0.4 - 0.4}{3(0.4)(0.6)} = -0.027778$$

On the right-most leaf,

$$\alpha \frac{\sum_{j=1}^n (Y_j - p_0)}{np_0(1 - p_0)} = 0.1 \times \frac{-0.4 - 0.4}{2(0.4)(0.6)} = -0.166667$$

Hence on the left-most leaf in DT(1), $x_{1j} = x_0 + \alpha \frac{\sum_{j=1}^n (Y_j - p_0)}{np_0(1 - p_0)} = -0.40547 + 0.11111 = -0.29435$. This means every node $j = 1, 2, 6$ in that

leaf has the same x_{1j} value of -0.29435 . On the second leaf from the left, $x_{1j} = x_0 + \alpha \frac{\sum_{j=1}^n (Y_j - p_0)}{np_0(1-p_0)} = -0.40547 + 0.041667 = -0.36380$. This means every node $j = 3, 7$ in that leaf has the same x_{1j} value of -0.36380 . On the third leaf from the left, $x_{1j} = -0.40547 - 0.02778 = -0.43324$. This means every node $j = 4, 5, 8$ in that leaf has the same x_{1j} value of -0.43324 . On the right-most leaf, $x_{1j} = -0.40547 - 0.16667 = -0.57213$. This means every node $j = 9, 10$ in that leaf has the same x_{1j} value of -0.57213 .

The corresponding p_{1j} 's as in $p_{1j} = \frac{e^{x_{1j}}}{1+e^{x_{1j}}}$ can then be computed. Then the updated pseudo residuals $Y_j - p_{1j}$'s can then be computed for all j . These are shown in the following table.

j	Y_j	$\alpha \frac{\sum_{j=1}^n (Y_j - p_0)}{np_0(1-p_0)}$	x_{1j}	$p_{1j} = \frac{e^{x_{1j}}}{1+e^{x_{1j}}}$	Updated pseudo residual $Y_j - p_{1j}$	$\Delta p_{1j} = p_{1j} - p_0$
1	0	0.111111	-0.29435	0.426938	-0.42694	0.026938
2	1	0.111111	-0.29435	0.426938	0.573062	0.026938
3	0	0.041667	-0.36380	0.41004	-0.41004	0.01004
4	0	-0.027778	-0.43324	0.393352	-0.39335	-0.00665
5	0	-0.027778	-0.43324	0.393352	-0.39335	-0.00665
6	1	0.111111	-0.29435	0.426938	0.573062	0.026938
7	1	0.041667	-0.36380	0.41004	0.58996	0.01004
8	1	-0.027778	-0.43324	0.393352	0.606648	-0.00665
9	0	-0.166667	-0.57213	0.360745	-0.36075	-0.03925
10	0	-0.166667	-0.57213	0.360745	-0.36075	-0.03925

Next the second boosting tree, DT(2), is built to fit/predict the new pseudo residuals or targets $Y_j - p_{1j}$. Nodes or leaves (not just end leaves) have updated log-odds as follows.

$$x_{2j} = x_{1j} + \alpha \frac{\sum_{j=1}^n (Y_j - p_{1j})}{\sum_{j=1}^n p_{1j} (1 - p_{1j})}$$

for j 's within each node. Basically, at each m th boosting tree, x_{mj} is updated. This leads to new fitted p_{mj} , new updated pseudo residual $Y_j - p_{mj}$, next tree fitting and so on.

The algorithm then continues in the same way until there is no significant improvement in the log odds, i.e., x_{nj} . At the end of n trees, i.e., at end of $DT(n)$, the probability estimates p_{nj} for each j are fitted and the GB trees $DT(1)$ up to $DT(n)$ can now be used to predict p_{nk} for each test sample point k .

An alternative way to think about it is as follows. At the end of n trees, i.e., at end of $DT(n)$, the probability estimates p_{nj} for each j are computed as $p_{nj} = \Delta p_{nj} + \Delta p_{n-1,j} + \dots + \Delta p_{3j} + \Delta p_{2j} + \Delta p_{1j} + p_0$. If $p_{nj} > 1/2$, then that j th case is predicted as $Y_j = 1$. Otherwise, it will be predicted as $Y_j = 0$. The number of boosting trees n can be large, e.g., 100, especially if the learning rate α is kept low and if the depths of the trees are kept small. It is common for an effective GB algorithm in a well-defined data set to enable a close to, if not, 100% training fit or accuracy in training.

Visually, GB classification trees for fitting training data can be seen as follows. The quantities on the top line denote the updated pseudo residuals. The quantities on the bottom line denotes the adjustments in predicted probabilities from the previous DT. See Figure 6.6.

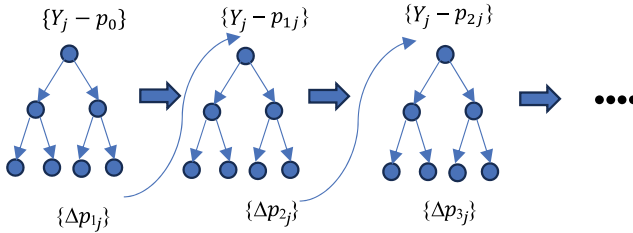


Figure 6.6. Gradient Boosting classification.

Note that the features are typically randomly selected for different boosting trees. GB probability predictor for each j is $p_0 + \Delta p_{1j} + \Delta p_{2j} + \Delta p_{3j} + \dots + \Delta p_{nj}$.

For the test sample or prediction of any sample point j given its features, the trained GB classification trees $DT(1)$ to $DT(n)$ are used to predict Δp_{1j} from $DT(1)$, Δp_{2j} from $DT(2)$, and so on. Then the GB probability predictor for each j is $p_0 + \Delta p_{1j} + \Delta p_{2j} + \dots + \Delta p_{nj}$. If $p_{nj} > 1/2$, then that j th case is predicted as $Y_j = 1$. Otherwise, it will be predicted as $Y_j = 0$.

There are several points worth noting. Firstly, a different loss function such as the MSE may not work as well here as $L(x) = \frac{1}{N} \sum_{j=1}^N (Y_j - p)^2 = \frac{1}{N} \sum_{i=j}^N (Y_j - \frac{e^x}{1+e^x})^2$ is non-convex given Y_j 's for some x . Non-convexity

does not allow a global minimum to be found and local minimum may not lead to effective predictors. If we try to find the minimum in $\frac{1}{N} \sum_{j=1}^N (Y_j - p)^2$ by directly taking p as the control variable, then its solution is just $\hat{p} = \frac{1}{N} \sum_{j=1}^N Y_j$ which is just the proportion of Y_j taking the value 1. $\hat{p}$ is also the initial predictor p_0 in the above example. It is the same for all sample points j , so it is not a strong or effective predictor.

Secondly, if in the terminal node of each tree we had used updated estimate of p directly as the empirical probability on that node, e.g., $p_1 = 2/3$ (based on $j = 1, 2, 6$ on the left-most leaf) without going through the gradient involving log odds, then the DT becomes an ordinary DT without boosting as the levels on the DT can continue to grow without the need to update the targets.

Thirdly, unlike RF where bagging uses bootstrapping – random sampling with replacements, GB may use random sampling of a smaller set of training sample points without replacements (i.e., no repeated sample points) for the boosting trees. The smaller set enables faster computations. This alternative is sometimes called stochastic GB. See Friedman (2002).

Fourthly, many different varieties of GB based on different loss functions, different split criteria, and different ways of combining the boosting trees, can be built. There are some general comparisons but specifically how one ensemble tree algorithm will perform relative to another depends also on the data.

Fifthly, GB trees may require more tuning of the hyperparameter(s) – setting learning rate, pruning some nodes, setting maximum number of levels per tree, etc.

6.2 Popular Alternatives

An example of a popular alternative to the GB algorithm is the LightGBM (light GB machine). LightGBM (from a different original builder and is not part of sklearn) generally has faster speed and places less burden on computer memory. In LightGBM (which can apply to both RF and boosting method), each tree grows not level-wise as in traditional trees, but leaf-wise. Instead of extending the levels down with two nodes at the next level, then four nodes, etc., the nodes are only extended down from the previous node with the largest information gain or minimum loss. In principle, if we can grow the trees as long as possible, the lightGBM should produce closely similar prediction result with the regular GB method. However,

with regularizations such as limiting the branching and limiting maximum depth of trees, the lightGBM can perform marginally better. The following Figure 6.7 shows how one lightGBM tree is constructed in the training data.

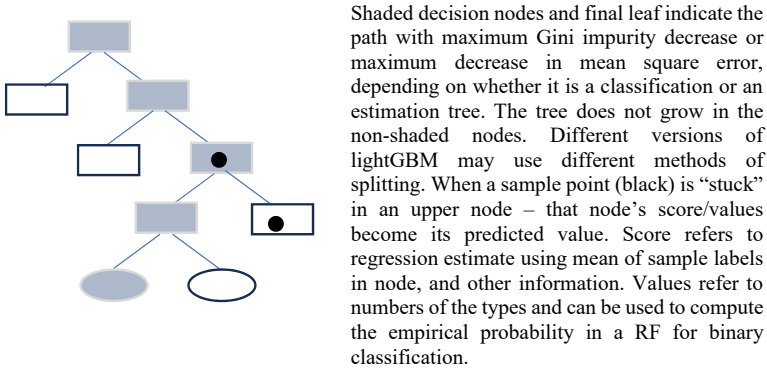


Figure 6.7. Illustrating a lightGBM algorithm.

Another popular alternative is the XGBoost (Extreme GB). It is in basic terms similar in approach to the GB method, but it offers some extra bells and whistle. XGBoost incorporates various regularization techniques to prevent overfitting and improve the generalization capability of the models. These regularizations occur where optimization procedures are required in the GB. Regularization methods such as $L1$ and $L2$ regularization (i.e., Lasso and Ridge regularization) penalize complex models, helping to control model complexity and reduce overfitting.

XGBoost employs tree pruning algorithms to control the size of DTs, reducing overfitting and improving computational efficiency. Tree pruning techniques such as depth-based pruning (to reduce too much depth) and weight-based (to reduce nodes with too little weights) pruning remove unnecessary branches from DTs. In addition, XGBoost employs parallel processing techniques to maximize the computational efficiency.

XGBoost also provides a built-in support for handling missing values in the dataset during training and prediction. It automatically estimates the best imputation strategy for missing values, and fills in the necessary missing data.

When comparing the performances of GBM and XGBoost, the former may be computationally slower as it lacks early stopping such as tree pruning. XGBoost also appears to have a more user-friendly interface and better customizations although as in most software development, competitors can adapt and catch up fast. Other variants of boosting methods can be seen in chapter 7 of Geron (2019).

6.3 Worked Example GB: Data

We continue with the credit rating accounting data set `corporate_rating2.csv` used in Chapter 5 for the DT and RF algorithms. The program file is `Chapter6-1.ipynb`. Recall there are 2029 firms/instances with either ratings BBB up to AAA (Class “1”) or ratings BB and below (Class “0”).

After standardizing the features using `sklearn StandardScalar`, the data set “`corporate_rating2.csv`” is split into 75% training set (`X_train, y_train`) with 1521 rows and 25% test set (`X_test, y_test`) with 508 rows in code lines [13] and [14]. The data consist of 25 features.

```
[13]: TT=pd.concat([df_ratios['Class'],df_ratios_new],axis=1)
      TT.shape
      TT.tail()
```

	Class	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding	netProfitMargin	
2024	1	0.186827	0.135444	1.783407	-0.074822	-0.036575	...
2025	0	-0.012870	-0.041268	-0.066739	-0.068183	-0.073966	...
2026	0	-0.060074	-0.054997	-0.120995	-0.041872	-0.047159	...
2027	0	-0.059442	-0.057857	-0.099558	-0.045460	-0.031518	...
2028	0	-0.055507	-0.049416	-0.129463	-0.040959	-0.084191	...

5 rows × 26 columns

```
[14]: Train, Test = train_test_split(TT,
                                     test_size=0.25,
                                     random_state=0)

X_train, y_train = Train.iloc[:,1:26], Train.iloc[:,0] ### choose the 25 features
X_test, y_test = Test.iloc[:,1:26], Test.iloc[:,0]
```

```
[15]: X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

```
[15]: ((1521, 25), (1521,), (508, 25), (508,))
```

`Sklearn GradientBoostingClassifier` is used with specification of 1000 DTs (`n_estimators = 1000`). We can set random selection of features, e.g., `max_features = “sqrt”` which means that each DT randomly uses $\sqrt{25} = 5$ features of the 25 total in constructing the branching in each of the 1000 DTs. However, because of the learning to solve for the net error in subsequent DTs, there is a learning rate just as in Gradient Descent. Here, we fine-tune the learning rate to attain a higher accuracy or lower loss function. See code line [17].

Gradient Boosting

```
[17]: from sklearn.ensemble import GradientBoostingClassifier
      ## Example:
      ## class sklearn.ensemble.GradientBoostingClassifier(*, loss='log_loss',
      ## learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse',
      ## min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0,
      ## max_depth=3, min_impurity_decrease=0.0, init=None, random_state=None,
      ## max_features=None, verbose=0, max_leaf_nodes=None, warm_start=False,
      ## validation_fraction=0.1, n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)

      GB_model = GradientBoostingClassifier(n_estimators=1000, random_state=1, \
      max_features="sqrt", learning_rate=0.01, max_depth=24)
      GB_model.fit(X_train, y_train)

      y_pred_GB = GB_model.predict(X_test)
      Accuracy_GB = metrics.accuracy_score(y_test, y_pred_GB)
      print("GB Accuracy:", Accuracy_GB)

      GB Accuracy: 0.8051181102362205
```

The confusion matrix and the classification report for the test are shown in code lines [18] and [19].

```
[18]: from sklearn.metrics import confusion_matrix
      confusion_matrix = confusion_matrix(y_test, y_pred_GB)
      print(confusion_matrix)

      [[151  64]
       [ 35 258]]
```

```
[19]: from sklearn.metrics import classification_report
      print(classification_report(y_test, y_pred_GB))
```

	precision	recall	f1-score	support
0	0.81	0.70	0.75	215
1	0.80	0.88	0.84	293
accuracy			0.81	508
macro avg	0.81	0.79	0.80	508
weighted avg	0.81	0.81	0.80	508

In this investment grade/speculative grade prediction problem, the accuracy of the GB here is 80.51%. The prediction performance is marginally better than RF with accuracy of 79.53% seen in Chapter 5.

The ROC curve is shown below after computing the various True positive rates and False positive rates based on different threshold probabilities.

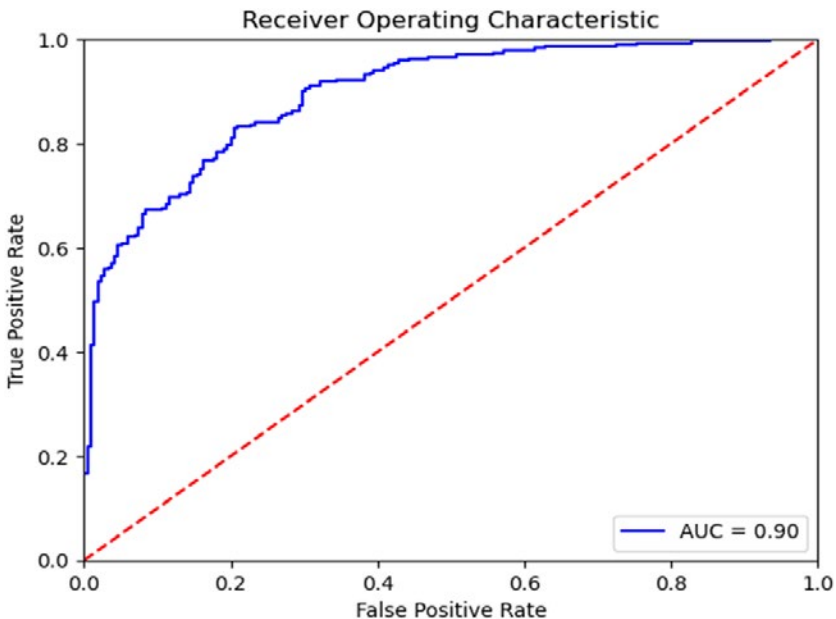
Code lines [20] and [21] show the computation of the AUC of the ROC as 89.57%. This is marginally better than the 88.69% in RF.

```
[20]: import sklearn.metrics as metrics
# calculate the fpr and tpr for all thresholds of the classification
preds_GB = GB_model.predict_proba(X_test)[:,-1]

fpr, tpr, thresholds = metrics.roc_curve(y_test, preds_GB)
### matches y_test of 1's and 0's versus pred prob of 1's for each
### of the 508 test cases
### sklearn.metrics.roc_curve(y_true, y_score,...) requires y_true as 0,1
### input and y_score as prob inputs
### This metrics.roc_curve returns fpr, tpr, thresholds (Decreasing thresholds
### used to compute fpr and tpr)
roc_auc_GB = metrics.auc(fpr, tpr)
### sklearn.metrics.auc(fpr,tpr) returns AUC using trapezoidal rule
roc_auc_GB
```

```
[20]: 0.89569013413763
```

```
[21]: import matplotlib.pyplot as plt
plt.title('Receiver Operating Characteristic')
plt.plot(fpr, tpr, 'b', label = 'AUC = %0.2f' % roc_auc_GB)
plt.legend(loc = 'lower right')
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.show()
```



Any one of the 1000 GB trees can be plotted. The first DT(1) is shown as follows.

Visualizing a boosting tree

```
[23]: ## Classifier
GB_model.fit(X_train, y_train)
## Get the tree number 1
sub_tree_1 = GB_model.estimators_[0,0]
## Visualization

from sklearn.tree import plot_tree
fig, axes = plt.subplots(nrows = 1,ncols = 1,figsize = (15,8), dpi=300)

## plot tree
plot_tree(sub_tree_1, fontsize=10) # set plot size (denoted in inches)
plt.title("Boosting sub_Tree_1")
plt.show()
```



A text representation can be obtained via:

“from sklearn import tree

text_representation = tree.export_text(GB_model.estimators_[n,0])

print(text_representation)”

where “ n ” indicates the $n + 1$ tree or DT($n + 1$). The first splitting into binary nodes from the root node uses the 20th feature (“companyEquity-Multiplier”) ≤ -0.044 with a minimum Friedman MSE criterion shown in the root node. Higher equity multiplier means that more of the firm’s assets are financed by shareholder equity rather than debt. The right node leads to higher credit rating probability or higher probability of Class “1”

in this context. The values (different apps will use slightly different representations) indicate sum of the pseudo residuals. As the branching produces clearer separation of the classes, the left node and the right node will indicate opposite signed numbers since the side with instances closer to Class “1” will show more positive residuals, being sum of pseudo residuals or $(1 - \text{estimated probability})$ over all instances in that node. The other side with instances further away from Class “1” will show more negative residuals, being sum of pseudo residuals or $(0 - \text{estimated probability})$ over all instances in that node.

In the case of the LightGBM (see code lines [26–33]), the prediction accuracy is 83.27%, higher than GB.

Light GBM

```
[26]: pip install --upgrade pip
Requirement already satisfied: pip in c:\users\vista\anaconda3\lib\site-packages (24.3.1)
Note: you may need to restart the kernel to use updated packages.
```

```
[27]: !pip3 install lightgbm
Requirement already satisfied: lightgbm in c:\users\vista\anaconda3\lib\site-packages (4.5.0)
Requirement already satisfied: numpy>=1.17.0 in c:\users\vista\anaconda3\lib\site-packages (from lightgbm) (1.26.4)
Requirement already satisfied: scipy in c:\users\vista\anaconda3\lib\site-packages (from lightgbm) (1.11.4)
```

```
[28]: import lightgbm as lgb
from lightgbm.sklearn import LGBMClassifier
```

```
[29]: # Creating an object for model and fitting it on training data set
LGBMModel = LGBMClassifier(learning_rate=0.1)
LGBMModel.fit(X_train, y_train)

# Predicting the Target variable
predLGBM = LGBMModel.predict(X_test)
print(predLGBM)
accuracy = LGBMModel.score(X_test, y_test)
print(accuracy)

0.8326771653543307
```

```
[30]: from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, predLGBM)
print(confusion_matrix)

[[160  55]
 [ 30 263]]
```

```
[31]: from sklearn.metrics import classification_report
print(classification_report(y_test, predLGBM))
```

	precision	recall	f1-score	support
0	0.84	0.74	0.79	215
1	0.83	0.90	0.86	293
accuracy			0.83	508
macro avg	0.83	0.82	0.83	508
weighted avg	0.83	0.83	0.83	508

The LightGBM precision and recall on the test data set are also slightly higher than those in GB, showing the advantages of regularization in complex trees.

In the case of the XGBoost (see code lines [35–40]), the prediction accuracy is 80.91%, marginally higher than GB.

XGBoost

```
[35]: pip install xgboost

Requirement already satisfied: xgboost in c:\users\vista\anaconda3\lib\site-packages (2.1.0)
Requirement already satisfied: numpy in c:\users\vista\anaconda3\lib\site-packages (from xgboost) (1.26.4)
Requirement already satisfied: scipy in c:\users\vista\anaconda3\lib\site-packages (from xgboost) (1.11.4)
Note: you may need to restart the kernel to use updated packages.
```

```
[36]: # XGBoost
from xgboost import XGBClassifier
XG_model = XGBClassifier(n_estimators=1000, random_state=1, \
    max_features="sqrt", learning_rate=0.02, max_depth=24, \
    objective='binary:logistic')

### Note for multiclass, objective='multi:softprob'
XG_model.fit(X_train,y_train)

### XG_model follows GB model to have n_estimators = 1000 and max_depth = 24
y_pred_XG = XG_model.predict(X_test)
Accuracy_XG = metrics.accuracy_score(y_test, y_pred_XG)
print("XG Accuracy:",Accuracy_XG)

XG Accuracy: 0.8090551181102362
```

```
[37]: from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, y_pred_XG)
print(confusion_matrix)

[[152  63]
 [ 34 259]]
```

```
[38]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_XG))
```

	precision	recall	f1-score	support
0	0.82	0.71	0.76	215
1	0.80	0.88	0.84	293
accuracy			0.81	508
macro avg	0.81	0.80	0.80	508
weighted avg	0.81	0.81	0.81	508

The XGBoost precision and recall on the test data set are only marginally higher than those in GB.

For comparison, we use the sklearn LogisticRegression for prediction. The results are shown below. It is seen that the various GB Trees perform significantly better than the logit regression – indicating the efficacy of non-parametric methods when there is a large number of features in a complex manner of data patterns.

Logistic Regression

```
[42]: # Logistic Regression
from sklearn.linear_model import LogisticRegression
LR = LogisticRegression(random_state=1,
                        multi_class='multinomial',
                        solver='newton-cg')
LR = LR.fit(X_train, y_train)
y_pred_LR = LR.predict(X_test)
Accuracy_LR = metrics.accuracy_score(y_test, y_pred_LR)
print("LR Accuracy:", Accuracy_LR)

LR Accuracy: 0.6476377952755905
```

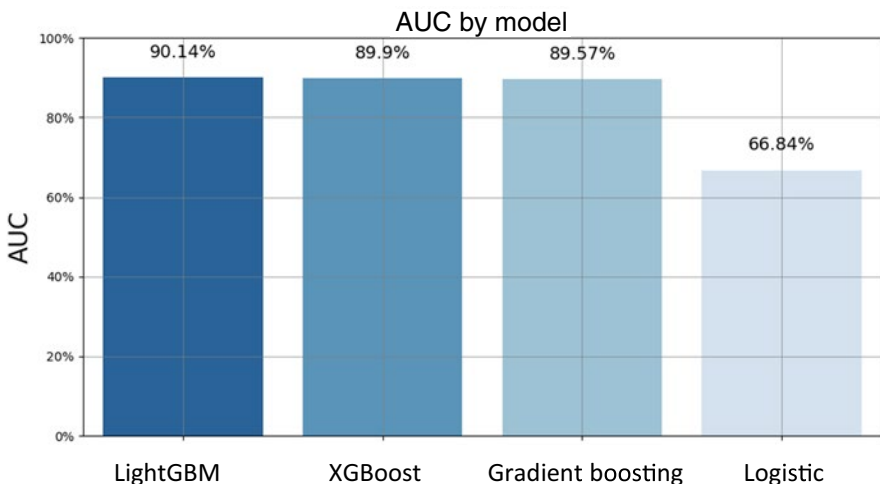
```
[43]: from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, y_pred_LR)
print(confusion_matrix)

[[ 70 145]
 [ 34 259]]
```

```
[44]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_LR))
```

	precision	recall	f1-score	support
0	0.67	0.33	0.44	215
1	0.64	0.88	0.74	293
accuracy			0.65	508
macro avg	0.66	0.60	0.59	508
weighted avg	0.65	0.65	0.61	508

The comparison of AUC performances of these various methods are shown as follows.



6.4 Shapley and SHAP Values

We have seen how feature importance is a way of measuring how significantly a feature contributes to the prediction accuracy of a DT and RF. The percentage contribution of a feature to the overall Gini importance or overall Gini weighted impurity decrease signifies its importance in a DT and a RF algorithm. In a similar way, the percentage contribution of a feature to the overall weighted MSE or Friedman MSE decrease signifies its importance in a GB Tree and its close relatives.

In XGBoost, Feature Importance is shown in code lines [51] and [52].

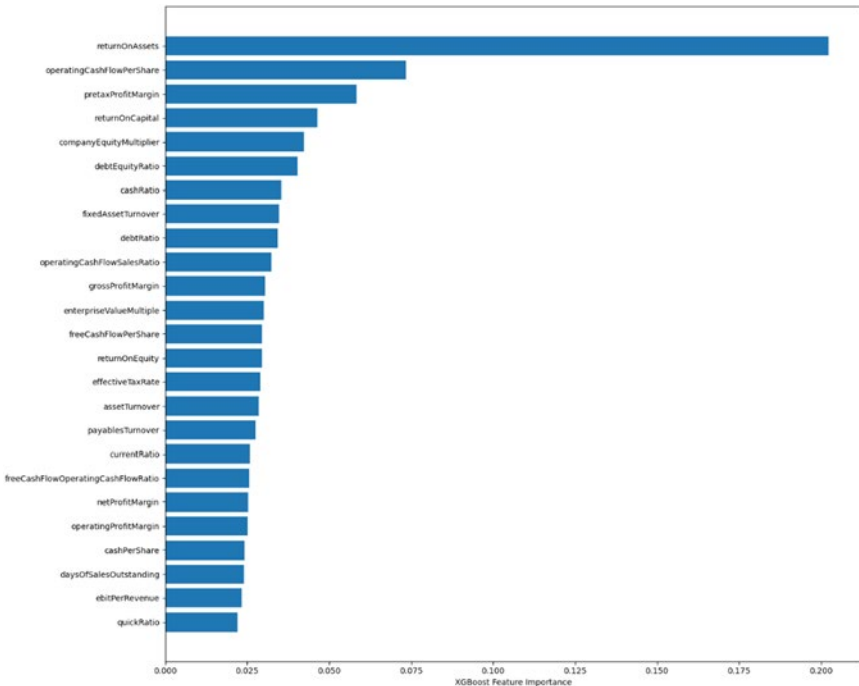
```
[51]: print(XG_model.feature_importances_)

[0.02575384 0.02205251 0.03534279 0.02407188 0.02522108 0.05827876
 0.03048301 0.02512895 0.20236763 0.04647086 0.02949365 0.02840674
 0.03474097 0.04035477 0.03425717 0.02903091 0.0256409 0.02956792
 0.02413125 0.04235359 0.02329668 0.03009087 0.07348198 0.03233119
 0.0276501 ]

[52]: ### representation

fig, ax = plt.subplots(figsize=(15,12))

sorted_idx = XG_model.feature_importances_.argsort()
plt.barh(X_train.columns[sorted_idx],XG_model.feature_importances_[sorted_idx])
plt.xlabel("XGBoost Feature Importance")
plt.figure(figsize=(20, 15), dpi=80)
fig.tight_layout()
```



In the XGBoost Tree, the percentage contributions of the four main features of “return on assets”, “operating cash flow per share”, “pretax profit margin”, and “return on capital” are 20.24%, 7.35%, 5.83%, and 4.64%. In the case of the RF in Chapter 5, the four main features of “operating cash flow per share”, “return on assets”, “return on capital”, and “net profit margin” are 8.06%, 6.06%, 5.75%, and 5.59%. Three of the main features are common to both tree algorithms. However, they are not exactly similar.

Though useful, Feature Importance, however, does not measure explicitly the impact of a feature on the predicted outcome, be it a target value or probability of a classification. This is unlike a linear regression where after fitting, we can state that a feature value times its weight (fitted coefficient) contributes their product value as a percentage of the expected outcome of the target or dependent variable.

We want to consider how much each feature explicitly contributes to the predicted outcome of a ML algorithm. This idea is similar to how much each player in a coalition game contributes to the outcome of a game. This contribution is called the Shapley value. In ML, it is equivalent to the contribution of a feature to the predicted outcome.

We first talk about the coalition (cooperative) game of N players. To understand the meaning of cooperation, let’s first think of a non-cooperative game that involves Nash equilibrium. A simple example is the well-known Prisoner’s Dilemma problem.

Two robbery suspects, X and Y , were caught and the police believe they are the culprits. However, there is insufficient evidence unless one or both of them testify against the other. Each suspect is interrogated independently, and the outcomes for each of them is given as follows, depending on their responses in the interrogations. If both pleaded innocence, they still have to be in a lock-up for three months pending more investigations.

	A : Y does not testify against X	B : Y testifies against X
X does not testify against Y	Lock-up for three months	X jailed for three years, Y goes free
X testifies against Y	Y jailed for three years, X goes free	Both jailed for two years

This is a non-cooperative game with a Nash equilibrium solution. X considers Y ’s possible responses (assuming Y makes a response and sticks

to it) if X does not testify against Y . In this case, X either gets lock-up for three months (event A) or jailed for three years (event B). Then X considers Y 's possible responses if X testifies against Y . In this case, X either goes free (event A) or jailed for two years (event B). In both A and B , X is always better off by choosing to testify against Y . By the same argument, Y will also choose to testify against X , so both were jailed for two years.

Now, think of a cooperative game involving $N > 0$ players trying to work together in a game of making earnings (such as in a sales force) denominated in \$. Assume that outside of the game, any member would make zero earning. But the team will make earnings $V[N] > 0$ and any player j in the total coalition of N will receive an allocation $\phi_j(V[N]) \geq 0$ with at least one team player having positive allocation. So, it makes sense for all N players to join the game together and there is no incentive for any to break away from the total coalition of N . We also assume $V([0]) = 0$ where $[0]$ denotes the null set of no players.

Let each unique player be denoted as player $\{j\}$ where j is one number of 1, 2, 3, ..., N . The problem here is how to determine an efficient and fair allocation $\phi_j(V[N]) \geq 0$ for every player $\{j\}$. Clearly, it is not enough to know just $V[N]$. And distribution such as $V[N]/N$ for each player may not be fair as some players could have contributed more toward earning V . In order to be able to find out who contributes more, more observations of between-player interactions are required. In Shapley cooperative game, it is assumed that we can observe the marginal (incremental) contribution by a player $\{j\}$ when he/she is added to various smaller coalitions. Whether practically this is observable or not is a separate matter.

Smaller coalitions or groupings consist of 0, 1, 2, 3, ..., $N - 1$ players respectively. For example, coalition $[S]$ consists of S players, for $0 \leq S \leq N - 1$. We include the null set or coalition $[0]$. There are ${}^{N-1}P_S = (N-1)!/(N-1-S)!$ number of permutations of selecting S players in an ordered sequence from a pool of $N - 1$ players to form coalition $[S]$.

In the Shapley game, the total coalition earnings $V[N]$ is made in the following way. A coalition $[1]$ (one player only – suppose player $\{7\}$) is formed and sales is $V(\{7\})$. Argument $\{7\}$ denotes the earning is obtained with player $\{7\}$. $\{7\}$'s marginal or incremental contribution to earnings is $V(\{7\}) - V([0]) = V(\{7\})$.

Next suppose a new player $\{3\}$ is added to form coalition $[2]$ (two players), and the earning is now $V(\{7\} \cup \{3\})$ where $\cup$ denotes “union” or united effort of $\{7\}$ and $\{3\}$. In this case $\{3\}$'s marginal contribution is $V(\{7\} \cup \{3\}) - V(\{7\})$. Suppose $\{5\}$ is next added to form coalition $[3]$,

then $\{5\}$'s marginal contribution is $V(\{7\} \cup \{3\} \cup \{5\}) - V(\{7\} \cup \{3\})$. And so on.

Suppose the addition of new players to a larger coalition till total coalition $[N]$ is in the order: $\{7\}$, $\{3\}$, $\{5\}$, ..., $\{N\}$, $\{4\}$, $\{6\}$. Eventually a total coalition earning consists of the marginal contributions of each added player as the coalitions increase to size N , i.e.,

$$\begin{aligned} V[N] = & V([0]) + [V(\{7\}) - V([0])] + [V(\{7\} \cup \{3\}) - V(\{7\})] \\ & + [V(\{7\} \cup \{3\} \cup \{5\}) - V(\{7\} \cup \{3\})] + \dots \\ & + [V(\{7\} \cup \{3\} \cup \{5\} \cup \dots \cup \{N\} \cup \{4\}) - V(\{7\} \cup \{3\} \\ & \cup \{5\} \cup \dots \cup \{N\})] + [V(\{7\} \cup \{3\} \cup \{5\} \cup \dots \cup \{N\} \cup \{4\} \\ & \cup \{6\}) - V(\{7\} \cup \{3\} \cup \{5\} \cup \dots \cup \{N\} \cup \{4\})] \end{aligned}$$

Thus, $V[N]$ of the total coalition is $V(\{7\} \cup \{3\} \cup \{5\} \cup \dots \cup \{N\} \cup \{4\} \cup \{6\})$.

In Shapley game (not other types of games), a coalition $[S]$ consisting of the same set of players will have the same value despite having different ordered sequences of the players. For example, coalition $[2]$ comprising $(\{7\} \cup \{3\})$ has value $V(\{7\} \cup \{3\}) = V(\{3\} \cup \{7\})$. But coalitions $[S]$ with different sets of players can have different values. For example, $V(\{7\} \cup \{3\}) \neq V(\{8\} \cup \{1\})$ though $(\{7\} \cup \{3\})$ and $(\{8\} \cup \{1\})$ are both coalition $[2]$.

In the sequence of coalition $[1]$, coalition $[2]$, coalition $[3]$, ..., coalition $[N]$, it is seen that $\{7\}$'s marginal contribution is $[V(\{7\}) - V([0])] = V(\{7\})$ which is observable by the Shapley game. $\{3\}$'s marginal contribution is $[V(\{7\} \cup \{3\}) - V(\{7\})]$ which is observable by the Shapley game. $\{5\}$'s marginal contribution is $[V(\{7\} \cup \{3\} \cup \{5\}) - V(\{7\} \cup \{3\})]$ which is observable by the Shapley game. It is important to note that we observe just one sequence $\{7\}$, $\{3\}$, $\{5\}$, ..., $\{N\}$, $\{4\}$, $\{6\}$ of many possible sequences based on different ways of forming the coalition $[S]$, for each S . For example, another sequence $\{2\}$, $\{4\}$, $\{N\}$, ..., $\{9\}$, $\{3\}$ will produce different marginal contributions. $\{4\}$'s marginal contribution is now $[V(\{2\} \cup \{4\}) - V(\{2\})]$. $\{3\}$'s marginal contribution is now $[V[N] - V(\{2\} \cup \{4\} \cup \{N\} \cup \dots \cup \{9\})]$. It is important to note that the sum of all marginal contributions in any sequence is always $V[N]$.

In general, player j or $\{j\}$ can enter the coalition $[S]$ in N number of ways at $S = 0, 1, 2, \dots, N - 1$. At each entry, its marginal contribution is $V([S] \cup \{j\}) - V([S])$ for different $S = 0, 1, 2, \dots, N - 1$. For different S , $V([S] \cup \{j\}) - V([S])$ will likely be different. Even for the

same S , $V([S] \cup \{j\}) - V([S])$ will likely be different for different player composition in the coalition $[S]$.

The number of different marginal contributions of player $\{j\}$ for coalition $[S]$ is ${}^{N-1}C_S = (N-1)! / [(N-1-S)!S!]$ where ${}^{N-1}C_S$ is the number of different combinations of making up the coalition $[S]$ from the remaining $N-1$ members of the team. We can denote the different values for same coalition $[S]$ as S_k . Hence, we denote value of coalition $[S]$ (for $S < N$) as $V[k, S]$ where k denotes a specific one of the ${}^{N-1}C_S$ combinations.

There are ${}^{N-1}P_S = (N-1)! / (N-1-S)!$ number of permutations of selecting S players in an ordered sequence from a pool of $N-1$ players to form coalition $[S]$. After $\{j\}$ is added as the $S+1$ th player, there are $(N-1-S)!$ permutations of the players to finish up the sequencing to total coalition $[N]$. The different permutations or combinations that come after $\{j\}$ is added do not affect the marginal contribution of $\{j\}$. The total number of permutations in N players is seen as $N! = N \times {}^{N-1}P_S \times (N-1-S)! = N \times {}^{N-1}C_S \times S! \times (N-1-S)!$ where multiplication by N indicates the number of the entry position $\{j\}$ can take.

The Shapley value of player j or $\{j\}$ refers to his/her fair contribution $\phi_j(V[N])$ to the game payout $V([N])$ as the average of his/her marginal contributions over all possible sequencings, $N!$.

$$\phi_j(V[N]) = \frac{1}{N!} \sum_{S=0}^{N-1} S! \times (N-1-S)! \left\{ \sum_{k=1}^{C_S^{N-1}} [V([k, S] \cup \{j\}) - V([k, S])] \right\}$$

where $0 \leq S \leq N-1$. (6.1)

The Shapley value is a unique solution concept for allocation of $V[N]$ that holds some nice properties.

- (1) *The allocation is efficient:* $\sum_{j=1}^N \phi_j(V[N]) = V[N]$. There is no leakage or wastage in distributing $V[N]$.
- (2) *Symmetry:* If two players i, j , contribute the same in joining every identical coalition $[S]$, i.e., $V([k, S] \cup \{i\}) = V([k, S] \cup \{j\})$, then $\phi_i(V[N]) = \phi_j(V[N])$.
- (3) *Nullity:* If any player j contributes nothing, i.e., $V([k, S] \cup \{j\}) = V([k, S])$, then $\phi_j(V[N]) = 0$.
- (4) *Linearity or additivity:* $\phi_j(aV_1[N] + bV_2[N]) = a\phi_j(V_1[N]) + b\phi_j(V_2[N])$ provided any player's marginal contribution scales linearly with total coalition payouts.

Hence, Shapley allocation is “fair”.

We want to use the concept in the Shapley game to measure how each feature (player) of a set of N features X (N players) in an algorithm (game) F contributes to the predicted outcome $F(X)$.

$F(\cdot)$ could be a nonlinear ML algorithm such as RF or NN. Feature space X is made of T number of sample points, $X_{T \times N}$. Each tabular row in X has N feature observations and is associated with a target value. However, the Shapley value or SHAP value has to do with computing contributions by features in a prediction and they do not require target or label in the computation $F(\cdot)$. Hence, Shapley and SHAP values can be computed on both the training as well as the test data sets.

Let one row be $(x_1, x_2, x_3, \dots, x_N)$. The ML algorithm produces a prediction outcome or output $F(x_1, x_2, x_3, \dots, x_N)$. We want to find out how much does a feature x_j explain output $F(x_1, x_2, x_3, \dots, x_j, \dots, x_N)$.

Let $(z_1, z_2, \dots, z_S)$ be a particular permutation choosing $S \leq N - 1$ elements of $(x_1, x_2, x_3, \dots, x_{j-1}, x_{j+1}, \dots, x_N)$ from any row of sample size T . Let $(z_{S+2}, z_{S+3}, \dots, z_N)$ be a particular permutation choosing $N - S - 1$ elements of the remaining features in $(x_1, x_2, x_3, \dots, x_{j-1}, x_{j+1}, \dots, x_N)$ from the same row of sample size T . Then $F(z_1, z_2, \dots, z_S, \mathbf{x}_j, z_{S+2}, z_{S+3}, \dots, z_N) - F(z_1, z_2, \dots, z_S, \mathbf{r}_j, z_{S+2}, z_{S+3}, \dots, z_N)$ is equivalent to $V(\{z_1\}, \{z_2\}, \dots, \{z_S\}, \{x_j\}) - V(\{z_1\}, \{z_2\}, \dots, \{z_S\})$, where $\mathbf{r}_j$ is a randomly selected element from feature j in any of the T rows. The full set of N features is included in computing the marginal contribution of feature $\mathbf{x}_j$ as required in the ML algorithm. In practice, for higher accuracy, the random selection of $\mathbf{r}_j$ can be repeated and the average is taken for $F(z_1, z_2, \dots, z_S, \mathbf{x}_j, z_{S+2}, z_{S+3}, \dots, z_N) - F(z_1, z_2, \dots, z_S, \mathbf{r}_j, z_{S+2}, z_{S+3}, \dots, z_N)$.

To compute Shapley value of feature x_j :

$$\phi(V[N]) \equiv \frac{1}{N!} \sum_{S=0}^{N-1} S! \times (N-1-S)! \\ \times \left\{ \sum_{k=1}^{C_S^{N-1}} \left[F(z_1, z_2, \dots, z_S, x_j, z_{S+2}, \dots, z_N) - F(z_1, z_2, \dots, z_S, \mathbf{r}_j, z_{S+2}, \dots, z_N) \right] \right\}$$

where $0 \leq S \leq N - 1$. (6.2)

Equation (6.2) follows Eq. (6.1) where the value addition by feature j is in

$$[F(z_1, z_2, \dots, z_S, x_j, z_{S+2}, \dots, z_N) - F(z_1, z_2, \dots, z_S, \mathbf{r}_j, z_{S+2}, \dots, z_N)].$$

The Shapley value of a feature value is the average change in the prediction when the feature value joins an existing coalition of features.

However, the above exact method in (6.2) is computationally too tedious for every x_j as it involves too many C_S^{N-1} (for each $S \leq N - 1$) number of combinatorial computations of $F(\cdot)$, especially when the number of features N is large. In practice some approximation methods are used.

One approximation method is to use Monte Carlo sampling. We want to find the approximate Shapley value/contribution of the j th feature of $x = (x_1, x_2, x_3, \dots, x_j, \dots, x_N)$. This is finding feature x_j 's Shapley value $\phi_j(V[N])$.

Let an iteration m involve the following.

- (1) Randomly draw a row from feature matrix $X_{T \times N}$. Call this draw $Z = (z_1, z_2, z_3, \dots, z_N)$.
- (2) Perform a random permutation of the order of the features – call this permutation “*”, but leaving out the j th feature of x_j . For example, $*(z_1, z_2, z_3, \dots, z_j, \dots, z_N)$ becomes $(z_5, z_7, z_1, \dots, z_j, \dots, z_6)$. This simulates the different sequencings or permutations in Shapley value calculation while keeping x_j intact.
- (3) Form two new instances based on a row of X (going from row 1 to T , or if more data are required, re-sampling some of these rows) and the randomly drawn Z . Permutate these instances based on “*” in step (2). Call these permutated sequences x_{j*} 's and z_{j*} 's.
- (4) Adjust one of the two permutated instances by replacing the $j + 1, j + 2, \dots, N$ elements of the permutated X sequence with $z_{j+1*}, \dots, z_{N*}$. Hence, $x_{+j} = (x_{1*}, x_{2*}, x_{3*}, \dots, x_{j-1*}, x_j, z_{j+1*}, \dots, z_{N*})$.
Adjust the other permutated instance by replacing the $j, j + 1, j + 2, \dots, N$ elements of the permutated X sequence with $z_{j*}, z_{j+1*}, \dots, z_{N*}$. Here, z_{j*} replaces x_j . Hence, $x_{-j} = (x_{1*}, x_{2*}, x_{3*}, \dots, x_{j-1*}, z_{j*}, z_{j+1*}, \dots, z_{N*})$.
- (5) Compute $\phi_j^m = F(x_{+j}) - F(x_{-j})$. This is marginal contribution of x_j in one constructed iteration.
- (6) Repeat iterations for $m = 1, \dots, M$ for a large M .

Compute approximate Shapley value as average: $\phi_j(x) = \frac{1}{M} \sum_{m=1}^M \phi_j^m$. In the above Shapley value computations, we assume that the value is dependent on the order of the features in the first coalition $[S]$. The formula in Eq. (6.2) and the approximation formula are, however, valid even if a particular ML algorithm does not differentiate amongst the order of entry in the feature input.

6.4.1 SHAP values (Shapley Additive exPlanations)

The concept in Shapley method is to measure how each feature (player) of a set of N features X (N players) in an algorithm (game) F contributes to the predicted outcome $F(x)$ when features in $x = (x_1, x_2, x_3, \dots, x_j, \dots, x_N)$ are used. In Eq. (6.2), suppose the features are arranged as $(x_3, x_8, x_5, x_{11}, \dots, x_N, x_4, \dots, x_7, x_9)$ which is some permutation of the original x .

This permutation of the N number of represented features will produce contribution of $F(x_3, r_8, r_5, \dots, r_N, \dots, r_9) - F(r_3, r_8, r_5, \dots, r_N, \dots, r_9)$ by feature 3, contribution of $F(x_3, x_8, r_5, \dots, r_N, \dots, r_9) - F(x_3, r_8, r_5, \dots, r_N, \dots, r_9)$ by feature 8, contribution of $F(x_3, x_8, x_5, r_{11}, \dots, r_N, \dots, r_9) - F(x_3, x_8, r_5, r_{11}, \dots, r_N, \dots, r_9)$ by feature 5, ..., contribution of $F(x_3, x_8, x_5, \dots, x_N, r_4, \dots, r_9) - F(x_3, x_8, x_5, \dots, r_N, r_4, \dots, r_9)$ by feature N , ..., and contribution of $F(x_3, x_8, x_5, x_{11}, \dots, x_N, x_4, \dots, x_7, x_9) - F(x_3, x_8, x_5, x_{11}, \dots, x_N, x_4, \dots, x_7, r_9)$ by feature nine where r_j represents a random draw from feature j . Suppose we write the sum of all the above items in a k th permutation as $Z_{(k)}$. Then $Z_{(k)} = F(x_3, x_8, x_5, x_{11}, \dots, x_N, x_4, \dots, x_7, x_9) - F(r_3, r_8, r_5, r_{11}, \dots, r_N, r_4, \dots, r_7, r_9)$. Now, suppose $F(x)$ value is independent of the order of the features in x . Then the average across all the $N!$ permutations of x , i.e., $\frac{1}{N!} \sum_{k=1}^{N!} Z_{(k)} = F(x) - \frac{1}{N!} \sum_{k=1}^{N!} F(r_{(k)})$ where $r_{(k)}$ is a k th random vector draw from feature vector of X .

With the independence of $F(x)$ from the order of the features in x , $\frac{1}{N!} \sum_{k=1}^{N!} Z_{(k)}$ can also be expressed $\sum_{j=1}^N \phi_j(F[x])$ from Eq. (6.2). Then $F(x) - \frac{1}{N!} \sum_{k=1}^{N!} F(r_{(k)}) = \sum_{j=1}^N \phi_j(F[x])$. Let $\phi_0 = \frac{1}{N!} \sum_{k=1}^{N!} F(r_{(k)})$. Thus,

$$F(x) = \phi_0 + \sum_{j=1}^N \phi_j(F[x]) \quad (6.3)$$

for a particular $x = (x_1, x_2, x_3, \dots, x_j, \dots, x_N)$ from a row of X . ϕ_0 can be considered as the expectation of the predictions by $F(\cdot)$ over random x . Eq. (6.3) provides the SHAP values $\phi_j(F[x])$ of different j features for the ML algorithm prediction $F[x]$. The SHAP values are additive and together help explain the predicted value of $F(x)$. SHAP values also obtain similar nice properties as in the Shapley game. This linearized construction still may not help directly in solving for the SHAP values ϕ_j . Some ML algorithm such as Trees, may however, provide for each instance that leads up to the final predicted probability the computation of contributions of features in arriving at that predicted probability. See Lundberg and Lee (2017).

In general, to solve for SHAP values, we can implement some approximation methods. For highly nonlinear ML algorithm where finding $\phi_j(F[x])$ directly is a problem due to the enormous amount of computations, approximations of $F(x)$ by a simpler $f(x)$ (with faster computing) in the neighborhood of each x in X may be done. One popular method is called LIME (Local Interpretable Model-agnostic Explanations) model.

Basically, we search for an interpretable surrogate or approximating $f(x)$ that minimizes the loss function L (square error loss) over all sample points i in the neighborhood of each x in X . $f(x)$ can be different for different x .

$$\min_f L(F, f, w) = \sum_j^T w_j [F(x_j) - \hat{f}(x_j)]^2 + \Omega(f)$$

where w is some weighting and $\Omega(f)$ is a penalty function that gets larger if the model f is more complex. For example, if f is linear regression, complexity could be reduced by limiting the number of and size of the coefficients. This leads to regularized linear regression. If f is DT method, complexity could be reduced by limiting the depth of the tree. Typically, the same type of LIME $f(x)$ is used for the X . In principle, as the LIME method can be applied to whatever $F(x)$, it is called a model-agnostic method.

Suppose the prediction or value function is $F(x_1, x_2, x_3, \dots, x_N)$ for a given row of features x in X . Suppose we approximate $F(x)$ by a linear ridge regression $f(x)$. For each x , randomly generate M local neighborhood points $x^{(1)}, x^{(2)}, \dots, x^{(M)}$ where each vector $x^{(i)}$ is $N \times 1$. Weigh more distant point $x^{(i)}$ from original x by smaller weight, e.g., $w^{(i)} = \exp(-\|x^{(i)} - x\|^2)/D$ where $D > 0$ is a hyperparameter denoting extent of circle of influence. Larger D means all weights increase. Hence, weighted point $x^{(i)}$ becomes $x^{*(i)} = w^{(i)}x^{(i)}$.

Using the ML algorithm $F(\cdot)$, find $F(x^{*(1)}) = Y_1$, $F(x^{*(2)}) = Y_2$, $\dots$, $F(x^{*(M)}) = Y_M$. Then perform linear regression $Y_i = b_0 + b_1 x_1^{*(i)} + b_2 x_2^{*(i)} + \dots + b_N x_N^{*(i)} + e_i$ for $i = 1, 2, \dots, M$, with coefficient constraints.

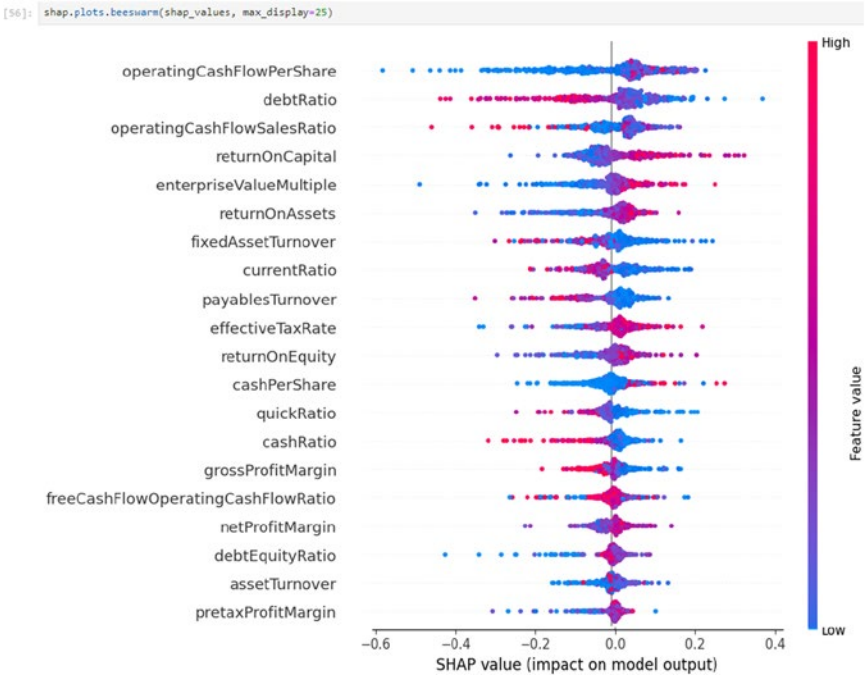
The regression produces estimates $\hat{b}_0, \hat{b}_j$'s from $x^{(i)}$'s. Then find $f(x) = \hat{b}_0 + \sum_{j=1}^N \hat{b}_j x_j$ with the original x_j 's. For this instance or sample point x , we can take expectation over the probability space of X in that neighborhood to obtain

$$E[f(X)] = \hat{b}_0 + \sum_{j=1}^N \hat{b}_j E(x_j).$$

Then, $f(x) - E[f(X)] = \sum_{j=1}^N \hat{b}_j[x_j - E(x_j)]$ or $f(x) = E[f(X)] + \sum_{j=1}^N \hat{b}_j[x_j - E(x_j)]$.

We can treat $E[f(X)] = \phi_0$ and $\sum_{j=1}^N \hat{b}_j[x_j - E(x_j)] = \sum_{j=1}^N \phi_j$ as in Eq. (6.3), thus obtaining $\hat{b}_j[x_j - E(x_j)]$ as the j th feature SHAP value for instance x . The sum of the SHAP values from all the N features in this instance x gives $\sum_{j=1}^N \phi_j = f(x) - E[f(X)]$.

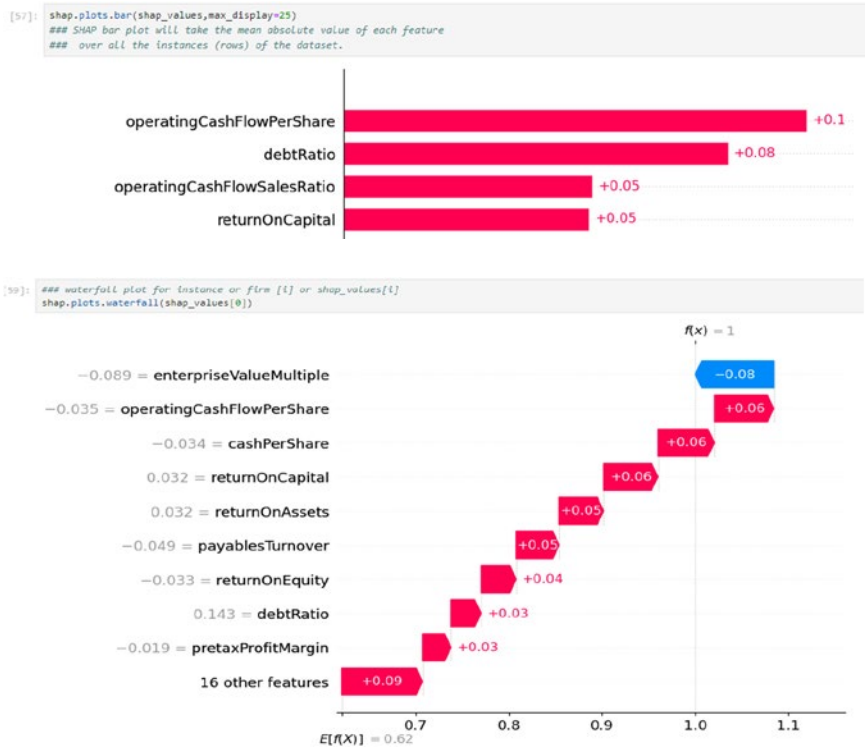
Continuing with Chapter6-1.ipynb, we analyse the XGBoost predictions on the test data set of 508 sample points. The percentage (mean) of the predicted values of “1”s in Class “1” is 0.615. Using the implementation of “import SHAP” and its SHAP explainer, we display the beeswarm plot (partial display) of SHAP values of each feature in each instance represented by the dots.



There are 508 points on each feature. For example, in the “return on capital” feature, most of the points in the oval have positive contributions to the predicted probability and they are features with high positive values.

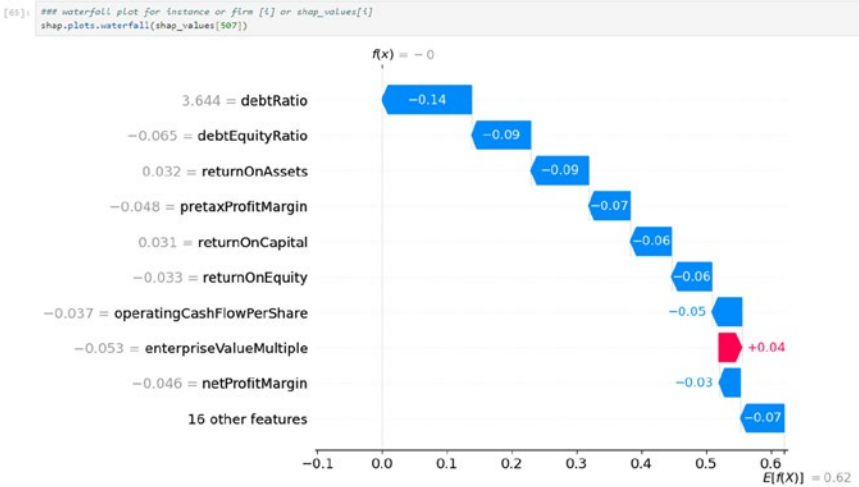
This is intuitively correct as high return on capital for a firm implies higher probability of being in Class “1” or of higher credit rating. In the “debt-ratio” feature, points in the rectangle have high debts and this feature on those points or instances or firms implies lower predicted probability of being in Class “1” and hence higher probability of being in Class “0” or of a lower credit rating.

Code line [57] shows the tabulation of average absolute SHAP value per instance across all the features. It is seen that “operating cash flow per share” and “debt ratio” have the largest contributions to predicted probability of being Class “1”. “operating cash flow per share” is also one of the four dominant features in features importance. Note that only the partial graph is shown due to space constraint.



We also show in code line [59] the ‘waterfall’ plot of the SHAP values of features in contribution to the predicted probability of Class “1” of the first firm or instance in the test data set. In this case, the predicted

probability $f(x) = 1$. The sum of SHAP values of all the 25 features is equal to $f(x) - E[f(x)] = 1 - 0.62 = 0.38$ where 0.62 is the mean of the predicted values of “1”s in Class “1”. For this first firm, the main features with highest SHAP value contribution to the predicted probability of Class “1” are “operating cash flow per share”, “return on capital”, “return on assets”, etc. The negative contribution of “enterprise value multiple” is due to a lower than average multiple.



We show in code line [65] the “waterfall” plot of the SHAP values of features in contribution to the predicted probability of Class “1” of the last firm or instance in the test data set. In this case, the predicted probability $f(x) = 0$. The sum of SHAP values of all the 25 features is equal to $f(x) - E[f(x)] = -0.62$. For this last firm, the main features with largest negative SHAP value contribution to the predicted probability of Class “1” are “debt ratio”, “debt-equity ratio”, “return on assets” (low feature value), etc.

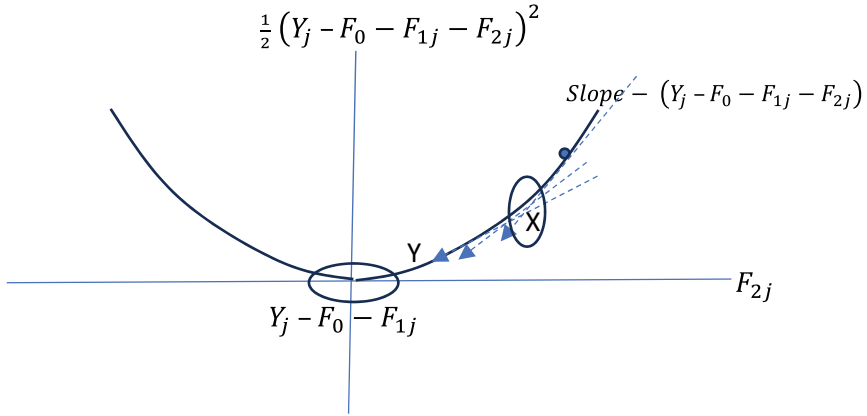
6.5 Concluding Thoughts

In summary, ensemble methods include the bagging approach of RF and the GB approach of GB, LightGBM, XGBoost, and others. Each DT in a RF of say 500 trees produces a prediction x_i . The prediction of the trees used the same decision rules formulated in the training sample for each of

the ensemble of trees. Thus, it is possible that the formulated rules could produce a biased estimate and hence biased prediction. For regression trees, the RF basically yields an averaged prediction from the 500 trees. The variance of this average is $\text{var}\left(\frac{1}{500} \sum_{i=1}^{500} x_i\right) \approx \frac{1}{500} \text{var}(x_i)$ which is much smaller than the variability of an individual DT prediction, assuming the predictions from different trees are approximately independent. Hence, bagging decreases variance in the prediction but may not decrease the bias if there is any.

For GB, the idea is to use gradient to boost subsequent trees to reduce the loss function toward zero. Typically, if the loss function gets to near-zero, the prediction would be very accurate. Hence, the GB approach reduces bias in trees toward zero, but the variability may be larger than in RF.

The above idea can be illustrated as follows in the regression tree for continuous or discrete target variable fitting. Suppose the loss function $L = \frac{1}{2} \sum_{j=1}^N \left(Y_j - \sum_{i=0}^n F_{ij}\right)^2$ for n trees is MSE or is quadratic in the fitted/predicted “error”. Suppose $n = 2$.



In RF, suppose most re-sampled test data trees based on the trained decision rules predict with a bias and end up close to point X (circled region). The averaged prediction of the RF ensemble would also in general be biased but with less variance. The GB trees however would sequentially improve by training on the gradients toward $Y_j - F_0 - F_{1j}$ so that the prediction would have less bias, but perhaps more variance (circled region). Note F_{1j} was also trained using gradient adjustment.

In GB, if the learning rate is too small, the algorithm will require many subsequent trees (iterations) to get close to the minimum. But if the learning rate is too high, the prediction may overshoot the target and end up having the next iteration or tree to revert the other direction. Thus, the learning may take longer.

XGBoost is GB method with added regularizations on the optimization of the loss function. With adequate hyperparameter tuning, it can produce slightly better predictive performance than the GBM. AdaBoost is another boosting method that is popular.

Besides the popular bagging via RF and the boosting gradient models in ensemble prediction, another ensemble approach is stacking. Stacking uses different models, e.g., RF, GB, SVM, NN to perform predictions. The different model predictions (from the different machine learners), called the intermediate predictions, are then possibly averaged to obtain the final ensemble prediction. This final prediction is said to be stacked on top of the intermediate models. More discussions of bias and variance in ensemble methods can be seen in chapter 6 of De Prado (2018).

In this chapter, we also see how the SHAP values can contribute a better understanding or explanation of the prediction in every instance, in addition to feature importance computations in trees. Specific additions or subtractions to predicted probability by features can help decision-making in improving features to directly affect the probability.

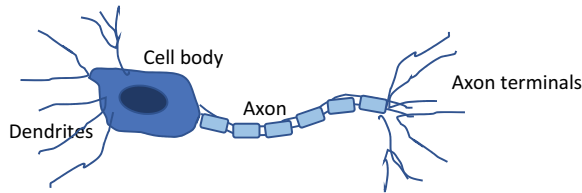
References

- De Prado, M. L. (2018). *Advances in Financial Machine Learning*. Wiley.
- Friedman J. H. (2002). Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4), 367–378.
- Geron A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras & Tensor Flow*. 2nd ed. O'Reilly Publisher.
- Lundberg, Scott M. and Su-In Lee (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*.

This page intentionally left blank

Chapter 7

Artificial Neural Network I



Neurons are nerve cells in the brain. They use electrical impulses and chemical signals to transmit information to other nerve cells, and between the brain and the rest of the nervous system. Neurons are composed of dendrites (signal inputs), cell body (processing), and axon with terminals (outputs). Neurons can connect with other neurons to form a chain of complex input and output responses. Signals pass from one neuron to another via synapses connecting axon terminals with dendrites of the next neuron. Refer to the diagram above.

Artificial Intelligence (AI) fundamentally is about harnessing computer programming to mimic human behavior such as choice and recognition using large data inputs. Machine Learning (ML) may be viewed as a specific AI domain to perform specific tasks such as prediction and classification. One major area of ML in this context is Neural Network (NN) that builds an architecture or structure of input-processing-output nodes mimicking the structure of human brain neurons. The processing of input information and the eventual output of prediction and classification is contained in the algorithm of the ML. Deep AI in such areas refers to more complex algorithms.

In this chapter, we study a basic NN called Multi-Layer Perceptron (MLP). This is sometimes referred to as Artificial Neural Network (ANN). In the next chapter we will study some other more complex NNs.

Understanding NNs mainly includes two aspects. One is the structure of NNs, and the other is the training and learning of NNs. It is just like how our brain structure is composed, and how we learn and recognize different things based on this composition.

A single layer perceptron may be characterized as follows in Figure 7.1 in the simple architecture with inputs, the computation process node in the middle, and output(s).

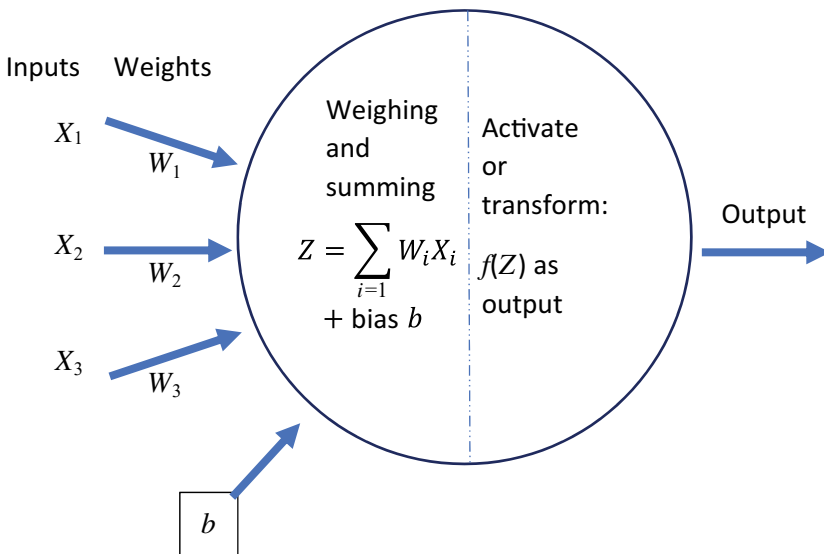


Figure 7.1. Single layer perceptron.

This embryonic version of today's NN essentially takes inputs or features of a case or instance, $\{X_1, X_2, X_3\}$, aggregates it with corresponding weights $\{W_1, W_2, W_3\}$, adds a bias term b , then computes the output

$$f\left(\sum_{i=1}^3 W_i X_i + b\right)$$

where the bias (or adjustment of mean) can be treated like an input of 1 with weight b . The weights W_j 's in a NN are initiated typically randomly

as small positive or negative numbers, e.g., between -1 and $+1$. $f(\cdot)$ is an activation or transformation function.

An example of an activation function is $f(Z) = 1$ if $Z \geq \theta$, and 0 otherwise, as seen in Figure 7.2. This could be an effective function in classifying cases with labels or targets that can be represented as binary 1 or 0 .

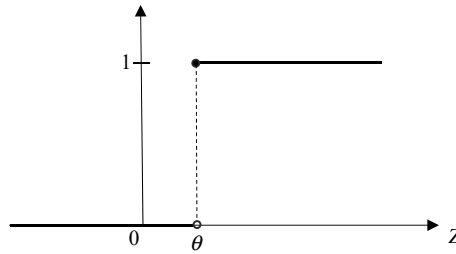


Figure 7.2. A simple activation function.

In a 2-dimensional problem with only two features X_1 and X_2 , if cases with label 1 and cases with label 0 are separated by dotted line $w_1x_1 + w_2x_2 + b = \theta$, then activation function $f(w_1x_1 + w_2x_2 + b) = 1$ or 0 depending on $w_1x_1 + w_2x_2 + b \geq \theta$ or $< \theta$ clearly serves the purpose of predicting the labels based on inputs x_1 and x_2 . See in Figure 7.3 such a separation of cases.

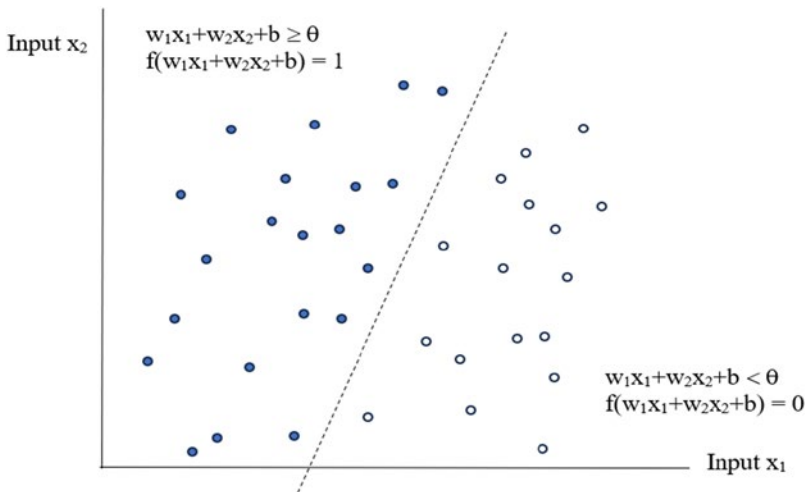


Figure 7.3. Predicting labels based on activation function.

In network language, the inputs are like nodes and the weights are edges.¹ (Sometimes the latter are referred to as the synapses.) The computation node may be called a neuron.

Suppose we use two layers of computations as follows. We call such NN MLP when there is more than 1 layer. Some authors prefer to call the following a single layer perceptron (not including the output layer as the second layer).

The same inputs or features are now fed to not one, but three different nodes or neurons $\#1^{[1]}$, $\#2^{[1]}$, $\#3^{[1]}$ in the first layer. Notationally, superscript $[j]$ denotes the j th computational layer. The first layer neurons each has output just like in a single layer perceptron and these outputs become inputs (with no change of values) to the next (output) layer of neuron. In the latter it is neuron $\#1^{[2]}$. Neuron $\#1^{[2]}$ then produces one output.

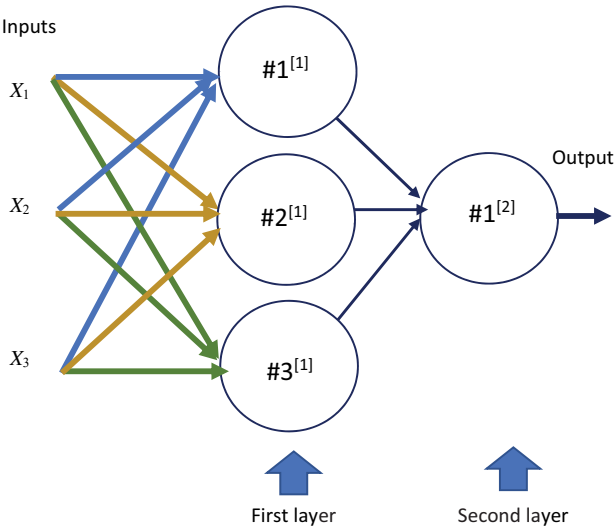


Figure 7.4. Illustrating a many-to-one ANN.

Figure 7.4 is a Many-to-One (many inputs toward one output) ANN. For a different problem, the structure could also be a Many-to-Many ANN

¹In mathematics and computer science, networks are structures that are represented by nodes, edges, and other abstract connections – they form a part of graph theory.

(see Figure 7.5) where there are now two nodes or neurons in the second (output) layer and then two outputs accordingly from neurons $\#1^{[2]}$ and $\#2^{[2]}$.

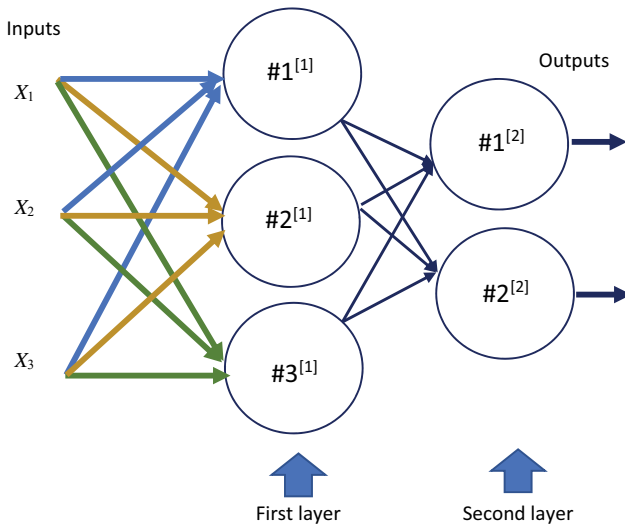
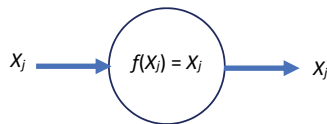


Figure 7.5. Illustrating a many-to-many ANN.

Note that we have not yet shown the weights and the bias in the diagram. In the following diagram, if we consider an input X_j producing identical output X_i (with a trivial identity activation function), then we can also treat an input as a neuron since we see that the last layer as outputs in Figure 7.5 has inputs as neurons in the first layer.



Then all the inputs can be represented as input nodes or neurons – now drawn similarly as circles. Moreover, we envelope neurons together that belong to the same forward step.

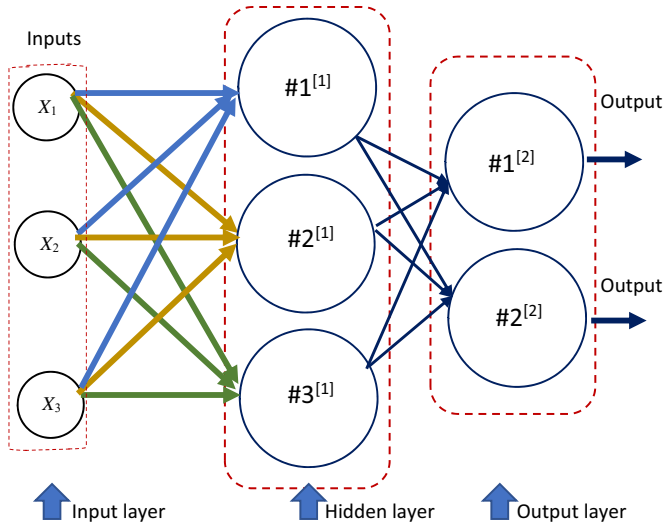


Figure 7.6. Input, hidden, and output layers.

Thus, there is an input layer of input nodes or neurons, an output layer of outputs or neurons, and a middle layer called “hidden layer” as the computed outputs from the nodes in this layer are not explicitly observed, unlike those in the input and output layer nodes. See Figure 7.6. For the same prediction or classification problem, an ANN can be made wider (larger number of input nodes or neurons in the same layer) and/or deeper (higher number of hidden layers). These affect the size (total number of neurons in the NN). A deeper NN can increasingly learn more nonlinear patterns. In this standard structure, each neuron will connect with all neurons in the right-side layers via the forward pass process denoted by the pointed arrows.

A rule of thumb is that the number of nodes in the hidden layer could be about $2/3$ of those in the input layer and the output layer combined, and less than twice the number in the input layer. These node width and depth could be hyperparameters to be tuned.

Pruning removes nodes where input weights are close to zero – pruning can occur during training of model. Normalizing features also avoids some weights being close to 0 as out-scale feature sizes are standardized.

Suppose an ANN has two hidden layers, an input layer of similarly three nodes (besides bias), and one output node. It can be represented diagrammatically as follows in Figure 7.7.

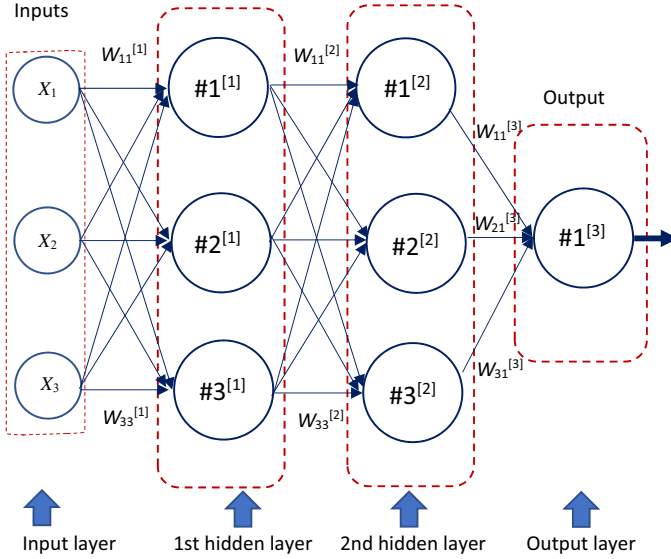


Figure 7.7. Notations of nodes in ANN.

The edges or synapses connecting different neurons in different layers continue to represent weights. We depict some weights in the diagram above (not all – so as not to clutter up the diagram). The weights of input X_1 on neurons $\#1^{[1]}$, $\#2^{[1]}$, $\#3^{[1]}$ are respectively $W_{11}^{[1]}$, $W_{12}^{[1]}$, and $W_{13}^{[1]}$. The weights of input X_2 on neurons $\#1^{[1]}$, $\#2^{[1]}$, $\#3^{[1]}$ are respectively $W_{21}^{[1]}$, $W_{22}^{[1]}$, and $W_{23}^{[1]}$. The weights of input X_3 on neurons $\#1^{[1]}$, $\#2^{[1]}$, $\#3^{[1]}$ are respectively $W_{31}^{[1]}$, $W_{32}^{[1]}$, and $W_{33}^{[1]}$.

The weights of output from neuron $\#1^{[1]}$ on neurons $\#1^{[2]}$, $\#2^{[2]}$, $\#3^{[2]}$ are respectively $W_{11}^{[2]}$, $W_{12}^{[2]}$, and $W_{13}^{[2]}$. The weights of output from neuron $\#2^{[1]}$ on neurons $\#1^{[2]}$, $\#2^{[2]}$, $\#3^{[2]}$ are respectively $W_{21}^{[2]}$, $W_{22}^{[2]}$, and $W_{23}^{[2]}$. The weights of output from neuron $\#3^{[1]}$ on neurons $\#1^{[2]}$, $\#2^{[2]}$, $\#3^{[2]}$ are respectively $W_{31}^{[2]}$, $W_{32}^{[2]}$, and $W_{33}^{[2]}$.

The weights of outputs from neurons $\#1^{[2]}$, $\#2^{[2]}$, $\#3^{[2]}$ on neuron $\#1^{[3]}$ are respectively $W_{11}^{[3]}$, $W_{21}^{[3]}$, and $W_{31}^{[3]}$. The weights are connections

from one neuron to another in the next forward layer. There is no connection between neurons in the same layer. In general, the weight from n th hidden layer $\#j^{[n]}$ neuron to the next layer $\#k^{[n+1]}$ neuron is $w_{jk}^{[n+1]}$.

The purpose of the ANN is to perform prediction or classification on a set of data containing cases or instances with labels or targets and are each associated with a set of features or characteristics. Examples are predictions of likely default (target status) given cases of borrowers with different background features/characteristics, of treatment success (target status) given patients (subjects) with different medical background features/conditions, of correctly identifying criminals (target status) given subjects' profile and history, of Click-Through Rate (CTR) in a digital marketing campaign with different campaign resources, of next stock price or corporate earnings given latest corporate financial and other information, of identifying an object whether it belongs to a particular classification given features/characteristics of the object, and so on.

To prepare a model for prediction with new data, the model must be trained using many known cases/subjects or instances with their known target statuses and with associated features/characteristics or conditions in the training data set. Hyperparameters in the model could be fine-tuned or optimally selected using a validation data set. Once the model is calibrated with optimal hyperparameters, the weights could be optimally trained again using a re-grouped training data set (discussed in earlier chapters). The ANN can then perform the necessary computations based on these optimal weights (including optimal biases), fine-tuned hyperparameters, and the fresh inputs to find the predicted output(s) or target value(s) in the test data set.

The ANN computes the output(s) in the output layer in a forward pass process called Forward Propagation – this is discussed as follows. At the end of the pass, the errors of the fittings of all training data points are measured by a loss function to be minimized. The reduction in fitting errors requires the revision of the weights and biases – this uses a process called Backward Propagation which will also be discussed later. One forward propagation together with one backward propagation of the entire training data set is called an epoch. Sometimes the training data set (whole batch) is broken down into (mini-) batches that are smaller subsets in a random partitioning. If the size of each mini batch is fixed at n , then the number of mini batches in an epoch is total training sample size divided by the mini batch size. The forward and backward propagations are then

done one mini-batch after another until the whole training data set has gone through one epoch. The number of mini batches implies the number of iterations in updating or revising the weights and biases.

Many epochs can be performed, which means a new epoch will require a new re-sampled training set (some may use the same sample). Each forward propagation on a new mini batch of same epoch or next epoch uses the updated weights and biases from the previous backward propagation. The total number of iterations is equal to the number of epochs multiplied by the number of mini batches in each epoch. Sometimes it may not be necessary to increase the number of epochs or mini batches per epoch when the fitting/prediction error in the training data set is sufficiently small.

7.1 Forward Propagation

We use a single hidden layer MLP to illustrate the forward propagation computation of a single output. The bias inputs are depicted as originating from the unfilled circles – we use a more general structure whereby the bias can be different for each of hidden and output layer neuron – shown as $b_j^{[n]}$ – corresponding to the $\#j^{[n]}$ neuron. (This is equivalent to using the same bias of value 1 for all neurons in the next forward layer and then putting different weights $b_j^{[n]}$ on them.) See Figure 7.8.

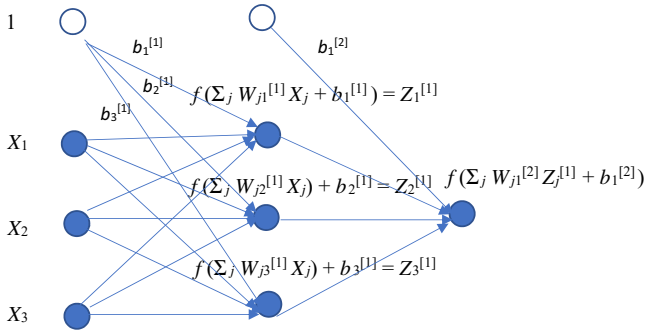


Figure 7.8. Forward propagation in MLP.

Given the various weights, biases, and inputs, the final output is

$$f(\sum_j W_{j1}^{[2]} Z_j^{[1]} + b_1^{[2]}) = \hat{Y}$$

for a particular case or training sample point with associated features (X_1, X_2, X_3) where superscript [1] to output Z_j denotes output is at hidden layer 1. In a standard ANN structure, the activation function $f(\cdot)$ remains the same for every layer.

7.1.1 Activation function $f(\cdot)$

The weighted inputs and biases are passed through activation function $f(Z)$. The latter is typically a nonlinear function with an S-curve such as the Sigmoid function (logistic function) $f(Z) = 1/(1 + e^{-Z})$, the Hyperbolic Tangent function (Tanh) $f(Z) = (e^Z - e^{-Z}) / (e^Z + e^{-Z})$, or the Rectifier Linear Unit function² (ReLU) that is $f(Z) = \max(Z, 0)$. The nonlinear functions are depicted graphically as follows in Figure 7.9. In the standard ANN, the activation function remains the same for each neuron in the same layer and remains the same also for different layers.

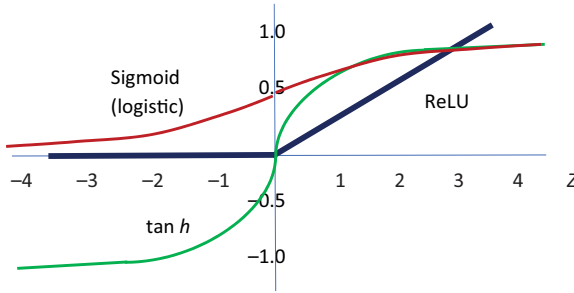


Figure 7.9. Different activation functions.

Suppose we have N number of data points (sample points or cases) in our training data set. Each case/subject/instance k has an actual output Y_k and associated features $(X_{1,k}, X_{2,k}, X_{3,k})$. The notation now includes subscript k to indicate association with one case or instance.

The forward propagation computations after an optimal number of revisions of weights and biases end up with a target prediction of

²An alternative version, the Leaky ReLu is sometimes used. Unlike ReLu, Leaky ReLu has a small positive slope instead of zero slope for negative Z values. This slope is pre-fixed before training. Using Leaky ReLu generally appears to help in reaching convergence faster than the standard ReLu.

$$\hat{Y}_k = f(\sum_j W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})$$

where $Z_{m,k}^{[1]} = f(\sum_j W_{jm}^{[1]} X_{j,k} + b_m^{[1]})$ denotes output node m of the hidden layer in the k th case. Note there is only one output in this NN, but we add subscripts k to denote the k th case or instance in the training set data. Also, the weights W_{jm} are constant for whichever k . As the final output or target prediction $\hat{Y}_k$ is computed in a forward pass manner, this ANN is also called a feedforward NN due to this simple architecture. We consider NNs with more complicated architecture in a later chapter – those are sometimes referred to as deep learning NNs.

7.1.2 Error or loss functions

The training fitting/prediction error is an error or loss function based on the training data set of all training sample points and measures “deviations” of NN outputs (target predictions) of all cases with the actual outputs of the associated cases. The idea in training the model is to find optimal weights and biases so that the training set error or loss function is minimized or reduced toward zero. A commonly used error/loss function for continuous variable targets is the average least squares of deviations (or Mean Square Error, MSE):

$$L(Y, \hat{Y}) \equiv \frac{1}{N} \sum_{k=1}^N L(Y_k, \hat{Y}_k) = \frac{1}{N} \sum_{k=1}^N (Y_k - \hat{Y}_k)^2 \quad (7.1)$$

where $Y = (Y_1, Y_2, Y_3, \dots, Y_N)$ and $\hat{Y} = (\hat{Y}_1, \hat{Y}_2, \hat{Y}_3, \dots, \hat{Y}_N)$. This is also called L^2 loss function and is common for target variables that are continuous variables and not categorical or binary numbers. Sometimes its square root, the Root Mean Square Error (RMSE) is used. An alternative is the mean absolute loss or Mean Absolute Error (MAE) or L^1 loss function

$$L(Y, \hat{Y}) = \frac{1}{N} \sum_{k=1}^N |Y_k - \hat{Y}_k|$$

In scenarios where there are outliers, MSE will usually be much larger than MAE because the difference between estimated target values and actual target values will be amplified after squaring. MAE is thus more robust in the presence of some outliers, i.e., producing more stable results.

However, MSE has better convergence properties when it comes to finding optimal weights.

The Huber loss function $L_\delta(Y, \hat{Y})$ is a combination of MSE and MAE, including a hyperparameter δ : $\frac{1}{N} \sum_{k=1}^N L_\delta(Y_k, \hat{Y}_k)$ where

$$L_\delta(Y_k, \hat{Y}_k) = \begin{cases} \frac{1}{2}(Y_k - \hat{Y}_k)^2, & \text{if } |Y_k - \hat{Y}_k| < \delta \\ \delta|Y_k - \hat{Y}_k| - \frac{1}{2}\delta^2, & \text{otherwise} \end{cases}$$

When $|Y_k - \hat{Y}_k| < \delta$, L_δ becomes MSE. When $|Y_k - \hat{Y}_k| \geq \delta$, it functions like MAE. Huber loss function overcomes the shortcoming of MSE when the deviation becomes larger perhaps due to an outlier.

When the actual target or output is categorical, different types of error/loss functions may be more practicable. In a binary classification where the target is either 1 or 0, e.g., it rains or it does not in weather prediction, or it is a bird or it is not based on picture pixels, or the firm has earnings increase or not, then error function could be 1 minus the accuracy where accuracy is the fraction of correct prediction. However, this function may not be easily differentiable which makes it harder to use for finding optimal weights.

Typically, a binary classification prediction using features would provide a probability estimate $\hat{P}$ of the target status whether it is 1. Otherwise, $1 - \hat{P}$ would be estimate of the probability the target status is 0. A useful error/loss function for the binary classification is binary cross-entropy that employs estimate $\hat{P}_k$ for each case k : $L(Y, \hat{P}) = \frac{1}{N} \sum_{k=1}^N L(Y_k, \hat{P}_k)$ where

$$L(Y_k, \hat{P}_k) = -\left[Y_k \ln \hat{P}_k + (1 - Y_k) \ln(1 - \hat{P}_k)\right]$$

This is also the log loss function for binary classification. $\hat{P}_k$ is the predicted probability that $Y_k = 1$, and $1 - \hat{P}_k$ is the predicted probability that $Y_k = 0$. Note that here when $Y_k = 1$, the loss function is reduced when $\hat{P}_k \in (0, 1)$ is higher. When $Y_k = 0$, the loss function is reduced when $\hat{P}_k \in (0, 1)$ is lower. This loss function therefore captures the idea of ensuring the estimated or predicted probability of $(Y_k = 1)$, $\hat{P}_k$, is high when $Y_k = 1$, and that the predicted probability of $(Y_k = 1)$, $\hat{P}_k$, is low when $Y_k = 0$. The log loss function above is positive. The smaller is the log loss

function or the smaller the binary cross-entropy measure toward zero, the less is the uncertainty and hence better prediction.

Recall that we may deem $\hat{Y}_k = 1$ or 0 if $\hat{P}_k > 0.5$ or else ≤ 0.5 . Note that unlike the MSE error/loss function where the predictor $\hat{Y}_k$ enters intuitively in the loss function $L(Y_k, \hat{Y}_k)$, the loss function in the categorical output cases is $L(Y_k, \hat{P}_k)$ where the predicted classification is not exactly the predicted output $\hat{P}_k$ but a function of it, $\hat{Y}_k(\hat{P}_k)$.

When there is more than one output, e.g., Q outputs for Q number of categories, then the (categorical) cross-entropy loss function for each case k is

$$L(Y_{k,c}, \hat{P}_{k,c}) = -\sum_{c=1}^Q Y_{k,c} \ln \hat{P}_{k,c}$$

where $Y_{k,c}$ is 1 (otherwise 0) if case Y_k belongs to class c , and $\hat{P}_{k,c}$ is the predicted probability of case k belonging to class c . We may in general use the notation of error/loss function as $L(Y_k, \hat{Z}_k)$ where $\hat{Z}_k \equiv \hat{Y}_k$ in the continuous variable case and $\hat{Z}_k \equiv \hat{P}_k$ in the categorical case.

7.2 Backward Propagation

In each epoch or training of the entire training data set from inputs to final output(s) in the output layer, the predicted outputs are compared with the actual (target) outputs and the loss function $L(Y, \hat{Z})$ is computed. Thus, the epoch involves passing the entire training set of points forward, then backward with revision of weights and biases before final passing forward to obtain the output and loss function. The second epoch will involve a resampled training data set, continued updating with a second backward propagation and then forward pass to obtain the updated loss function. This may continue for many epochs where necessary to obtain satisfactory convergence in the loss function.

When the loss function or binary cross entropy measure is too high, the model needs further improvement – we need to train the NN further, by updating/revising the weights and biases and run more epochs and so on, and hopefully end with an acceptable loss function outcome.

The Backward Propagation is the mechanism of training the NN toward an acceptable loss before proceeding for application to the validation and then the test data set. While forward propagation performs

computations by moving forward in the process from inputs to the final output, backward propagation involves backwards pass that performs computations by moving backward from the loss function at the final output to adjusting the weights and biases.

As in Figure 7.8, the weights and biases feeding into each layer forward are initially obtained by random draws from usually a small range $(-1, +1)$. In the single hidden layer MLP, the weights are $\{W_{11}^{[1]}, W_{21}^{[1]}, W_{31}^{[1]}, b_1^{[1]}\}$ (including the bias) for neuron #1^[1] with output $Z_{1,k}^{[1]}$ for case or instance k . The weights are $\{W_{12}^{[1]}, W_{22}^{[1]}, W_{32}^{[1]}, b_2^{[1]}\}$ (including the bias) for neuron #2^[1] with output $Z_{2,k}^{[1]}$, and $\{W_{13}^{[1]}, W_{23}^{[1]}, W_{33}^{[1]}, b_3^{[1]}\}$ (including the bias) for neuron #3^[1] with output $Z_{3,k}^{[1]}$. In the output layer, the weights are $\{W_{11}^{[2]}, W_{21}^{[2]}, W_{31}^{[2]}, b_1^{[2]}\}$ (including the bias) for neuron #1^[2] with output $Z_{1,k}^{[2]} \equiv f(\sum_j W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]}) = \hat{Y}_k$. Hence, in this structure there are altogether 16 parameters of weights and biases to update/revise: $\{W_{11}^{[1]}, W_{21}^{[1]}, W_{31}^{[1]}, b_1^{[1]}, W_{12}^{[1]}, W_{22}^{[1]}, W_{32}^{[1]}, b_2^{[1]}, W_{13}^{[1]}, W_{23}^{[1]}, W_{33}^{[1]}, b_3^{[1]}, W_{11}^{[2]}, W_{21}^{[2]}, W_{31}^{[2]}, b_1^{[2]}\}$.

To update $W_{11}^{[1]}$ toward minimizing the error/loss objective function, a standard approach is to use the gradient descent method/algorithm or its variants. The gradient descent method requires the computation of the partial slope of the objective function with respect to each of the weights and biases. The estimation uses the chain rule. In the single layer MLP, Figure 7.10 below shows how

$$\frac{\partial L(Y, \hat{Z})}{\partial W_{11}^{[1]}} = \frac{\partial \sum_{k=1}^N \frac{1}{N} L(Y_k, \hat{Z}_k)}{\partial W_{11}^{[1]}}$$

is computed. For each case k ,

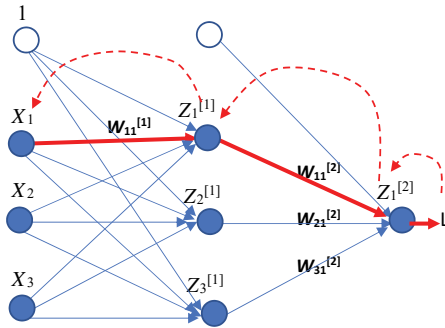


Figure 7.10. Backward propagation.

$$\begin{aligned}
\frac{\partial L(Y_k, \hat{Z}_k)}{\partial W_{11}^{[1]}} &= \left(\frac{\partial L(Y_k, \hat{Z}_k)}{\partial Z_{1,k}^{[2]}} \times \frac{\partial Z_{1,k}^{[2]}}{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})} \right) \\
&\times \left(\frac{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})}{\partial Z_{1,k}^{[1]}} \times \frac{\partial Z_{1,k}^{[1]}}{\partial \sum j(W_{j1}^{[1]} X_j + b_1^{[1]})} \right) \\
&\times \frac{\partial \sum j(W_{j1}^{[1]} X_j + b_1^{[1]})}{\partial W_{m1}^{[1]}}
\end{aligned} \quad (7.2)$$

for $m = 1, 2, 3$. In the case $m = 1$, Figure 7.10 shows the backwards pass along the path of the forward pass process starting at X_1 with edge or synapse weight $W_{11}^{[1]}$ (see bold line). The partial derivatives components follow the dotted lines. Similarly,

$$\begin{aligned}
\frac{\partial L(Y_k, \hat{Z}_k)}{\partial W_{m2}^{[1]}} &= \frac{\partial L(Y_k, \hat{Z}_k)}{\partial Z_{1,k}^{[2]}} \times \frac{\partial Z_{1,k}^{[2]}}{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})} \times \frac{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})}{\partial Z_{2,k}^{[1]}} \\
&\times \frac{\partial Z_{2,k}^{[1]}}{\partial \sum j(W_{j2}^{[1]} X_j + b_2^{[1]})} \times \frac{\partial \sum j(W_{j2}^{[1]} X_j + b_2^{[1]})}{\partial W_{m2}^{[1]}}
\end{aligned}$$

for $m = 1, 2, 3$.

$$\begin{aligned}
\frac{\partial L(Y_k, \hat{Z}_k)}{\partial W_{m3}^{[1]}} &= \frac{\partial L(Y_k, \hat{Z}_k)}{\partial Z_{1,k}^{[2]}} \times \frac{\partial Z_{1,k}^{[2]}}{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})} \times \frac{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})}{\partial Z_{3,k}^{[1]}} \\
&\times \frac{\partial Z_{3,k}^{[1]}}{\partial \sum j(W_{j3}^{[1]} X_j + b_3^{[1]})} \times \frac{\partial \sum j(W_{j3}^{[1]} X_j + b_3^{[1]})}{\partial W_{m3}^{[1]}}
\end{aligned}$$

for $m = 1, 2, 3$.

And

$$\begin{aligned}
\frac{\partial L(Y_k, \hat{Z}_k)}{\partial b_1^{[1]}} &= \frac{\partial L(Y_k, \hat{Z}_k)}{\partial Z_{1,k}^{[2]}} \times \frac{\partial Z_{1,k}^{[2]}}{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})} \times \frac{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})}{\partial Z_{1,k}^{[1]}} \\
&\times \frac{\partial Z_{1,k}^{[1]}}{\partial \sum j(W_{j1}^{[1]} X_j + b_1^{[1]})} \times \frac{\partial \sum j(W_{j1}^{[1]} X_j + b_1^{[1]})}{\partial b_1^{[1]}}
\end{aligned}$$

$$\begin{aligned} \frac{\partial L(Y_k, \hat{Z}_k)}{\partial b_2^{[1]}} &= \frac{\partial L(Y_k, \hat{Z}_k)}{\partial Z_{1,k}^{[2]}} \times \frac{\partial Z_{1,k}^{[2]}}{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})} \times \frac{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})}{\partial Z_{2,k}^{[1]}} \\ &\quad \times \frac{\partial Z_{2,k}^{[1]}}{\partial \sum j(W_{j2}^{[1]} X_j + b_2^{[1]})} \times \frac{\partial \sum j(W_{j2}^{[1]} X_j + b_2^{[1]})}{\partial b_2^{[1]}} \end{aligned}$$

$$\begin{aligned} \frac{\partial L(Y_k, \hat{Z}_k)}{\partial b_3^{[1]}} &= \frac{\partial L(Y_k, \hat{Z}_k)}{\partial Z_{1,k}^{[2]}} \times \frac{\partial Z_{1,k}^{[2]}}{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})} \times \frac{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})}{\partial Z_{3,k}^{[1]}} \\ &\quad \times \frac{\partial Z_{3,k}^{[1]}}{\partial \sum j(W_{j3}^{[1]} X_j + b_3^{[1]})} \times \frac{\partial \sum j(W_{j3}^{[1]} X_j + b_3^{[1]})}{\partial b_3^{[1]}} \end{aligned}$$

and so on.

For example, if $L(Y, \hat{Y}) = \frac{1}{N} \sum_{k=1}^N (Y_k - \hat{Y}_k)^2$, then for each k :

$$\frac{\partial L(Y_k, \hat{Y}_k)}{\partial Z_{1,k}^{[2]}} = -2(Y_k - \hat{Y}_k).$$

Note that if activation function $f(X) = 1/(1 + e^{-X})$, $df(X)/dX = e^{-X}/(1 + e^{-X})^2 = (1 - f(X))f(X)$. Note that $Z_{1,k}^{[2]} = f(\sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]}), Z_{1,k}^{[2]} \equiv \hat{Y}_k$. Then,

$$\begin{aligned} \frac{\partial Z_{1,k}^{[2]}}{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})} &= (1 - \hat{Y}_k) \hat{Y}_k \\ \frac{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})}{\partial Z_{1,k}^{[1]}} &= W_{11}^{[2]} \end{aligned}$$

$$\frac{\partial Z_{1,k}^{[1]}}{\partial \sum j(W_{j1}^{[1]} X_j + b_1^{[1]})} = (1 - Z_{1,k}^{[1]}) Z_{1,k}^{[1]}$$

and $\frac{\partial \sum j(W_{j1}^{[1]} X_j + b_1^{[1]})}{\partial W_{11}^{[1]}} = X_1$.

Hence,

$$\frac{\partial L(Y_k, \hat{Y}_k)}{\partial W_{11}^{[1]}} = -2(Y_k - \hat{Y}_k) \times (1 - \hat{Y}_k) \hat{Y}_k \times W_{11}^{[2]} \times (1 - Z_{1,k}^{[1]}) Z_{1,k}^{[1]} \times X_1$$

and

$$\frac{\partial L(Y, \hat{Y})}{\partial W_{11}^{[1]}} = -\frac{1}{N} \sum_{k=1}^N 2(Y_k - \hat{Y}_k) \times (1 - \hat{Y}_k) \hat{Y}_k \times W_{11}^{[2]} \times (1 - Z_{1,k}^{[1]}) Z_{1,k}^{[1]} \times X_1$$

For a shorter forward pass from $Z_1^{[1]}$ to L , the partial derivative of

$$\begin{aligned} \frac{\partial L(Y_k, \hat{Y}_k)}{\partial W_{11}^{[2]}} &= \left(\frac{\partial L(Y_k, \hat{Y}_k)}{\partial Z_{1,k}^{[2]}} \times \frac{\partial Z_{1,k}^{[2]}}{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})} \right) \\ &\times \frac{\partial \sum j(W_{j1}^{[2]} Z_{j,k}^{[1]} + b_1^{[2]})}{\partial W_{11}^{[2]}} \end{aligned} \quad (7.3)$$

which is just part of Eq. (7.2). For longer forward pass involving a deeper NN such as the following with three hidden layers and two final outputs, the partial derivative of error/loss function (possibly weighted linear combination of the loss functions L_1 and L_2 of the two final outputs) with respect to $W_{11}^{[1]}$ (the first edge or weight from input X_1) is the sum of each of $3^2 \times 2$ backward derivative chain computations – one such backward pass is shown in bold below in Figure 7.11.

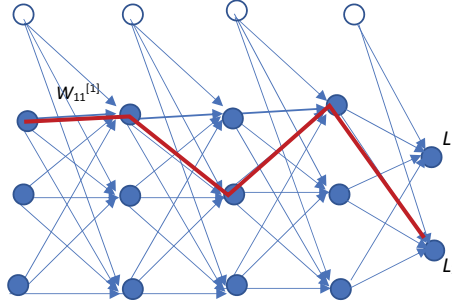


Figure 7.11. Updating weights in backward pass.

The purpose of finding all derivatives of loss function with respect to all the weights and biases in the ANN is as follows. Suppose for a particular weight (or bias) W , $\partial L / \partial W > (<) 0$, then we could try to attain a lower loss by decreasing (increasing) the weight (or bias). To do so, we could find a revised weight (or bias) by adjusting it lower (higher) from the

previous one. Hence, the adjustment for each iteration (over mini batches and over epochs) or updating of all weights at the same time is as follows.

$$W_{ij}^{*[r]} = W_{ij}^{[r]} - \alpha \frac{\partial L}{\partial W_{ij}^{[r]}}$$

where updated weight is $W_{ij}^{*[r]}$, and $\alpha > 0$ is a learning rate that determines how fast is the descent toward the minimum. α is a hyperparameter in this algorithm. These adjustments toward a smaller loss function form the gradient descent approach in the loss function minimization.

In Eqs. (7.1) and (7.2), we see that one backward propagation computation (and the initial forward pass) in one epoch (across all data points N in the training set) involves at least N times the number of computations of the partial derivatives for one case/instance k such as in Eq. (7.2). The computing memory resource may be overstretched with the large number of sample point derivatives. With E epochs, there would only be E number of iterations or updates on the parameters and this may not be adequate if the NN is highly nonlinear.

A variant of the (batch) gradient descent, is to perform the backward propagation using not the entire training sample of size N in one epoch, but smaller (mini) batches of size $n < N$. This is called the “mini-batch” gradient descent approach.

In this way, the computational memory burden for each iteration involving only n sample points and correspondingly fewer partial derivatives is lesser. The number of mini batches in an epoch is then N/n . There are then N/n number of iterations in each epoch, i.e., all the data need to pass through before one epoch is counted. If E number of epochs is run, then in total there will be $E \times N/n > E$ number of iterations in updating the parameters in the training. This will provide more accuracy if the NN is highly nonlinear.

Another popular variant is the Stochastic Gradient Descent (SGD) that randomly (by default random generator in the algorithm) sequentially computes the gradients and update for each sample point within the batch – it is like the mini-batch with a batch size of 1. The updating of the SGD partial derivatives for each sample point can cause the convergence to the minimum error/loss function to be more volatile than when using the (full) batch or mini-batch training data set. However, there are far more iterations. SGD typically has a constant learning rate. Different

platforms such as TensorFlow may, however, have related SGD algorithms that are built a bit differently.

Besides batch size strategy in achieving computationally efficient convergence, another related algorithm is in adjusting the learning rate with each parameter/weights update/iteration instead of keeping it constant throughout the training. One popular algorithm is Adam (adaptive moment estimation). See Kingma and Ba (2015). The algorithm computes the exponentially decaying (moving) averages of past gradients and past squared gradients, viz.

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) G_t$$

and

$$V_t = \beta_2 V_{t-1} + (1 - \beta_2) G_t^2$$

where t is each iteration step forward, G_t and G_t^2 are the computed gradient and squared gradient at t for a particular weight updating, i.e., $G_t = \partial L / \partial W_{ij,t}^{[r]}$. M_0 and V_0 are typically initialized at 0. M_t and V_t are moving averages of G_t and G_t^2 , respectively.

Now, $M_1 = (1 - \beta_1)G_1$, $M_2 = \beta_1(1 - \beta_1)G_1 + (1 - \beta_1)G_2$, $M_3 = \beta_1^2(1 - \beta_1)G_1 + \beta_1(1 - \beta_1)G_2 + (1 - \beta_1)G_3$.

In general, $M_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} G_i$. Taking unconditional expectation,

$$E(M_t) = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} E(G_i)$$

Assuming $E(G_i)$ for every i is approximately constant, then

$$E(M_t) \approx E(G_t)(1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} = E(G_t)(1 - \beta_1^t)$$

The approximate mean of G_t , $E(G_t) \approx E(M_t) / (1 - \beta_1^t)$. It is said that the M_t process supposedly converging over long-term to some exponentially (decreasing) weighted averages of past and current G_t is biased away from $E(G_t)$ and M_t needs to be adjusted to $\widehat{M}_t = M_t / (1 - \beta_1^t)$ so that $E(\widehat{M}_t) = E(M_t) / (1 - \beta_1^t) \approx E(G_t)$. Now, $\widehat{M}_t$ is (approximately) unbiased so that its long-term weighted averaging of past and current G_t should lead to $E(G_t)$.

In a similar analysis,

$V_1 = (1 - \beta_2)G_1^2, V_2 = \beta_2(1 - \beta_2)G_1^2 + (1 - \beta_2)G_2^2, V_3 = \beta_2^2(1 - \beta_2)G_1^2 + \beta_2(1 - \beta_2)G_2^2 + (1 - \beta_2)G_3^2, \dots, V_t = (1 - \beta_2)\sum_{i=1}^t \beta_2^{t-i}G_i^2$. Taking unconditional expectation,

$$E(V_t) = (1 - \beta_2)\sum_{i=1}^t \beta_2^{t-i}E(G_i^2)$$

Assuming $E(G_i^2)$ for every i is approximately constant, then

$$E(V_t) \approx E(G_i^2)(1 - \beta_2)\sum_{i=1}^t \beta_2^{t-i} = E(G_i^2)(1 - \beta_2^t)$$

The approximate “uncentered variance” (second moment) of G_t , $E(G_t^2) \approx E(V_t)/(1 - \beta_2^t)$. It is said that the V_t process supposedly converging over long-term to some exponentially (decreasing) weighted averages of past and current G_t^2 is biased away from $E(G_t^2)$ and V_t needs to be adjusted to $\hat{V}_t = V_t/(1 - \beta_2^t)$ so that $E(\hat{V}_t) = E(V_t)/(1 - \beta_2^t) \approx E(G_t^2)$. Now, $\hat{V}_t$ is (approximately) unbiased so that its long-term weighted averaging of past and current G_t^2 should lead to $E(G_t^2)$.

The Adam update rule for gradient descent in the weight adjustment during backward propagation is:

$$W_{i,j,t}^{[r]} = W_{i,j,t-1}^{[r]} - \theta \frac{M_t}{\sqrt{V_t + \epsilon}}$$

where given a step size θ (hyperparameter to be tuned – it is typically a small positive number), adjustment or descent is faster with higher $\hat{M}_t$ and lower $\hat{V}_t$ (more persistent gradient in one direction with less variability). In most implementations, default values for β_1 and β_2 are 0.9 and 0.999. ϵ is included to prevent G_t (hence also G_t^2) disappearing to 0 when division otherwise becomes not possible in the denominator. Typical value for ϵ is 10^{-8} . A comparison with the gradient descent shows the learning rate in Adam may be construed as changing according to $\alpha_t = \theta \hat{M}_t / \left[G_t \left(\sqrt{\hat{V}_t} + \epsilon \right) \right]$. Hence, using the Adam “optimizer” or the updating or learning rule involves more computations of these hyperparameter-related parameters in the NN.

Any algorithm can also set a limit time to the iterations or stop the iterations when the loss function is close to minimum and not reducing

anymore with iterations. Other discussions of backpropagation can be seen in <https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/> by M. Mazur (2015). MLP application example in sentiment analysis can be seen in <https://towardsdatascience.com/multilayer-perceptron-explained-with-a-real-life-example-and-python-code-sentiment-analysis-cb408ee93141/> by C. Bento (2021).

7.3 Worked Example: Data

The data set – “Churn_Modelling.csv” – is found in public resource: <https://www.kaggle.com/datasets/>. The data set contains a bank’s 10,000 customers’ details, viz., nationality, gender, credit score (higher score means more credit-worthy), age, tenure (years with the bank), USD bank balance in the period of the data, the number of the bank products used by the customer, whether the customer has the bank credit card, is active customer, and estimated USD salary of the customer. These serve as features of each customer that could explain the target variable – which is whether the customer exited or left the bank. In the latter the customer had closed his/her bank account during the data period and the classification of exit is “1”. Otherwise, the customer continues with the bank and the classification of exit is “0”. For ease of interpretation, assume the exit prediction is for a year forward.

The ability to predict if an existing customer is likely to exit the bank (or analogously to exit as customer of a store or as employee of a company) is important as customer exit or attrition or loss of company clients could be highly damaging to the bottom line or even reputation of a company. If a customer is predicted to be likely to exit, or those customers likely to exit possess certain types or characteristics, then the bank or company can step up certain strategies on those characteristics to try to retain such customers and promote customer loyalty.

The following code lines [1–3] in demonstration file Chapter7-1.ipynb show the data set. Note that TensorFlow (incorporating Keras) is the key platform and API/algorithms used for the building the MLP NNs here.

```
[1]: #Importing necessary Libraries
import numpy as np
import pandas as pd
import tensorflow as tf
```

```
[2]: #Loading Dataset
data = pd.read_csv("Churn_Modelling.csv")
```



```
[3]: pd.options.display.max_columns = None
data.head()
```

```
[3]:
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age
0	1	15634602	Hargrave	619	France	Female	42
1	2	15647311	Hill	608	Spain	Female	41
2	3	15619304	Onio	502	France	Female	42

In code lines [4–6], the Gender variable and the Geography variable are transformed to dummy or 1, 0 variables.

```
[4]: ### Generating Dependent Variable Vectors and Features
y = data.iloc[:,1].values
### ".values" returns the data as a NumPy array
X = data.iloc[:,3:13]
X.head()
```

```
[4]:
```

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
0	619	France	Female	42	2	0.00	1	1	1	101346.88
1	608	Spain	Female	41	1	83807.86	1	0	1	112542.58
2	502	France	Female	42	0	159660.00	3	1	0	113931.57
3	699	France	Female	39	1	0.00	2	0	0	93826.63
4	850	Spain	Female	43	2	125510.82	1	1	1	79064.10

```
[5]: X['Gender'] = X['Gender'].map({'Female':0, 'Male':1})
### above is used instead of a more complicated package involving --
### from sklearn.preprocessing import LabelEncoder, which converts
### Female to 0, Male to 1, i.e. hot-encoding categorical variables
print (X['Gender'])
```

```
0      0
1      0
2      0
3      0
4      0
..
9995    1
9996    1
9997    0
9998    1
9999    0
Name: Gender, Length: 10000, dtype: int64
```

```
[6]: ### Encoding Categorical variable Geography
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [1])], \
                           remainder="passthrough")
X = np.array(ct.fit_transform(X))
### Geography is transformed into France:1,0,0;Spain:0,0,1;Germany:0,1,0
### Moreover, this encoded vector of ones-zeros is now put in first 3 cols.
### Credit Score pushed to 4th col.
```

Code line [8] shows the scaling of the feature data in the whole sample (both training and test set combined) which is useful if the features are data with different orders of magnitudes.

```
[8]: ### Performing Feature Scaling
      from sklearn.preprocessing import StandardScaler
      sc = StandardScaler()
      X = sc.fit_transform(X)
```

In cross-sectional feature data, we can do same mean and standard deviation transforms of each feature before shuffling into train and test set, since each random shuffling should be invariant to the scaling.

Each feature in the training set is scaled to mean 0 and variance 1. In `sklearn.preprocessing.StandardScaler()`, centering and scaling operations happen independently on each feature. The fit method is calculating the mean and variance of each of the features present in the data. The transform method is transforming all the features (and target variable) using the respective features' (target's) means and variances. Note that normalizing the features may avoid some weights being close to 0 as out-scale feature sizes are standardized.

Code line [10] shows split of the dataset into training (80%) and test (20%) data sets.

```
[10]: ### Splitting dataset into training and testing dataset
       from sklearn.model_selection import train_test_split
       X_train,X_test,Y_train,Y_test = train_test_split(X,Y,test_size=0.2,\
                                                       random_state=1)
```

Note that it is assumed a validation set would have been already used to find appropriate hyperparameters, including the type of hyperparameter computation such as “Adam”. Then the training and validation data sets are re-grouped into this current training data set for training and then testing.

In this Chapter7-1.ipynb example, the depth and width of the NN are kept small to show comparative improvements in later examples in this chapter. Besides the learning rate, the other hyperparameters are the number of hidden layers (depth) and the number of neurons in each layer (width).

Code lines [11] and [12] use the `tf.keras.models.Sequential()` API to build the MLP NN model.

```
[11]: ### This is the very first step while creating NNmodel -- you can rename this model.
### Here we are going to create NNmodel object by using a certain class of Keras
### named Sequential. As a part of tensorflow 2.0, Keras is now integrated with
### tensorflow and is now considered as a sub-library of tensorflow. The Sequential class
### is a part of the models module of Keras Library which is part of tensorflow library now.
### It used to be "import tensorflow as tf; from tensorflow import keras;
### from tensorflow.keras import layers"
### See documentation at https://keras.io/guides/sequential_model/

#Initialising the NN model name -- NNmodel
NNmodel = tf.keras.models.Sequential()
### Sequential specifies to keras that the model NNmodel is created sequentially and
### the output of each layer added is input to the next specified layer.
### Note that keras Sequential is not appropriate when the model has multiple outputs

[12]: ### Creating a network with 1 hidden layer, 1 input layer and 1 output layer
### Adding First Hidden Layer
NNmodel.add(tf.keras.layers.Dense(units=2,activation="sigmoid"))
### units = 2 refer to 2 neurons in hidden layer
### modelname.add is used to add a layer to the neural network
### -- need to specify as an argument what type of layer
### Dense is used to specify the fully connected layer -
### i.e. all neurons are forward connect to all forward layer nodes
```

In the above, first hidden layer is created using the Dense class which is part of the “layers” module. A Dense class denotes a dense layer in the NN. A dense layer is also called a “fully connected” layer. This means that every neuron in one layer is connected to every neuron in the next layer. This class accepts 2 inputs – (1) units: number of neurons that will be present in the respective layer (2) activation: specifying which activation function to be used. This example uses first input as 2. We start with a small width and will increase this to show how the NN will become more effective. In general, there is no correct answer which is the exactly right number of neurons in the layer – it will be by trial and error or some mechanical iterations till optimal accuracy or predictive effectiveness is attained. It would not be too small to be ineffective and also not be too large to be computationally too expensive. For the second input, we try the sigmoid or logistic function as an activation function for hidden layers. We can also try “relu” [rectified linear unit].

Then we create the output layer as follows.

```
[14]: ### now we create the output layer
      #Adding Output Layer
      NNmodel.add(tf.keras.layers.Dense(units=1,activation="sigmoid"))
      ### Only 1 output neuron
```

For a binary classification problem as above, actual case output is 1 or 0. Hence, we require only one neuron to output layer – output could be estimated probability of case’s actual output = 1. For multiclass classification problem, if the output contains m categories, then we need to create $m - 1$ different neurons for the different categories. In the binary output case, the suitable activation function is the sigmoid function. For multiclass classification problem, the activation function is typically “softmax”. The “softmax” function predicts a multinomial probability distribution. Next we need to compile the NN model using Keras.

```
[16]: ### After creating the layers - require compiling the NNmodel. Compiling allows the
      ### computer to run and understand the program. It adds other elements or link other
      ### libraries, and does optimization, such that after compiling the results are readily
      ### computed e.g. in a binary executable program as an output.
      ### Compiling NNmodel
      NNmodel.compile(optimizer="adam",loss="binary_crossentropy",metrics=['accuracy'])
      ### Note optimizer here is a more sophisticated version of the Mean Square Loss
```

The Compile method above accepts as inputs – (1) optimizer: specifies which optimizer to be used in order to perform gradient descent, (2) error/loss function, e.g., “binary_cross-entropy” here. (For multiclass classification, it should be categorical_crossentropy), and (3) metrics: the performance metrics to use in order to compute performance. “accuracy” is one such performance metric.

Next, we check the dimension of X_{train} .

```
[18]: X_train.shape
```

```
[18]: (8000, 12)
```

In code line [19], we run the training of the NN on the training data set. This uses the “.fit” in TensorFlow-Keras.

```
[19]: ### Last step in creation of NNmodel NNmodel is trained on the training set here
      ### with Tensor-Keras .fit based on Compiled NNmodel
      history=NNmodel.fit(X_train,Y_train,batch_size=8000,epochs = 100)
      ### Note that tf.keras.models.Sequential() by default uses glorot initializer -
      ### - drawing intial weights from a uniform distribution -- see other possibilities
      ### in https://keras.io/api/layers/initializers/
      ### Or you could try own customized wts inputs using for layer in model.layers:
      ### init_layer_weight = []
      ### layer.set_weights(init_layer_weight)
```

We show the output as follows. Since we specify in [19] batch size of 8000 on a 8000 sample point training set, the entire training set is run in one iteration to update the parameters. This constitutes one epoch. Then the next epoch is run. The outputs show the updated accuracy and loss each epoch 1/100, 2/100, ..., till the 100/100 epoch. Within each epoch, there is only one batch, so it completes 1/1 in each epoch. (If there were two mini-batches, then it would show completion of 2/2 before moving to the next epoch.)

```
Epoch 1/100
1/1 ————— 1s 589ms/step - accuracy: 0.5250 - loss: 0.6954
Epoch 2/100
1/1 ————— 0s 25ms/step - accuracy: 0.5276 - loss: 0.6945
Epoch 3/100
1/1 ————— 0s 19ms/step - accuracy: 0.5293 - loss: 0.6936
```

```
.....

Epoch 98/100
1/1 ————— 0s 20ms/step - accuracy: 0.6920 - loss: 0.6198
Epoch 99/100
1/1 ————— 0s 24ms/step - accuracy: 0.6933 - loss: 0.6191
Epoch 100/100
1/1 ————— 0s 19ms/step - accuracy: 0.6957 - loss: 0.6184
```

It is noted that after 100 epochs or iterations, the loss is 0.6184 and accuracy (% of correct predictions of exit or retention) is 0.6957 (69.57%). There is one more computation after this when the updated weights after the 100th epoch is used in the .evaluate computation of the prediction – see below.

Code line [20] provides a summary of the architecture/structure of this NN.

```
[20]: NNmodel.summary()
```

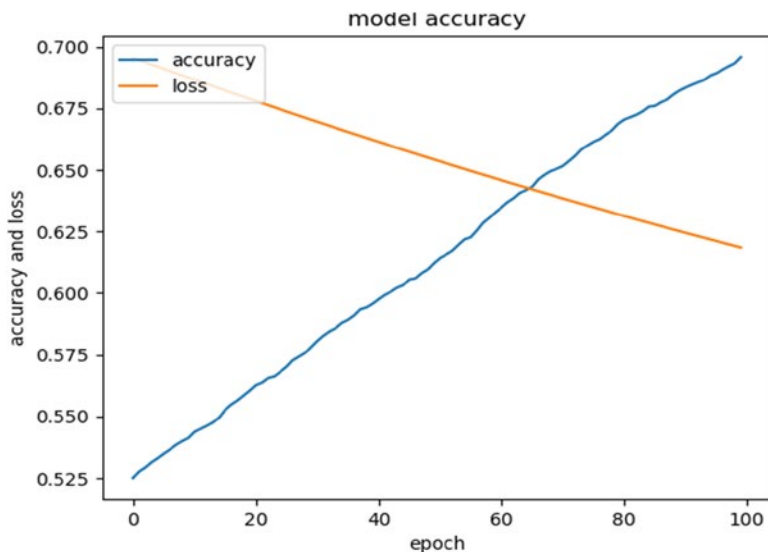
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(8000, 2)	26
dense_1 (Dense)	(8000, 1)	3

As there are 12 features as inputs, and one bias as input to each of the two neurons in the first hidden layer, there are $(12 + 1) \times 2 = 26$ weight parameters in the first dense (fully connected) hidden layer. In the 2 outputs of the first hidden layer, adding the next bias, there are $2 + 1 = 3$ weight parameters in the next dense layer which is also the output layer.

The outputs of losses and accuracies in code line [19] can be captured in the 'history' output of the NN model. They are plotted as shown in [21].

```
[21]: import matplotlib.pyplot as plt
print(history.history['accuracy'])
print(history.history['loss'])
plt.plot(history.history['accuracy'])
plt.plot(history.history['loss'])
plt.title('model accuracy')
plt.ylabel('accuracy and loss')
plt.xlabel('epoch')
plt.legend(['accuracy', 'loss'], loc='upper left')
plt.show()
```



The plot shows that as the number of training iterations (epochs) increases, the loss decreases, and the accuracy increases to about 70%.

We can obtain the final iteration weights and biases of the trained NN and use these to compute any predicted output (which is a probability of exit) given the features' values. These are shown in code lines [22–26].

```
[22]: ### Below are the weights in the final iteration
first_layer_weights = NNmodel.layers[0].get_weights()[0]
first_layer_biases = NNmodel.layers[0].get_weights()[1]
second_layer_weights = NNmodel.layers[1].get_weights()[0]
second_layer_biases = NNmodel.layers[1].get_weights()[1]
```

```
[23]: print(first_layer_weights)
first_layer_weights.shape
```

```
[[ 0.38304186 -0.4201266 ]
 [ 0.6633316 -0.09484755]
 [ 0.536904  0.48574576]
 [-0.19971466 -0.09429155]
 [ 0.22192973 0.60840285]
 [-0.22986005 -0.02993411]
 [ 0.37355074 -0.10736771]
 [ 0.39336446 -0.6348443 ]
 [ 0.19186138 -0.45866606]
 [ 0.5351657  0.3326683 ]
 [-0.25606754 0.5614511 ]
 [-0.36787778 0.01808825]]
```

```
[23]: (12, 2)
```

Code line [23] shows the trained weights for the (first) hidden layer neurons. There are 2 units (width of 2) or 2 neurons in the hidden as indicated in code line [12]. The first column shows weight on each input feature to the first or top neuron. The second column shows weight on each input feature to the second or bottom neuron. Code line [24] shows the two biases' weights for the two neurons in the (first) hidden layer.

```
[24]: print(first_layer_biases)
first_layer_biases.shape
### (2,) here basically means 2 elements in a 1-dim array.
### .T has no effect on 1d array
```

```
[-0.09503298 0.09826887]
```

```
[24]: (2,)
```

Code lines [25] and [26] show the weights in the (second) or output layer.

```
[25]: print(second_layer_weights)
      second_layer_weights.shape
      [[ 0.83238465]
       [-1.1146983 ]]
[25]: (2, 1)
```

```
[26]: print(second_layer_biases)
      second_layer_biases.shape
      [-0.09636633]
[26]: (1,)
```

Now, we can use the trained weights and biases to predict on a new case with features (1, 0, 0, 500, 1, 40, 3, 60000, 2, 1, 1, 100000). These are standardized as in [8]. See code line [27]. (Note sometimes we may not need to standardize dummy variables.)

```
[27]: ## Now use trained weights and biases to predict based on a new case
      tr=sc.transform([[1, 0, 0, 500, 1, 40, 3, 60000, 2, 1, 1, 100000]])
      print(tr) ## tr.shape is (1,12)
      [[ 0.99720391 -0.57873591 -0.57380915 -1.55748773  0.91241915  0.10281024
        -0.69598177 -0.26422114  0.80773656  0.64609167  0.97024255 -0.00156918]]
```

Based on the NN architecture, the output is then computed in code line [28] using “.predict” as 0.3890. This is verified using code lines [29–33] showing the forward pass computations with the activation function as the sigmoid or logistic function.

```
[28]: ## Example
      ## Predicting result for Single Observation
      print(NNmodel.predict(tr))
      ## note in each recompute -- this no. will change slightly because of
      ## the random initiation of the weights
      1/1 ————— 0s 34ms/step
      [[0.38903028]]
```

```
[29]: ## now we compute the predicted prob of 1, manually
      tr.dot(first_layer_weights) ## gives a 1 x 2 matrix
```

```
[29]: array([[0.06886018, 0.68774673]])
```



```
[30]: Flayerneurons_sum=tr.dot(first_layer_weights) + first_layer_biases
      print(Flayerneurons_sum) ### 1 x 2 matrix

      [[-0.0261728  0.7860156]]

[31]: Flayerneurons_act=1/(1+np.exp(-Flayerneurons_sum))
      print(Flayerneurons_act) ### 1 x 2 matrix: output of neurons in hidden Layer

      [[0.49345717 0.68697516]]

[32]: Slayerneurons_sum=Flayerneurons_act.dot (second_layer_weights)+second_layer_biases
      print(Slayerneurons_sum)

      [[-0.4513902]]

[33]: predprob=1/(1+np.exp(-Slayerneurons_sum))
      print(predprob)
      ### Note this is the same output as NNmodel.predict(tr). This manual computation
      ### of the forward pass should have output same as in NNmodel.predict(tr)

      [[0.38903029]]
```

In code line [34], we use “evaluate” (based on the trained NN in [19]) to predict outputs for X_train and then compare with the actual outputs in Y_train. This returns a loss of 0.6178 and accuracy of 0.6974. Thus “evaluate” produces loss and accuracy while “predict” in [28] produces the probability of case output being “1”.

```
[34]: ### Now we use the trained NNmodel to predict output in X_train sample
      NNmodel.evaluate(X_train,Y_train)
      ### evaluates the loss and accuracy as specified in the Compiler

      250/250 ————— 0s 450us/step - accuracy: 0.6913 - loss: 0.6199

[34]: [0.6178003549575806, 0.6973749995231628]
```

The above is based on the final weights after the 100 epochs and should be consistent with “predict”. Code lines [35–44] check the predicted outputs and compare with the actual outputs and returns an accuracy of 0.697375 in [44] which is the same as that in [34].

```
[35]: ### Now use the trained NNmodel to predict output in X_train sample
      ### - computing manually via .predict
      TE=NNmodel.predict(X_train) ### note X_train has 8000 data points

[36]: TE.shape

[36]: (8000, 1)
```

TE is a 8000×1 vector.

```
[37]: h=(TE > 0.5).astype(int) ### Convert TE>0.5 == true ==> 1, False to 0

[38]: print(h)
      h.shape

      [[0]
       [1]
       [0]
       ...
       [0]
       [0]
       [0]]

[38]: (8000, 1)

[39]: ### replace all predicted elements in numpy array of value 0 with value -1
      h[h==0]=-1

[41]: ### replace all actual elements in numpy array of value 0 with value -1
      Y_train1=Y_train
      Y_train1[Y_train1==0]=-1

[44]: J=np.multiply(Y_train1.T,h.T) ### element by element multiplication
      c=np.count_nonzero(J > 0)
      print(c,c/8000)

      5579 0.697375
```

In code line [45], we finally use the trained NN to perform prediction of exits in the test sample (of 2000 points).

```
[45]: ### Now use the trained NNmodel to predict output in X_test sample
      NNmodel.evaluate(X_test,Y_test)
      ### evaluates the loss and accuracy as specified in the Compiler

      63/63 ----- 0s 646us/step - accuracy: 0.7261 - loss: 0.6145

[45]: [0.6181555986404419, 0.69900000009536743]
```

The accuracy is verified in code lines [46] and [47] as 69.90%.

```
[46]: ### Now use the trained NNmodel to predict output in X_test sample -
      ### - computing manually via .predict
      TE1=NNmodel.predict(X_test) ### note X_test has 2000 data points

      63/63 ----- 0s 511us/step
```

```
[47]: h1=(TE1 > 0.5).astype(int) ### Convert TE1>0.5 == true ==> 1, False to 0
      h1[h1==0]=-1
      Y_test1=Y_test
      Y_test1[Y_test1==0]=-1
      J1=np.multiply(Y_test1.T,h1.T) ### element by element multiplication
      c1=np.count_nonzero(J1 > 0)
      print(c1,c1/2000)

1398 0.699
```

Note the “.evaluate” here used the trained weights in [19] “.fit”. It should be noted that as the weights and biases feeding into each layer forward are initially obtained by random draws from usually a small range (−1, +1), a MLP will produce slightly different predictive results for each new run unless we fix the random seed.³ Re-sampling with each new epoch can also create a difference. Different results may be more obvious in NN with less depth and less width.

The TE1 in code line [46] of keras provides the vector of predicted probabilities in the context of binary cross entropy loss in predicting the binary class of “1” or “0”. The vector also translates into a vector of predicted 1’s or 0’s depending on whether the predicted probability (of “1”) is > 0.5 or not. These vectors can be used to compute the confusion matrix, the classification table, and the ROC AUC using sklearn.metrics.

7.4 Improving the NN and Understanding the Results

Although the accuracy of the prediction of bank customer exit in the single hidden layer (2 neurons) MLP shown in the last section for the test data set is 69.90%, which is reasonably good, it can perhaps be improved. Also, sometimes a single batch may not yield robust or stable results. A telltale sign of the possibility of improvement is indicated in [21] where the training performance graph shows the loss is still decreasing and accuracy still climbing at the end of the training iterations.

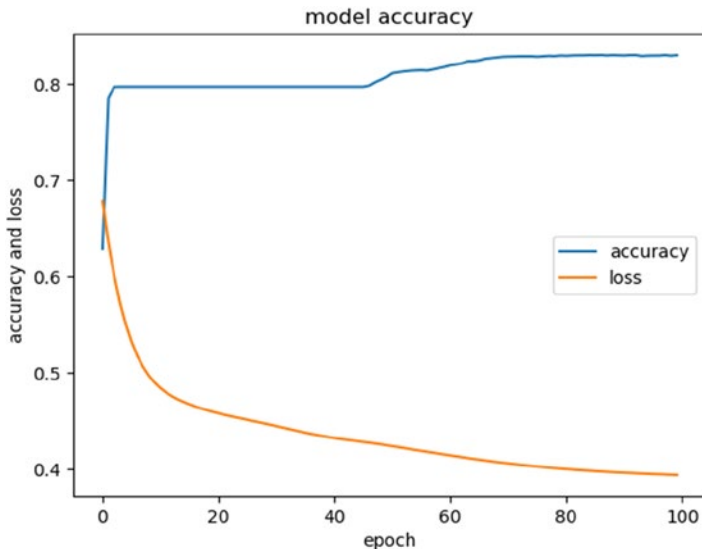
We increase the number of backward passes and iterations, hence parameter revisions, by using “mini-batches” of batch size = 100 (instead

³This can be done by putting in the first code line of the notebook the following. `import json \ import numpy as np \ import tensorflow as tf \ import keras \ from keras import layers \ from keras import initializers\ keras.utils.set_random_seed(5) ## set a seed number 5 or something else \ tf.config.experimental.enable_op_determinism()`.

of the entire batch size of 8000), and still using 100 epochs. See demonstration file Chapter7-2.ipynb.

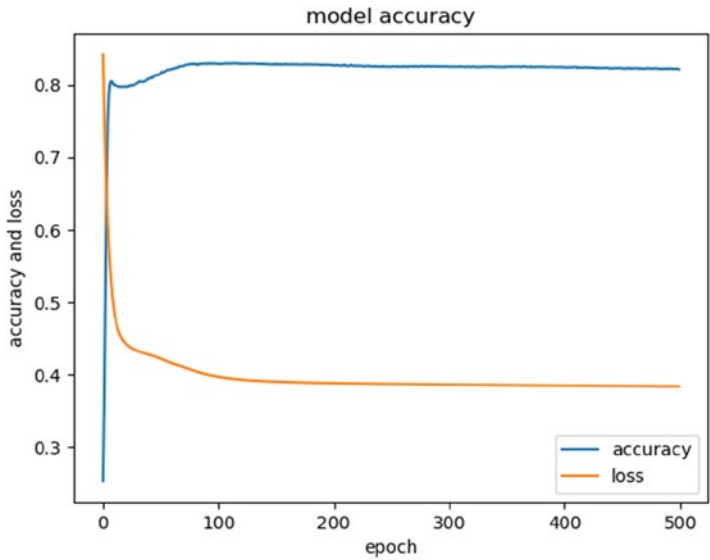
```
[18]: ### Last step in creation of NNmodel NNmodel is trained on the training set
      ### here with Tensor-Keras .fit based on Compiler Fitting NNmodel
      history=NNmodel.fit(X_train,Y_train,batch_size=100,epochs = 100)
```

This yields the following loss and accuracy graph.



It is seen that after 1 epoch or 80 iterations (each epoch has 80 mini-batch iterations; $80 = 8000/100$), the accuracy had already climbed to over 77%. The loss and accuracy stabilize at higher epochs, ending with training accuracy of about 82.99% (see output to code line [33] and [43]). The test accuracy is 83.10% (see output to code line [46]). Both accuracies are better than those of 69.74% and 69.90% in the previous case of batch size = 8000 and 100 epochs. Hence, higher number of iterations is useful to improve accuracy.

We also document the case of keeping batch size = 100 but increasing the number of epochs to 500. This created a total of $80 \times 500 = 40,000$ iterations. See demonstration file Chapter7-3.ipynb. This yields the following loss and accuracy graph.



The loss and accuracy stabilize after about 6 epochs, ending with training accuracy of 82.25%. The test accuracy is 82.60%. Both accuracies are about similar to the case of batch size = 100 and 100 epochs. The initial few epochs saw a sharp hike in accuracy from a starting low of about 24%. The latter is possible as the initiating weights are random draws and could start off with first several iterates of large error/loss. Overall, it is seen that having 100 epochs in this case and a batch size of 100 may already clock in a sufficient number of iterations to adequately train the NN.

We employ the same batch size = 100 and 500 epochs, but attempt 3 different models (1), (2), and (3). In (1) (see demonstration file Chapter7-4.ipynb), we adjust the optimizer to SGD instead of Adam. In (2) (see demonstration file Chapter7-5.ipynb), we add a second hidden layer with now 4 neurons in both the first and second hidden layers. We create an even deeper NN in (3) (see demonstration file Chapter7-6.ipynb) where the number of neurons in the first and the second hidden layers are increased from 4 to 8. In (3) there is therefore a total of $(12 + 1) \times 8 = 104$ neurons in the first hidden layer, $(8 + 1) \times 8 = 72$ neurons in the second hidden layer, and $8 + 1 = 9$ neurons in the output layer.

For (3), the code lines for building the NN are shown as follows. Note how the second hidden layer is built in the Keras-Sequential algorithm.

```
#Initialising the NN model name -- NNmodel
NNmodel = tf.keras.models.Sequential()
```

```
### Creating a network that has 2 hidden layers together with 1 input Layer and 1 output Layer.
#Adding First Hidden Layer
NNmodel.add(tf.keras.layers.Dense(units=8,activation="sigmoid"))
### units = 8 refer to 8 neurons in hidden Layer
```

```
### Creating 2nd hidden Layer
#Adding Second Hidden Layer -- note this is added sequentially to the first hidden Layer
NNmodel.add(tf.keras.layers.Dense(units=8,activation="sigmoid"))
### units = 8 refer to 8 neurons in hidden Layer
```

```
### now we create the output Layer -- this is added sequentially
#Adding Output Layer
NNmodel.add(tf.keras.layers.Dense(units=1,activation="sigmoid"))
### Only 1 output neuron
```

```
NNmodel.compile(optimizer="adam",loss="binary_crossentropy",metrics=['accuracy'])
```

```
NNmodel.fit(X_train,Y_train,batch_size=100,epochs = 500)
```

The minimized (after the specified number of iterations) loss value and the accuracy of the predictions for the different models of (1), (2), (3) are reported below. Note the results may change a little for any new run since the initialization of the parameters is random and each epoch may see a re-sampling.

Metrics	(1)	(2)	(3)
Training set			
Loss	0.3951	0.3355	0.3229
Accuracy (%)	83.00	86.00	86.46
Test set			
Loss	0.3879	0.3331	0.3280
Accuracy (%)	83.10	86.30	86.75

In (1), it is seen that the SGD optimizer performs about the same as Adam with the same number of iterations in batch size = 100, epochs = 500, one hidden layer and 2 neurons in the layer. The former, (1), has slightly larger loss but slightly higher accuracy. Model (2) with 2 hidden layers of 4 neurons each improves accuracy by about 3% compared to the case of only

1 hidden layer. The deeper NN with 8 neurons in each hidden layer in (3) improves further by about 0.5% on both training and test accuracies from (2).

During the training, pruning or reducing the total number of neurons or nodes could occur in order to improve the prediction performances. `tensorflow_model_optimization` module in tensorflow can be used to prune away weights that are close to zero. The weights from one layer to another can be viewed as matrices. When the weights are too small and close to zero, they can be seen as sparse matrices. The module can prune based on indicated final level of sparsity to be attained where sparsity is the percentage number of weights that is set to zero. In general, pruned NN can perform better.

We also tap on the Scikit-learn (Sklearn) python library to use the LogisticRegression codes to perform classification test on the training data set. Logistic regression performance outcomes can be used for base-line comparison. See demonstration file Chapter7-7.ipynb. The code lines are shown as follows.

```
[12]: from sklearn.linear_model import LogisticRegression
      LogReg=LogisticRegression()

[13]: lresult = LogReg.fit(X_train, Y_train)
      print(lresult.coef_, lresult.intercept_)
      ### SKlearn logisticRegression adds regularization so the results
      ### are a bit different from the statsmodel package above

      [[-0.12256364  0.21736622 -0.07633822 -0.05348885 -0.26533692  0.74822048
        -0.02792812  0.16934491 -0.04022361 -0.03669323 -0.54902098  0.01605643]] [-1.6584463]
```

Code lines [14] and [15] show the prediction accuracy of the logistic regression method.

```
[14]: Lpredict=LogReg.predict(X_test)

[15]: ### Using Score method to obtain accuracy (% correct prediction) of model
      score = LogReg.score(X_test,Y_test)
      print(score)

      0.8125
```

The prediction accuracy using the estimated logit regression coefficients (where estimated probability > 0.5 is designated as predicting 1,

otherwise 0) shows 81.25%. This is better than the first case in the last section with 1 hidden layer and epoch = 100. It is just slight below case (1) with 1 hidden NN layer but batch size = 100 and epoch = 500. The better trained and deeper NNs in (2) and (3) appear to perform better than logistic regression prediction.

We also perform a logit regression of the exit variable (1 or 0) on the feature variables using package “statsmodel” in python. See code line [19]. The regression outputs are summarized as follows. Note that sklearn Logistic Regression adds regularization, so the results are a little bit different from statsmodel package results shown here.

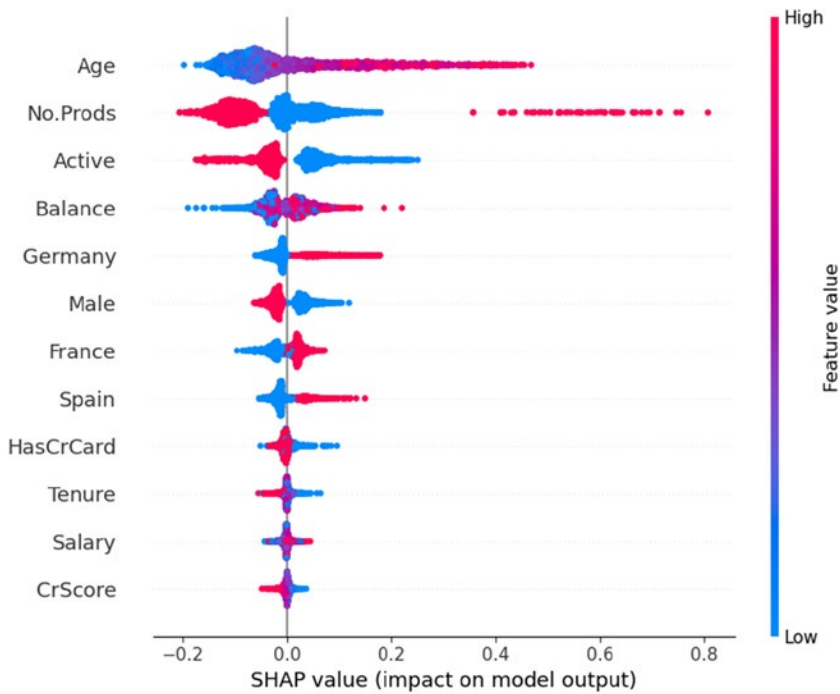
Logit Regression Results						
=====						
Dep. Variable:	y	No. Observations:	8000			
Model:	Logit	Df Residuals:	7987			
Method:	MLE	Df Model:	1			
Date:	Tue, 26 Nov 2024	Pseudo R-squ.:	0.1490			
Time:	19:59:49	Log-Likelihood:	-3432.6			
converged:	False	LL-Null:	-4033.5			
Covariance Type:	nonrobust	LLR p-value:	6.935e-250			
=====						
	coef	std err	z	P> z	[0.025	0.975]

Constant	-1.6590	0.035	-47.936	0.000	-1.727	-1.591
France	-0.1227	nan	nan	nan	nan	nan
Spain	0.2174	nan	nan	nan	nan	nan
Germany	-0.0764	nan	nan	nan	nan	nan
CrScore	-0.0535	0.030	-1.771	0.077	-0.113	0.006
Gender	-0.2656	0.030	-8.768	0.000	-0.325	-0.206
Age	0.7491	0.030	24.800	0.000	0.690	0.808
Tenure	-0.0280	0.030	-0.924	0.355	-0.087	0.031
Balance	0.1695	0.036	4.715	0.000	0.099	0.240
Products	-0.0402	0.030	-1.319	0.187	-0.100	0.020
CrCard	-0.0367	0.030	-1.220	0.222	-0.096	0.022
Active	-0.5498	0.032	-16.996	0.000	-0.613	-0.486
Salary	0.0161	0.030	0.530	0.596	-0.043	0.076
=====						

The estimated coefficients and their statistical significance provide some observations to help understand how the features influence the prediction of “exit” in the NN. Recall that if a customer had closed his/her bank account during the data period, then the classification of exit is “1”. Hence the predicted probability of Class “1” being higher means that it would be more likely that the customer closed his/her bank account. Predicted probability of Class “1” being lower means that the customer is likely to continue his/her bank account.

It is seen that (at 10% 2-tailed significance level or a p-value of < 0.10), higher credit score (CrScore), Gender being Male, lower Age, being an active customer with the bank are features that predicted a lower probability of exit. A larger bank balance, however, predicted a higher probability of exit.

In Chapter7-8.ipynb, model (3) with 8 neurons in the first and the second hidden layer, and an output layer, the Shap values are obtained as follows.



It is seen that instances/individuals that have feature values that are circled such as using a large number of the bank products (“No.Prods”), being active, being Male, and having a high credit score (“CrScore”) have negative Shap values for the features. These features directly contributed to lower predicted probability of exit. Instances/individuals that have higher ages show the age feature producing positive Shap values, i.e., higher age contributed to higher predicted probability of exit from the bank. The Shap value results are consistent with the coefficient estimates from the logistic regression. Moreover, for each instance/individual, we

can find how much does each feature contributed to the predicted probability of exit, whether increasing it or decreasing it.

7.5 Concluding Thoughts

One major application of ML is the use of NN that processes input information and produces output(s) of prediction and classification. The MLP or ANN is seen to be able to predict close to 87% accuracy in the bank customer exit data set when the NN is adequately structured with sufficiently deep number of hidden layers and adequate training iterations. When the training accuracies are high, it is more likely that the testing accuracies are also high. The trained NN can also help to provide understanding of how the various features affect the final output such as the predicted probability in the case of classification.

References

- Kingma D. P. and Jimmy Lei Ba (2015). ADAM: A method for stochastic optimization. Conference ICLR.
<https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/>.
<https://towardsdatascience.com/multilayer-perceptron-explained-with-a-real-life-example-and-python-code-sentiment-analysis-cb408ee93141>.

This page intentionally left blank

Chapter 8

Artificial Neural Network II

In this chapter, we discuss a major type of neural network especially for financial analyses with time series – the Recurrent Neural Network (RNN). This involves a more complicated architecture, and is sometimes referred to as deep learning NN to distinguish them from the feedforward NN (also called ANN) discussed in the previous chapter. We also discuss variants of the RNN such as the Long Short-Term Memory (LSTM) NN, the Bidirectional RNN (BRNN), the Gated Recurrent Unit (GRU), and the Convolutional Network (CNN).

In Chapter 7, we see that when cases k are independent of one another, i.e., their inputs and output do not affect other cases, then the common (across k) input weights and biases, and weights and biases in hidden layer(s) are revised via backward propagation to minimize the error/loss function $L(.)$ that combines all cases $\sum_{k=1}^N \frac{1}{N} L(Y_k, \hat{Z}_k)$ or else via mini-batches on smaller numbers of cases each iteration, or via stochastic gradient descent iteratively for each case. Y_k is the actual output in case k , and $\hat{Z}_k$ is the corresponding predicted output. For each forward and then backward propagation (one iteration) when weights and biases of the ANN are revised before the next forward propagation to recompute the new prediction, these same weights and biases are applied to every case k in order to arrive at the loss function $\sum_{k=1}^N \frac{1}{N} L(Y_k, \hat{Z}_k)$. These computations in a single iteration can be done in parallel for each case k since the ANN is the same for each k .

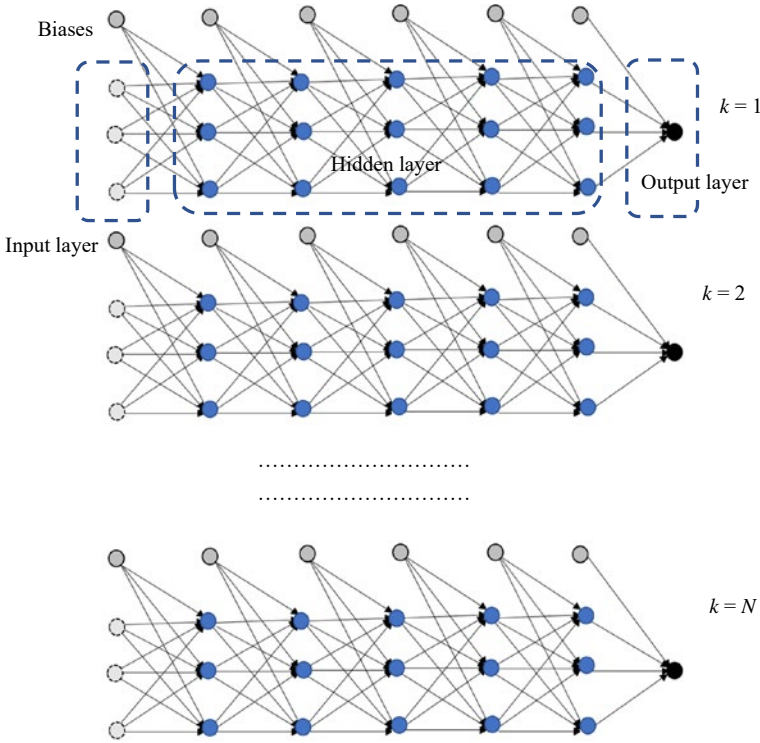


Figure 8.1. Multiprocessing MLP.

In Figure 8.1, we show the parallel multiprocessing using N identical MultiLayer Perceptrons (MLP), each to forward-propagate each case/subject or sample point k in the training data set of size N or N sample points.

For each k , the inputs $\{X_{i,k}\}$ are features ($i = 1, 2, 3, \dots$) related to subject k . The features/inputs are marked as dotted circles in the input layer. For examples, $\{X_{i,k}\}$ could be attributes/features i of customer k of a bank in the prediction of the outcome/output of exit or not ($Y_k = 1$ or 0); $\{X_{i,k}\}$ could be characteristics/features i of a house k in a particular region in the prediction of the house k price in the region (during a specified period).

For each k , there are $3 \times (3 + 1) = 12$ parameters with forward pass to the first hidden layer – each first hidden layer neuron takes a forward pass

of three input weights and one bias. Then there are $4 \times [3 \times (3 + 1)] = 48$ parameters in the next 4 hidden layers. Finally, there are $(3 + 1) = 4$ parameters in the output layer. Altogether there are $12 + 48 + 4 = 64$ different parameters in this ANN. Note that the parameters in edges to each hidden layer neuron are all different. The trained 64 parameters are, however, the same for each k since the training is across the total loss function $\sum_{k=1}^N \frac{1}{N} L(Y_k, \hat{Z}_k)$ or else in mini-batches or stochastic gradient descent across each k . The final iteration set of parameters as common to all k . In the parallel set of N MLPs shown in Figure 8.1, each iteration means all the N MLPs are updated simultaneously with the new set of revised weights and biases. This trained set of parameters is then applied to predict the test data cases j .

8.1 Recurrent Neural Network

Suppose now the input features for each case, instead of given altogether at the initial input layer, are given one at a time in a sequence. In a common setup to this new way of input, the number of hidden layers is now equal to the number of sequential input features so that at each additional hidden layer, there is a fresh sequential input feature – see dotted arrow in Figure 8.2. We consider the simple case where each sequential input feature is a scalar number. More general cases include an input feature which is a vector of numbers. This new architecture is common in time series prediction such as predicting the next day stock price (label or target) using lagged prices in the past five days. These lagged prices form the features of this label. Each of these lagged prices becomes one feature in this one training case. A different window of price with lagged five prices forms another case, and so on.

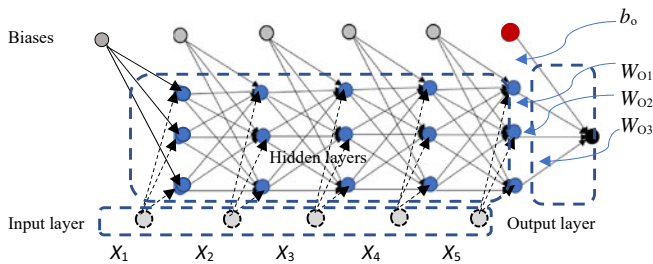


Figure 8.2. Recurrent neural network.

Figure 8.2 shows an architecture of three neurons per hidden layer whereby one training case is passed through the NN. We call this architecture RNN. The inputs for this case consist of 5 sequential features X_1 , X_2 , X_3 , X_4 , and X_5 . There is one output for the case in the output layer of one neuron. As in ANN or MLP, many cases in a batch or mini batch are required to pass through this RNN before the loss function, comprising sum of loss in each case, is to be reduced via revising the parameters in the model that include all the weights and biases. The following Figure 8.3 shows the labeling convention we adopt for the RNN.

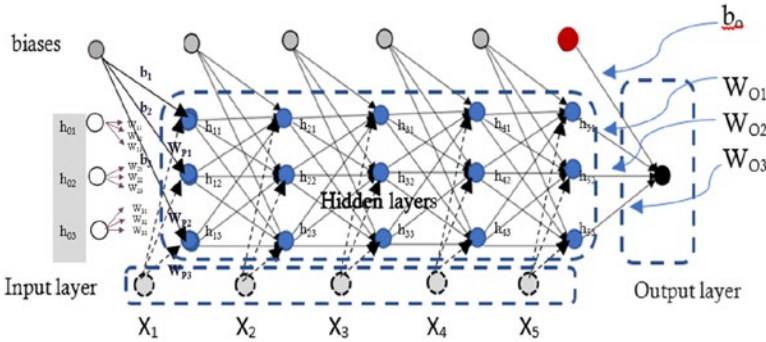


Figure 8.3. Notations for weights and biases.

In RNN, the weights and biases at each neuron level do not change from one hidden layer to another. This is unlike that in the ANN. Consider the first hidden layer in this RNN in Figure 8.3. Each of the three neurons in this layer receives five inputs and thus five associated weights/parameters in the edges/synapses. In the first neuron on the first (top) level, weight on the input feature X_1 is W_{p1} where “P” stands for “input”. Weights from the prior layer of three neurons are W_{11} , W_{21} , and W_{31} , respectively where subscripts pq in W_{pq} is such that p represents the level position of the previous output neuron and q represents the level position of the current neuron. Hence weights from prior layer of neurons to the second level neuron are W_{12} , W_{22} , and W_{32} , respectively. Weights from prior layer of neurons to the third level neuron are W_{13} , W_{23} , and W_{33} , respectively. Bias or weight on value 1 to the top-level neuron is b_1 , to the second level neuron is b_2 , and to the third level neuron is b_3 . So far there are three input weights (W_{p1} , W_{p2} , and W_{p3}), 3×3 weights W_{pq} from prior layer of neurons, and 3 biases. In addition, there are the weights W_{o1} , W_{o2} ,

W_{o3} and b_0 representing weights and bias from last hidden layer to the output neuron. Hence in total there are $3 + 3 \times 3 + 3 + (3 + 1)$ or $(3 + 2) \times 3 + (3 + 1)$ or $(3 + 2) \times (3 + 1) - 1 = 19$ parameters to be fitted in this RNN. In general, when the hidden layer has m neurons, then the total number of parameters to be fitted is $(m + 2)(m + 1) - 1$. This number is far less than that in ANN when each hidden layer has a different set of parameters indexed by superscript $[n]$ indicating n th hidden layer.

As another example, consider the second hidden layer in the NN in Figure 8.3. Each neuron in this layer receives five inputs and thus five associated weights/parameters in the edges/synapses. For example, the parameters to the first (uppermost) neuron in the layer are (1) W_{p1} , the weight on exogenous input X_2 from the input layer I, (2) the bias b_1 , (3) W_{11} , the weight on forward pass from previous hidden layer first neuron, (4) W_{21} , the weight on forward pass from previous hidden layer second neuron, and (5) W_{31} , the weight on forward pass from previous hidden layer third neuron.

The parameters to the second neuron in the layer are (1) W_{p2} , the weight on exogenous input X_2 from the input layer I, (2) the bias b_2 , (3) W_{12} , the weight on forward pass from previous hidden layer first neuron, (4) W_{22} , the weight on forward pass from previous hidden layer second neuron, and (5) W_{32} , the weight on forward pass from previous hidden layer third neuron.

The parameters to the third neuron in the layer are (1) W_{p3} , the weight on exogenous input X_2 from the input layer I, (2) the bias b_3 , (3) W_{13} , the weight on forward pass from previous hidden layer first neuron, (4) W_{23} , the weight on forward pass from previous hidden layer second neuron, and (5) W_{33} , the weight on forward pass from previous hidden layer third neuron. At the output layer, there are the bias parameter b_0 , and the three weight parameters W_{o1} , W_{o2} , and W_{o3} .

In Figure 8.3, the output from the first neuron in the first hidden layer is

$$h_{11} = \tan h(W_{p1} \times X_1 + b_1 + W_{11} h_{01} + W_{21} h_{02} + W_{31} h_{03}) \quad (8.1)$$

where h_{ij} is a hidden state (explicitly unobserved) at sequence step t and from neuron j counting from the top. The initialized values of h_{01} , h_{02} , h_{03} could be zeros or it could be set by a random function in Keras App based on random normal numbers. In subsequent iterative computations based on revised weights, these values (h_{01} , h_{02} , h_{03}) will take on values (h_{51} , h_{52} , h_{53}) of the last hidden layer neurons based on each case/subject.

The output as the hidden state h_{11} will be passed forward to the sequence at $t = 2$. Note that $\tanh$ is commonly used as the activation function in this type of architecture. Output from the second neuron in the first hidden layer is

$$h_{12} \equiv \tan h(W_{p2} \times X_1 + b_2 + W_{12} h_{01} + W_{22} h_{02} + W_{32} h_{03})$$

Output from the third neuron in the first hidden layer is

$$h_{13} \equiv \tan h(W_{p3} \times X_1 + b_3 + W_{13} h_{01} + W_{23} h_{02} + W_{33} h_{03})$$

The output from the first neuron in the second hidden layer is

$$h_{21} \equiv \tan h(W_{p1} \times X_2 + b_1 + W_{11} h_{11} + W_{21} h_{12} + W_{31} h_{13})$$

Output from the second neuron in the second hidden layer is

$$h_{22} \equiv \tan h(W_{p2} \times X_2 + b_2 + W_{12} h_{11} + W_{22} h_{12} + W_{32} h_{13})$$

Output from the third neuron in the second hidden layer is

$$h_{23} \equiv \tan h(W_{p3} \times X_2 + b_3 + W_{13} h_{11} + W_{23} h_{12} + W_{33} h_{13}), \text{ and so on.}$$

The hidden states (or output from last hidden layer neurons) are h_{51} , h_{52} , and h_{53} . Therefore, the output is $\tan h(W_{O1} \times h_{51} + W_{O2} h_{52} + W_{O3} h_{53} + b_o)$ where the subscript “O” denotes “output”.

It is noted that the hidden states (at each level of neuron) change with each step in the sequence. Each hidden state at sequence step t , h_{ij} , contains information of lagged exogenous input features X_{t-1} , X_{t-2} , X_{t-3} , and so on. This type of architecture is important if indeed lagged input features also have useful information in predicting the output. This type of neural network that allows for recurrence of influences of past features is thus called RNN.

The RNN of Figure 8.3 may be represented in a concise manner as follows in Figure 8.4. Each box represents a hidden layer of neurons.

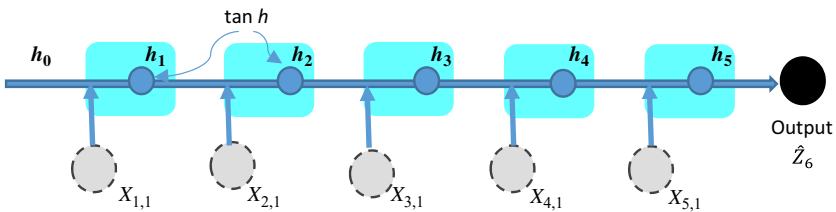


Figure 8.4. Concise representation of RNN.

The additional “1” in the input box reflects the “input” to multiply with the bias coefficient. It is sometimes also represented schematically as follows in Figure 8.5 by compacting hidden layers and sequential inputs. In this diagram, the recurrence of the hidden layers of inputs is made more obvious.

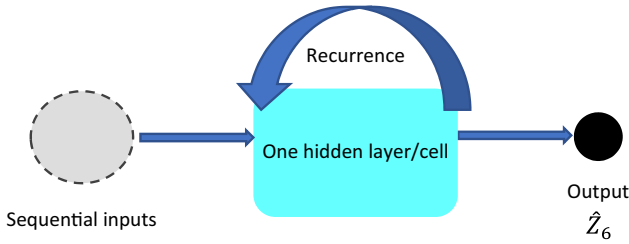


Figure 8.5 Illustration of recurrence.

Figures 8.3 and 8.4 are sometimes called the unfolded NN of the above diagram. In Figure 8.4, there is concatenation of the inputs (X_t , 1) and $h_{t-1} \equiv (h_{t-1,1}, h_{t-1,2}, h_{t-1,3})$ at their junction, and this is passed through the hidden layer (neurons inside layer and synapses or edges of weights and biases are not shown) where the dark circle represents the activation function on the dot product of the concatenated inputs and parameters as in Eq. (8.1). The output of that hidden layer is then h_t which enters the computations at the next hidden layer. Each of the hidden layer may be viewed as a recurrent cell since the computations are recurrent on the updated hidden states as the sequence progresses until the output.

In time series, the input features at each time point t , viz., X_{t-1} , X_{t-2} , X_{t-3} , and so on, may not be independent. For example, the vector X_t may be autocorrelated with X_{t-1} and so on, e.g., X_t is a Vector autoRegressive (VaR) stochastic process. This means that some, if not all, of each variable in the vector, e.g., X_{t-j} , may be autocorrelated. In general, X_t is a function of X_{t-1} , X_{t-2} , and so on, for every t . Thus, when the features are not independent across time, past history of X_t such as X_{t-1} , X_{t-2} , X_{t-3} , and so on, besides current input X_t , more likely also affect predicted output $\hat{Y}_{t+1}$. This can be modeled by adjusting the architecture of the NN such that the activation function at the hidden layer at t , $f_t(t)$ includes not just input X_t but also lagged X_{t-1} , X_{t-2} , X_{t-3} , and so on. Hence, the RNN is suitable for use in predicting time series sequences. It is also useful in other tasks such as speech recognition, optical music recognition with sequences of music scores, emotional facial recognition using snapshots of facial

expressions, language translation via text sequences, gesture recognition (useful in robotics), sentiment classification, and so on. Popular applications incorporating such software technologies are Siri, Voice Search, Google Translate, ChatGPT, and so on.

In the training of the RNN shown in Figures 8.3 and 8.4, we treat each of the five recurrent cells like five hidden layers and use backward propagation to compute the required partial derivatives of the loss function with respect to the $(m + 2)(m + 1) - 1$ number of parameters where m is the number of neurons in each hidden layer and there is only one output neuron. In general, a training data set could have a time series or a sequence of T data points. The T sample points are divided into cases or packs each with R , e.g., 5, number of sequential features or recurrent cells/hidden layers. The number of cases or packs is then T/R .

A certain number of cases/packs are grouped into mini batches with batch size S or S number of cases in each mini batch. So, there are $T/(RS)$ number of mini batches (each with size S). Each case in a mini batch yields one set of partial derivatives of the loss function with respect to the $(m + 2)(m + 1) - 1$ number of parameters. While the sequence within each case or pack clearly considers dependence and recurrence, the different cases or packs are treated as independent. Thus, S number of cases in each mini batch yield S sets of partial derivatives of the loss function. The aggregate loss function is the sum of losses of the S number of cases in each batch. Thus, over each mini batch, the forward propagation and then backward propagation in updating the parameters based on the sum of S partial derivatives is considered one iteration.

The training sample cases can be run in parallel for each iteration. Over E number of epochs and $T/(RS)$ number of mini batches, there will be $E \times T/(RS)$ number of iterations to update/revise the parameters. Just as in MLP, when $S = 1$, it is similar to the stochastic gradient descent method. The Adam (with changing learning rate) can also be applied to adjust the parameters to minimize the loss function via iterations.

Consider the loss function of one case, e.g., $L(Y_{t+R}, \hat{Y}_{t+R}) = (Y_{t+R} - \hat{Y}_{t+R})^2$ where $\hat{Y}_{t+R} \equiv \hat{Z}_{t+R}$, $R > 0$, and the sequence of inputs are $(X_t, X_{t+1}, X_{t+2}, \dots, X_{t+R-1})$. In the backward propagation, each of the parameters $W_{p1}, W_{p2}, \dots, W_{pm}, W_{11}, W_{21}, \dots, W_{m1}, W_{12}, W_{22}, \dots, W_{m2}, W_{13}, W_{23}, \dots, W_{m3}, \dots, W_{1m}, W_{2m}, \dots, W_{mm}, b_1, b_2, \dots, b_m$, occurs as the same constants in every hidden layer or time-sequence step, and parameters $W_{01}, W_{02}, W_{03}, \dots, W_{0m}, b_o$, occur at the output layer.

The partial derivative with respect to each of the output layer parameter is, where $m = 3$, $R = 5$, and $\hat{Y}_{t+6} = \tan h(W_{01} \times h_{51} + W_{02} h_{52} + W_{03} h_{53} + b_0)$, for $j = 1, 2$, or 3 :

$$\begin{aligned} & \frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial W_{oj}} \\ &= \frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial \hat{Y}_{t+6}} \times (1 - \hat{Y}_{t+6}^2) \\ & \quad \times \frac{\partial (W_{01} \times h_{51} + W_{02} \times h_{52} + W_{03} \times h_{53} + b_0)}{\partial W_{oj}} \\ &= \frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial \hat{Y}_{t+6}} \times (1 - \hat{Y}_{t+6}^2) \times h_{5j} \end{aligned}$$

Note that $d \tan h(z) / dz = 1 - (\tan h(z))^2$, so $\partial \tan h(W_{01} \times h_{51} + W_{02} h_{52} + W_{03} h_{53} + b_0) / \partial (W_{01} \times h_{51} + W_{02} h_{52} + W_{03} h_{53} + b_0) \approx 1 - [\tan h(W_{01} \times h_{51} + W_{02} h_{52} + W_{03} h_{53} + b_0)]^2$.

$$\begin{aligned} & \frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial b_o} \\ &= \frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial \hat{Y}_{t+6}} \times (1 - \hat{Y}_{t+6}^2) \\ & \quad \times \frac{\partial (W_{01} \times h_{51} + W_{02} \times h_{52} + W_{03} \times h_{53} + b_o)}{\partial b_o} \\ &= \frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial \hat{Y}_{t+6}} \times (1 - \hat{Y}_{t+6}^2) \end{aligned}$$

Now,

$$h_{51} \equiv \tan h(W_{p1} \times X_5 + b_1 + W_{11} h_{41} + W_{21} h_{42} + W_{31} h_{43})$$

$$h_{52} \equiv \tan h(W_{p2} \times X_5 + b_2 + W_{12} h_{41} + W_{22} h_{42} + W_{32} h_{43})$$

$$h_{53} \equiv \tan h(W_{p3} \times X_5 + b_3 + W_{13} h_{41} + W_{23} h_{42} + W_{33} h_{43})$$

$$h_{41} \equiv \tan h(W_{p1} \times X_4 + b_1 + W_{11} h_{31} + W_{21} h_{32} + W_{31} h_{33})$$

$$h_{42} \equiv \tan h(W_{p2} \times X_4 + b_2 + W_{12} h_{31} + W_{22} h_{32} + W_{32} h_{33})$$

$$h_{43} \equiv \tan h(W_{p3} \times X_4 + b_3 + W_{13} h_{31} + W_{23} h_{32} + W_{33} h_{33})$$

...

$$h_{11} \equiv \tan h(W_{p1} \times X_1 + b_1 + W_{11} h_{01} + W_{21} h_{02} + W_{31} h_{03})$$

$$h_{12} \equiv \tan h(W_{p2} \times X_1 + b_2 + W_{12} h_{01} + W_{22} h_{02} + W_{32} h_{03})$$

$$h_{13} \equiv \tan h(W_{p3} \times X_1 + b_3 + W_{13} h_{01} + W_{23} h_{02} + W_{33} h_{03})$$

$$h_{01} = h_{02} = h_{03} = 0 \text{ or some small random numbers in } (-1, +1)$$

The partial derivative with respect to each of the hidden layer parameters for each case is

$$\begin{aligned} \frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial W_{p1}} &= \frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial \hat{Y}_{t+6}} \times \frac{\partial \hat{Y}_{t+6}}{\partial W_{p1}} = \frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial \hat{Y}_{t+6}} \times (1 - \hat{Y}_{t+6}^2) \\ &\quad \times \frac{\partial (W_{01} \times h_{51} + W_{02} \times h_{52} + W_{03} \times h_{53} + b_0)}{\partial W_{p1}} \end{aligned} \quad (8.2)$$

Where

$$\begin{aligned} \frac{\partial h_{51}}{\partial W_{p1}} &= (1 - h_{51}^2) \frac{\partial (W_{p1} X_5 + W_{11} h_{41} + W_{21} h_{42} + W_{31} h_{43} + b_1)}{\partial W_{p1}} \\ &= (1 - h_{51}^2) \left(X_5 + W_{11} \frac{\partial h_{41}}{\partial W_{p1}} + W_{21} \frac{\partial h_{42}}{\partial W_{p1}} + W_{31} \frac{\partial h_{43}}{\partial W_{p1}} \right) \\ \frac{\partial h_{52}}{\partial W_{p1}} &= (1 - h_{52}^2) \frac{\partial (W_{p2} X_5 + W_{12} h_{41} + W_{22} h_{42} + W_{32} h_{43} + b_2)}{\partial W_{p1}} \\ &= (1 - h_{52}^2) \left(0 + W_{12} \frac{\partial h_{41}}{\partial W_{p1}} + W_{22} \frac{\partial h_{42}}{\partial W_{p1}} + W_{32} \frac{\partial h_{43}}{\partial W_{p1}} \right) \\ \frac{\partial h_{53}}{\partial W_{p1}} &= (1 - h_{53}^2) \frac{\partial (W_{p3} X_5 + W_{13} h_{41} + W_{23} h_{42} + W_{33} h_{43} + b_3)}{\partial W_{p1}} \\ &= (1 - h_{53}^2) \left(0 + W_{13} \frac{\partial h_{41}}{\partial W_{p1}} + W_{23} \frac{\partial h_{42}}{\partial W_{p1}} + W_{33} \frac{\partial h_{43}}{\partial W_{p1}} \right) \end{aligned}$$

$$\begin{aligned}\frac{\partial h_{41}}{\partial W_{p1}} &= (1 - h_{41}^2) \frac{\partial(W_{p1} X_4 + W_{11} h_{31} + W_{21} h_{32} + W_{31} h_{33} + b_1)}{\partial W_{p1}} \\ &= (1 - h_{41}^2) \left(X_4 + W_{11} \frac{\partial h_{31}}{\partial W_{p1}} + W_{21} \frac{\partial h_{32}}{\partial W_{p1}} + W_{31} \frac{\partial h_{33}}{\partial W_{p1}} \right)\end{aligned}$$

and so on. Hence, Eq. (8.2) $\frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial W_{p1}}$ clearly involves summation of partial derivative terms across each time-step or each sequential recurrent unit or hidden layer, e.g., $\frac{\partial h_{5j}}{\partial W_{p1}}, \frac{\partial h_{4j}}{\partial W_{p1}}, \frac{\partial h_{3j}}{\partial W_{p1}}, \frac{\partial h_{2j}}{\partial W_{p1}}, \frac{\partial h_{1j}}{\partial W_{p1}}$ for every j , involving terms of inputs $X_1, \dots, X_5$ and previous parameter values $W_{ij}^{[t]}$, and so on.

This is unlike the feedforward NN case where a partial derivative would involve summation only across paths originating from the parameter edge and not across all edges in each layer. Firstly, input features a long time in the past such as X_1 can have almost zero contributions to the gradient $\frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial W_{p1}}$ when the W_{ij} 's are small, i.e., product of small W_{ij} 's

vanishes. Secondly, for a many-layered deep RNN, the gradients $\frac{\partial L(Y_{t+6}, \hat{Y}_{t+6})}{\partial W_{pj}}$ lose components involving products of small W_{ij} 's that vanish and hence may not update well. This is the “vanishing gradient” problem in this version of RNN. The flip side of the problem is when W_{ij} 's are large, i.e., >1 . In this case, there is “exploding gradient” problem, and updates to weights are again problematic.

The above partial derivatives are for one case. They are summed across all cases in one mini batch size S . If loss across all cases in the mini batch of size S is $L = 1/S \sum^S L(Y_{t+R}, \hat{Y}_{t+R})$, then $\partial L / \partial W_{ij}^{[t]} = 1/S \sum^S L(Y_{t+R}, \hat{Y}_{t+R}) / \partial W_{ij}^{[t]}$ at the t th iteration. After the updated partial derivatives are found, the usual adjustments of the parameter weights are done, viz.

$$W_{ij}^{[t+1]} = W_{ij}^{[t]} - \alpha(t+1) \frac{\partial L}{\partial W_{ij}^{[t]}}$$

While the advantage of RNN over MLP is the ability to recognize the input of lagged (historical) information in affecting current output, and the ability to take in long sequence of inputs (since the number of parameters is not blown up as the parameters are constant at each recurrent cell), the computational time can be longer as the backward propagation involves more computations of the complicated partial derivatives. RNN also has

two drawbacks: (1) the partial derivatives in the RNN backward propagation may in some situations progressively “collapse to zero” or else they may explode exponentially (“vanishing” or else “exploding” gradient with deeper layers); (2) the design does not allow it to consider future or forward input for training since the time-sequence uses information or input from the past or back till the current and is meant to predict the future or forward data. This limitation is real for problems such as predicting a picture by neighboring pixels or a word sequence as in a sentence. However, in general, limitation (2) is appropriate for financial time series where known future state cannot be used to predict an earlier state as this may introduce serious issues of spuriously high accuracies.

Problem (1) may be remedied by pruning the model such as reducing the number of recurrent cells to make the model less complex. It may also be remedied by a variant RNN called the Long Short-term Memory NN (LSTM) or use additional gates to control/reduce loss of gradient. Problem (2) may be remedied by using features at current time but are other market-based forecasts of the future state, e.g., a forward contract price on a future spot price on the same underlying commodity.

Besides the example in Figure 8.2 of many inputs to one output (many-to-one structure), there are also other structures such as one-to-many, e.g., for music generation or predicting a sequence of notes from one input, where at each step the activation becomes an explicit output that can be trained with actual output, and many-to-many as in text translation. We can also use the structure in Figure 8.2 for many-to-many if there is training on an output at the end of each hidden layer in a case.

8.2 Worked Example I: Data

The CBOE Volatility Index (VIX) is a real-time index derived from the prices of SPX (S&P 500) index options with near-term (approximately one month) expiration dates. It is market expectation of the SPX risk-neutral volatility over the short term. VIX has been called a “fear gauge” and is often seen as a measure of negative market sentiment. Data is downloaded from Yahoo Finance.

The following exercise uses daily VIX data from 2 Jan. 1990 to 26 Oct. 2022 (8270 time-sequenced sample points) to perform a RNN prediction of next day VIX based on the observed VIX of the past 10 days (approximately two trading weeks), the inputs. See demonstration file Chapter8-1.ipynb.

```
[1]: ### this notebook is about RNN on predicting daily S&P500 VIX using non-overlapping
### X-train data, epochs = 30
import pandas as pd
import numpy as np
from keras.models import Sequential
from keras.layers import Dense, SimpleRNN
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
import math
import matplotlib.pyplot as plt
```

```
[2]: Vx = pd.read_csv("VIX.csv")
print(Vx)
```

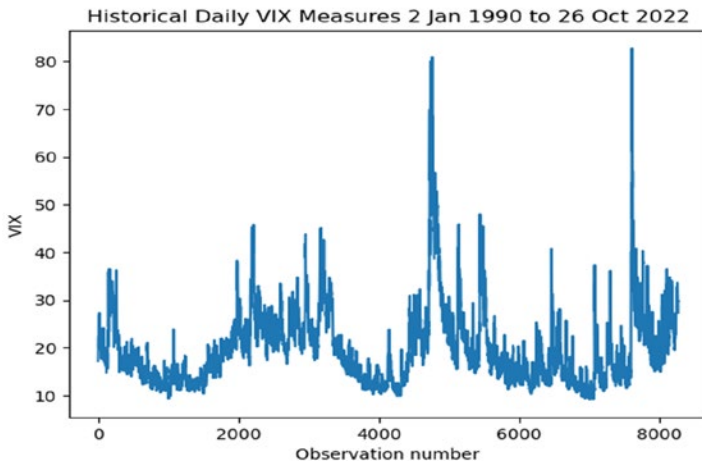
	Date	Open	High	Low	Close	Adj Close	Volume
0	2/1/1990	17.240000	17.240000	17.240000	17.240000	17.240000	0
1	3/1/1990	18.190001	18.190001	18.190001	18.190001	18.190001	0
2	4/1/1990	19.219999	19.219999	19.219999	19.219999	19.219999	0
3	5/1/1990	20.110001	20.110001	20.110001	20.110001	20.110001	0
4	8/1/1990	20.260000	20.260000	20.260000	20.260000	20.260000	0
...	...	...	...	...	...	...	...
8265	20/10/2022	31.299999	31.320000	29.760000	29.980000	29.980000	0
8266	21/10/2022	30.209999	30.440001	29.240000	29.690001	29.690001	0
8267	24/10/2022	30.650000	30.950001	29.780001	29.850000	29.850000	0
8268	25/10/2022	29.799999	30.000000	28.219999	28.459999	28.459999	0
8269	26/10/2022	28.440001	28.520000	27.270000	27.280001	27.280001	0

[8270 rows x 7 columns]

The adjusted close VIX prices or “fear gauge” are chosen. This should be similar to the VIX close (in the case of stock prices, they would be different).

```
[4]: ### view time series graph of data -- check if it is stationary
### perhaps by some unit root test
VIX.plot()
plt.xlabel('Observation number')
plt.ylabel('VIX')
plt.title('Historical Daily VIX Measures 2 Jan 1990 to 26 Oct 2022')
```

[4]: Text(0.5, 1.0, 'Historical Daily VIX Measures 2 Jan 1990 to 26 Oct 2022')



The total data set is split into 80% training data (6616 observations) and 20% test data (1654 observations) – see code line [5] and [6]. Note that the split is done with the training data set occurring prior to the test data set. It is not a random selection. The financial time series has the property that under reasonable market efficiency, past information and not future information should be part of an investor's consideration for future forecast. Future observations could not have been perfectly known beforehand.

```
[5]: ### we cannot use 'train_data = VIX.sample(frac=0.8, random_state=1); \
      ### test_data = VIX.drop(train_data.index)' as this is time series data
      ## that is sequenced and cannot be randomly chopped up
      split_percent=0.8
      n = len(VIX) ### len() is a numpy array function
      split = int(n*split_percent) ### point for splitting data into train and test
      train_data = VIX[range(split)]
      test_data = VIX[split:]

[6]: train_data.shape, test_data.shape ### 0.8 x 8270 = 6616

[6]: ((6616,), (1654,))
```

Snapshot of the data is shown as follows.

```
In [8]: train_data.values.reshape(-1, 1)

Out[8]: array([[17.24 ],
               [18.190001],
               [19.219999],
               ...,
               [13.95 ],
               [13.1 ],
               [14.12 ]])
```

Next, the data is scaled to between 0 and 1 using the MinMaxScaler in sklearn. Note that scaling is done separately for the training data and test data as they are separated in time, and there may be drifts over time. It is preferable not to mix them up in using just one scaling.

```
[9]: #Performing Feature Scaling
      #from sklearn.preprocessing import StandardScaler
      from sklearn.preprocessing import MinMaxScaler
      scaler = MinMaxScaler(feature_range=(0, 1))
      ## putting all positive is suitable as vola are all pos nos.
      train_data = scaler.fit_transform(train_data.values.reshape(-1, 1))
      ## (-1,1) reshapes it to a 2D array; without .values - it may not work;
      ## .values .values convert to np array
      test_data = scaler.fit_transform(test_data.values.reshape(-1, 1))
      ## (-1,1) reshapes it to a 2D array
```

In code line [13], the training data and the test data from total sample size T are arranged for the training and then for prediction as shown in Figure 8.6.

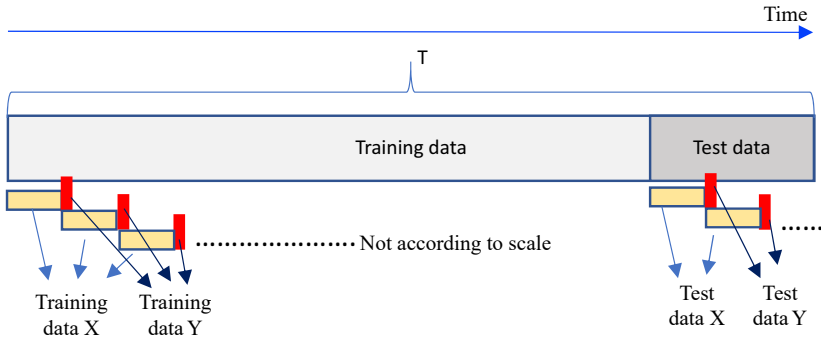


Figure 8.6. Structure of training and test data sets.

```
[13]: ### Preparing the input X and target Y
def get_XY(dat, time_steps):
    ### Indices of target array
    C_ind = np.arange(time_steps, len(dat), time_steps)
    ### example np.arange(start=1, stop=10, step=3) gives array([1, 4, 7]),
    ### ends up to/before stop
    C = dat[C_ind]
    ### example: ray=np.arange(2, stop=10, step=3); print(ray) --- gives [2 5 8]
    ### c=np.array([1,3,6,8,9,10,12,15,18,20]); c[ray] --- gives array([ 6, 10, 18])
    ### with elements from the 2nd, 5th, 8th positions of c. c's 1st position starts at '0'
    ### Prepare X
    rows_x = len(C) ### here len(C) is 661
    X = dat[range(time_steps*rows_x)]
    ### range(L) is 0,1,2,..., L-1. L is 10 x 661 = 6610. X is array 1,2,...,6610
    X = np.reshape(X, (rows_x, time_steps, 1))
    ### X reshaped as (661,10,1)
    return X, C

time_steps = 10
### hence C_ind = array ([10, 20, 30, 40, ...,661]), 661 number of 10 steps
### C = array(10th position, 20th position of dat, etc.)
### -- approx two weeks (10 trading days) interval for one prediction point of VIX
trainX, trainY = get_XY(train_data, time_steps)
testX, testY = get_XY(test_data, time_steps)
```

In this case we use a sequence of 10 steps (approximately two weeks of 10 trading days) to form one pack or case to feed into 10 recurrent cells/hidden layers. The 10 daily VIX prices of training data X (inputs) are then followed by the following 11th day of VIX price as the training data for output Y .

In code line [18], the SimpleRNN app in Keras Sequential is used to define the structure including the loss function of mean squared error for prediction and Adam for optimizing algorithm.

```
[18]: def create_RNN(hidden_units, dense_units, input_shape, activation):
      model = Sequential()
      model.add(SimpleRNN(hidden_units, input_shape=input_shape, activation=activation[0]))
      ### See SimpleRNN apps in https://www.tensorflow.org/api_docs/python/tf/keras/Layers/SimpleRNN
      model.add(Dense(units=dense_units, activation=activation[1]))
      ### Using model = Sequential() defines no. of inputs, neurons in hidden & in output layer
      model.compile(loss='mean_squared_error', optimizer='adam')
      ### .compile in Sequential carries loss and optimizer options
      return model
```

```
[19]: # Create model and train
      modell = create_RNN(hidden_units=8, dense_units=1, input_shape=\
                          (time_steps,1), activation=['tanh', 'tanh'])
      ### calls function create_RNN, fills in the arguments that were sub-defined
      ### in last codeline via .add that defines operations
      ### at the input layer and at the hidden layer and at dense/output layer
      ### hidden_units = 8 means that there are 8 neurons in each hidden layer
      ### input_shapes = (time-steps,1) with time_steps=10 means that each time-step
      ### in a pack of 10 is an input (the output for that time-step is ignored)
      ### - only the end of pack time variable is used in trainY
```

Code line [19] shows the specifications of the structure in [18], e.g., $m = 8$ neurons in each recurrent hidden cell/layer, output neuron (called “dense_units” here) = 1, and activation function for hidden layer and for output layer that are both the tanh function. These arguments are defined in the Keras SimpleRNN App.

Modell.fit is then called (via the Sequential app) to execute the training fit (minimizing loss based on number of iterations specified in number of epochs (30) and batch size in one batch, 661 where $661 = (T/10)$, to find the optimal $(m + 2)(m + 1) - 1 = 10 \times 9 - 1 = 89$ parameters.

```
modell.fit(trainX, trainY, epochs=30, batch_size=1, verbose=2)
### time series of trainX is reshaped as (661,10,1), trainY is (661,)
### time series of testX is (165, 10, 1), testY is (165,)
### The 6616 sequence points in trainX are separated into 661 "separate"
### non-overlapping packs of 10 sequential points each.
### In the training, trainY is matched against predicted using trainX
### of each of the 661 "separate" packs
```

This is followed by making predictions on trainY based on trainX (using the optimized .fit parameters), and then making predictions on testY based on testX (using the optimized .fit parameters).

```

### make predictions
train_predict = modell.predict(trainX) ### Using the fitted model with trainX, trainY
test_predict = modell.predict(testX)  ### Using the same fitted model with trainX, trainY

### above, test_predict is run after train_predict error is minimized or loss
### is minimized after the training set trainX, trainY are used in .fit

### Dense unit implements the operation: output = activation(dot(input, kernel) + bias)
### where activation is the element-wise activation function passed as the activation
### Dense occurs at hidden and output layers
### By setting verbose 0, 1 or 2 you just say how do you want to 'see' the training
### progress for each epoch. verbose=0 will show you nothing (silent)/ verbose=1 will show
### an animated progress bar like this: progress_bar. verbose=2 will be most detailed

```

After computing over 661 cases (each is a pack of 10 days) each epoch, and then repeatedly over 30 epochs, the optimized .fit parameters produce the following root-mean-square-errors (RMSE) in the prediction of training data Y (trainY) and test data Y (testY) respectively:

```

[21]: def print_error(trainY, testY, train_predict, test_predict):
      ### Error of predictions
      train_rmse = math.sqrt(mean_squared_error(trainY, train_predict))
      test_rmse = math.sqrt(mean_squared_error(testY, test_predict))
      ### Print RMSE
      print('Train RMSE: %.3f RMSE' % (train_rmse))
      print('Test RMSE: %.3f RMSE' % (test_rmse))

      print_error(trainY, testY, train_predict, test_predict)

      Train RMSE: 0.022 RMSE
      Test RMSE: 0.025 RMSE

```

The RMSEs are about 2.2% for the training and 2.5% for the testing. This level of accuracy is not significantly improved in the RNN for this study even if we perform overlapping data using all 6606 number of cases for training and increasing the number of epochs to 100. The plot of the rescaled (inverse of scaling in [9]) actual versus predicted outputs in both the training set and the test set are shown below.

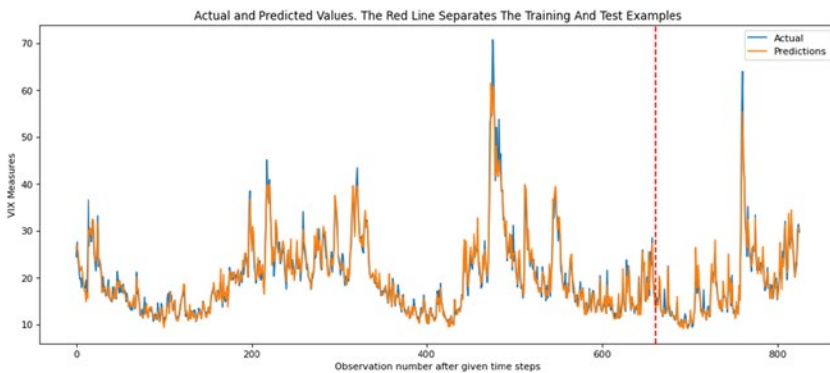
```

[24]: trainY1 = scaler.inverse_transform(trainY)
      testY1 = scaler.inverse_transform(testY)
      train_predict1 = scaler.inverse_transform(train_predict)
      test_predict1 = scaler.inverse_transform(test_predict)

```



```
[25]: ### See also https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.axvline.html
def plot_result(trainY1, testY1, train_predict1, test_predict1):
    actual = np.append(trainY1, testY1)
    predictions = np.append(train_predict1, test_predict1)
    rows = len(actual)
    plt.figure(figsize=(15, 6), dpi=80)
    plt.plot(range(rows), actual)
    plt.plot(range(rows), predictions)
    plt.axvline(x=len(trainY1), color='r', linestyle='--')
    ### note: vertical red line separates training part from test part
    plt.legend(['Actual', 'Predictions'])
    plt.xlabel('Observation number after given time steps')
    plt.ylabel('VIX Measures')
    plt.title('Actual and Predicted Values. The Red Line Separates The Training And Test Examples')
    ### Plot result -- this puts together Actual = (trainY, testY) or 187+46 points,
    ### and Predicted = (train_predict, test_predict)
    plot_result(trainY1, testY1, train_predict1, test_predict1)
```



8.3 Variants of RNN

As mentioned, the traditional RNN may run into vanishing gradient problem (when the partial derivatives in the RNN backward propagation collapse toward zero) – when this happens, the effect of earlier inputs in the sequence becomes negligible, so the network does not learn from the long past – it has only short-term memory with more recent inputs. To mitigate this problem, especially when long term memory is important, the LSTM NN, a variant of RNN, can be used.

In the LSTM architecture, the recurrent cell or layer in Figure 8.4 is redesigned as follows in Figure 8.7. There is an additional state (also not explicitly observed) called the “cell state”. Like the hidden states h_t in a traditional RNN as in Figure 8.4 that stores “short-term” memory of effects

of past inputs, the cell state c_t stores “long-term” memory of effects of past inputs. As in Figure 8.4, each “box” with each fresh input represents one hidden layer in the RNN. Each “box” can contain multiple neurons as width.

Four gates are built into the cell to regulate or control flow strength of the data. The concatenated inputs $(X_t, 1)$ and the previous hidden states (outputs of last hidden layer) h_{t-1} are passed through (1) the Forget Gate before they flow through to update the last cell state c_{t-1} , (2) the Input Gate before they flow through to make a Hadamard product with output from the (3) Cell Update Gate, and then to make the final update on the cell state that is preliminarily updated by (1), (4) the Output Gate before they update the hidden state in conjunction with the cell state c_t .

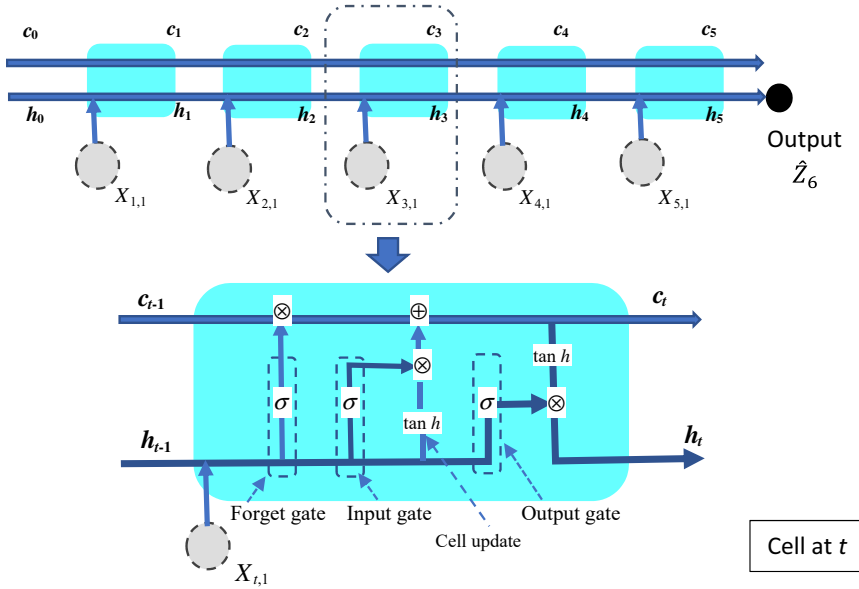


Figure 8.7. LSTM NN architecture.

Suppose we create 50 neurons in the hidden layer or the cell. In the Forget Gate (1), concatenated inputs $(X_t, 1)$ after weighting is transformed by activation function (typically the sigmoid function, represented by the symbol σ in the diagram) on the weighting, viz.

$$\sigma(W_{FX}X_t + W_{Fh}h_{t-1} + b_F)$$

where weights W_{FX} and W_{Fh} have appropriate dimensions to match the dimensions in X_t and h_{t-1} . For example, at each time of input, X_t could be 1×1 and h_{t-1} could be 50×1 (typically the dimension of the hidden states is the same as the number of neurons). Then there would be 50×1 W_{FX} and 50×50 W_{Fh} and 50×1 b_F so that $\sigma(W_{FX}X_t + W_{Fh}h_{t-1} + b_F)$ is a 50×1 output from the Forget Gate, each element being a sigmoid function of the respective linear weightings. There will typically be a similar dimension as the hidden states in the cell states, i.e., 50×1 vector in c_{t-1} . The output from the Forget Gate is a long-term retention that lies between 0 and 1 due to the nature of the sigmoid function. A value close to zero means that most of the past memory will be discarded. This Forget Gate output fraction vector $\sigma(W_{FX}X_t + W_{Fh}h_{t-1} + b_F)$ is then multiplied by c_{t-1} (symbol $\otimes$ in Figure 8.7) using the Hadamard product or element to element multiplication. This provides the preliminarily updated cell state.

In Input Gate (2), concatenated inputs ($X_t, 1, h_{t-1}$) after weighting $\sigma(W_{IX}X_t + W_{Ih}h_{t-1} + b_I)$ is used in Hadamard product with cell update candidate $\tanh(W_{CX}X_t + W_{Ch}h_{t-1} + b_C)$ to feed as addition to update the preliminarily updated cell state c_{t-1} from (1) to c_t . This Input Gate identifies only important impacts from the concatenated inputs to be added to the long-term memory cell c_t where

$$c_t = \sigma(W_{FX}X_t + W_{Fh}h_{t-1} + b_F) \otimes c_{t-1} + \sigma(W_{IX}X_t + W_{Ih}h_{t-1} + b_I) \otimes \tanh(W_{CX}X_t + W_{Ch}h_{t-1} + b_C)$$

In Output Gate (3), the $\tanh$ activation on the final updated cell state c_t is then multiplied (Hadamard product $\otimes$) by $\sigma(W_{OX}X_t + W_{Oh}h_{t-1} + b_O)$ to yield the next hidden state h_t that flows into the next LSTM layer, viz.

$$h_t = \sigma(W_{OX}X_t + W_{Oh}h_{t-1} + b_O) \otimes \tanh(c_t)$$

In a many-to-one structure as in Figure 8.7, the final output could be a scalar predicted number provided by the output layer comprising 51 weights (dimension of output h_t and the bias), viz. $f(W_{Yh}h_t + b_Y)$ where W_{Yh} and b_Y denote final output weights (“Y” being the output state) from a RNN/LSTM and $f(\cdot)$ is the final activation function.

If in this LSTM NN with one hidden layer in each input point, there are 50 neurons (Keras App names it as number of units in the hidden layer/cell), then there is a total of $4 \times [(50 + 1) \times 50 + 50]$ parameters in the first hidden layer. In general, for N neurons or units in the first hidden layer of LSTM, and J number of inputs/features, there are

$4 \times [(N + J) \times N + N]$ parameters to be optimized, where the $(N + J)$ denotes the number of previous hidden state and feature inputs. This is multiplied into N , the number of neurons in the hidden layer. As said, the number of hidden states is typically structured to be the same as the number of neurons in the layer. The remaining N in the $[\]$ denotes the N biases, one for each neuron in the hidden layer. Thus, for the 50 neurons (units) in one layer of LSTM, there are $4 \times [(50 + 1) \times 50 + 50] = 10,400$ parameters. All adjacent layers of 50 neurons in same LSTM share the same $4 \times [(50 + 1) \times 50 + 50] = 10,400$ parameters.

However, single LSTM (even with a large number of neurons per layer or width, or a large number of adjacent layers) and many epochs may not be able to produce good training results. A deeper version is the stacked LSTM model with more hidden layers (that are stacked up at an input time-sequence).

The LSTM with two stacked hidden layers at each time-sequence is shown below in Figure 8.8. Superscripts to hidden states h_t and cell states c_t denote the first hidden layer 1 at t and the second (stacked) layer at t . The key idea is that the hidden state output $h_t^{[1]}$ at t in level 1 passes up to the second level layer as input together with the hidden state input $h_{t-1}^{[2]}$ at level 2. Note $h_{t-1}^{[2]}$ (level 2 hidden state at $t-1$) needs not be the

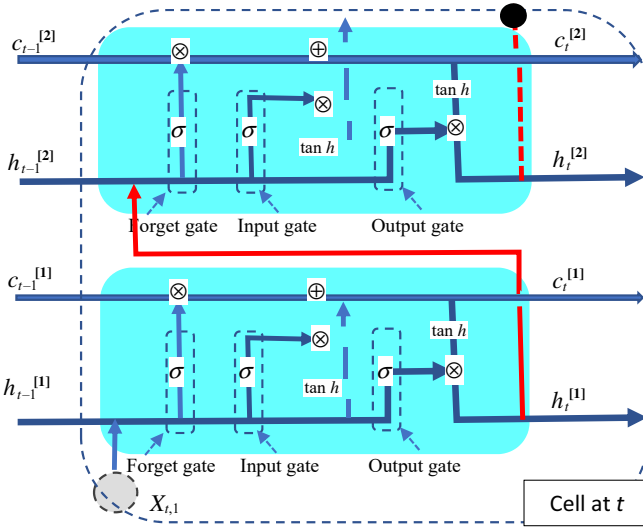


Figure 8.8. Stacked LSTM architecture.

same as $h_{t-1}^{[1]}$, the level 1 hidden state at $t-1$. The last output layer, if any, at t uses the level 2 hidden state $h_t^{[2]}$, i.e., $f(W_{y_h} h_t^{[2]} + b_y)$ where $h_t^{[2]} = \sigma(W_{Ox}^{[2]} h_t^{[1]} + W_{Oh}^{[2]} h_{t-1}^{[2]} + b_O^{[2]}) \otimes \tan h(c_t^{[2]})$.

Suppose we have similarly 50 neurons in each LSTM cell – 50 neurons in the first level cell and 50 neurons in the second level cell of the same cell at t , as in Figure 8.8.

The Output Gate from first level cell provides 50 hidden states at level 1, viz.

$$h_t^{[1]} = \sigma(W_{Ox}^{[1]} X_t + W_{Oh}^{[1]} h_{t-1}^{[1]} + b_O^{[1]}) \otimes \tan h(c_t^{[1]})$$

where the superscripts to the parameters denote association with the cell level. Together with $h_{t-1}^{[2]}$, these are inputs to the Forget Gate, Input Gate, Candidate for Cell State Update, and the Output Gate at level 2. Since $N = 50$, and dimension of $h_{t-1}^{[2]}$ (same dimension as $h_{t-1}^{[1]}$) and of $h_t^{[1]}$ are both N , then $(N + N)$ inputs are connected to N , the number of neurons in the stacked second hidden layer.

At the same time, there are N biases added, so there are $(N^2 + N^2 + N)$ number of parameters at the Forget Gate. The Forget Gate output fraction vector $\sigma(W_{FX}^{[2]} h_t^{[1]} + W_{Fh}^{[2]} h_{t-1}^{[2]} + b_F^{[2]})$ is then multiplied by $c_{t-1}^{[2]}$ using the Hadamard product or element to element multiplication, where $W_{FX}^{[2]}$ is 50×50 , $W_{Fh}^{[2]}$ is 50×50 , and $b_F^{[2]}$ is 50×1 . This provides the preliminarily updated cell state at level 2 at t .

The same operations as in level 1 occur at the Input Gate, the candidate for cell update, and the Output Gate. Hence in total there are $4 \times [(N + N) \times N + N]$ parameters at the second level LSTM cell/stacked layer. If $N = 50$, then there is a total of $4 \times [(50 + 50) \times 50 + 50] = 20,200$ parameters at the second level LSTM cell.

Other descriptions of the RNN and LSTM NN can be found in <https://stanford.edu/%7Eshervine/teaching/cs-230/cheatsheet-recurrent-neural-networks> and in https://d2l.ai/chapter_recurrent-modern/lstm.html.

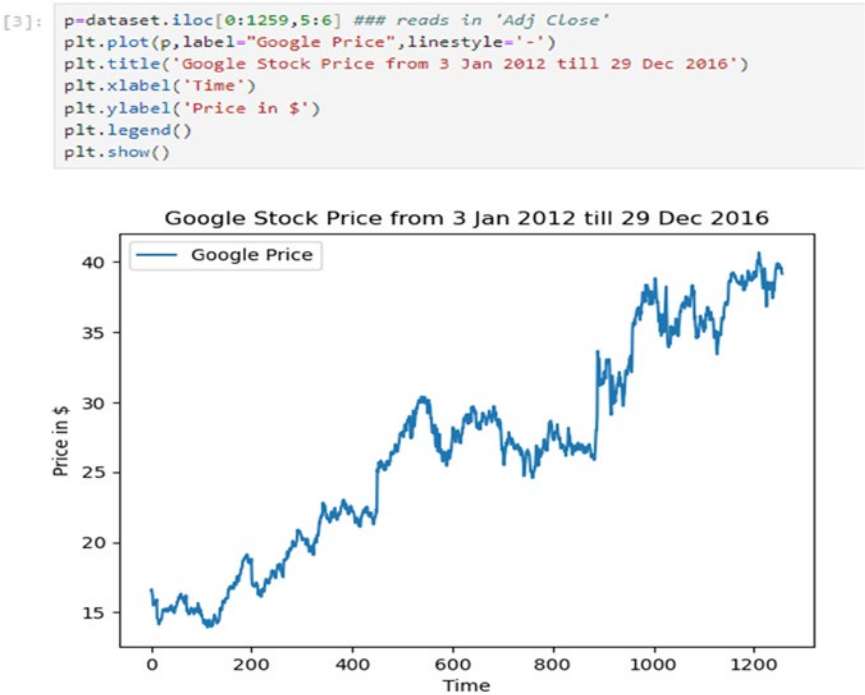
8.4 Worked Example II: Data

In this second worked example, we show how a LSTM RNN can be used to predict Google stock prices. Historical data are collected from Yahoo Finance consisting of daily (distribution and split) adjusted closing Google stock prices from 3 Jan. 2012 till 29 Dec. 2016, with a total of 1257 sample points. See demonstration file Chapter8-2.ipynb.

```
[2]: ### import training set
dataset=pd.read_csv('GOOG.csv')
dataset.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	3/1/2012	16.262545	16.641375	16.248346	16.573130	16.573130	147611217
1	4/1/2012	16.563665	16.693678	16.453827	16.644611	16.644611	114989399
2	5/1/2012	16.491436	16.537264	16.344486	16.413727	16.413727	131808205
3	6/1/2012	16.417213	16.438385	16.184088	16.189817	16.189817	108119746
4	9/1/2012	16.102144	16.114599	15.472754	15.503389	15.503389	233776981

The graph of the price data series is shown as follows.



These are split into the front-end 1100 sample points as training set, and the back-end 157 sample points as test set. These are then scaled using Sklearn App MinMaxScaler to scale them to within (0,1), i.e., X is transformed to $(X - \min)/(\max - \min)$, where min, max refer to the minimum

and maximum of the training set feature associated with X . The test set data are scaled using the same MinMaxScaler.

```
[4]: training_set=dataset.iloc[0:1100,5:6].values
      trgsset=pd.DataFrame(training_set)

      test_set=dataset.iloc[1100:1259,5:6].values
      tstset=pd.DataFrame(test_set)

      print(dataset.shape, trgsset.shape, tstset.shape)
      print(trgsset.head(),trgsset.tail(), tstset.head(),tstset.tail())

      (1257, 7) (1100, 1) (157, 1)
```

```
[5]: ### Feature Scaling
      from sklearn.preprocessing import MinMaxScaler
      sc=MinMaxScaler(feature_range=(0,1))
      training_set_scaled=sc.fit_transform(training_set)
      test_set_scaled=sc.transform(test_set)
      type(training_set_scaled), type(test_set_scaled)

[5]: (numpy.ndarray, numpy.ndarray)
```

Next, the data structure is created to feed the inputs to the NN. As in Figure 8.6, $T = 1100$ for the training data set. The first 60 data points are used as a case in training data X (X_{train}) while the next data point (point 61) is used as the corresponding label Y or Y_{train} . The concatenated X_{train} is reshaped into 1040 rows (first dimension size) each with 60 columns (second dimension size) of timed inputs each.

```
[7]: ### creating data structure with 1040 cases, each with 60 time-steps and 1 output
      X_train=[]
      y_train=[]
      for i in range(60,1100):
          X_train.append(training_set_scaled[i-60:i, 0])
          y_train.append(training_set_scaled[i, 0])
      X_train, y_train = np.array(X_train), np.array(y_train)
      X_train=np.reshape(X_train, (X_train.shape[0], X_train.shape[1],1))
      ### This last step converts X_train to 3D from (1040,60) to (1040,60,1)
      ### for input to the keras app
      print(X_train.shape, y_train.shape)

      (1040, 60, 1) (1040,)
```

As shown below, stacked LSTM NN is used with four stacked LSTM cells at each timed input for each case of 60 inputs followed by prediction of

the output at the end of 60 time-sequenced inputs. There are 1040 cases as we use overlapping cases here. Each stacked LSTM cell contains 50 neurons. The output layer specified one neuron, so there is only one predicted scalar output. Notice that activation functions throughout are built into the Keras LSTM App as discussed in the last section. The dropout rate is a regularization tool applied to reduce overfitting. In general, it is used in deep learning NN with many neurons and layers. If the dropout rate is set to say 0.20 at a particular layer (including input layer), then at that layer, 20% of the neurons or nodes will be randomly selected to “disappear” in that iteration of forward and backward propagation through that layer. In that iteration, there is no forward pass from the temporarily dropped out neurons, and there is also no updates or revisions in weights connected to those neurons during the backward propagation. In the next iteration, other neurons will be dropped out and revisions of weights continue as before.

When dropout rate is added to the training, it is sometimes suggested that the remaining neurons in that layer with dropped-out neurons have weights that are scaled up by $1/(1 - \text{dropout rate})$ so that the sum of weights of the neurons in that layer is undiminished.

Dropout can prevent overfitting by avoiding over-use of some neurons in providing the huge weights to an output. This may facilitate better prediction in generalized data.

```
[8]: from keras.models import Sequential
    from keras.layers import Dense
    from keras.layers import LSTM
    from keras.layers import Dropout

[10]: ### Initializing RNN
    model = Sequential()

[11]: ### Add first LSTM Layer and add Dropout Regularization

    model.add(LSTM(units=50,return_sequences=True,input_shape=(X_train.shape[1],1)))

    ### Sequential reads input as 3D. Add return_sequences=True for all LSTM layers except
    ### the last one. Setting this flag to True Lets Keras know that LSTM output should
    ### be 3D. So, next LSTM Layer can work further on the data. If this flag is false, then
    ### LSTM only returns last output (2D). Such output does not feed to another LSTM layer.

    model.add(Dropout(0.2))
    ### This is a hyperparameter that prevents overfit by dropping 20% of nodal values
```

```
[12]: ### Add second LSTM Layer and Dropout
model.add(LSTM(units=50,return_sequences=True))
model.add(Dropout(0.2))

[13]: ### Add third LSTM Layer and Dropout
model.add(LSTM(units=50,return_sequences=True))
model.add(Dropout(0.2))

[14]: ### Add fourth LSTM Layer and Dropout
model.add(LSTM(units=50))
### note: Last LSTM Layer does not carry argument 'return_sequences=True'
model.add(Dropout(0.2))

[15]: ### Add output layer
model.add(Dense(units=1)) ### not capital "U"nit

[16]: ### Compiling the RNN
model.compile(optimizer='adam',loss='mean_squared_error')

[17]: ### Run the training set with the LSTM (specialized RNN here)
model.fit(X_train,y_train,epochs=100,batch_size=10)
```

In code line [16], we employ optimizer “Adam” for the optimal adjustment toward loss function minimum in the backward propagation, and mean squared error MSE as the loss function to be minimized. Code line [17] performs the fitting or optimization of loss function based on the built stacked LSTM model. The batch size = 10, which means that 10 cases are grouped together into one batch for parameter update each time, so there are $1040/10 = 104$ batches. Updating through the 104 batches is one epoch. Repeating the updates involve more epochs – we use 100 epochs.

```
[18]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 50)	10,400
dropout (Dropout)	(None, 60, 50)	0
lstm_1 (LSTM)	(None, 60, 50)	20,200
dropout_1 (Dropout)	(None, 60, 50)	0
lstm_2 (LSTM)	(None, 60, 50)	20,200
dropout_2 (Dropout)	(None, 60, 50)	0
lstm_3 (LSTM)	(None, 50)	20,200
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

Next we perform the prediction (getting the LSTM output layer output) by feeding in the feature data in `X_train`. See code line [19]. As the output “`predict_train`” is multiple array format, we use code line [21] to retrieve the column containing the 1040 predicted outputs with each case.

```
[19]: predict_train=model.predict(X_train)
      print(predict_train.shape)
      ### this output is (1040,60,1), we want only the first no. in each row of 1040
      ### this 3D structure makes it more difficult to interpret comparison of prediction
      ### in training set vs output in trg set

33/33 ————— 2s 43ms/step
(1040, 1)

[20]: print(predict_train)

[[0.08865437]
 [0.08858573]
 [0.08465725]
 ...
 [0.86496216]
 [0.86246455]
 [0.87109554]]

[21]: predict_train=predict_train[:, 0]
      ### select first column or first day of 60 days of each day prediction from 1 to 1040

[22]: print(predict_train)

[0.08865437 0.08858573 0.08465725 ... 0.86496216 0.86246455 0.87109554]
```

By this step, the model is trained using the training set data. We now structure the test set data to perform prediction using the built/trained stacked LSTM model. Inputs to the model from the test set are in `X_test` and the prediction is compared with actual `y_test` using the MSE loss function.

```
[23]: ### creating data structure with 60 time-steps and 1 output
      X_test=[]
      y_test=[] ### here y_test is to collect the predicted y values
      for i in range(60,157):
          X_test.append(test_set_scaled[i-60:i, 0])
          y_test.append(test_set_scaled[i, 0])
      X_test, y_test = np.array(X_test), np.array(y_test)
      ### convert to np arrays as X_test is "list"
      X_test=np.reshape(X_test, (X_test.shape[0], X_test.shape[1],1))
```



```
[24]: y_test.shape ### now (97,) is 1 Dim -- convert to two dim

[24]: (97,)
```

```
[25]: y_test = np.reshape(y_test, (-1, 1))
      y_train=np.reshape(y_train,(-1,1))
```

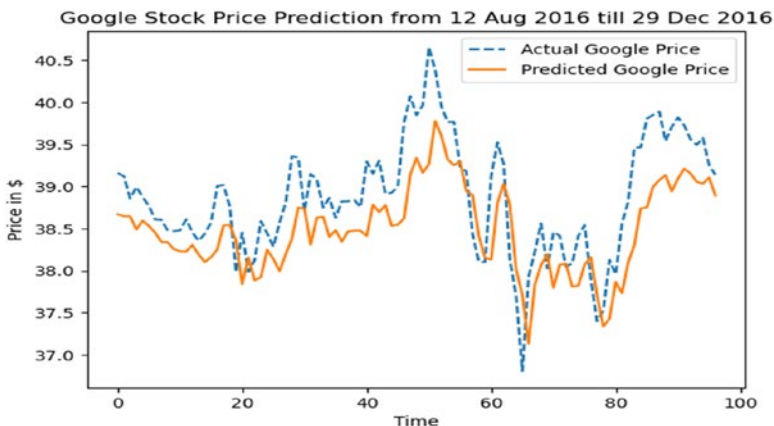
```
[26]: ### Prediction
      predicted_stock_price=model.predict(X_test)
      predicted_stock_price=predicted_stock_price[:,0]
      #predicted_stock_price1=sc.inverse_transform(predicted_stock_price)
      #predicted_stock_price.shape
```

```
[28]: from sklearn.metrics import mean_squared_error
      import math
      def print_error(trainY, testY, train_predict, test_predict):
          ### Error of predictions
          train_rmse = math.sqrt(mean_squared_error(trainY, train_predict))
          test_rmse = math.sqrt(mean_squared_error(testY, test_predict))
          ### Print RMSE
          print('Train RMSE: %.3f RMSE' % (train_rmse))
          print('Test RMSE: %.3f RMSE' % (test_rmse))

      print_error(y_train, y_test, predict_train, predicted_stock_price)

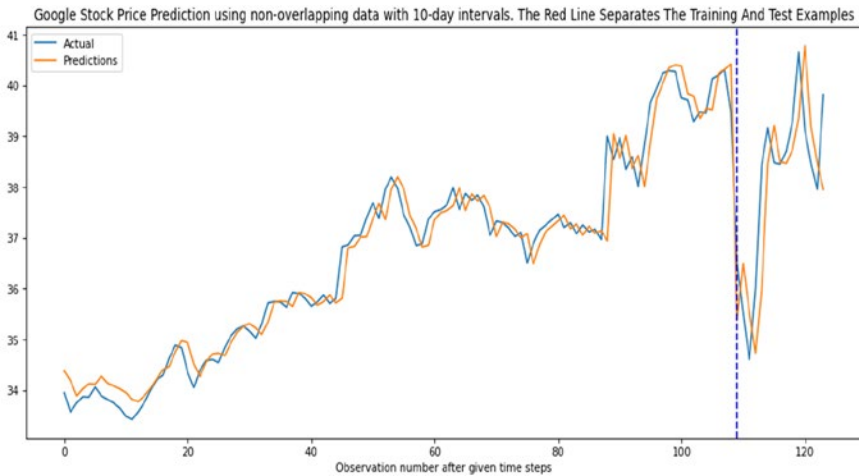
      Train RMSE: 0.021 RMSE
      Test RMSE: 0.022 RMSE
```

The RMSE for the training set fitting and then the test set prediction are about 2.1%, and 2.2%, respectively. Recall that a different run will produce slightly different RMSEs due to the random seeds in the NN. The time series of the actual Google price from 12 Aug. 2016 till 29 Dec. 2016 is compared with the predicted price based on the LSTM model above – this is shown in the following matplotlib.pyplot graph. It is seen that the predicted price tended to follow/lag the actual price.



If in the above example, non-overlapping cases are used, i.e., we use $\text{time-steps} = 10$, so every 10 daily inputs gives rise to one predicted price at the 11th day. The next case has daily inputs from day 11 to day 20 with output on day 21, and so on. See demonstration file Chapter8-3.ipynb. Then we have a total of 109 predicted prices in the training set to be matched with the actual prices in the training set, and a total of 15 predicted prices in the test set to be matched with the actual price in the test set. Number of neurons in similar four stacked LSTM layers/cells is 50.

The RMSE for the training set fitting and then the test set prediction are about 5.3% and 16.3% respectively. The time series of the actual Google price from 12 Aug 2016 till 29 Dec 2016 is compared with the predicted price based on the LSTM model above – this is shown in the following matplotlib.pyplot graph. It is seen that the predicted price tended to follow/lag the actual price.



8.4.1 *Bidirectional*

As mentioned, the traditional RNN can be improved by taking into consideration future inputs. This is not as common in the financial context as prediction of the future with some future information either means that the future events are pre-committed, e.g., a mandatory or committed public policy to be effected, or there is private information on future events so that the private investor can use that to help the prediction. However, in other AI applications involving physical or else sensory situations, future items in a sequence can be naturally helpful in prediction, e.g., predicting

missing words in a sentence when later parts of the sentence can help with a fuller context, predicting or language translation when the entire sequence of words including later words would help to make sense, and predicting or recognizing handwriting when the last sequence of strokes in pixel inputs are considered.

To enable past, current, and future inputs, BRNNs, are used. A BRNN is a combination of two RNNs (also LSTMs) – one RNN LSTM moving forward from the start of the data sequence, and the other moving backward from the end of the data sequence. The usual forward hidden state updates are h_t^f based on inputs ($X_t, 1$) and past hidden state h_{t-1}^f . See Figure 8.9.

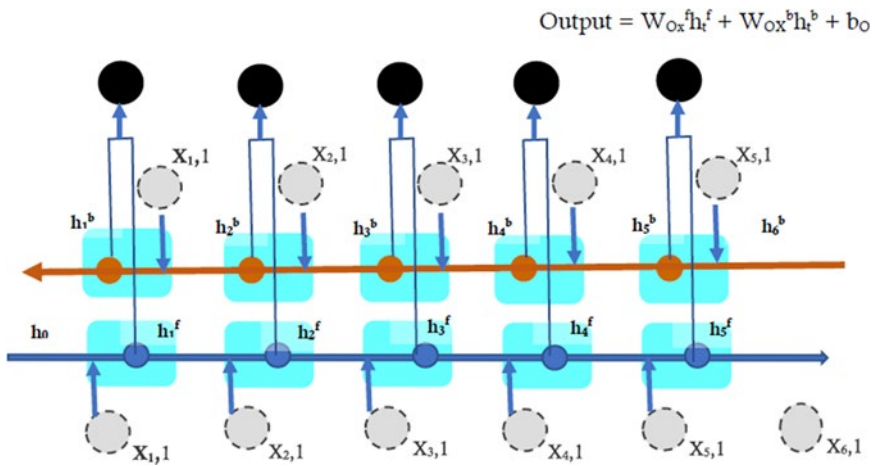


Figure 8.9. Bidirectional RNN.

The additional backward hidden state updates are now h_t^b based on inputs ($X_t, 1$) and future hidden state h_{t+1}^b . Their outputs are concatenated for final output. The diagram shows a many-to-many structure where each input is related to one output. If it is a many-to-one structure as in Figure 8.3, then the forward and the backward updating should be synchronized to yield the output at the same time point.

Other descriptions of the BRNN can be seen in https://d2l.ai/chapter_recurrent-modern/bi-rnn.html.

8.4.2 Gated recurrent unit

Just like the LSTM, the GRU is a structure to retain memory so that the gradient (hence parameter updates) will not disappear. The GRU structure

is seen as follows. This is a fully gated unit; there are other variations that are simpler. See Figure 8.10.

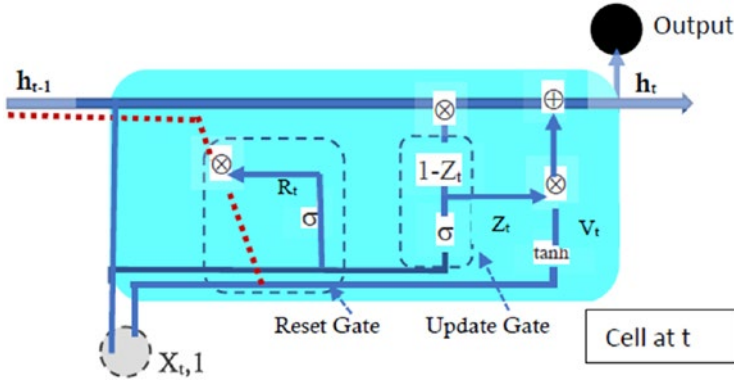


Figure 8.10. Gated recurrent unit.

GRU has less gates than in LSTM. It has a Reset Gate and an Update Gate. In the Reset Gate, the output R_t is given by $R_t = \sigma(W_{RX}X_t + W_R h_{t-1} + b_R)$. This R_t is multiplied with h_{t-1} in a Hadamard product (see dotted line) and then activated using tanh function to create a candidate activation vector, V_t . R_t is between 0 and 1 and could be close to zero, implying small V_t , where $V_t = \tanh(W_{VX}X_t + W_V(R_t \otimes h_{t-1}) + b_V)$.

The Reset Gate decides how much of the past information in h_{t-1} is to be neglected or if the hidden state is important or not. In the Update Gate, the output Z_t is given by $Z_t = \sigma(W_{UX}X_t + W_Z h_{t-1} + b_Z)$. The output hidden state $h_t = Z_t \otimes V_t + (1 - Z_t) \otimes h_{t-1}$. The Update Gate determines a weighted average of V_t and past h_{t-1} . A detailed explanation of an alternative variation – the GRU can be found in <https://towardsdatascience.com/understanding-gru-networks-2ef37df6c9be>.

8.4.3 Convolutional neural network

Another type of deep-learning neural network is Convolutional Neural Network (CNN) that is frequently applied in image identification and classification. The CNN approach can also be applied to time series data for prediction and classification. In image recognition or classification, the input is typically a 2-dimensional matrix of pixel values. A pixel value could be the representation of shade of a pixel (smallest unit or building

block of a digital image) at a location. A rugby ball could perhaps be represented as follows on the left and the figure 5 could be represented by 5×5 matrix on the right.

0	0	1	0	0
0	2	3	2	0
0	3	5	3	0
0	2	3	2	0
0	0	1	0	0

0	1	1	1	0
1	1	0	0	0
0	2	1	2	0
0	0	0	3	0
0	3	2	0	0

A colored picture could be represented by 3 sets of matrices, each corresponding to a color (RGB). In the color context, the 3 sets of filters can also be termed 3 channels. An input picture with pixel values in a huge, e.g., $10,000 \times 10,000$ matrix, could be reduced to a much smaller matrix using several operations such as convolution filters, activation, and pooling. As an example, a 7×7 input matrix on the left when convolved with a 3×3 filter (or sometimes called “kernel”) based on element by element multiplication and then summing all elements in the matrix produces 0, as seen in the output matrix.

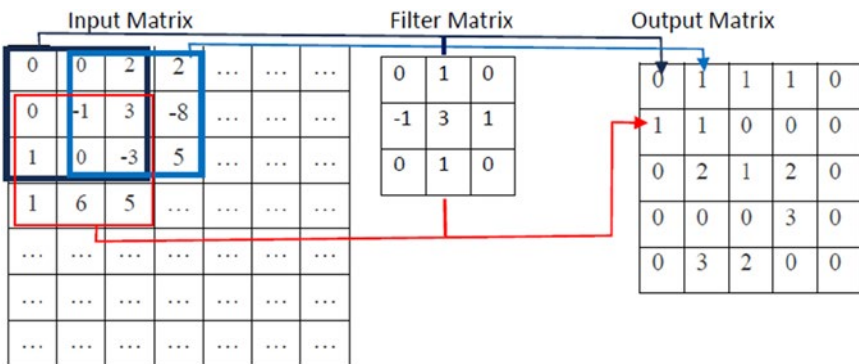


Figure 8.11. Convolution layer in CNN.

This operation is a convolution. The output matrix is also called a feature map. Often, an activation function such as ReLU is put on this summing operation to ensure the output is a positive number. Feature maps capture important and essential details of the inputs for effective learning by the algorithm.

The 3×3 filter works across the input matrix on each 3×3 sub-matrix from left to right, then top to bottom like a magnifying glass. There are two hyperparameters in the convolution operation: padding and stride. A padding of size p ($p \geq 0$) means adding an extra p first and p last rows of zeros and also an extra p first and p last columns of zeros to the matrix. Thus, a padding of $p = 2$ would mean an output matrix of 7×7 without shrinking the original image size. A stride of 1 means that the shift of the filter is one column right or one row down at a time. This is the case shown in Figure 8.11. A stride of 2 means that the shift of the filter is two columns right or two rows down at a time, and so on.

The above makes up a convolution and is a convolution layer in the CNN. It serves to sharpen the image. Often many filters (randomly initiated) are put onto one input image to derive many feature maps for use later. One could be to detect line shapes, another to detect curve shapes, and so on. Together they aggregate to form more features to identify or classify the input image or input time series.

Next the feature map is typically subject to a Pooling layer in which the feature map or output matrix in the convolution layer is reduced (reducing the number of feature inputs) via a $k \times k$ filter. The pooling filter could be a Max Pooling operation whereby the output is the maximum of all elements in the $k \times k$ sub-matrix. Or it could be a Sum Pooling or else Averaging Pooling operation.

Each filter in a convolution layer typically produces one output feature map. The outputs or feature maps could be passed through several alternating convolution and pooling layers before they are then flattened into a single dimension array of feature inputs into a dense (fully connected) neural network (layer(s) of neurons). The flattening concatenates all the final feature maps. So, if each final feature map is of dimension $k \times k$, then m feature maps mean a flattening to an input feature array of $k^2 \times m$. In a single time series prediction, the original image could be a one-dimensional array of numbers such as stock prices. The filter could be also a $k \times 1$ vector that shifts down the array of numbers to produce the feature map. This is shown in Figure 8.12.

The final output is in the output layer which is either a classification or a regression result. The CNN can be adapted to work with a dense ANN or a more complicated NN such as LSTM by coupling the convolutional and pooling layers with the NN.

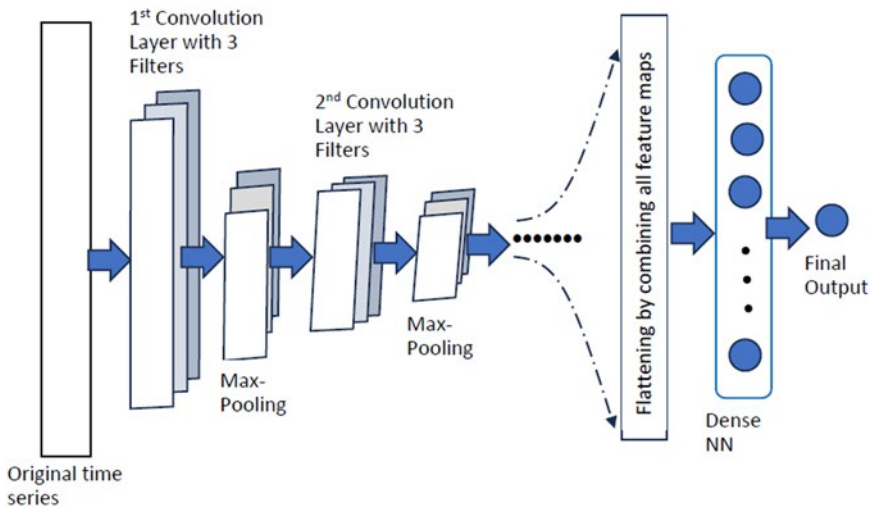


Figure 8.12. Convolutional NN.

In CNN, filters in convolution layers are initialized using random numbers in a small grid. These filter numbers are like weights, and they are updated via the NN backward propagation.

Other details of the CNN can be found in <https://towardsdatascience.com/convolutional-neural-networks-explained-9cc5188c4939/> by M. Mishra (2020).

Using the same data set of Google prices in Worked Example II (demonstration file Chapter8-2.ipynb), the CNN is applied to predict the following day stock price. The algorithm uses 12 1-D filters with zero padding, filter (kernel) size of 3, and activation function ReLU. MaxPooling (selecting maximum in non-overlapping kernel) is done with a pool size equal to 2. These are fed into a dense 100 neurons NN. Execution involves 1,000 epochs and batch size of 10. The code lines are shown as follows.

```
[39]: # Define model architecture
model = Sequential()
model.add(Conv1D(filters=12, kernel_size=3, activation='relu', input_shape=(60,1), padding='valid'))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(100, activation='relu'))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
```

```
[40]: # fit CNN model
history = model.fit(X_train,y_train,epochs=1000,batch_size=10,verbose=1)
print(history)
```

Prediction is shown in code lines [41] and [42], and the RMSE results are reported in code line [44].

```
[42]: CNN_y_train = model.predict(X_train, verbose=0)
      print(CNN_y_train)
```

```
[[0.08787928]
 [0.08102182]
 [0.08562031]
 ...
 [0.85767883]
 [0.8520642 ]
 [0.8435601 ]]
```

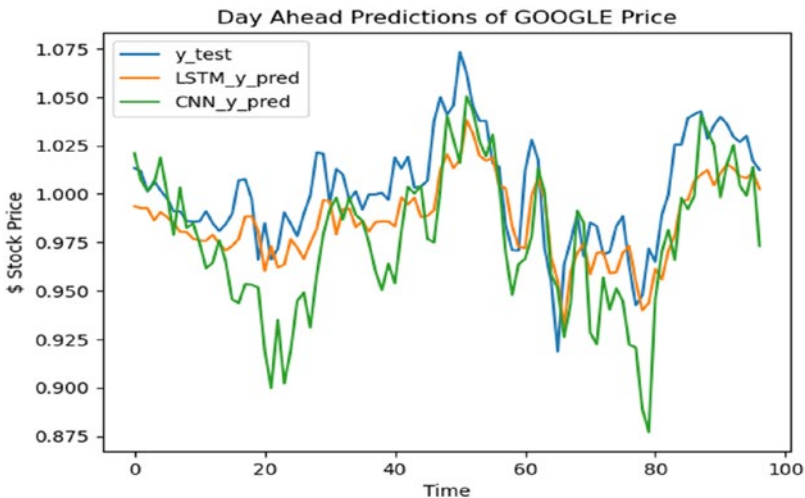
```
[43]: # Testing the model
CNN_y_pred = model.predict(X_test, verbose=0)
print(CNN_y_pred)
```

```
[[1.0208956 ]
 [1.0069052 ]
 [1.0016748 ]
 [1.0052946 ]
 .....
 .....
 .....]
```

```
[44]: from sklearn.metrics import mean_squared_error
import math
def print_error(trainY, testY, train_predict, test_predict):
    ### Error of predictions
    train_rmse = math.sqrt(mean_squared_error(trainY, train_predict))
    test_rmse = math.sqrt(mean_squared_error(testY, test_predict))
    ### Print RMSE
    print('Train RMSE: %.3f RMSE' % (train_rmse))
    print('Test RMSE: %.3f RMSE' % (test_rmse))

print_error(y_train, y_test, CNN_y_train, CNN_y_pred)

Train RMSE: 0.011 RMSE
Test RMSE: 0.036 RMSE
```



8.5 Concluding Thoughts

In this chapter, we see some of the deep Neural Networks and also see examples of using them to predict VIX and also a stock price. The standard ML process is used involving training the model (or fitting the model on the training data) and then testing the trained model. Hyperparameters that are involved in the model fitting are assumed to be optimized in the training process. Otherwise, a validation data set would have to be used to do the hyperparameter fine-tuning. Doing it all on the training data set and not cutting off a piece of the data for validation has the advantage of using enough data on a time series. However, it has the disadvantage of losing some robustness as validation and also a testing provide a double check before the model is rolled out for real action in generalized data and on real problems that may yield costly decisions if the model is not performing.

In the training and testing for a next day stock price prediction, we show that it is suitable to deploy a rolling window with a sequence of training data prices followed by a test data point. Thus, it is not the usual deployment of a sequential training data set and a sequential test data set. The one-period ahead training obviously uses the most recent information and is in line with the idea of market informational efficiency, in whatever degree.

For stock price prediction for trading purposes, there are two major caveats. One is transactions costs. Any apparent trading profits can be

decimated or vanquished by transaction fees/costs, impact costs/widening bid-ask spread if more want to buy (ask goes up) or if more want to sell (bid goes down), liquidity risk – when it is not possible to trade as frequently as in the training/testing using past data, and slippages in market order – getting higher purchase price or lower selling price for your order (effect same as in execution risk when latency – time delay between order and actual trade – is not as fast as competitors).

Two is the technical aspect of the prediction by minimizing mean-square errors. The results for Google indicate that the NNs are able to provide forward predictions with a small MSE. However, a closer look may reveal that most of the time, the algorithm is trend chasing. Hence the predicted curve appears to be a forward shift of the past price curve. Trading profit may be possible if the trend persists and that there is no major price reversals. Otherwise, a buy on an upward trend (or a sell on a downward trend) in the face of a major market fall (rise) would face huge losses (unless one has the capital to hold on and suffer marked-to-market losses) that can easily wipe out all the previous trend-chasing gains.

More meaningful ML models for trading could include training based not just on past prices, but using market microstructure features in high frequency trading (HFT) such as queue order level 2 quotes (market depth) or even level 3 (who are the buyers/sellers in the queue), the momentum such as average price movements in the last few intervals, bid-ask spread, volume, a market-sentiment measure, firm size, price-earnings ratios, price-to-book ratios, profit news, competitor prices, industry price, market movements, interest rates, and so on. And importantly, the model is effective only when back-testing on a trading strategy, e.g., buy low and sell high or sell high and buy low, can yield a significant profit.

References

- <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>.
- https://d2l.ai/chapter_recurrent-modern/lstm.html.
- https://d2l.ai/chapter_recurrent-modern/bi-rnn.html.
- <https://towardsdatascience.com/convolutional-neural-networks-explained-9cc5188c4939/>.

Index

A

- accounting features, 146
- accounting ratios, 90
- accounting statements, 65
- accuracy, 239
- accuracy measure, 36, 71
- activation, 279
- activation function, 216
- Adam (adaptive moment estimation), 225
- Adam update rule, 226
- AI applications, 276
- annualized return, 14
- architecture, 250
- area under the curve (AUC), 75
- area under the receiver operating characteristics (AUROC), 75
- Artificial Neural Network (ANN), 208
- averaging pooling, 280

B

- back-test, 34
- backward derivative, 223
- backward hidden state, 277
- backward propagation, 214, 247
- backwards pass, 220–221

- bagging approach, 169
- balance, 138
- balance sheet, 66
- batch size, 238, 254
- Bayes' rule, 95
- Bayes' theorem, 95
- beeswarm plot, 201
- Bernoulli distribution, 79
- biases, 223, 268
- bidirectional RNN (BRNN), 247
- binarizes, 164
- binary classification, 70
- binary cross-entropy, 218
- boosting regression trees, 171
- boosting trees, 173
- bootstrap aggregation ("bagging"), 152
- bootstrapping, 57
- boundary line, 109
- branching, 139

C

- cashflow statement, 66
- categorical, 99
- categorical variable DT, 138
- cell state, 265
- cell update gate, 266

classification and regression tree (CART), 144
 classification report, 132
 click-through rate (CTR), 214
 coalition game, 193
 concatenated inputs, 253, 267
 conditional heteroskedasticity, 34
 conditional probability, 93
 conformable vector, 22
 confusion matrix, 71
 constant residual eigenvalue method, 34
 constrained optimization, 116
 continuously compounded, 8
 continuous variable DT, 138
 convolutional network (CNN), 247
 convolution filters, 279
 convolution layer, 280
 cooperative game, 193
 covariance matrix, 3
 credit ratings, 146
 cross-sectional prediction, 38
 cross-validation, 39
 cross-validation score, 59
 customer loyalty, 227

D

decision boundary, 111
 decision node, 138
 decision rules, 137
 decision tree, 137
 DecisionTreeClassifier, 143
 DecisionTreeRegressor, 143
 deep learning NN, 247
 denoised, 31
 denoising, 29
 dense ANN, 280
 dense layer, 230
 depth, 229
 dichotomous classes, 112
 dimension reduction, 89

dropout rate, 272
 dummy value, 103

E

edges, 213
 efficient portfolio frontier, 3
 eigenvalue, 22, 87
 eigenvectors, 22, 88
 elastic net regression, 42
 empirical pdf, 28
 empirical probability, 183
 ensemble, 152
 epoch, 214, 254
 epsilon tube, 125
 Euclidean distance, 107
 Euclidean norm, 113
 Euclidean space, 114
 ex-dividend date, 7
 exogenous input features, 252
 explained variance, 88
 exploding gradient, 257

F

F1-score, 74
 false negative, 70
 false positive rate, 70, 75
 fear gauge, 258
 feature importance, 159, 193
 feature map, 279
 feature selection, 55
 features, 35, 214
 feedforward NN, 247
 filter, 280
 final iteration, 234
 fine-tuning, 38
 first hidden layer, 248
 fitted hyperplane, 124
 fitted model, 35
 fitting errors, 35
 forget gate, 266
 forward hidden state, 277
 forward layer, 215

forward prediction, 32
 forward propagation, 214
 fraud detection, 127
 frequency counts, 137
 Friedman MSE, 177

G

gated recurrent unit (GRU), 247
 Gaussian kernel, 120
 Gaussian Naïve Bayes (GNB), 101
 Gaussian radial basis function, 120
 generalized data, 35
 Gini impurity, 140
 gradient adjustment, 204
 gradient boosting, 167, 169
 GradientBoostingClassifier, 185
 gradient descent, 185
 gradient descent method, 220
 “greedy” algorithm, 146

H

Hadamard product, 267
 hidden layer, 212, 215, 229
 hidden state, 252
 hidden state output, 268
 high dimensionality, 166
 hinge loss function, 122
 historical returns, 32
 hold-out data, 35
 housing prices, 44
 Huber loss function, 218
 Hyperbolic Tangent function, 216
 hyperparameter, 35

I

imbalanced data, 71
 income statement, 66
 information gain, 176, 183
 input gate, 266
 in-sample fit, 37
 iteration, 224

K

kernel, 25, 279
 kernel trick, 111
 k-fold, 57
 k-Nearest Neighbors (k-NN), 95

L

L1-norm, 43
 L2-norm, 43
 labels, 36, 143, 214
 Lagrange duality, 116
 Lagrange multipliers, 117
 Lagrangian, 117
 Lasso regression, 43
 lazy learning, 107
 leaf, 139
 least squares, 40
 LightGBM, 183
 likelihood function, 79, 178
 limiting the depth, 140
 linear kernel, 120
 linear regression, 35
 Local Interpretable Model-agnostic Explanations (LIME) model, 200
 logistic regression, 79, 242
 LogisticRegression, 190
 logit model, 79
 log loss, 141, 178
 log odds, 178
 long short-term memory (LSTM), 247
 loss function, 37, 178
 lower entropy, 141

M

machine learning model, 35
 macro average of F1-score, 74
 macro average of precision, 74
 macro average of recall, 74
 majority voting, 108
 Manhattan distance, 107
 many-to-many, 210

many-to-one, 210
 Marcenko–Pastur theorem, 22
 margin, 112
 marginal contribution, 196
 market sentiment, 258
 marksman, 138
 Matplotlib, 1
 matrix multiplication, 2
 maximum entropy, 141
 maximum margin, 109
 maximum separation, 109
 mean absolute error (MAE), 173, 217
 mean squared errors (MSE), 143, 170, 217
 mean-variance analysis, 3
 Mercer’s theorem, 119
 Micro average precision, 74
 Micro average recall, 74
 microstructure features, 284
 mini-batch, 215, 224, 254
 minimum cross entropy, 35
 minimum mean square error, 35
 minimum variance portfolio, 3
 Minkowski distance, 107
 MinMaxScaler, 63, 270
 missing data, 184
 model parameters, 35
 Monte Carlo sampling, 198
 multiclass classification, 231
 multi-class problems, 146
 multi-class separation, 123
 multi-collinear, 41
 multi-layer perceptron (MLP), 208
 multinomial, 99

N

Naïve Bayes algorithm, 95, 99
 Nash equilibrium, 193
 natural language processing, 97
 nearest-neighbor algorithm, 107
 neighborhood, 200

neural network (NN), 207
 neurons, 207
 non-convexity, 182
 nonlinear kernel, 119
 non-overlapping, 262
 non-overlapping data, 35
 non-parametric algorithm, 108
 nonparametric estimation, 137
 normalized, 22
 null hypothesis, 70
 numerical iterative procedure, 79
 numpy, 1

O

one-vs.-all (OvA), 123
 one-vs.-rest (OvR), 123
 optimal boundary, 109
 optimal splitting, 172
 optimal weights, 14
 orthogonal, 22
 orthogonality, 87
 outliers, 41
 out-of-sample, 21, 37
 output gate, 266
 output layer, 215, 240
 overfit, 37
 overshooting, 171

P

packs, 262
 padding, 280
 Pandas, 1
 parallel hyperplanes, 109
 parallel processing, 184
 penalty, 122
 penalty function, 55, 200
 perfect discrimination, 140
 performance metrics, 154
 permutations, 196
 perpendicular projection, 115
 pixel values, 278
 polynomial function, 110

polynomial kernel, 119–120
 pooling layer, 279–280
 positive-definite matrix, 22, 87
 precision, 72
 predicted outputs, 264
 prediction error, 171
 prediction performance, 37, 40
 prediction rules, 140
 primal, 117
 principal component, 24
 Prisoner's Dilemma problem, 193
 pruning, 212
 pseudo residuals, 176

R

R^2 test score, 37
 random forest, 138
 RandomForestClassifier, 156
 random matrix, 22
 random sampling, 183
 recall, 72
 receiver operating characteristics (ROC), 75
 rectified linear unit, 230
 rectifier linear unit function (ReLU), 216
 recurrence, 252
 recurrent cells, 254
 recurrent neural network (RNN), 247
 recursive distribution, 142
 regression tree model, 143
 regularization, 37
 resampling procedure, 56
 reset gate, 278
 residual noise, 39
 ridge regression, 42
 risk-free rate, 4
 robustness check, 37
 root mean square error (RMSE), 40, 217
 root node, 150

S

scatter plot, 18
 sensitivity, 72
 separating hyperplane, 115
 sequential errors, 176
 sequential input, 249
 SGD optimizer, 241
 Shapley value, 193
 SHAP value, 197, 199, 244
 Sharpe ratio, 4
 short-sale constraint, 3
 shrinkage estimators, 43
 sigmoid function, 216, 266
 single hidden layer MLP, 220
 single layer perceptron, 208
 skewness, 33
 smoker, 138
 softmax, 231
 specificity, 75
 stacked layer, 269
 stacked LSTM, 268
 stacking, 205
 standardization, 93
 standardized, 235
 StandardScaler, 63
 stepwise regression, 42
 stochastic gradient descent (SGD), 224
 stock dividend, 8
 stock price prediction, 283
 stock split, 8
 stride, 280
 sum pooling, 280
 supervised learning, 138
 supporting vector, 113
 support vector, 115
 support vector machine (SVM), 95
 support vector regression (SVR), 95

T

tanh function, 278
 target errors, 174

- target variable, 35
- Taylor series expansion,
 - 178
- terminal nodes, 139, 180
- test data, 35
- test set data, 21
- testing, 35
- threshold, 76, 139
- time series, 247
- total coalition, 195
- total errors, 171
- trace, 24
- training data, 35
- training iterations, 234
- tree pruning, 184
- true negative, 70, 85
- true negative rate, 75
- true positive, 70, 85
- true positive rate, 72
- type 1 error, 70

U

- underfitting, 37
- update gate, 278

V

- validation data set, 35, 214
- validation R2 score, 42
- vanishing gradient, 257
- Vector autoRegressive (VaR), 253
- volatility index (VIX) , 258
- volatility errors, 21

W

- “waterfall” plot, 203
- weak predictor, 170
- weights, 213
- width, 229

X

- XGBoost (Extreme GB), 184